Privacy-Preserving Shopping Intelligence via Federated and On-Device Learning

October 20, 2026

Group Members:

1. Ahmed Sherif
2. Eid Abdelrihem
3. Ahmed Abdelhamed

Supervised by

Professor Rongxing Lu
School of Computing
Queen's University
Kingston, Canada

Professor Mona Farouk
Computer Engineering
Cairo University
Cairo, Egypt

Professor Motaz El-Saban
Industry Mentor
Microsoft

A thesis submitted in partial fulfillment of the requirements for the MSc in Artificial Intelligence and Data Science, School of Computing, Queen's University, Kingston, Canada.

Dedication

To our parents and families, whose patience, encouragement and sacrifices carried us through every late night of this work. Their belief in us has been our greatest source of strength.

Acknowledgements

We would like to express our sincere gratitude to our supervisors. Prof. Rongxing Lu of the School of Computing, Queen's University, and Prof. Mona Farouk of the Computer Engineering Department, Cairo University, guided the academic direction of this work. Prof. Motaz El-Saban of Microsoft, our industry mentor, framed the problem around a real browser product and kept us close to the questions a deployment team would actually ask. Their feedback shaped both the research question and the way we chose to answer it.

We thank the School of Computing at Queen's University for access to the Lobot GPU cluster. Every large experiment in this report ran on it, and the technical support team helped us when we needed more GPU memory and storage.

We are grateful to the Ministry of Communications and Information Technology of Egypt and to the Digilians presidential initiative, whose scholarship and programme made this Master's degree possible, and to the Egyptian Military Academy for its role in delivering the programme.

Finally, we thank our families for their unconditional moral and practical support throughout this journey.

Abstract

A browser that recommends products could learn from the shopping behaviour that happens inside it. Collecting that behaviour on a server, however, is exactly what privacy-conscious users and regulators increasingly reject. This thesis asks how much next-product recommendation quality can be retained when moving from cloud-based models to on-device or federated models. We study the question on REES46, a public e-commerce log from October and November 2019. The task is to rank the full catalogue of 158,486 products. The study covers 131,072 simulated client devices, a server-side population of 1,217,610 users and a held-out test population of 26,254 users. The models are SASRec-family sequential recommenders. They are trained centrally, with federated learning (FedAdam), from a server-side pretrained start, on the device only, and with on-device fine-tuning. Differential privacy, 8-bit uploads and INT8 inference were added on top, and secure aggregation was simulated.

To protect the validity of the answer, the main comparison was pre-registered. Every analysis decision was locked before the single confirmatory look at the test set, and each later exploratory look was gated and recorded.

On the registered recipe, the realistic edge system retains 88.49% of a full-data cloud model in client-macro MRR@20 (95% CI 87.73–89.18); this system is federated training warm-started from a server pretrained model. Federation adds no value over the pretrained model alone (−0.000508 MRR@20), and a tuned follow-up is statistically equivalent to it. On-device fine-tuning gives a small, significant gain (+0.002283). Training on the device alone retains only 2.10% of a cloud model trained on the same devices' users (1.64% of the full-data model).

A post-hoc diagnosis found that the registered FedAdam setting ($\tau = 10^{-3}$) almost froze federated training; the corrected recipe still adds nothing measurable over the 1.2-million-user pretrained model on REES46 (+0.00022 [−0.00011, 0.00057]). In exploratory looks with a corrected recipe ($\tau = 10^{-6}$ together with other changes), federated models reach 75–92% of the large central model (C_FULL) on six public benchmarks. On REES46, the value of federation depends on how much data the server already has: it adds +0.0091 MRR@20 over a pretrained model built on 100,000 users and nothing over one built on 1.2 million.

In an exploratory audit, a federated model trained from scratch with the corrected optimiser leaks membership (attack AUC 0.747), whereas the warm model built on 100,000 users showed no material leakage (AUC 0.537). A later audit found that the legacy differential-privacy implementation does not establish a formal guarantee, that an aggregate-update attack exceeds our 2-percentage-point threshold, and that secure aggregation was only simulated.

We conclude that on-device and federated shopping recommendation is feasible and retains most of the

cloud quality, but whether federation pays off depends on the server-side pretraining and on a correctly scaled optimiser. Privacy has to be engineered and audited, not assumed. We report every experiment of the project in sequence, together with what each one did and did not teach us.

Keywords: federated learning; on-device learning; sequential recommendation; SASRec; differential privacy; membership inference; secure aggregation; pre-registration; e-commerce; REES46.

Contents

Acknowledgements . . . . . **ii**

Abstract . . . . . **iii**

Table of Contents . . . . . **v**

List of Figures . . . . . **x**

List of Tables . . . . . **xii**

List of Abbreviations and Arm Names . . . . . **xv**

1 Introduction **1**

1.1 Background 1

1.2 Problem statement 2

1.3 Motivation 2

1.4 Research question and objectives 3

1.5 Scope and delimitations 3

1.6 Contributions 4

1.7 The research journey at a glance 4

1.8 Organisation of the report 5

2 Background and Literature Review **7**

2.1 Sequential recommendation 7

2.2 Evaluating recommenders 10

2.3 Federated learning 11

2.4 Federated recommendation 13

2.5 Privacy in federated learning 15

2.6 On-device inference 16

2.7 Pre-registration and statistics 18

2.8 The research gap 19

3 Methodology **21**

3.1 Research design 21

3.2 Data 22

3.3 Task and metrics 24

3.4 Populations and splits . . . 25
3.5 The model . . . 26
3.6 Training regimes and arms . . . 27
3.7 The federated recipe . . . 27
3.8 Privacy mechanisms . . . 30
3.9 On-device measurement . . . 31
3.10 Statistics . . . 32
3.11 Pre-registration and governance . . . 33
3.11.1 Rules, amendments and locks . . . 33
3.11.2 The single test evaluation and the exposure register . . . 33
3.11.3 The follow-up analysis . . . 34
3.11.4 Operating model with AI agents . . . 34
3.12 External benchmarks . . . 35
3.13 Tools and compute . . . 36

4 Design and Implementation . . . 38
4.1 Architecture overview . . . 38
4.2 The prototype platform on GitHub . . . 38
4.2.1 Repository, workstreams and environment . . . 38
4.2.2 The data pipeline . . . 40
4.2.3 The model workstream . . . 40
4.2.4 Federated plumbing with Flower . . . 41
4.3 The system, deployment and security workstream . . . 42
4.3.1 Environment audit and simplification . . . 42
4.3.2 Repository safety, threat model and secret handling . . . 42
4.3.3 Deterministic features . . . 43
4.3.4 Measurement discipline . . . 43
4.3.5 ONNX export and the dynamic-batch fix . . . 44
4.3.6 INT8 quantisation and the reversal . . . 44
4.3.7 Continuous integration and the demonstration specification . . . 45
4.4 The main-study research platform . . . 46
4.4.1 Packages and frozen code trees . . . 46
4.4.2 The evaluation harness . . . 46
4.4.3 The federated engine: FedAdam and the DP accountant . . . 47
4.4.4 Privacy tooling and the Edge device bench . . . 47
4.5 Governance tooling . . . 48
4.6 Infrastructure and operations (Lobot) . . . 49
4.7 Verification and testing . . . 49
4.8 Delivery and consolidation . . . 51

4.8.1 The merged folder of 2026-10-17 . . . 51
4.8.2 The public release, PRs #103–#114 . . . 51

5 Results and Evaluation . . . 53

5.1 Overview of the evaluation . . . 53
5.2 The prototype stage on GitHub . . . 56
5.2.1 Data audit, cohort and task headroom . . . 56
5.2.2 The GRU model workstream and the architecture comparison . . . 58
5.2.3 The T2 label defect and its correction . . . 59
5.2.4 Negative transfer and the joint loss . . . 59
5.2.5 T3 learned heads and the miss decomposition . . . 60
5.2.6 Federated plumbing and system measurements . . . 60
5.2.7 The scoped prototype and the 88.38% quality retention . . . 61
5.2.8 Why the project moved to a stronger, pre-registered main study . . . 62
5.3 The pilot studies . . . 63
5.3.1 A frozen dataset . . . 63
5.3.2 The GRU pilot study . . . 64
5.3.3 The SASRec pilot study . . . 65
5.4 Foundations of the main study: census, population, catalogue and features . . . 67
5.4.1 Census, population and catalogue . . . 67
5.4.2 The feature search and the input schema . . . 68
5.4.3 Engineering measurements and the first tuning result . . . 69
5.5 The backbone contest and the architecture lock . . . 70
5.5.1 Equal-effort tuning . . . 70
5.5.2 The eight contest runs on MAIN . . . 71
5.5.3 Baselines, the lock and the incidents . . . 72
5.6 Calibration and the federated recipe tuning stages . . . 73
5.6.1 References and rules of the tuning stages . . . 73
5.6.2 The local learning rate and the grid edge (tuning stages 0 and 1) . . . 74
5.6.3 Server optimiser, client optimiser and local computation (tuning stages 2 to 4) . . . 75
5.6.4 The warm line: pretraining and frozen item tables (tuning stages 7 and 8) . . . 76
5.6.5 Device arms, DP constants and the MAIN launches . . . 77
5.7 Benchmark design and the cold-start chain . . . 78
5.7.1 Choosing the data sets and cutting the scope . . . 79
5.7.2 Protocol and the reproduction of published baselines . . . 79
5.7.3 Amendment 07, the design gate and the runner . . . 80
5.7.4 The cold-start chain and amendment 06P . . . 80
5.8 MAIN completion, the VALIDATION read and the follow-up analysis design . . . 82
5.8.1 Finishing the MAIN runs and the endgame gates . . . 82

5.8.2 The single VALIDATION read . . . 82
5.8.3 Was the warm recipe at fault? . . . 84
5.8.4 The follow-up analysis design . . . 84
5.8.5 Building the follow-up analysis . . . 85
5.9 Follow-up analysis runs, device measurements and the membership audit . . . 86
5.9.1 Part B reduced . . . 86
5.9.2 Selection, futility and the confirmatory MAIN runs . . . 87
5.9.3 The membership audit and the secure-aggregation simulation . . . 88
5.9.4 Device measurements on the tuned model . . . 89
5.9.5 Defects found before the test evaluation . . . 90
5.10 Confirmatory TEST results . . . 90
5.10.1 The test evaluation . . . 91
5.10.2 The main analysis: the key contrasts . . . 92
5.10.3 The follow-up analysis: Q1 is equivalent . . . 94
5.10.4 The benchmark TEST looks . . . 96
5.10.5 Comparison with published full-ranking results . . . 98
5.11 Diagnosing the FedAdam tau effect . . . 100
5.12 The fixed recipe on the benchmarks and on REES46 . . . 104
5.13 How much federation is worth depends on the server's data . . . 107
5.14 Privacy evaluation . . . 111
5.15 On-device feasibility . . . 115
5.16 Closure experiments . . . 117
5.17 Summary of all experiments . . . 120

6 Discussion . . . 123
6.1 Answering the research question . . . 123
6.2 Were the objectives met? . . . 123
6.3 Interpreting the main findings . . . 124
6.4 Practical implications for a browser shopping assistant . . . 125
6.5 Relation to prior work . . . 126
6.6 Limitations and threats to validity . . . 126
6.7 Lessons about the research process . . . 127

7 Conclusion and Future Work . . . 128
7.1 Summary of the work . . . 128
7.2 Answer to the research question . . . 128
7.3 Contributions . . . 129
7.4 Future work . . . 129
7.5 Closing remarks . . . 129

References **131**

A Pre-registration lineage, locks and gates **136**

A.1 Governing rules 136

A.2 Amendment timeline 136

A.3 Locks, freezes and result files 138

A.4 The single VALIDATION read and the confirmatory test evaluation 138

B Complete confirmatory results **140**

C TEST exposure register **146**

D Incident and fix log **147**

E Experiment register **149**

F Reproducibility and source code guide **152**

F.1 Running the public repository 152

F.2 Result files, data and checkpoints 153

G Contribution and AI-use statements **154**

List of Figures

1.1 Online shopping happens on the shopper's own device: browsing, comparing and paying in a web browser. The events of such sessions are the data this project learns from. . . . 1
1.2 Three ways to deploy a shopping recommender 2
1.3 The research journey in fourteen phases 5

2.1 A data-centre server room. In the cloud-only setting, every shopper's events are copied to machines like these and the model is trained there; federated learning was proposed to avoid that copy. 11
2.2 The device side of the system. In the federated and on-device settings, the shopper's history and the model that ranks products stay on a phone or laptop such as this one. . . 17

3.1 The October 2019 REES46 log in the data-audit notebook 23
3.2 Populations, CALIB, TEST and the task 25
3.3 SASRec model and sizes 26
3.4 One FedAdam round and the role of τ 28
3.5 Gated TEST protocol and exposure register 33

4.1 System architecture 39

5.1 Prototype temporal split, cohort trade-off and task headroom 57
5.2 First GRU run on real REES46 sessions 58
5.3 The cold-start federated recipe tuning stages 0 to 4 75
5.4 Confirmatory retention against the cloud reference 93
5.5 Forest plot of the key deltas 96
5.6 Our cloud reference against published results 99
5.7 Update norms of the follow-up federated runs 103
5.8 Raw trajectories of the tuned warm-start federated model 103
5.9 Registered and tau-fixed federated arms on six benchmarks (TEST) 106
5.10 Value of federation against the size of the server pretrain 108
5.11 Disjoint server band, 15% and 50%, warm against cold (validation) 109
5.12 Capacity decomposition (validation) 110
5.13 Membership-inference AUC and DP utility retention 113
5.14 Simulated secure-aggregation upload overhead 115

5.15 Edge WebAssembly latency, FP32 against INT8 116
5.16 Fine-tuning on moving models and the second pretrain seed 118
5.17 Training-free baselines on REES46 119

List of Tables

2.1 Sequential recommenders used as reference points . . . 8
2.2 Published full-ranking NDCG@10 of neural and transition models . . . 9
2.3 Federated recommenders discussed in this chapter . . . 14
2.4 Privacy tools for federated learning and their limits . . . 15
2.5 Research gap relative to the cited literature . . . 19

3.1 Populations and roles. Ranks refer to the pseudonymous selection key. Source: `HANDOVER/02_DATA.md`. . . . 26
3.2 Systems and arms. "Leaves the device" lists what a user's device sends. Levels follow the privacy ladder of amendment 06L_C; L2 carries no formal guarantee. Variants on the frozen-table model carry the suffix `_WARM_FROZEN`. Source: `HANDOVER/01_RESEARCH_QUESTION_AND_DESIGN.md`. . . . 28
3.3 The cold federated recipe tuning stages on CALIB (*descriptive*). Value: E2E client-macro MRR@20 at PB, `FA`, seed 2026, full 6.0 budget, CALIB validation users. Source: `contracts/fl/` and `FL_IMPROVEMENT_LADDER.md`. . . . 29
3.4 Contrast hierarchy of the registry `ANA_CONTRASTS_v4` (hash prefix `f1769f0b`). Source: `HANDOVER/01_RESEARCH_QUESTION_AND_DESIGN.md`. . . . 32
3.5 The six public benchmark datasets. Source: `results/benchmarks/datasets.csv`. . . . 36

4.1 Federated runtime scale profile . . . 41
4.2 FP32 against dynamic INT8 . . . 45
4.3 Main-study packages . . . 47
4.4 Technical-closure verification counts . . . 50

5.1 The three evaluation regimes of the chapter . . . 54
5.2 Roadmap of the fourteen phases . . . 55
5.3 Prototype data audit and cohort . . . 57
5.4 Prototype headroom and model-free baselines . . . 58
5.5 Prototype architecture comparison . . . 59
5.6 Prototype federated and system measurements . . . 61
5.7 Scoped prototype pilots . . . 62
5.8 The pilot-study dataset . . . 64

5.9 Learning curves of the pilot-study models . . . 65
5.10 Pilot-study selection and retrospective looks . . . 66
5.11 Main-study census and populations . . . 68
5.12 Permutation importance of input groups . . . 69
5.13 Tuning outcomes on CALIB . . . 71
5.14 Contest runs on MAIN validation . . . 71
5.15 The federated recipe tuning stages 0 to 8 on CALIB . . . 74
5.16 Registered warm-start screens of tuning stages 7 and 8 on CALIB . . . 77
5.17 Amendments that changed the federated recipe . . . 78
5.18 Benchmark candidates that were dropped . . . 79
5.19 Defects caught before the first benchmark run . . . 81
5.20 The endgame gate chain . . . 83
5.21 Headline VALIDATION contrasts . . . 83
5.22 Follow-up selection on CALIB and the MAIN run facts . . . 87
5.23 Membership-audit status of the follow-up targets . . . 88
5.24 E4 secure-aggregation simulation . . . 89
5.25 Browser latency of the tuned model . . . 89
5.26 The confirmatory test evaluation . . . 91
5.27 Key contrasts of the main analysis on TEST . . . 92
5.28 VALIDATION against TEST . . . 93
5.29 Absolute MRR@20 on MAIN TEST . . . 94
5.30 Follow-up contrasts on TEST . . . 95
5.31 Benchmark TEST looks . . . 97
5.32 Best federated arm against the cloud reference . . . 97
5.33 Our cloud reference against published results . . . 98
5.34 Exploratory TEST looks after the confirmatory test evaluation . . . 101
5.35 Measurements behind the FedAdam tau diagnosis . . . 102
5.36 Fixed-recipe federated arms on the six benchmarks (TEST) . . . 105
5.37 Tau-fixed REES46 contrasts on TEST . . . 106
5.38 Value of federation with a 100,000-user pretrain (TEST) . . . 108
5.39 Disjoint server band: warm federated learning on TEST . . . 110
5.40 Membership-inference audit . . . 113
5.41 Update-leakage audit . . . 114
5.42 Edge WebAssembly latency of the exported model . . . 116
5.43 Second pretrain seed: matched warm contrasts . . . 117
5.44 Summary of all experiments, phases 01 to 14 . . . 121

6.1 Assessment of the research objectives . . . 124

A.1 Amendment timeline of the pre-registration 136
A.2 Locks, freezes and result files with sha prefixes 138

B.1 All 33 main-analysis confirmatory contrasts on the REES46 TEST set (client-macro MRR@20, practical-best checkpoints, 26,254 users). Source: `results/rees46/confirmatory_phase1_contrasts.csv`, generated from `RETENTION_RESULT.json` of the confirmatory test evaluation. 140
B.2 Follow-up confirmatory contrasts (registered follow-up analysis, TEST, client-macro MRR@20). Source: `results/rees46/confirmatory_phase2_contrasts.csv`. 142
B.3 Absolute TEST metrics of every main-analysis arm and baseline (mean of the registered seeds). Source: `results/rees46/confirmatory_phase1_arms.csv`. 142
B.4 Confirmatory benchmark results, track A (TEST, full ranking, mean of seeds 2026 and 2027). Source: `results/benchmarks/confirmatory_track_a.csv`. 143

C.1 TEST exposure register, entries 1 to 23 146

D.1 Condensed incident and fix log 147

E.1 Condensed experiment register 149

List of Abbreviations and Arm Names

Term	Meaning
AUC	Area under the ROC curve (used for membership-inference attacks)
CALIB	Calibration subset of the federated population, used for tuning and selection
CB / PB	Controlled-budget / practical-best checkpoint rule
CI	Confidence interval (paired user bootstrap unless stated otherwise)
DL	Decision log of the project (append-only record of every action)
DP	Differential privacy; DP8 / DP1 denote $\varepsilon = 8$ / $\varepsilon = 1$
Epochs (passes)	Training-budget unit: one pass is one exposure of every eligible TRAIN example
FL	Federated learning
FedAvg, FedProx, FedAdam	Federated optimisation algorithms (Chapter 2)
GRU	Gated recurrent unit (GRU4Rec family)
HR@k	Hit rate at cut-off k
INT8 / FP32	8-bit integer / 32-bit floating-point model precision
MIA	Membership-inference attack
MRR@20	Mean reciprocal rank of the true next item within the top 20
NDCG@10	Normalised discounted cumulative gain at cut-off 10
ONNX	Open Neural Network Exchange model format
REES46	Public e-commerce behaviour log (multi-category store, Oct–Nov 2019)
SASRec	Self-attentive sequential recommendation model
SecAgg	Secure aggregation
TOST	Two one-sided tests (equivalence testing)
τ (tau)	FedAdam adaptivity constant that bounds the server step
Arms (trained systems)	
C_FULL	Central SASRec (d=256, 3 blocks) trained on all 1,348,682 users — the cloud reference

Term	Meaning
C_BIG	Central SASRec (d=256, 3 blocks) trained on the 131,072 federated users only
C_SMALL	Central device-size SASRec (d=64, 2 blocks) trained on the federated users
PRE ($\theta_{\mathbf{pre}}$)	Device-size model pretrained on the 1,217,610 server-side users
FA	Federated FedAdam model from a random (cold) start
FA_WARM_FROZEN	Federated FedAdam model warm-started from PRE, item tables frozen — the realistic edge system
FP / PF	Federated variants of the tuned recipe: with a FedProx proximal term / with a per-user personal vector
FA_1024, DP8, DP1	Poisson-sampled federated schedule (about 1,024 clients per round) without noise, and its differentially private twins at $\varepsilon = 8$ and $\varepsilon = 1$
FA_Q8	Federated model with 8-bit quantised uploads
LO	Device-only model trained on each user's own history
FT_PRE, FT_C, FT_FA	On-device fine-tuning of PRE, of a central model, of a federated model
W_STAR (W*)	Tuned warm-start federated model of the follow-up analysis, selected on calibration data
PB100	Variant whose server pretraining used only 100,000 users

Chapter 1: Introduction

1.1 Background

Online shopping is guided by recommendation. When a shopper opens a product page, the list of "related" or "you may also like" items is produced by a model that has learned from the sequences of views, carts and purchases of millions of earlier sessions. Modern sequential recommenders such as GRU4Rec [1] and the self-attentive SASRec [2] read a user's recent history as a sequence and predict the next item; they are accurate, but they are trained on servers that hold the complete behavioural history of every user. Figure 1.1 shows this setting from the shopper's side.



Figure 1.1: Online shopping happens on the shopper's own device: browsing, comparing and paying in a web browser. The events of such sessions are the data this project learns from.
Photo: Bogdan Hoyaux / European Commission, "Shopping online with bank card", CC BY 4.0, via Wikimedia Commons.

A web browser occupies a special position in this landscape. Microsoft Edge, the product that framed this project, runs on the user's own device and observes the shopping journey across many stores. It could therefore offer shopping assistance that no single retailer can offer, but only if it learns from behaviour that users increasingly regard as private. Regulation and user expectations point in the same direction: personal browsing and purchase histories should not leave the device unless there is a strong reason. Two families of techniques promise recommendation without central collection of raw data. *On-device learning* ships a model to the device and adapts it locally. *Federated learning* (FL) [3, 4] trains a shared model across many devices by exchanging model updates instead of data. Figure 1.2 contrasts these options with the cloud-only baseline.

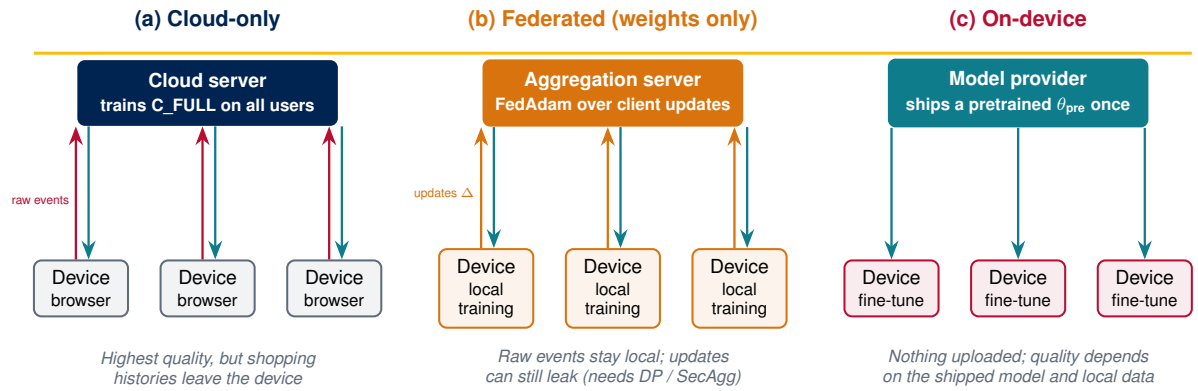


Figure 1.2: Three ways to deploy a shopping recommender. (a) Cloud-only training moves raw events to a server and gives the quality ceiling. (b) Federated learning keeps raw events on the device and uploads model updates, which can still leak information. (c) On-device learning ships a pretrained model once and adapts it locally without uploading anything. Every comparison in this report is expressed relative to the cloud reference.

1.2 Problem statement

Moving recommendation away from the cloud has a price. A device holds only one user's short history; a federation of devices sees many users but only through noisy, aggregated updates; privacy mechanisms such as differential privacy [5] add further noise. How large that price is for real shopping behaviour is not known. Many of the federated-recommendation studies we reviewed evaluate on small public datasets and rank a handful of sampled candidates instead of the full catalogue, a practice that can change the ranking of methods [6]. Few studies separate the contribution of a server-side pretrained model from the contribution of federation itself, and fewer still measure leakage and on-device cost in the same setting. A product team therefore cannot tell from the literature whether an on-device or federated shopping recommender would keep most of the cloud quality, what federation would add, and what privacy would cost.

1.3 Motivation

The motivation of this project is practical and methodological at the same time. Practically, a browser vendor needs a quantitative answer before it can decide whether to invest in on-device recommendation, and it needs to know which ingredients matter: the size of the shipped model, a server-side pretraining step, federated rounds, local fine-tuning, quantisation and privacy protection. Methodologically, a comparison of many training regimes on a large log invites selective reporting: with dozens of model variants, several metrics and two random seeds, it is easy to find a favourable reading after the fact. We therefore decided to treat the main comparison as a confirmatory experiment. Its analysis plan was registered before the test data was opened, in the spirit of pre-registration in empirical science [7], and every later look at the test data was recorded and labelled as exploratory.

1.4 Research question and objectives

The project proposal stated a single research question, which this report answers:

> *"How much shopping recommendation quality can be retained when moving from cloud-based to on-device or federated models?"*

To make the question measurable we divided it into five sub-questions:

RQ1 How much next-product ranking quality (client-macro MRR@20 over the full catalogue) does each architecture—federated, on-device and centrally trained models of device size—retain relative to a full-data cloud model?

RQ2 How much of that quality comes from a server-side pretrained model (a *warm start*), and what does federated training add on top of it?

RQ3 What does on-device adaptation add, both training from scratch on one user's data and fine-tuning a shipped model?

RQ4 What does privacy protection cost in quality, and how much do federated model updates leak about their users?

RQ5 Can the device model run inside a browser at an acceptable cost in size, latency and upload volume?

These questions translate into six objectives:

O1 Measure the retention of federated and on-device systems relative to cloud references on a large real shopping log, with full-catalogue ranking.

O2 Quantify the value of a server-side pretrained start and of federation on top of it.

O3 Measure on-device adaptation: device-only training and on-device fine-tuning.

O4 Measure the utility cost of differential privacy, the empirical leakage of model updates and the overhead of secure aggregation.

O5 Assess deployability: INT8 model size and quality, browser inference latency and per-round upload size.

O6 Carry out the study under a pre-registered, auditable protocol and release a reproducible code base and record.

1.5 Scope and delimitations

The study is deliberately bounded. The main dataset is REES46 [8], a public e-commerce event log of a multi-category store, used for October and November 2019; six public benchmarks serve as a second check. Clients are *simulated*: 131,072 users of the log play the role of devices, and federated rounds are executed on GPU servers, not on a live fleet. Evaluation is offline, on held-out future behaviour.

Device cost is measured with a real browser runtime on a laptop, a phone and a server CPU, but no recommender was deployed to real users. Secure aggregation was implemented as a simulation, and the differential-privacy implementation is reported together with the limitations that a later audit found (Chapters 5 and 6). The task is next-product ranking; other shopping tasks explored in the first prototype, such as purchase prediction, are reported only as part of the project history.

1.6 Contributions

The report makes the following contributions:

C1 A pre-registered, full-catalogue comparison of cloud, federated and on-device sequential recommendation on a large real e-commerce log, with a single sealed confirmatory look at the test data (Chapters 3 and 5).

C2 The confirmatory answer: on the registered recipe, the realistic edge system—federated training warm-started from a server-pretrained model—retains 88.49% of the full-data cloud model, federation adds nothing measurable over the pretrained model, and on-device fine-tuning gives a small, significant gain (Chapter 5).

C3 The diagnosis of a silent failure mode: the registered FedAdam adaptivity constant was mis-scaled relative to the update magnitudes and nearly froze federated training. With the constant corrected, federation learns, and its value depends on how much data the server already holds (Chapter 5).

C4 A privacy evaluation that combines the utility cost of differential privacy, membership-inference and update-leakage audits, a secure-aggregation simulation, and an honest record of the implementation limits found by a later audit (Chapters 5 and 6).

C5 An on-device pipeline—privacy-motivated feature encoding (hashed identifiers and price bands, not an anonymity claim), ONNX export with INT8 quantisation, a communication byte ledger and browser latency measurements (Chapters 4 and 5).

C6 A reproducible open-source code base, the full decision log of the project and a merged, verified archive of all experiments (Chapter 4, Chapter F).

1.7 The research journey at a glance

The results in this report were not planned in one piece. They were reached step by step, and each step answered a question raised by the previous one. Figure 1.3 shows the fourteen phases of the project, which Chapter 5 follows in order.

From prototype to protocol. The team first built a prototype on GitHub between August and September 2026: a data pipeline for the REES46 log, three shopping tasks, a GRU model workstream, federated plumbing and the system, deployment and security tooling. The prototype worked, but it also exposed

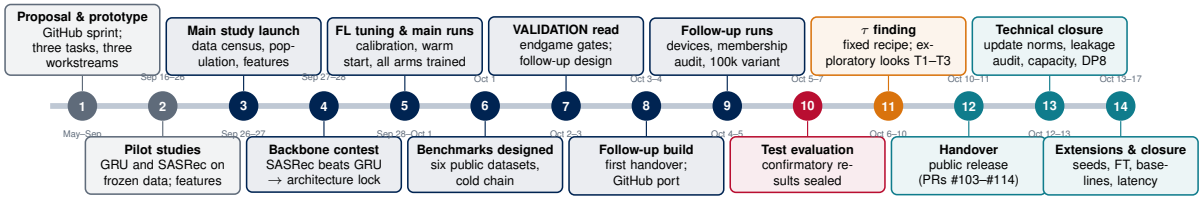


Figure 1.3: The research journey in fourteen phases, from the proposal (May 2026) to the closure of the project (17 October 2026). Grey: the first prototype and the pilot studies. Navy: the main pre-registered study up to the test evaluation. Red: the confirmatory TEST evaluation. Orange: post-hoc exploratory work. Teal: handover, technical closure and extensions.

how fragile conclusions are: a label defect and two evaluator conventions changed the apparent value of the models. Short pilot studies then compared GRU and SASRec models on frozen data and studied input features.

The pre-registered study. These lessons led to the main study. It fixed the task as next-product ranking over the full catalogue, ran a census of the data, chose the population, held a backbone contest that SASRec won in our setting, and locked the architecture. A federated recipe was then calibrated, including a warm start from a server-side pretrained model, and every arm was trained. After the validation results were read once, a follow-up analysis was designed, and all analyses were sealed in a single confirmatory test evaluation on 5–7 October 2026.

Explaining the answer. The confirmatory answer was clear but surprising: federation added nothing over the pretrained model. The post-hoc investigation that followed found that the scaling of the FedAdam optimiser had almost frozen federated training, showed that federation learns once the scaling is corrected (although it still adds nothing over a strong pretrain), measured how its value depends on the server's data, and audited privacy leakage once federation was really learning. Each of these looks at the test data was gated and recorded.

Closure. After the study lead left the project, the remaining team verified every saved result and ran a set of closure experiments: a second pretraining seed, fine-tuning on moving models, training-free baselines and latency measurements. They documented what was stopped or not run, and merged the whole project into a single verified archive.

1.8 Organisation of the report

Chapter 2 reviews sequential recommendation, federated learning, federated recommendation, privacy in federated learning, on-device inference and pre-registration, and identifies the gap this project addresses. Chapter 3 describes the methodology: data, task, populations, models, training regimes, the federated recipe, privacy mechanisms, statistics and the pre-registration protocol. Chapter 4 presents the design and implementation of the system, from the prototype platform to the research platform of the main study and the final delivery. Chapter 5 reports every experiment in chronological order, separating confirmatory from

exploratory evidence, and closes each step with what it taught us. Chapter 6 interprets the findings, checks them against the objectives, draws practical implications and lists the limitations. Chapter 7 concludes and outlines future work. The appendices contain the pre-registration lineage, the complete confirmatory tables, the test-exposure register, the incident log, the experiment register, a reproducibility guide and the contribution and AI-use statements.

Chapter 2: Background and Literature Review

This chapter collects the prior work on which the rest of the report depends, and it ends by stating what that work leaves open. The project asks how much shopping-recommendation quality survives when a model moves from the cloud to the device, so the chapter follows the question in the order in which it has to be answered. Section 2.1 reviews the sequential recommenders that could serve as the model, and Section 2.2 reviews how such models are evaluated, because a retention figure is only as meaningful as the protocol behind it. Sections 2.3 and 2.4 cover federated learning (FL) in general and federated recommendation in particular. Section 2.5 treats the privacy machinery that FL is usually paired with, Section 2.6 covers on-device inference, and Section 2.7 introduces the pre-registration and statistical tools that govern how results are reported. Section 2.8 brings these strands together in a comparison table and states the gap that this project addresses. Papers are described only at the level of their published main idea; their numbers are quoted only where this chapter reproduces a table that we checked against the source.

2.1 Sequential recommendation

A sequential recommender receives the time-ordered list of items a user has interacted with and ranks candidate items for the next interaction. The session-based variant, in which the sequence is a short visit rather than a long history, is surveyed by Wang et al. [9]. Our task is next-product ranking over the full catalogue of 158,486 products on REES46, an e-commerce behaviour log from October to November 2019 [8]; Chapter 3 defines it precisely. Public benchmarks in this literature are mostly built from Amazon product reviews [10, 11] and from MovieLens [12], which is one reason why REES46 is a useful complement: it records behaviour (views and purchases) rather than ratings.

Recurrent models. GRU4Rec [1] introduced recurrent networks to session-based recommendation. It runs a gated recurrent unit over the session, trains with session-parallel mini-batches, and optimises ranking losses (BPR and TOP1) over sampled negative items. A later paper [13] showed that the choice of loss and the number of negative samples strongly affect top-k quality, and proposed the BPR-max and TOP1-max losses together with additional negative samples.

Attention-based models. SASRec [2] replaces the recurrence with causal self-attention [14]. Every position of the sequence predicts the next item, and the original objective scores the true next item against one randomly sampled negative with a binary cross-entropy. BERT4Rec [15] instead applies bidirectional attention and trains by masking items in the sequence and predicting them from both sides, the Cloze task, with a softmax over the catalogue. Table 2.1 summarises these models and the later objective variants discussed below.

Table 2.1: The sequential recommenders discussed in this section. The descriptions follow the cited papers at the level of their main idea; the last column states why each matters for this project. No result of this project is shown.

Model	Architecture	Training objective	Relevance here
GRU4Rec [1, 13]	gated recurrent unit over the session	ranking losses over sampled negatives (BPR, TOP1; BPR-max, TOP1-max in the later paper)	recurrent comparator in the backbone contest
SASRec [2]	causal self-attention	binary cross-entropy of the next item against one sampled negative per position	backbone family of the device and cloud models
BERT4Rec [15]	bidirectional self-attention	masked-item (Cloze) prediction with a softmax over items	shows that the objective can matter as much as the architecture
SASRec with full cross-entropy [16]	SASRec architecture	softmax cross-entropy over the whole catalogue	strong objective, but cost grows with catalogue size
gSASRec [17]	SASRec architecture	generalised binary cross-entropy (gBCE) with many sampled negatives and a calibration parameter	addresses over-confidence caused by negative sampling
eSASRec [18]	SASRec with several modular changes to objective and layers	as released by the authors	strong full-ranking baseline with public splits and code

The loss debate. For some years BERT4Rec was reported to outperform SASRec, which suggested that bidirectional context was valuable. The picture changed when the training objective was examined. Klenitskiy and Vasilev [16] showed that SASRec trained with a softmax cross-entropy over the full catalogue reaches or exceeds the effectiveness of BERT4Rec, which points to the loss rather than to bidirectionality as a main source of the difference. Petrov and Macdonald [17] approached the same question from the other direction. When a model is trained with many sampled negatives, it over-estimates the probability of positive items; their gSASRec adds a calibration parameter and a generalised binary cross-entropy, and it reports effectiveness competitive with BERT4Rec at lower training cost. The eSASRec study [18] collects several modular improvements of this kind into one strong SASRec variant and evaluates it with full ranking on the standard benchmarks.

The trade-off between the two families of loss is computational. A full softmax needs a score for every catalogue item at every position, so its cost grows with the catalogue; sampled objectives avoid that cost but have to be corrected for the bias they introduce. This matters in an FL setting more than in a data centre. With 158,486 products, the item tables of our device model are large: the untied input and output item tables hold about 98% of its parameters (*descriptive*). The project's early cost benchmark found that every client visit pays for the full-catalogue output head and for the optimiser over all parameters (summarised in ADR-001, the project's first architecture decision record), which is one reason why communication and compute are measured explicitly later in the report.

Simple transition models are competitive on standard benchmarks. A recent study from Spotify [19], published at RecSys'26, asks how much of the progress on these benchmarks is genuinely sequential. Its Table 1 compares neural models (SASRec and eSASRec variants, labelled SAS+ and eSAS) with simple transition models under one matched protocol, whose caption states that all values are the authors' own

runs: the full catalogue is ranked, items already in the user's history are removed, no sampled negatives are used, and the last interaction of each user is held out. The transition models are a first-order Markov chain (MC), a sequential-rules model (SeqRules) and a further transition model labelled PCTM; two factorised Markov chains, omitted here, are also reported. Table 2.2 reproduces five of the seven columns for NDCG@10.

Table 2.2: Full-catalogue NDCG@10 from Table 1 of the Spotify RecSys'26 study [19], four decimals, best value per row in bold. The columns for the two factorised Markov chains are omitted. These are published values, not results of this project, and carry no label. Each neural cell is a single run (neural seed 32); MC, SeqRules and PCTM are deterministic. Source: arXiv 2608.19833, Table 1, read from the paper and the authors' repository.

Dataset	MC	SAS+	eSAS	SeqRules	PCTM
Beauty	0.0492	0.0537	0.0524	0.0605	**0.0635**
Sports	0.0251	0.0315	0.0324	**0.0371**	0.0368
Toys	0.0548	0.0575	0.0533	0.0730	**0.0738**
ML-1M	0.1240	0.1662	0.1739	0.1505	**0.1815**

On each of the four datasets the best cell is a transition model: PCTM on Beauty, Toys and ML-1M, and SeqRules on Sports. On Toys even the first-order MC (0.0548) exceeds eSAS (0.0533). As we read the paper, such models serve as probes of how much sequential capacity the standard benchmarks actually demand, and the table suggests that the demand is modest. Two cautions apply. First, every neural cell is a single run with no reported spread. Comparing the SAS+ and eSAS cells with the NDCG@10 values published by the eSASRec authors on the same splits, the differences are at most 0.0033 (largest on ML-1M for eSAS, 0.1739 against 0.1772), which suggests a run-to-run spread of roughly 0.003 (*descriptive*). Second, we re-ran MC, SeqRules and PCTM on CPU from the authors' code, and all 12 values equalled Table 1 to four decimals (*descriptive*), so the transition-model side of the table is reproducible.

For this project the finding has three consequences. The choice of SASRec as backbone was made inside the project, by a contest between a recurrent and an attention model on our own protocol, and not by appeal to the literature; in our contest SASRec scored higher, and this report does not claim that attention models are better than recurrent ones in general. Simple baselines must be reported next to neural models. The training-free rule LAST, which repeats the last viewed item, reaches about 44% of the cloud model trained on the same 131,072 users on the validation week (about 51% on TEST) (*descriptive*), and every arm in the study is therefore also reported against it. And protocol details decide what a number means: the training catalogues of these public benchmarks hold 3,704 (ML-1M), 11,917 (Toys), 12,092 (Beauty) and 18,347 (Sports) items (*descriptive*), an order of magnitude below the 158,486 products of our task, so published full-ranking scores cannot be compared with ours without that caveat (Figure 5.6 places our cloud reference `C_FULL` beside published full-ranking results). The next section sets out the evaluation protocol that makes such comparisons meaningful.

2.2 Evaluating recommenders

The previous section ended with the observation that the same model can look strong or weak depending on how it is scored. This section fixes the vocabulary of evaluation and explains the choices that the rest of the report makes. A ranking metric is computed from the rank r that the model gives to the true next item among all candidates; the definitions are collected in the box below.

Ranking metrics for one next-item decision

Let a decision have one true next item whose rank in the model's ordering of the candidates is $r \geq 1$, and let k be a cut-off. Then

$$\mathrm{HR@}k = \mathbb{1}[r \leq k], \qquad \mathrm{MRR@}k = \frac{\mathbb{1}[r \leq k]}{r}, \qquad \mathrm{NDCG@}k = \frac{\mathbb{1}[r \leq k]}{\log_2(r+1)}.$$

With a single relevant item the ideal discounted gain is 1, so NDCG needs no further normalisation. HR (hit rate) asks only whether the item is in the top k; MRR (mean reciprocal rank) and NDCG (normalised discounted cumulative gain) also reward placing it higher. In this report the primary metric is client-macro MRR@20 (written MRR@20), and NDCG@10 and HR@10 are secondary. Whether items the user has already seen are removed before ranking is a protocol choice that has to be stated with every number.

Full versus sampled metrics. A common shortcut ranks the true item against a small random sample of candidates, for example 100, instead of against the whole catalogue. Krichene and Rendle [6] showed that metrics computed on such samples are in general not consistent with the full-catalogue metrics, and that the ordering of two models can change depending on whether sampling was used. We therefore rank over the full catalogue throughout, which is also the protocol of the eSASRec and Spotify studies reviewed above [18, 19]. Full ranking is more expensive, but it removes a source of ambiguity from every comparison, including that between a federated model and its cloud reference.

Even under full ranking, details of the split change the meaning of a score. In the public eSASRec splits the held-out item is each user's last interaction, dropped if the user or the item is absent from the training part or if the user has already interacted with the item. The timestamps of the Amazon sets are synthetic positions within each user's sequence and contain no ties, whereas on ML-1M 2,483 of 6,040 users have tied timestamps at their last training event (*descriptive*).

Averaging over users. A second choice is what is averaged. A micro average takes the mean over all decisions, so users with long histories dominate it; a macro average over clients first averages within each user and then across users, so that every user counts equally. The macro choice follows the setting of this project, where each device is one client and the question is what the typical user experiences. The primary metric is therefore client-macro MRR@20 over the full catalogue of 158,486 products: a user's reciprocal ranks are averaged over their decisions, then averaged over users, a rank beyond 20 scores 0, and a target outside the vocabulary counts as a miss, so that models are penalised for items they cannot

represent. Quality is reported as retention, 100× the metric of a model divided by the metric of a reference, computed with the same checkpoint rule and the same seed and then averaged over seeds.

Uncertainty. Published sequential-recommendation tables often report a single number per cell. The spread of roughly 0.003 noted in Section 2.1 shows that this can hide differences of the size that matter. We therefore attach an interval to every contrast, using the paired bootstrap over users introduced in Section 2.7. The confirmatory TEST set contains 26,254 users and 229,742 decisions (Chapter 3).

2.3 Federated learning

Section 2.1 described models that are trained where the data is collected. The premise of this study is that a browser vendor may prefer not to collect browsing behaviour, which is the setting for which FL was proposed. Kairouz et al. [4] define it as training a model across many clients under the coordination of a central server while the training data stay with the clients, and they distinguish the cross-silo setting (few reliable organisations) from the cross-device setting (very many unreliable devices, only a small fraction of which participate in any round). A shopping assistant in a browser is a cross-device case. Frameworks such as Flower [20] lower the cost of experimenting with it; Chapters 3 and 4 describe the simulator used here. Figure 2.1 shows where such central training usually takes place.



Figure 2.1: A data-centre server room. In the cloud-only setting, every shopper's events are copied to machines like these and the model is trained there; federated learning was proposed to avoid that copy.
Photo: BalticServers.com, "BalticServers data center", CC BY-SA 3.0, via Wikimedia Commons.

Federated averaging. FedAvg [3] is the baseline algorithm. In each round the server samples a set of clients and sends them the current global model; each client trains it on its local data for a few steps and returns the updated weights; the server replaces the global model with the average of the returned models, weighted by local data size. Raw data never leave the device, but the communication cost is the full model in both directions per participating client in each round. For the 20.7 million-parameter device model used here, the FP32 transfer of one visit is 165,669,672 bytes (download plus upload) (*descriptive*), against a registered payload cap of 170 MB per visit. Cost per visit is therefore a first-order design constraint and

not an afterthought.

Non-identical data and client drift. FedAvg works well when client data are similar and degrades when they are not. Hsu et al. [21] studied the effect of non-identical class distributions on federated visual classification and showed that accuracy falls as the distributions diverge. Recommendation data are non-identical by nature: each user has their own taste, item popularity follows a long tail, and most users have short histories. Local training on such data pulls each client towards its own optimum, so that the average of the client models is not the minimiser of the global objective; this effect is called client drift. Two well-known remedies change the local problem. FedProx [22] adds a proximal term to the local objective that penalises distance from the current global model and tolerates partial local work. SCAFFOLD [23] corrects the drift with control variates that estimate the update direction of each client and of the server. The price of SCAFFOLD is state: a control variate has the size of the model, doubles the communication, and has to persist on clients that in a cross-device setting may participate only once. Both remedies address drift that grows with the number of local steps, and a client that makes few local steps per visit offers them little to correct.

Why FedBN does not apply. FedBN [24] handles feature shift between clients by keeping the batch-normalisation layers of each client local, so that only the remaining weights are averaged. The original research proposal listed FedBN among the federated variants to compare. SASRec, however, normalises with layer normalisation, which computes its statistics per token at run time and keeps no running mean or variance that could differ between clients; the mechanism that FedBN relies on has nothing to act on in this architecture. The charter of the main pre-registered study, dated 2026-09-26 (ADR-001), therefore dropped FedBN from it. This is a statement about applicability, not a finding; we did not run FedBN.

Server-side optimisers. A line of work treats the averaged client update as a gradient and applies an optimiser to it on the server. Hsu et al. [21] add momentum (FedAvgM), and Reddi et al. [25] propose adaptive variants (FedAdagrad, FedYogi and FedAdam) and show that they can improve convergence. Because FedAdam is the server optimiser of the realistic edge system studied here, and because one of its constants turned out to decide whether FL learned at all, we state the update in full.

FedAdam and the role of τ

In round t the server samples a set S_t of clients; client k starts from the global model θ_t, trains locally on n_k examples and returns θ_t^k. The pseudo-gradient is the weighted mean update

$$\Delta_t = \sum_{k \in S_t} \frac{n_k}{\sum_{j \in S_t} n_j} \left(\theta_t^k - \theta_t\right),$$

and FedAvg sets $\theta_{t+1} = \theta_t + \Delta_t$. FedAdam [25] instead feeds Δ_t to Adam [26]:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\Delta_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2)\Delta_t^{\odot 2}, \qquad \theta_{t+1} = \theta_t + \eta_s \frac{m_t}{\sqrt{v_t} + \tau}.$$

Here η_s is the server step, β_1 and β_2 are moment decays, and τ is a constant that controls the degree of adaptivity. Per coordinate, if $\sqrt{v_t} \gg \tau$ the step has size about η_s whatever the scale of Δ_t, which is the scale-free behaviour of Adam. If $\sqrt{v_t} \ll \tau$ the update reduces to $(\eta_s/\tau)\, m_t$, a rescaled momentum step whose size follows the pseudo-gradient. FedAvgM [21] is the momentum-only case, $m_t = \beta m_{t-1} + \Delta_t$ and $\theta_{t+1} = \theta_t + \eta_s m_t$.

The constant τ is not scale-free: it is compared with the magnitude of the pseudo-gradient, which depends on the model, the loss, the number of local steps and the number of clients per round. In this project the registered value was $\tau = 10^{-3}$, while the client pseudo-gradients were about 10^{-7}, so that the server step was 20 to 100 times too small; the tuned warm-start federated model of the follow-up analysis moved only about 0.06% from the pretrained model in 13,654 rounds (*exploratory*). With $\tau = 10^{-6}$ FL learns. The warm run with τ fixed on REES46, compared with the pretrained model, differs by +0.00022 [−0.00011, 0.00057] (*exploratory*), an interval that includes 0, so the confirmatory answer is robust to this error. The re-runs changed several ingredients at once (τ together with the server step η_s, unfrozen item tables or the training budget), so we do not attribute the gain to τ alone. The lesson from the literature is practical: an adaptive server optimiser must be configured against measured update norms, and the measurement must be reported (Chapter 5).

Pretraining for FL. Nguyen et al. [27] and Chen et al. [28] studied initialisation in FL and reported that starting from a pretrained model, rather than from random weights, makes FL markedly more robust to client heterogeneity and reduces the number of rounds needed. Neither study concerns recommendation, but the idea transfers directly. A browser vendor that already has a population of opted-in users, or public data, can pretrain a model on the server and ship it as the starting point of federated rounds. In this project the server-pretrained model θ_{pre} (PRE) is trained on 1,217,610 users who are not among the 131,072 simulated devices, and the cloud reference C_FULL on all 1,348,682 users. Pretraining changes the question that FL has to answer: not whether federated rounds can train a model from scratch, but what they add to a model that is already strong. The confirmatory answer for our setting is that they add nothing measurable: the contrast of the realistic edge system FA_WARM_FROZEN with PRE is −0.000508 [−0.000933, −0.000037] (*confirmatory*) (K21). Exploratory looks, reported in Chapter 5, indicate that the value of FL depends on how strong the pretraining is. We therefore state the result in the form permitted by the evidence: FL adds nothing over a strong 1.2 million-user pretrain on REES46, and adds a lot when the pretrain is weak or absent.

2.4 Federated recommendation

The general FL literature of the previous section is written for classification and language tasks. Recommendation differs in ways that matter for design. Part of the model is user-specific (user embeddings or a personal head) and can stay on the device, the shared part is dominated by large item tables, and the signal in an update is sparse because a user touches few items. The last property has a privacy side: when

item embeddings are updated, the non-zero rows of an update identify the items the user interacted with. This is a consequence of the model structure and not a flaw of one paper, and Section 2.5 returns to it. Table 2.3 lists the federated recommenders that frame this project.

Table 2.3: Federated recommenders discussed in this section, described at the level of their main idea. The column "Sequence order" states whether the model uses the order of a user's interactions. No result of these papers is reproduced.

Method	Model family	Sequence order	Main idea
FCF [29]	matrix factorisation, implicit feedback	no	user factors are trained and kept on the device; updates of the item factors are sent to the server and aggregated
FedRec [30]	matrix factorisation, explicit feedback	no	federated factorisation of ratings, with a step intended to hide which items a user has rated
FedeRank [31]	pairwise ranking for top-N	no	federated learning to rank in which users keep control of their data and of what they contribute
FedFast [32]	federated recommenders, faster convergence	no	informed client sampling and aggregation based on user similarity, instead of uniform averaging
PFedRec [33]	personalised federated recommendation	no	item embeddings are shared and fine-tuned locally, together with a user-specific scoring function
FELLAS [34]	federated sequential recommendation	yes	a large language model is used as an external service to enhance the sequence representations

Federated collaborative filtering (FCF) [29] was an early demonstration that implicit-feedback matrix factorisation can be trained without centralising interactions. The user factors never leave the device, and only item-factor updates are aggregated. FedRec [30] treats the explicit-feedback case. Its distinctive step is to obscure which items a user has actually rated, because the set of rated items is itself informative. FedeRank [31] moves from rating prediction to pairwise ranking, which is closer to the top-N lists produced in practice, and gives users control over what they contribute. FedFast [32] attacks the slow convergence of plain averaging with informed client sampling and aggregation, and PFedRec [33] personalises the item embeddings and the scoring function on each device. FELLAS [34] is closest to our setting in one respect, since it treats federated *sequential* recommendation; it adds a large language model as an external service, which is a different design axis from the one studied here.

Three observations follow. First, the earlier systems are built on factorisation or pairwise-ranking models, not on the transformer-based sequential models that are the strong baselines of Section 2.1. Second, in the papers reviewed here the federated system is compared with centralised or federated variants of the same model; none of them, to our reading, asks what federation adds to a model pretrained on a much larger server-side population, which is the question of the previous section. Third, the privacy of the update itself, rather than the locality of the raw data, is a separate concern, to which we now turn. The research proposal behind this project cited these works as motivation, but we do not use any efficiency figure attributed to them there, because we could not verify it.

2.5 Privacy in federated learning

Keeping raw data on the device is a necessary step but not a sufficient one. Model updates are functions of the training data, and a server that sees an individual update may learn about the user who produced it. The threat model of this study is an honest-but-curious browser vendor: the server follows the protocol but inspects whatever it receives. This section reviews the defences that the literature offers against such a server, and the attacks used to test them. Table 2.4 gives the overview.

Table 2.4: Privacy tools for FL, what each protects against, and what it does not. The last column states how the tool appears in this project. Sources: the cited papers; project status from Chapters 3 and 5.

Tool	Protects against	Does not protect against	In this project
Differential privacy (client level) [5, 35]	inference about the presence of any one client in the released model, bounded by (ε, δ)	loss of utility; leakage if the noise or sampling randomness is predictable	nominal $\varepsilon = 8$ and $\varepsilon = 1$ arms; no formal guarantee for the legacy implementation
Secure aggregation [36, 37]	a server reading individual updates	what the aggregate itself reveals; the final model	simulation of upload overhead only
Membership inference [38, 39]	(an attack, not a defence) measures whether a record can be recognised as a training example	—	AUC and true-positive rate at low false-positive rate
Gradient leakage [40]	(an attack) reconstructs inputs from shared gradients	—	item-set probes on single uploads and on the aggregate
Auditing [41, 42]	(a test) estimates how large ε an implementation really delivers	—	canary design with a ceiling below the nominal ε (see below)

Differential privacy. A randomised mechanism $\mathcal{M}$ is (ε, δ)-differentially private if, for every pair of datasets D, D' that differ in one unit of adjacency and every set of outputs S, $\Pr[\mathcal{M}(D) \in S] \leq e^{\varepsilon} \Pr[\mathcal{M}(D') \in S] + \delta$ [5]. DP-SGD [43] makes gradient-based training private by clipping each example's gradient to a norm bound, adding Gaussian noise, and tracking the cumulative privacy loss with a moments accountant. DP-FedAvg [35] lifts the unit of adjacency from the example to the user: each client's update is clipped, the sum is noised, and the guarantee protects the presence of an entire client (add/remove adjacency). In this central form the aggregator is trusted to see the individual clipped updates before noising; the guarantee concerns what is released afterwards.

The accounting uses Rényi differential privacy [44], under which the privacy loss of a composition of mechanisms adds up in a convenient way and is converted to (ε, δ) at the end, and the analysis of subsampled mechanisms [45], which quantifies the amplification obtained when only a random subset of clients takes part in a round. Two assumptions behind these tools are easy to violate in practice. The amplification result presumes that the sampling is random in the sense required (clients drawn independently with a fixed probability) and that the sampling and the noise are not predictable to the adversary. Our nominal target for the DP arms is $(\varepsilon, \delta) = (8,\ 5 \times 10^{-6})$, with a stricter $\varepsilon = 1$ arm, under central DP with client-level add/remove adjacency; the tuning on the calibration clients is not accounted. Differential privacy also costs utility, since clipping and noise perturb the update, and the size of that cost

is measured and not assumed (Chapter 5).

Secure aggregation. Secure aggregation lets the server learn only the sum of the client updates. In the protocol of Bonawitz et al. [36] every pair of clients agrees on a mask; the masks cancel in the sum, and secret sharing makes the protocol robust to clients that drop out. The cost per client grows with the number of participants in a round, and Bell et al. [37] reduced it to polylogarithmic overhead by letting each client communicate with a sparse set of neighbours. Secure aggregation hides individual updates but does not make the aggregate private, so it is complementary to DP and not a substitute for it. In this project it is a simulation of the upload overhead and not production cryptography.

Attacks and audits. Shokri et al. [38] showed that an adversary can train shadow models and infer whether a record was part of the training set. Carlini et al. [39] argued that average-case measures such as accuracy or AUC can hide the risk to individual records, and that attacks should be evaluated by their true-positive rate at low false-positive rates, using a likelihood-ratio test per example. Zhu et al. [40] showed that training inputs can be reconstructed from shared gradients; for recommenders the same leakage takes a simpler form (our own observation, not a claim of that paper): the set of items a client touched can be read from the non-zero rows of its update. Finally, because a DP claim is only as good as its implementation, auditing methods estimate empirically how large an ε an implementation actually delivers: Jagielski et al. [41] audit DP-SGD with poisoning-style attacks, and Steinke et al. [42] reduce the cost to a single training run by inserting many canaries at once. Such a one-run audit can certify only values of ε up to a ceiling set by its design.

Scope of the privacy claims in this report

Four statements limit what the report can say, and later chapters repeat them where they apply. (i) Without DP, "only weights leave the device" is data minimisation and not privacy: the non-zero rows of an update identify the viewed items, and arms without DP are not called private. (ii) There is no formal DP guarantee. The legacy DP implementation used a disclosed public seed, so its noise and Poisson-sampling streams can be reconstructed; the utility numbers of those runs remain measurements, but the runs do not establish the randomized-DP guarantee. (iii) Secure aggregation is a simulation, not production cryptography. (iv) The canary design of the audit has a ceiling of $\varepsilon \approx 5.213$ (*descriptive*), below the nominal 8, and it therefore cannot reject $\varepsilon = 8$. In addition, the update-leakage audit, an attack on the aggregate input, shows an advantage of about 2.625 percentage points (95% CI 2.312 to 2.953) (*exploratory*), above the fixed 2 percentage-point threshold, and we report it as a negative finding. DP-FTRL and the restarted canary audit were stopped by the study lead, and the cold-start DP calibrations failed before training.

2.6 On-device inference

If the model is trained in the cloud or by federated rounds, it still has to run on the user's device, with the latency, memory and download size that a browser can tolerate. This section reviews the three technologies

the deployment path relies on. The target is the shopper's own phone or laptop (Figure 2.2).

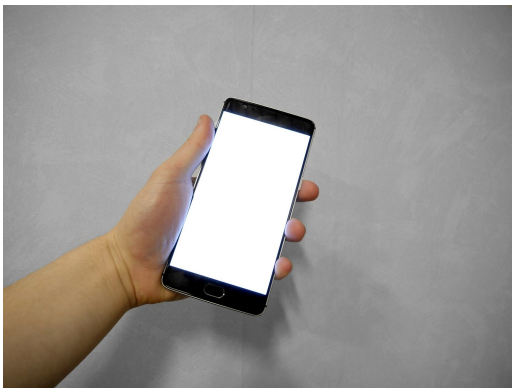


Figure 2.2: The device side of the system. In the federated and on-device settings, the shopper's history and the model that ranks products stay on a phone or laptop such as this one.
Photo: Santeri Viinamäki, "Hand holding smartphone with blank white screen", CC BY-SA 4.0, via Wikimedia Commons.

Quantisation. Quantisation represents weights and, optionally, activations with low-precision integers. Jacob et al. [46] describe the affine scheme in which a real value is represented by an 8-bit integer through a scale and a zero point, with arithmetic carried out on integers, and show that it can reduce latency and model size with a small loss of accuracy; they also train with the quantisation simulated, whereas post-training quantisation converts a finished model. In dynamic quantisation, the form used in this project, the weights are converted to INT8 ahead of time and the scale of each activation is computed at run time. The effect on quality is not guaranteed and has to be measured per model; for a recommender whose parameters are mostly item tables, the saving depends on which tensors are quantised. Chapter 4 describes our export, and Chapter 5 reports the INT8 versus FP32 comparison.

ONNX Runtime. ONNX Runtime [47] is a cross-platform inference engine for models in the ONNX exchange format. A model exported from a training framework [48] is executed by the engine on a chosen back end, so that the same file serves a server, a laptop or, through the WebAssembly build, a browser. In this project the model is exported at ONNX opset 17 and quantised dynamically to INT8.

WebAssembly. WebAssembly [49] is a portable binary instruction format that browsers execute in a sandbox at close to native speed. It is what makes in-browser inference plausible for the Edge setting: the model and the runtime are downloaded as ordinary web assets, and no native code needs to be installed. Which accelerations (vector instructions, multiple threads) are available depends on the browser and on how the page is served, so the latency of a given model must be measured on the target devices. This report distinguishes two kinds of latency number. The in-browser WebAssembly latencies of FP32 and INT8 models are measured on a laptop, a phone and a server; the L6 latency is a single-thread CPU latency measured on a shared server and is not a laptop or phone measurement. A third caveat concerns the comparison of a recurrent and a small attention model on CPU, which is a CPU rehearsal and not a device test.

2.7 Pre-registration and statistics

Sections 2.1 to 2.6 describe what is built and measured. This section describes how the claims about it are protected against the most common failure of empirical machine learning, the quiet adaptation of the analysis to the data. Pre-registration [7] commits to the hypotheses, the metrics and the analysis plan before the outcome data are seen, and it separates confirmatory results, which test the registered plan, from exploratory ones, which were designed after seeing data and generate hypotheses. In machine learning the outcome data that must stay unseen is the test set. This project registered its protocol before training, evaluated the TEST set once, in the gated confirmatory test evaluation (from 2026-10-05T21:03Z to 2026-10-07T10:09Z), and recorded every later look in a test-exposure register; Chapter 3 describes the protocol (Figure 3.5). The result labels CONFIRMATORY, CONFIRMATORY-P2, EXPLORATORY, VALIDATION, DESCRIPTIVE and STOPPED, attached to every reported number, make the status of each result visible.

Bootstrap intervals. We use the bootstrap of Efron and Tibshirani [50]: users are resampled with replacement, the metric is recomputed on each resample, and the percentile interval of the resampled values is the confidence interval. A paired bootstrap resamples the same users for both arms of a contrast, so that the difference is computed on matched users, which is much more sensitive than comparing two independent intervals. The registered analysis uses 1,000 resamples with seed 20260926 and 95% percentile intervals; ratios such as retention are bootstrapped directly, not derived from the intervals of their parts.

Multiple comparisons. A study with many contrasts will find some that look significant by chance. Holm's step-down procedure [51] controls the probability of any false rejection across a family of tests, and it is uniformly more powerful than the Bonferroni correction. It requires the family to be named in advance, which is what pre-registration provides.

Equivalence testing. A non-significant difference is not evidence that two models are equal. The two one-sided tests procedure (TOST) [52] reverses the burden: it declares two quantities equivalent only if the difference is shown to lie inside a pre-specified margin on both sides. Equivalently, equivalence at the 5% level holds when the 90% interval of the difference lies inside the margin, which is the registered test; the wider 95% interval that we report also lies inside the margin for Q1. In this project the margin is ±0.0005 in MRR@20. The contrast Q1 of the follow-up analysis, the tuned warm recipe minus PRE, is −0.000081 [−0.000179, 0.000024] (*confirmatory, follow-up*), which lies inside the margin and is read as EQUIVALENT. By contrast, the interval of K21, quoted in Section 2.3, excludes 0 on the negative side, so that result is a small but detectable loss of the federated stage relative to the pretrained model. Reading the two together is exactly the distinction that equivalence testing is meant to enforce. The question was "how much is retained", and the study lead set no pass or fail threshold.

2.8 The research gap

The preceding sections suggest a gap that is narrower than "federated recommendation is understudied". Table 2.5 sets what the cited literature establishes against what remains open for a browser vendor that would deploy a sequential recommender, and it states where this report addresses each point.

Table 2.5: What the cited literature establishes, what remains open for the setting of this project, and where the report addresses it. The second column characterises the cited work as we read it; the third column states the open point as a gap in the work we reviewed, not as a claim about all literature.

Aspect	Established by the cited literature	Open for this setting	Addressed in
Model family	federated recommenders built on factorisation or pairwise ranking [29–33]; one federated sequential model with an external language model [34]	a transformer sequential ranker over a catalogue of 158,486 products, with a device model of 20.7 million parameters and a cloud model of 83.9 million	Chapters 3 and 4
Evaluation	full-catalogue ranking is the sound protocol [6]; benchmark cells are single runs [19]	client-macro MRR@20 with paired bootstrap intervals, reported next to simple baselines and against published full-ranking results	Chapters 3 and 5
Pretraining	pretrained initialisation helps FL on non-recommendation tasks [27, 28]	what federation adds to a recommender pretrained on 1,217,610 server-side users, and how that depends on the strength of the pretrain	Chapter 5
Server optimiser	adaptive server optimisers help [21, 25] but depend on the adaptivity constant τ	configuring τ against measured update norms, and the cost of getting it wrong	Chapter 5
Privacy	DP, secure aggregation, attacks and audits [35, 36, 38, 42]	these tools applied together to a federated sequential recommender, with a statement of what each arm does and does not guarantee	Chapters 5 and 6
On-device inference	quantisation, ONNX Runtime and WebAssembly are available [46, 47, 49]	latency, size and quality of the exported recommender in a browser runtime	Chapters 4 and 5
Evidence standard	pre-registration separates confirmatory from exploratory work [7]	a gated TEST protocol with a public exposure register and labelled results for the whole study	Chapter 3

The gap has a practical form. A browser vendor who considers federated learning needs to know three things together: how much quality a deployable edge system keeps compared with a cloud model, whether federation contributes anything beyond what a server-side pretrain already provides, and what privacy statement can honestly be attached to each variant. The literature offers pieces of each answer, for other model families or other tasks, but to our reading not the three together for a sequential transformer recommender on a realistic e-commerce log, under a protocol that was fixed before the test data were seen. This report supplies that evidence for one dataset and one recipe, and it states the limits of the claim.

How much shopping recommendation quality can be retained when moving from cloud-based to on-device or federated models? The question is answered as a retention figure with an interval and not as a pass or fail verdict.

As a first orientation, the headline confirmatory number of the study is that the realistic edge system

FA_WARM_FROZEN retains 88.49% [87.73, 89.18] (*confirmatory*) of the cloud reference C_FULL (K20b), whereas the federated stage adds nothing measurable over the server-pretrained model, as discussed in Section 2.3. The recipe behind this system is defined in Chapter 3, and the story of how the project arrived at it, including what did not work, is told in order in Chapter 5.

Key finding

Five points shape the rest of the report. (1) Architecture comparisons depend on the loss and the protocol, and simple transition models are competitive on standard benchmarks, so every neural arm is reported beside simple baselines. (2) Full-catalogue ranking is used because sampled metrics can reorder models. (3) In FL, pretraining and the constants of the server optimiser deserve as much care as drift correction, and FedBN does not apply to a layer-normalised model. (4) Keeping data on the device is not privacy; DP, secure aggregation, attacks and audits answer different questions, each with a stated limit. (5) Pre-registration, paired bootstrap intervals, Holm's procedure and equivalence testing give each result a status a reader can check.

Chapter 3: Methodology

Chapter 2 placed the research question in the literature. This chapter explains how we turned it into a study in which every claim traces to a rule that was written down before the data it governs were read. It states the research design and evidence labels (Section 3.1); the data, task, populations, model and arms (Sections 3.2 to 3.6); the federated, privacy, on-device and statistical methods (Sections 3.7 to 3.10); and the governance, benchmarks and tools (Sections 3.11 to 3.13). Chapter 4 describes the implementation and Chapter 5 the results.

3.1 Research design

How much shopping recommendation quality can be retained when moving from cloud-based to on-device or federated models?

The question comes unchanged from the project proposal. The study lead asked how much quality is retained, not whether a system is acceptable, so the registered plan contains no pass or fail threshold (amendment 04). Each answer is a measured quantity with an interval: the retention of an arm relative to a reference model, or the paired difference between two arms. Whether a retention is good enough for a product is a decision that this thesis informs but does not make.

A second principle, which the study lead called realism first, shaped the list of arms. Every experiment simulates what a browser vendor could deploy: a large model on the server, trained on the data the server is allowed to hold; federated learning (FL), in which devices send model updates and never raw events [3, 4]; and a smaller model of the same architecture on the device. Federated averaging needs one parameter shape on both sides, so a larger server-only model was left to future work. Table 3.2 maps the resulting systems onto privacy levels L0 (cloud-only training) to L6 (device-only training).

Two facts determine how the word “private” may be used. First, without differential privacy (DP) an update reveals the viewed items: the input and output item tables are untied and hold about 98% of the parameters, so the non-zero rows of an update identify the items a user saw (gradient-leakage attacks show more generally that updates can expose training data [40]). “Only weights leave the device” is therefore data minimisation, not privacy, and we do not call any arm without DP private. Second, the threat model is an honest-but-curious browser vendor, that is, the server. DP here is central DP [5] with a trusted aggregator and client-level add/remove adjacency at $\delta = 5 \times 10^{-6}$, conditional on a public bootstrap block (the catalogue, the feature-transform statistics and the initial weights θ_0 and θ_{pre}). The tuning of the FL recipe on CALIB is not accounted for.

The evidence has four parts: the main analysis, the pre-registered deployment study (PREG1 and its amendments 01 to 07); the follow-up analysis (PREG2), registered after the first weak signal on the

pretrained line and scored in the same single test evaluation; exploratory looks after that evaluation, which asked why the federated gain was small; and the external benchmarks, which are descriptive. Six labels keep the parts apart and are attached to every reported number.

Evidence labels

A label records *when* the governing rule was registered relative to *when* the value was read; it does not depend on how the value turned out.

(*confirmatory*) A main-analysis contrast whose arms, checkpoint rule, seeds and decision rule were registered, hashed and logged before its value existed, and which was scored once on the TEST week in the single confirmatory test evaluation.

rmatory, follow-up) The same for the contrasts of the registered follow-up study PREG2, scored in the same test evaluation.

(*exploratory*) Anything added after the VALIDATION read or the TEST look, including every look logged in the exposure register after the test evaluation. It may raise questions; it carries no confirmatory claim.

(*validation*) A value measured on the validation week, including the central contest that chose the backbone and the single registered VALIDATION read. Values at the practical-best rule (Section 3.3) take the best of 12 marks and are therefore optimistic.

(*descriptive*) A value reported without a claim: tuning reads on CALIB, baselines, panels and the external benchmarks.

(*stopped*) A run or analysis stopped by the study lead, or by a registered reallocation, before completion. It is listed so that nothing is hidden and is never evidence.

A confirmatory contrast that comes out close to zero remains confirmatory, and an exploratory look that shows a large gain remains exploratory. We never move a result between labels after seeing it.

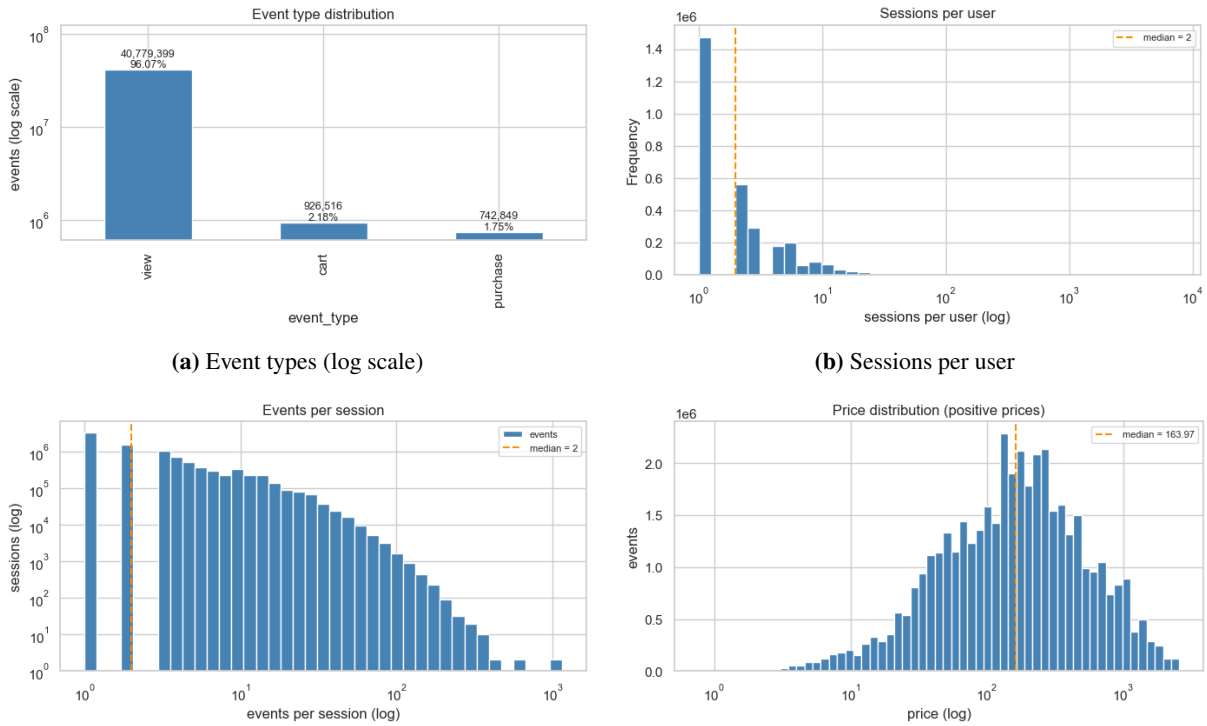
3.2 Data

The data are the REES46 e-commerce behaviour logs of one multi-category online store [8], recorded in October and November 2019. Each event is a view, a cart addition or a purchase, with a timestamp, product, category, brand, price, user and session identifier. The raw October file holds 42,448,764 events (5,668,612,855 bytes). Later months of the same store exist but were never used, and there are no December 2019 data (no TEST2). Figure 3.1 shows the shape of one month of the log.

All experiments read one frozen dataset, `rees46_nextitem_v1`, with the version hash

`860e5a1c3a81b8e4bbfeff4f245fa37638a43e4f260df179585964d2d4da8dbd`

covering 409 files (8.6 GiB) and listing only the source months 2019-10 and 2019-11. The dataset is immutable: it is never edited or copied per person, and any change of preprocessing would need a new



(a) Event types (log scale)

(b) Sessions per user

(c) Events per session

(d) Product prices

Figure 3.1: The October 2019 month of the REES46 log, as plotted by the data-audit notebook of the prototype stage (`S1_DS_04_Data_Audit.ipynb`); the panels are the notebook's own output. Views are 96.07% of the 42,448,764 events, carts 2.18% and purchases 1.75%. Sessions per user and events per session are heavy-tailed with a median of 2, and the median positive price is 163.97.

version and the study lead's approval. The data version of the main pre-registered study built from it is `CLEAN_V2_POP5100_V2`, produced as 15 manifests with a complete 98/98 receipt and sealed by the lock `DATA_PROTOCOL_LOCK_V2`. An earlier lock (v1) is historical: its population of 262,144 MAIN users was superseded when the study lead restricted the study to users with 5 to 100 eligible training examples.

That restriction defines four strata, computed from the TRAIN period only: TRUE COLD (0 eligible examples), SPARSE (1 to 4), the 5–100 pool, and HEAVY (more than 100). Together they contain 4,804,933 users and 40,180,608 examples. The 5–100 pool has 1,446,986 users with 28,623,386 examples, which is 71.24% of all supervision, with a median of 13 examples per user. A simulated device needs some history, and users with thousands of events are often automated clients.

Preprocessing follows the locked feature schema S*, with 25 information channels (24 input columns plus the output catalogue). It came from a registered feature search on a separate cohort (the FS band) that required the worse model family to lose at most 1.0% relative to the full schema (95% upper bound at most 2.0%); S* passed, with drops of −0.90% for the GRU family and −0.13% for SASRec (*descriptive*). Feature statistics are fitted once on the full TRAIN period for all arms; for FL arms this is disclosed as one federated-statistics round within the public bootstrap block. In the prototype the client-side transforms were written to run on the device: categorical identifiers are hashed with blake2b-64 [53], prices are mapped to bands by `price_transform_v1`, and the time gap is capped as $\log(1 + \min(\text{gap}, 86400))$ with

the gap in seconds.

Limitations

The data come from one store over two months, so the findings are specific to one catalogue and one traffic pattern. The TEST week contains Black Friday (29 November 2019); the change of the cloud model from VALIDATION to TEST is reported as a shift indicator and not corrected. Earlier GRU and SASRec pilot studies had inspected TEST at period level (register entry 3), so all main-study TEST results are retrospective. At TEST, some user-context features replay TRAIN and then TEST and skip the validation week.

3.3 Task and metrics

The task is next-product ranking over the full catalogue. A *decision* is one prediction point of a user; its target is the next *different* product that the user views in the same session (views only). The input is the user's last 50 representative events of the whole prior stream, across sessions and partition fences, strictly causal and identical for TRAIN, VALIDATION and TEST (history policy `USER_HISTORY_L50`). The catalogue holds every product with global TRAIN support of at least 5, which is 158,486 products, and every prediction ranks all of them with pessimistic ties. We do not sample negatives, because sampled item sets can change the ordering of methods [6].

The primary metric is client-macro MRR@20. For a target of rank r among the 158,486 products, the reciprocal rank is RR $= 1/r$ if $r \le 20$ and 0 otherwise; a target outside the catalogue counts as a miss, which is the registered end-to-end (E2E) denominator. For an arm a with user set $\mathcal{U}$ and decision sets $\mathcal{D}_u$,

$$\mathrm{MRR@20}(a) \;=\; \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \frac{1}{|\mathcal{D}_u|} \sum_{d \in \mathcal{D}_u} \mathrm{RR}_{u,d}(a), \tag{3.1}$$

so each user weighs equally. NDCG@10 and HR@10 are secondary, and HR, NDCG and MRR at cut-offs 1, 5, 10 and 20 are reported descriptively. The *retention* of an arm against a reference is

$$R \;=\; 100 \times \frac{\mathrm{MRR@20(arm)}}{\mathrm{MRR@20(reference)}}, \tag{3.2}$$

computed per seed at the same checkpoint rule and then averaged over seeds. The cloud model C_BIG, trained on the same 131,072 users as the devices, is the reference for the random-start (cold) line. The cloud reference C_FULL, trained on 1,348,682 users, is the reference for the pretrained (warm) line, because the pretrained model saw nine times more users than C_BIG (amendment 06K). The recipe of C_FULL was not re-tuned at that scale, so ratios against it carry an upward bias, which we disclose.

A model that needs no training gives a floor for interpretation. The baseline LAST ("repeat the last viewed item") reaches about 44% of C_BIG on the validation week (about 51% on TEST), so every arm is also reported as a difference to LAST and as the headroom $R^* = (\mathrm{arm} - \mathrm{LAST})/(C_{\mathrm{ref}} - \mathrm{LAST})$. An arm whose

difference to LAST has a confidence interval below zero carries the flag BELOW_DEVICE_HEURISTIC. On the MAIN validation users the baselines scored 0.11138 for session kNN, 0.07111 for LAST and 0.01943 for popularity (*descriptive*).

Two checkpoint rules govern which model of a run is scored. *PRACTICAL_BEST* (PB) is the strictly best of 12 evaluation marks, one every 0.5 budget units within the 6.0 budget of Section 3.7, where a new best needs an improvement of more than 0.0002; the marks are evaluated on the users' own validation rows. PB is the registered primary rule for every contrast except the DP ones. Because it takes a maximum, PB values on VALIDATION are optimistic, and the TEST week is the only unbiased number. *CONTROLLED_BUDGET* (CB) is the fixed 6.0 endpoint or, for DP arms, the round-384 endpoint, a data-independent release. CB is confirmatory for the four DP contrasts and reported beside PB elsewhere.

3.4 Populations and splits

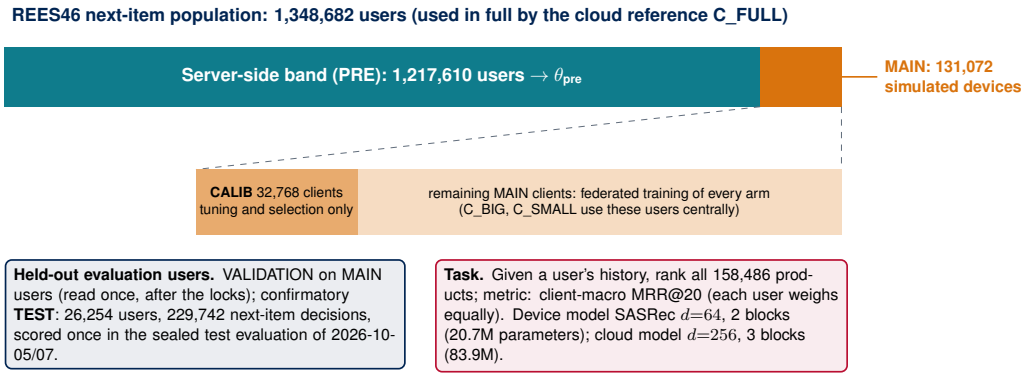


Figure 3.2: The user populations, the nested CALIB subset of MAIN, the TEST set and the prediction task. Counts are MEASURED from the registered manifests (source: `tikz/d4_populations.tex`).

Each user receives a pseudonymous key, the SHA-256 of the salted user identifier. Users of the 5–100 pool are ranked by this key, and every population is a deterministic rank range chosen without looking at outcomes (Figure 3.2 and Table 3.1). *MAIN* (ranks 0 to 131,072) is the population of simulated devices. *CALIB* is the first 32,768 users of MAIN and the only population on which tuning values were read. The *PRE band* holds the 1,217,610 users beyond MAIN who are assumed to have opted in to server-side training; it supplies the pretrained model θ_{pre}. MAIN together with the PRE band, 1,348,682 users, is the population of C_FULL (about 93% of the pool; the rest is held out).

The splits follow one global timeline in UTC, so there is no future leakage across users. TRAIN covers 2019-10-01 to 2019-11-16, VALIDATION the week to 2019-11-23, and TEST (RETRO_TEST) the week to 2019-11-30; the last day, 2019-11-30, supplies targets only and emits no decisions. Windows are half-open. VALIDATION was read once in the registered end-game.

Users *listed* in a VALIDATION manifest differ from users with at least one *evaluation row*. MAIN has 51,211 manifest users, of which 40,764 have an E2E row (519,714 decisions); CALIB has 12,715 manifest and 10,070 E2E users (127,530 decisions); the CALIB-disjoint part of MAIN, a diagnostic, has

Table 3.1: Populations and roles. Ranks refer to the pseudonymous selection key. Source: `HANDOVER/02_DATA.md`.

Population	Definition	Users	Role
5–100 pool	5 to 100 eligible TRAIN examples	1,446,986	source of all bands
MAIN	ranks [0, 131,072)	131,072	simulated devices; 2,587,776 TRAIN decisions
CALIB	ranks [0, 32,768), inside MAIN	32,768	tuning only; 646,067 TRAIN decisions
PRE band	ranks [131,072, 1,348,682)	1,217,610	server pretraining of θ_{pre}; 24,089,824 decisions
C_FULL population	ranks [0, 1,348,682)	1,348,682	cloud reference; 26,677,600 decisions
UNSEEN_VAL	ranks [1,348,682, 1,381,450)	32,768	device-recipe selection; 12,792 validation users
UNSEEN_HOLDOUT	ranks [1,381,450, 1,414,218)	32,768	new-user panel; 12,862 validation users
FS band	ranks from 1,414,218	32,768	feature search only

38,496 manifest and 30,694 E2E users. The TEST set contains 26,254 users and 229,742 decisions. The other bands are disjoint rank ranges, so the selection panel UNSEEN_VAL does not overlap the C_FULL population, but CALIB is the first quarter of MAIN and its validation rows lie in the week that was read. The device recipes were therefore selected on the unseen panel, and every CALIB-tuned arm carries a CALIB-disjoint diagnostic. The strata TRUE COLD, SPARSE and HEAVY form descriptive panels.

3.5 The model

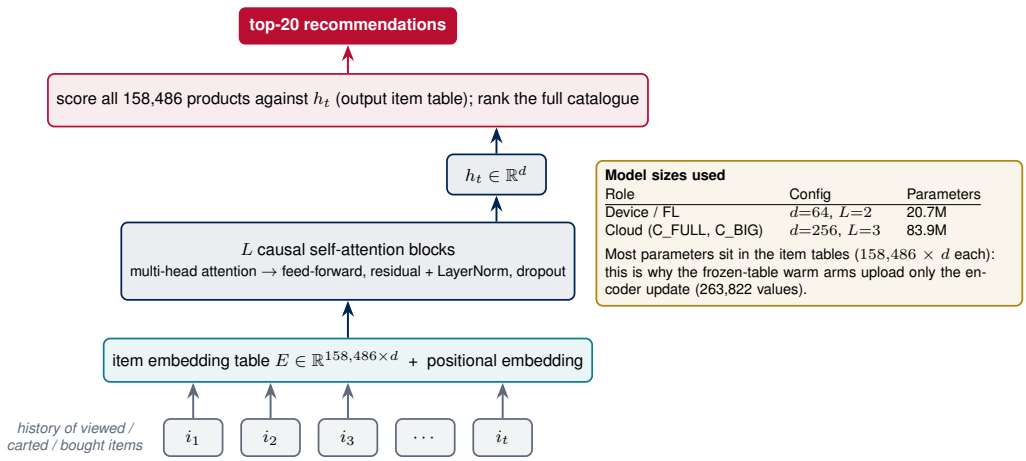


Figure 3.3: The SASRec model and its two sizes: the device and FL model (width 64, 2 blocks, 20.7 M parameters) and the cloud model (width 256, 3 blocks, 83.9 M parameters). Source: `tikz/d7_sasrec.tex`.

The backbone was chosen by a registered central contest. The contest compared a GRU in the style of GRU4Rec [1, 13] with SASRec [2], a unidirectional self-attention model built from Transformer blocks [14]. Amendments 02R1 and 02R2 put the model size inside the search at the full 6.0 budget and stratified the learning-rate grid within each size. In our contest SASRec scored higher at the registered rule PB: the GRU-minus-SASRec difference was −0.004271 with a 95% interval of [−0.005180, −0.003261] (*validation*). The verdict depends on the checkpoint rule; at the fixed 6.0 endpoint (CB) the GRU with

width 128 was significantly better, which we recorded as a diagnostic and did not use for selection. We therefore claim only that SASRec scored higher in our contest at the registered rule, not that it is the better architecture in general. The choice was sealed in the architecture lock. A later GRU-versus-C_SMALL comparison, run in the extensions, is a CPU rehearsal and is reported as such in Chapter 5.

Figure 3.3 shows the model. The last 50 events, each represented by the embedded channels of S*, pass through a stack of causally masked self-attention blocks, and the hidden state at the last position is scored against an untied output item table, giving one score for each of the 158,486 products. The cloud model has width 256 and 3 blocks (83,881,764 parameters). The device and FL model has width 64, 2 blocks and 4 heads, with 20,710,048 parameters when trained centrally and 20,708,708 shared parameters in the FL rows (MEASURED). It differs from the cloud model in depth as well as width, so the comparison of the two is not a pure width effect. The same architecture is the global model on the server and the model on the device.

Because the item tables hold about 98% of the parameters, the choice of which parameters to train and send decides the communication cost. The registered device payload cap is 170 MB per visit (amendment 03). Dense FL moves 165,669,672 bytes per visit (FP32, download plus upload; MEASURED). The frozen-table variant of Section 3.7 moves 2,110,584 bytes per visit, plus a one-time download of 81,779,544 bytes. The training objective and the software structure are described in Chapter 4.

3.6 Training regimes and arms

An arm combines a model, a training regime and a rule for what leaves the device (Table 3.2). Unless stated otherwise, every arm was trained with seeds 2026 and 2027; C_FULL has seed 2026 as confirmatory and 2027 as descriptive, and PRE has one seed (2026).

Two lines of FL arms exist. The cold line starts from random weights; it was the original design and now serves as an ablation. The warm line starts from the server-side pretrained model θ_{pre}, which a vendor could train on the data of users who agreed to server-side training. The study lead made the warm line primary after the first runs (amendment 06N), because real systems start from a model trained on permitted data. The realistic edge system is FA_WARM_FROZEN: federated training from θ_{pre} with the item tables frozen and never sent. The cold method set was re-run from scratch at the final recipe (the "N33" re-runs); the earlier partial cold runs of FedProx and of the non-private DP twin (*stopped*) were halted for reallocation and are never evidence. Several registered arms were declared not run, among them the device panels on SPARSE, HEAVY and TRUE COLD users.

3.7 The federated recipe

The first registered FL recipe was weak: on CALIB it reached only about 20 to 25% of a matched central model. A research audit found five causes: no server optimiser; a fresh client AdamW [26, 54] at every visit; a local learning rate that was too small; the central learning-rate shape mapped onto rounds; and a

Table 3.2: Systems and arms. "Leaves the device" lists what a user's device sends. Levels follow the privacy ladder of amendment 06L_C; L2 carries no formal guarantee. Variants on the frozen-table model carry the suffix `_WARM_FROZEN`. Source: `HANDOVER/01_RESEARCH_QUESTION_AND_DESIGN.md`.

Arm	What is trained, and where	Leaves the device	Level
C_BIG, C_FULL, C_SMALL	Cloud model on raw events: the 131,072 MAIN users, all 1,348,682 users, or the device-size model on MAIN	raw events	L0
PRE (θ_{pre})	Device-size model trained centrally on the 1,217,610 PRE-band users, shipped and evaluated alone	raw events of opted-in users only	L1
C_SMALL_FP32, C_SMALL_INT8	C_SMALL exported to ONNX, run on CPU	raw events once, for the base	L1
FA	FL from random weights θ_0, all weights	dense FP32 update	L2
FA_WARM, FA_WARM_FROZEN	FL from θ_{pre}: all weights, or encoder only with item tables frozen	dense or encoder-only update	L2
FP, FA_Q8	FedProx ($\mu = 0.01$); 8-bit stochastic-rounding uploads	update (8-bit for FA_Q8)	L2
PF	FL plus a personal vector p_u ($\lambda = 10^{-4}$) kept on the device	shared update; p_u stays	L4
FA_1024, DP8, DP1	Poisson-sampled FL with clipping; no noise, $\varepsilon = 8$, $\varepsilon = 1$	clipped update; noise added to the sum	L3
FT_FA, FT_FA_WARM, FT_FA_WARM_FROZEN	FL model downloaded, then fine-tuned on each device	as FL; personal parameters stay	L4
FT_C, FT_PRE	C_SMALL or θ_{pre} downloaded, then fine-tuned on each device	nothing after the download	L1, L5
LO	each device trains its own model from θ_0	nothing	L6
LAST, POP, SKNN, RULE, KNN_EMB	baselines	nothing	—

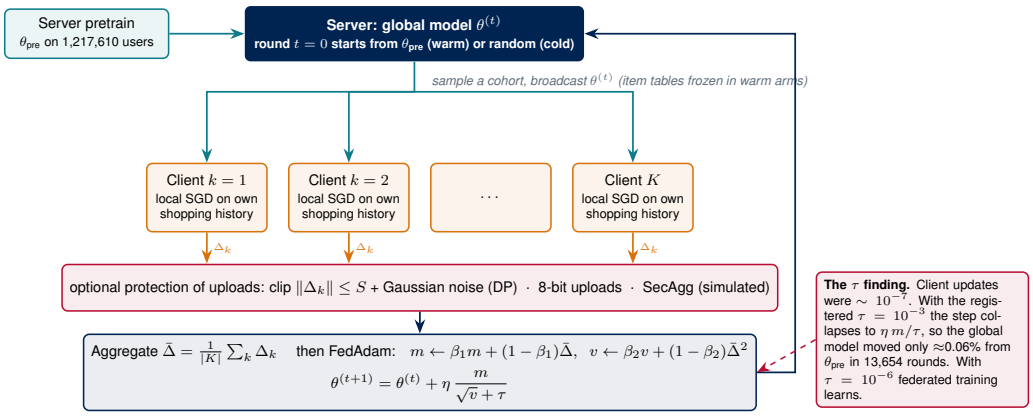


Figure 3.4: One federated round with the FedAdam server step and a callout on the stabiliser τ, whose registered value is examined in the box below. Source: `tikz/d3_fl_round.tex`.

random start. We removed them through a registered, strictly sequential series of federated recipe tuning stages on CALIB, so that the thesis can say what each change gained.

Figure 3.4 shows one round. The server sends the global model θ_t to a cohort S_t of clients. Each client trains on its own examples for the registered number of local passes and returns the difference Δ_k between its local weights and θ_t. The server averages the deltas into $\bar{\Delta}_t$ and applies the adaptive optimiser FedAdam [25],

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1)\,\bar{\Delta}_t, \\ v_t &= \beta_2 v_{t-1} + (1-\beta_2)\,\bar{\Delta}_t^{\odot 2}, \\ \theta_{t+1} &= \theta_t + \eta_s \frac{m_t}{\sqrt{v_t} + \tau}, \end{aligned} \tag{3.3}$$

with $\beta_1 = 0.9$, $\beta_2 = 0.99$ and $v_0 = \tau^2$ (a clarification registered in the decision log). The server step η_s was 0.01 on the cold line and 0.001 on the warm line, and the stabiliser τ was registered as 10^{-3}. We also screened server momentum (FedAvgM [21]).

Table 3.3: The cold federated recipe tuning stages on CALIB (*descriptive*). Value: E2E client-macro MRR@20 at PB, FA, seed 2026, full 6.0 budget, CALIB validation users. Source: `contracts/fl/` and `FL_IMPROVEMENT_LADDER.md`.

Stage	Change relative to the previous stage	MRR@20
0	FedAvg as registered (fresh client AdamW, no server optimiser)	0.023473
1a	local learning rate ×2	0.025562
1b	local learning rate ×8 (the ×2 winner sat on the grid edge)	0.028796
2	FedAdam server optimiser, $\eta_s = 0.01$, $\tau = 10^{-3}$	0.034247
3	constant-peak learning-rate shape (FL_CONST) and client SGD, momentum 0.9, learning rate 0.1	0.094936
4	one local pass instead of two, batch 16	0.107562

Table 3.3 summarises the federated recipe tuning stages. The rules were registered before the first screen was read: a challenger replaced the incumbent only if it gained at least 0.002, differences within 0.0005 were ties, and a winner on the edge of its grid triggered an extension of the grid. The last rule came from an early lesson: the winner at ×2 sat at the top edge and was still rising, the MAIN workstreams started on it were stopped and superseded, and the extended grid placed the optimum in the interior, at ×8. The cumulative change from tuning stage 0 to tuning stage 4 is a factor of 4.58 on CALIB (*descriptive*). We interpret the large step at tuning stage 3 as follows: the decline of the run of tuning stage 2 after its peak came from the fresh client AdamW, whose near sign-like dense updates touch all 158k output rows, and not mainly from overfitting. Every FL arm uses the recipe of tuning stage 4, which we call FA. The Poisson DP arms keep two local passes and batch 16, because their rounds and noise multipliers were registered with that setting.

The warm start has two registered screens at the point reached by tuning stage 4, both starting from θ_{pre}. Tuning stage 7 (FA_WARM) trains all weights; its winner used a quarter of the local learning rate (0.025) with a server step of 0.001 and reached 0.170582 on CALIB (*descriptive*). Tuning stage 8 (FA_WARM_FROZEN) freezes the three item-indexed tables, so they are neither trained nor uploaded; its

winner used the full local learning rate (0.1) with the same server step and reached 0.170709 (*descriptive*). Tuning stage 8 had the higher value and became the warm recipe, written FA_WARM* in the contrast registry. Only the encoder, with 263,822 parameters (MEASURED), is trained and sent, which gives the payload quoted in Section 3.5. A later review recorded that the screens moved the client learning rate only upward and that the winner sat at the smallest server step of the grid, which is the starting point of the follow-up analysis.

The training budget is measured in passes over the training data (epochs): 1.0 pass is one exposure of every eligible TRAIN example of the cohort. All arms train for 6.0 passes over the training data with 12 marks, one every 0.5 passes; in FL this is about three client visits with two local passes and about six with one. On CALIB the full budget is 1,707 rounds over 32,768 clients with two local passes (3,414 with one), and the cold FA arm, which uses one local pass, runs 13,654 rounds on MAIN. Poisson cohort sampling in the DP arms is described in Section 3.8. FedProx [22], personalised FL (PF) and 8-bit uploads (FA_Q8) use the settings of Table 3.2.

Limitations

The registered τ was too large. The stabiliser was registered as $\tau = 10^{-3}$, chosen in a CALIB screen at tuning stage 2, where the edge cell $\tau = 10^{-5}$ scored 0.024374 against 0.034247. The tuning stages were strictly sequential, so τ was not re-opened when tuning stage 3 replaced the client optimiser. After the test evaluation, an inspection of update norms (*exploratory*) showed that client pseudo-gradients were about 10^{-7}, so the server step was 20 to 100 times too small. The tuned warm-start federated model of the follow-up analysis moved only about 0.06% away from θ_{pre} in 13,654 rounds on MAIN (*exploratory*). With $\tau = 10^{-6}$, FL learns. The tau-fixed REES46 warm run minus PRE is +0.00022 with a 95% interval of $[-0.00011, 0.00057]$ (*exploratory*), which includes zero, so the confirmatory answer is robust to the fault. The re-runs changed several ingredients at once (τ together with η_s, with unfrozen item tables, or with the budget), so we never state that τ alone explains a gain.

3.8 Privacy mechanisms

The DP arms follow DP-FedAvg [35], the federated form of DP-SGD [43]. In every round each sampled client's delta is clipped to the norm S, the clipped deltas are summed, Gaussian noise is added to the sum, and the result is divided by a fixed denominator m,

$$\bar{\Delta}_t \;=\; \frac{1}{m}\Big(\sum_{k\in S_t} \Delta_k \min\big(1, \tfrac{S}{\|\Delta_k\|_2}\big) \;+\; \mathcal{N}\big(0,\, z^2 S^2 \mathbf{I}\big)\Big), \tag{3.4}$$

where $m = 1{,}024$ is the expected cohort size. Clients are chosen by Poisson sampling with $q = 2^{-7}$ for $T = 384$ rounds at $\delta = 5 \times 10^{-6}$. The noise multipliers are $z = 0.5642$ for $\varepsilon = 8$ and $z = 1.3094$ for $\varepsilon = 1$, computed with a user-level Rényi-DP accountant [44] for the subsampled Gaussian mechanism [43, 45]. The clip norm of the cold arms is $S = 0.284363$, the median client-delta norm of a calibration run; the

warm frozen arms use their own record. The arm FA_1024 is the non-private twin, with the same sampling, rounds and clipping but $z = 0$, so that the ratio of DP8 to FA_1024 isolates the cost of the noise. The best-of-12 choice of a mark on the users' own validation data lies outside the accountant, so the DP contrasts are confirmatory under CB.

Sampling mattered. Amendment 06C recorded, before any DP run, that the registered accountant assumes Poisson sampling while the first design of the DP arms used a fixed-size sampler without replacement. At $z = 0.5642$ that design would have given $\varepsilon \approx 1{,}388$, not 8. The arms were changed to Poisson sampling with a fixed denominator.

Limitations

No formal DP guarantee is established for the legacy DP runs. The legacy implementation used a disclosed public seed, so its noise and Poisson streams can be reconstructed. The utility numbers of the DP arms remain measurements of what the mechanism does to quality, but these runs do not establish the randomized-DP guarantee. We read $\varepsilon = 8$ and $\varepsilon = 1$ as the targets of the accountant and not as achieved guarantees.

Secure aggregation [36, 37] would let the server see only the sum of the updates. No main-analysis arm simulates it. In the extensions, a simulation estimates the upload overhead of masked updates (Figure 5.14); it is a simulation only, not production cryptography, and it does not make a non-DP arm private. Two exploratory audits examine what the mechanisms protect. The membership-inference audit [38, 39] reports the attack AUC (Figure 5.13). The update-leakage audit measures the advantage of an attacker who sees the aggregate input, against a fixed threshold of 2 percentage points (pp); the advantage is about 2.625 pp (95% CI 2.312 to 2.953) (*exploratory*), above the threshold, so the update-leakage audit gives a negative finding. An empirical canary audit in the manner of [41, 42] was also designed, but its design has a ceiling of $\varepsilon \approx 5.213$, below 8, so it cannot reject $\varepsilon = 8$. DP-FTRL and the canary restart were stopped by the study lead (*stopped*), and the ColdDP calibrations failed before training and produced no result.

3.9 On-device measurement

A deployed model must also fit and run on the device, so three quantities were measured: the quality cost of quantisation, the payload in bytes and the latency. The prototype exported models to ONNX at opset 17 and applied INT8 dynamic quantisation [47], based on the 8-bit affine scheme of [46]. In the main study, C_SMALL was exported to ONNX and run on CPU with ONNX Runtime [47] as an FP32 and an INT8 arm. The contrast K16 is the retention of INT8 against FP32 on CPU; for reasons of time it was computed on the first 16,384 E2E users in selection-key order (a registered fallback subset). Its timing was measured twice: pass 1 combined the accuracy run with timing on a loaded host, and pass 2 repeated the latency alone on a quiet CPU.

Latency uses the WebAssembly backend [49] in Microsoft Edge (Figure 5.15). The headless Edge on the

server (SERVER_BROWSER) is not a consumer device. The closure records also contain a single-thread CPU latency measurement on a shared server, labelled L6 there; it is not a laptop or a phone. We report no battery or energy measurement.

3.10 Statistics

All uncertainty comes from a paired user bootstrap [50]: we resample users with replacement 1,000 times (seed 20260926; 20261004 for the follow-up analysis), evaluate both arms of a contrast on the same resample and read the percentile 95% interval. Ratios are bootstrapped directly. Beside every contrast the analysis reports the per-seed points; the seed spread, with a flag SEED_SENSITIVE when it exceeds half the interval width; an interval over users and seeds; the share of harmed clients (users with a negative difference) against a seed-versus-seed null; the CALIB-disjoint diagnostic; and the split of targets into REPEAT (inside the user's 50-event window) and NOVEL (outside it).

The registry `ANA_CONTRASTS_v4` holds 33 confirmatory items in three tiers (Table 3.4). The primary pair is the retention of the realistic edge system against the matched cloud model (K20b) and its gain over the shipped pretrained model (K21). It is read unadjusted and outside the Holm family, and a claim about the deployment branch needs both. The 18 secondary contrasts form one Holm step-down family [51]; the 13 tertiary contrasts are unadjusted and carry a descriptive Holm band. Equivalence is tested with two one-sided tests (TOST) [52]: for 8-bit uploads (K14, K14w) the 90% interval of the retention must lie inside $[99, 101]$, and for the follow-up-analysis contrast Q1 the margin is ± 0.0005 in MRR@20. Several contrasts are algebraically linked and never independent evidence: K3 equals K2 divided by K1, the contrasts K17 to K19 equal the matching ratios times K15, and K20b minus K20c equals $100\,\mathrm{K21}/C_{\mathrm{FULL}}$.

Table 3.4: Contrast hierarchy of the registry `ANA_CONTRASTS_v4` (hash prefix `f1769f0b`). Source: `HANDOVER/01_RESEARCH_QUESTION_AND_DESIGN.md`.

Tier	Items	Contrasts	Multiplicity rule
Primary	2	K20b, K21	unadjusted, outside Holm; both required
Secondary	18	K1, K2, K3, K8, K9b, K9c, K11b, K12, K17, K20, K20c, K_INIT, P5, K4w, K5w, K12w, K13w, K14w	Holm step-down over 18
Tertiary	13	K4, K5, K6, K7, K9, K10, K11, K13, K14, K15, K16, K18, K19	unadjusted, with a descriptive Holm band

The claim vocabulary was frozen together with the analysis script. "A exceeds B" is used only when the paired 95% interval lies above the reference (0 for differences, 100 for ratios). An interval that includes the reference reads "no detectable difference", and only the TOST contrasts may say "equivalent". Every retention sentence names the recipe and the reference, a SEED_SENSITIVE or BELOW_DEVICE_HEURISTIC flag appears in the same sentence as the result it qualifies, and non-DP FL is never called private.

3.11 Pre-registration and governance

The study has more than one hundred arms, and AI agents helped write and review its code, so a post-hoc story would have been easy to tell. We therefore followed the pre-registration idea of fixing the analysis before seeing the outcome [7]. The invariant is that no rule changes after the value it governs is read.

3.11.1 Rules, amendments and locks

Any rule, grid, arm, contrast, seed set or interpretation is written into an amendment (01 to 07, 06A to 06P and PREG2) or a decision-log (DL) row, hashed and logged before a value that it governs exists. Amendments are never edited: a correction goes into a new file, an erratum appended before any read, or a new row. Tuning happens only on CALIB, device recipes are selected only on the unseen panel, MAIN_VALIDATION is read once and TEST is looked at once. Selections, locks, freezes, run outputs and register entries are written once, and new code runs only from a new immutable tree after tests and an independent review. Chapter A lists the amendments.

The gates ran in a fixed order, from the data lock through the architecture lock and the FL selections to the contrast registry, the analysis freeze, the single VALIDATION read and the lock of all methods. The check-only run of the analysis refused twice, and both refusals show the machinery working. On 2026-10-03 it expected the CALIB, MAIN and disjoint user counts {51,211; 12,715; 38,496} but counted {40,764; 10,070; 30,694}: the registered constants held *manifest users* and the check counted *E2E users*, both exact under their own definitions, and no value was read. The second refusal came from a field `n_norms`, which the registered writer defines as the number of client-delta norms (20,625) behind a median and not as the number of rounds. Each refusal was diagnosed from code alone, fixed by a small reviewed packet and followed by a new freeze (the final one is `59c83c86`); the superseded freezes were never used for a read. The check then passed and the single VALIDATION read followed. Everything new after that point was exploratory. The lock of all methods (hash prefix `04a3cd19`) recorded 131 arms, 56 sealed FL checkpoints and the TEST plan: 109 re-evaluations of the locked checkpoints under PB and CB, 39 panel re-evaluations, 14 device runs, 6 baselines and the ONNX step.

3.11.2 The single test evaluation and the exposure register

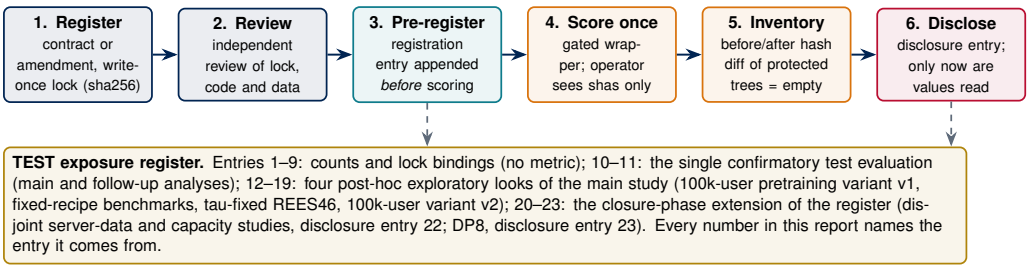


Figure 3.5: The gated protocol that led to the single test evaluation, and the hash-chained TEST exposure register that logs every access. Source: `tikz/d5_gated_protocol.tex`.

Every access to the TEST week is logged in a hash-chained exposure register that is never edited (Figure 3.5). Its first six entries, all before the test evaluation, record counts-only censuses, the prior exposure of the pilot studies, the manifest build and an integrity check; no model was scored. After the manifests were built, the study lead put the TEST look on hold because the first signal on the pretrained line was weak. When it resumed, the main analysis and the follow-up analysis shared ONE confirmatory test evaluation under binding conditions: both locks and the two plan entries had to exist before the first re-evaluation, all scoring for the main analysis came before all scoring for the follow-up analysis, no TEST output was opened before both analyses existed, and no second test evaluation would be held. A look at the main analysis first and a TEST2-only route were refused. The test evaluation ran from 2026-10-05T21:03Z to 2026-10-07T10:09Z and produced register entries 10 (main analysis) and 11 (follow-up analysis).

Entries after the test evaluation are exploratory looks. Entries 12 to 19 are the study lead's: the first server-data-size experiment (the 100k-user pretraining variant, v1, which varies the size of the server-side pretraining set), the T1 benchmarks with a fixed recipe, the T2 tau-fixed REES46 run and the second server-data-size experiment (the 100k-user pretraining variant, v2). Entries 20 to 23 belong to the closure-phase extension of the register. Each look uses a pair of entries: a pre-scoring registration entry fixes the hash of the plan before the evaluation runs, and a post-scoring disclosure entry records that it took place. The disjoint server-data study and the capacity study used registration entry 20 and disclosure entry 22, and the DP8 audit used registration entry 21 and disclosure entry 23.

3.11.3 The follow-up analysis

The VALIDATION read showed K21, the gain of federated training over the shipped pretrained model, to be slightly negative, and a diagnostic of the distance between the FL model and θ_{pre} followed. PREG2 (hash prefix `be0561ab`) then registered a separate follow-up study. It tests whether a warm-FL recipe specific to fine-tuning, with the tuned warm-start federated model chosen in a registered grid, adds value over θ_{pre}; the primary contrast Q1 is the difference between the tuned warm-start federated model and PRE. Two central controls (C1, C2) separate "the MAIN data add nothing" from "the FL recipe failed". On the TEST week the primary pair of the main analysis read K20b = 88.49% [87.73, 89.18] (*confirmatory*) and K21 = −0.000508 [−0.000933, −0.000037] (*confirmatory*), and Q1 = −0.000081 [−0.000179, 0.000024] (*confirmatory, follow-up*) is read as EQUIVALENT within ±0.0005. Chapter 5 interprets these numbers; they are quoted here only to show what the registered tests could say.

3.11.4 Operating model with AI agents

The way the work was done affects how the records should be read. The prototype stage (GitHub, August to September 2026) was a team effort. It was organised in three workstreams: a data and model workstream, a federated-training workstream that also ran the pilots, and a system, deployment and security workstream: the blake2b-64 hashing, the price bands, the capped log time gap, the ONNX export, the INT8 dynamic quantisation, the communication byte ledger, the quality-retention registry, continuous integration and the

check script. The main study (2026-09-26 to 2026-10-11) was then run under a single study lead, working with AI agents. A technical-closure phase (2026-10-12 to 2026-10-17) verified the saved results and ran the extensions, and a final consolidation phase merged the records and set up the shared Lobot workspace.

Claude models and, early on, OpenAI Codex assisted with code and analysis under the team's supervision. In the main study the study lead steered several agent sessions: an operations session (the AI operations agent, a Claude Opus session) that planned, launched and logged work; a chief-scientist session (the AI gate reviewer, a Claude instance) that was the binding reviewer at the gates and, where the study lead delegated, acted as his proxy; Opus and Sonnet AI worker agents that implemented numbered packets; and separate reviewer sessions that wrote the independent code reviews. Codex was resumed once at the study lead's instruction and then stopped. Work passed through written packets, hand-offs and the decision log, under the rule of one owner per file set. The study lead delegated the end-game decisions to the AI operations agent and the chief scientist, and let the AI operations agent run later end-game stages once the chief scientist returned a GO. The independent reviews were AI reviews, not human expert reviews.

Limitations

The registrations are team-internal records: hashed files, a decision log and a hash-chained register. We do not claim third-party timestamps. The process also made mistakes, which the records keep: a registered amendment was overwritten by a second session and restored byte-identically (amendment 06F); two sessions appended the same decision-log row number; and an organisation spending limit stopped several jobs. None changed a result that we report, but they show the limits of an agent-driven process. The incident log is in Chapter D, and the registered contracts are summarised in Chapter A.

3.12 External benchmarks

Every REES46 result depends on one store. To test whether the recipe transfers, amendment 07 added benchmarks on public datasets. By decision of the study lead only the REES46 recipe was run on them; we implemented no new method, such as eSASRec [18]. All benchmark values are DESCRIPTIVE with respect to every PREG1 claim, they never enter a REES46 selection, register or lock, and retention ratios on leave-one-out data are not comparable in magnitude to those on REES46. Table 3.5 lists the six datasets: Amazon Beauty, Sports and Toys [10] and MovieLens-1M [12] (Wave 1, byte-identical to the eSASRec release), and the Amazon Reviews 2023 collection (Video Games and Baby Products, as released with the BLaIR paper [11]); the real fix is in references.bib: make hou2024bridging the arXiv:2403.03952 BLaIR record, or add a second key for it (Wave 2, converted from the official 5-core files). Other candidates were dropped for size or form; FL was infeasible on Beauty_and_Personal_Care, with 75,998 rounds.

The protocol follows the eSASRec construction of leave-one-out evaluation. The user's last training row is the inner validation target, training uses the training file minus that row for every arm, and the holdout row is scored from the full training history without retraining. Ranking covers the full catalogue with seen-item filtering and ties broken by ascending item id. The primary metric is NDCG@10. Our

Table 3.5: The six public benchmark datasets. Source: `results/benchmarks/datasets.csv`.

Dataset	Source	Split	Users	Items	Interactions	TEST users
Amazon Beauty	eSASRec release (byte-identical)	leave-one-out	22,363	12,101	198,502	22,311
Amazon Sports	eSASRec release (byte-identical)	leave-one-out	35,598	18,357	296,337	35,539
Amazon Toys	eSASRec release (byte-identical)	leave-one-out	19,412	11,924	167,597	19,365
MovieLens-1M	eSASRec release (byte-identical)	leave-one-out	6,040	3,706	1,000,209	6,038
Amazon 2023 Video Games	Amazon Reviews 2023 official 5-core (last_out_w_his)	leave-one-out	94,762	25,612	814,586	94,762
Amazon 2023 Baby Products	Amazon Reviews 2023 official 5-core (last_out_w_his)	leave-one-out	150,777	36,013	1,241,083	150,777

copy of the Wave-1 data and protocol reproduced the published MC, SeqRules and PCTM results of the full-ranking comparison [19] in 12 of 12 cells, to four decimals (*descriptive*). Wave 2 has no published numbers and supports only comparisons between our own arms.

The benchmark arms mirror the REES46 arms, with PRE trained on a seeded 15% user band that stands for the consenting data. FL clients are all users of the dataset, and the number of rounds is $T = \lceil 60N/(576P) \rceil$ for N users and P local passes. The matched-recipe protocol (the headline) runs the recipe unchanged with seeds 2026 and 2027. The tuned protocol, secondary and limited to Wave 1, tries a small grid of history length and dropout on a validation basis; the two protocols are never mixed. Against a published single run, a difference is stated only if $|\Delta\,\text{NDCG@10}| > 0.005$ and every seed lies on the same side; otherwise the result is "on par". Each dataset has one lock and one TEST look. Two schedule decisions are disclosed: a clock backstop for the MovieLens tuned-protocol grid (B13) was rescinded before it triggered (B15), and MovieLens-1M was moved from CPU to a deterministic A40 GPU class before any MovieLens value had been read (B16); the CPU runs were sealed unopened.

3.13 Tools and compute

All models were implemented in PyTorch [48] (version 2.13.0 with CUDA 13.0 on the server) and exported with ONNX for on-device runs [47]. The study ran on Lobot, the JupyterHub group server of Queen's University, under the group `group-digi2026-g8`. The contest ran on two NVIDIA A16 GPUs (15,356 MiB each); the downstream phase moved to two NVIDIA A40 GPUs (46,068 MiB each), with a CPU guarantee of 16 cores, 128 visible processors and a 384 GiB memory limit. The evaluation device class is part of the comparison key, so the cloud references were re-evaluated on a pinned deterministic A40 class (`A40-FP32-DET`). Strict determinism holds independently of what else runs on the card, so jobs could share the cards; with ten FL jobs in parallel the measured throughput was about 550 rounds per hour per job, or about 5,500 rounds per hour in total.

Code reached the server only as immutable trees whose files are listed with their hashes. The prototype contributed the continuous-integration workflow, the check script, the byte ledger and the quality-retention

registry; Chapter 4 describes how these parts fit together, and Chapter F lists what is needed to reproduce the results.

Chapter 4: Design and Implementation

Chapter 3 fixed the question, the populations, the models and the gated protocol. This chapter describes the software and the working practices that carried those decisions out. The project produced two code bases in sequence: the prototype platform, built in the GitHub repository in August and September 2026 as a team effort, and the research platform of the main pre-registered study, built from 2026-09-26 under explicit governance and run on the Queen's Lobot server, which produced the results of Chapter 5. Most numbers below describe software, files, tests and measurements and carry the (*descriptive*) label. The few prototype model-quality numbers are (*validation*) values on a window that was also used for model selection; they belong to a different task (588 product categories) and are not comparable with the main study.

4.1 Architecture overview

The system has four layers (Figure 4.1). The *data layer* turns the raw REES46 log [8] into canonical events and compact input features (blake2b-64 identifiers, price bands, capped log time gaps), and freezes the dataset `rees46_nextitem_v1` with its populations PRE, MAIN and TEST. The *training layer* holds the central trainer for the references C_FULL, C_BIG and C_SMALL, the server pretraining of θ_{pre} on 1,217,610 users, the federated simulator (FedAdam with its adaptivity parameter τ, Poisson sampling, clipping and noise for the DP arms, 8-bit uploads), the on-device arms and the secure-aggregation simulation, all on the Lobot GPUs (A16 for the backbone contest, A40 downstream). The *evaluation layer* ranks the full catalogue of 158,486 products without sampling, computes the client-macro MRR@20 and the retention against the references, and runs the privacy audits. The *deployment and governance layer* exports ONNX models at opset 17, quantises them to INT8, measures payloads in a byte ledger, and holds the pre-registration, locks, gates, the TEST exposure register and the decision log.

Four rules run through every layer. Raw behaviour stays on the (simulated) client, and only model payloads cross the boundary; the simulation runs Flower and Ray on one shared host, which shows logical message separation but neither device isolation nor protection against a hostile co-tenant (Section 4.3.2). The system measures instead of estimating. Code is frozen before it is scored, and TEST data are read only through a gate. Every document states what the system does not do.

4.2 The prototype platform on GitHub

4.2.1 Repository, workstreams and environment

No code existed before 2026-08-11, when the repository was created and the team agreed a two-week sprint (the "prototype stage") that compressed the technical stages W1–W4 under the principle "Assemble, Don't

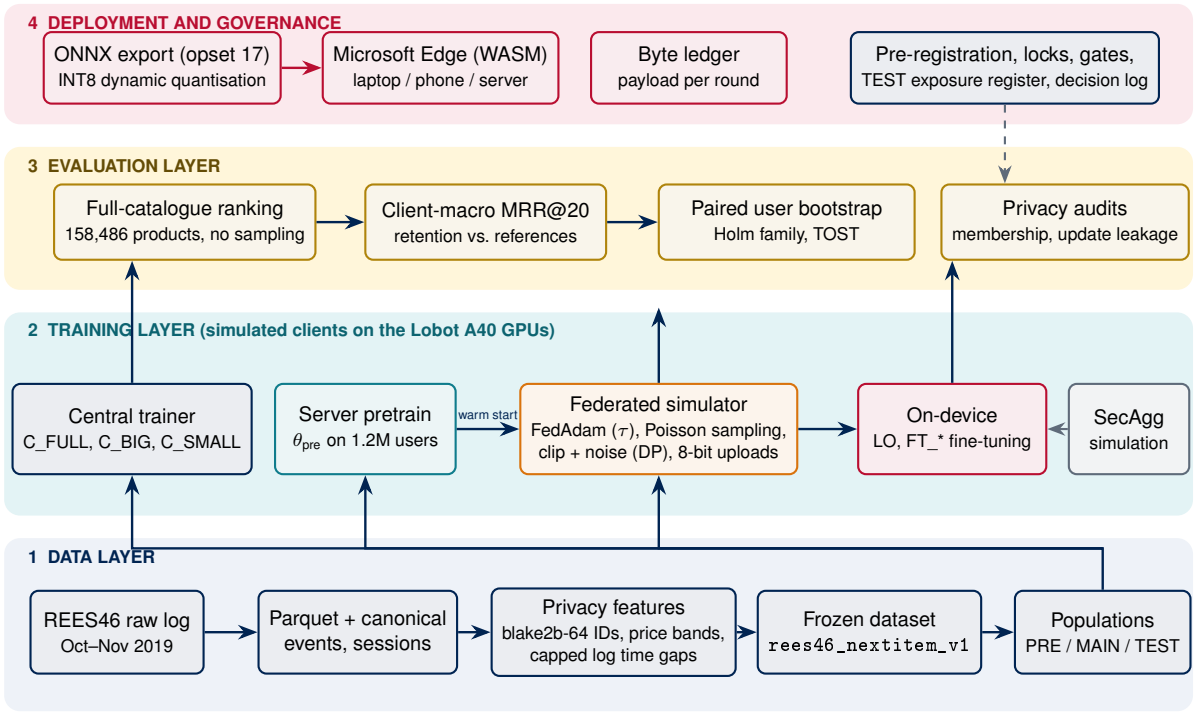


Figure 4.1: The four layers of the system: data (raw log, canonical events, privacy features, frozen dataset, populations); training (central trainer, server pretraining, federated simulator, on-device arms, secure-aggregation simulation); evaluation (full-catalogue ranking, client-macro MRR@20, paired bootstrap, membership-inference and update-leakage audits); deployment and governance (ONNX and INT8, Edge WASM, byte ledger, pre-registration, locks, gates, TEST exposure register, decision log). Source: `figures/tikz/d2_system_architecture.tex`.

Build". Three workstreams were fixed without rotation: a data and model workstream, a federated-training workstream with the pilots, and a system, deployment and security workstream. The workflow was Issue, Branch, Pull Request, Validation, Merge, with review optional. (*descriptive*) The board held 56 issues; the plan of two weeks took about 32 days; eight issues were open at the end; and the sprint closed on 2026-09-12T02:15:05Z with PR #92 (merge commit `bfc90a0`, 99 commits, 39 merged pull requests). We therefore call the result a *scoped* prototype, not a completed one. The locked environment used Python 3.11.14 through `uv`, CPU PyTorch 2.13.0 [48], Flower 1.33.0 [20], Ray 2.55.1, Polars 1.43.2, scikit-learn 1.9.0 and LightGBM 4.7.0.

4.2.2 The data pipeline

The raw file `2019-Oct.csv` [8] was acquired on 2026-08-15 and its identity recorded before processing: (*descriptive*) 5,668,612,855 bytes, 42,448,764 rows, nine columns, the event types `view`, `cart` and `purchase` only, and the range 2019-10-01 to 2019-10-31 UTC. The Parquet conversion accepted all rows, rejected none and wrote 85 row groups. The CanonicalEvent v1 schema has 12 columns. Because `category_code` is missing in 31.84% of rows and `brand` in 14.40%, the schema makes missing values explicit (`__UNK__`), namespaces every identifier (`rees46:user:123`), orders events by time and then source row, and keeps price raw. A `validate()` function returns a list of problems; it passed on the real sample and failed, as intended, on two deliberately broken copies. A fixture of 1,499 users and 23,109 events (281 users with 20 or more events; median 4 events per user, as in the full data) lets the team test interfaces without the large file. The data also showed that no `remove_from_cart` event exists, although the plan had assumed one.

The temporal protocol uses half-open UTC windows (TRAIN 10-01 to 10-22, VALIDATION 10-22 to 10-27, TEST 10-27 to 10-31, with 10-31 as an observation-only grace day) and excludes the 4,995 sessions that cross a boundary. Cohort C1 (at least 10 events, 2 sessions and 3 active days) has 388,789 clients; because its 17.6 million TRAIN events do not fit in memory, all modelling used a deterministic 25% slice of 97,279 clients. The price transform was frozen from median TRAIN prices (edges 29.58, 77.20 and 194.17; band 0 for the 693 items without a usable price). Three tasks were defined: T1 predicts the category of the next different item, T2 whether the first-viewed item is bought later in the session, and T3 reranks a frozen candidate list. A *feature gate* (G1) asserts that the example builders reproduce the frozen VALIDATION counts before any TEST row is read. Its first run stopped because T3 rebuilt 719,983 examples against 1,096,774 frozen ones; after the fix it returned GO on 9 of 9 criteria.

4.2.3 The model workstream

The model workstream trained on 3,113,814 TRAIN and 438,185 VALIDATION decisions (31,576 clients), with a history of 20 events, seeds 13, 42 and 2026 and a comparison threshold of 0.0033, the measured run-to-run spread. The selected model is a one-layer GRU with hidden size 128, dropout 0.3 and a cosine schedule from 0.001 over 30 epochs, whose history channels are category, event type and log time gap. A series of single changes (product, brand, price band, a second layer) gave no gain outside the

spread; the problem was undertraining. (*validation*) Three defects shaped later practice. The T2 label mask marked 549,185 negatives as censored, so the published PR-AUC of 0.1797 fell to 0.1424 after correction (the model was sound; the number was wrong). Fine-tuning for T2 destroyed T1, whose slice macro fell from 0.3479 to 0.1791, which led to a joint loss with $\lambda_{T2} \in \{0.1, 0.3, 1.0\}$ fixed in advance. The Recall@100 of the candidate lists had counted unreachable positives (0.6718 against 0.8916 after correction), which a blind replication found. Learned T3 rerankers did not beat the frozen retrieval order, so the shipped T3 component is model-free. The architecture decision (`shared_session_encoder_v1`) records that three separate GitHub approvals were not recorded. Chapter 3 gives the disclosure on AI assistance and AI review.

4.2.4 Federated plumbing with Flower

The federated workstream wrote the contracts that make a run reproducible: 13 schemas, a run-identifier grammar, immutable terminal attempts, seeds restricted to 13, 42 and 2026, and a regime catalogue (R1 centralised, R2A FedAvg [3], R2B FedAdam [25], R3 personalised, R4 local, R5 pretrain then adapt), with 219 tests at PR #63. A client identifier is `client-v1-` followed by a SHA-256 of a versioned label and the user identifier, and the per-round sampler is seeded by a SHA-256 of sampler version, seed and round. The frozen T1 evaluator fixes 588 dense categories and the headline `t1.next_distinct.mrr_at_20.macro`. A real-data smoke test (seed 13, 3 rounds, 4 clients per round) walked the whole path from frozen examples through Flower and Ray FedAvg to a validated result; it is explicitly non-scientific.

The scale profile asked how a fixed workload behaves as the declared population grows (Table 4.1). (*descriptive*) Every point registered exactly N nodes, selected 20 clients in each of six rounds, passed an independent FedAvg oracle within 10^{-6} and changed the model. Coverage is reported, not implied: in the accepted $N = 5{,}000$ trial, 120 fit calls reached 118 unique users. A repeat in a fresh process gave identical selection digests and a maximum parameter difference of 0.0.

Table 4.1: Federated runtime scale profile ((*descriptive*)): 20 clients per round, concurrency 2, six rounds, warm rounds 2 to 6, seed 13. Source: `docs/federated-scaling.md`.

Declared nodes N	Status	Warm p50 (s)	Warm p95 (s)	Peak tree RSS (GiB)
1,000	STABLE	0.525	0.540	1.775
2,500	STABLE	0.618	0.662	1.779
5,000	STABLE	0.644	0.688	1.784
5,000 (confirmation)	STABLE	0.635	0.763	1.785

The scoped pilots (PR #88 and #92) compared R1 with R2A on T1 (1,000 clients, 20 rounds of 50) and T2 (200 clients, 10 rounds of 20, single seed after a predeclared memory fallback). The first T1 attempt failed the frozen aggregation tolerance of 10^{-6} (deviation 1.43e-06, float32 noise at 50 clients) and the second exposed a contract defect; the third ran under a scale-aware float32 policy, with no model, data or training setting retuned between attempts. The pilot retention was 88.38% on T1 and 58.71% on T2 ((*validation*)), against a matched centralised pilot and not a final R1; these values must not be set beside the main-study retention figures. The verdict was `GO_T1_MVP_ONLY`. Not delivered were the final multi-task R1 and R2A,

the regimes R2B to R5, any differential privacy or secure aggregation, and any TEST evaluation.

4.3 The system, deployment and security workstream

This section describes the system, deployment and security workstream, from the GitHub record. Its goals were to keep the main branch safe without making the team wait, to state the real privacy boundary, to give every experiment honest measurements of bytes and latency, and to export the model to a portable format and check that it still computes what the training framework computes. (*descriptive*) The workstream delivered 20 of the 39 merged pull requests and 33 of the 99 commits, between 2026-08-31 and 2026-09-12 (PRs #65–#67, #69–#73, #80–#83, #85–#87, #89, #90 and #93–#95).

4.3.1 Environment audit and simplification

The first task (PR #69) audited the author's machine (Windows 11, Ryzen 7 PRO 4750U, 15.33 GiB RAM, no NVIDIA GPU). Three probes passed: the locked environment (77.907 s), a Polars scan of the fixture (23,109 rows) and a Flower and Ray synthetic smoke (299.323 s, 4,493.57 MiB peak) that reduced the global loss from 1.9444400072 to 1.1246404648. Ray created more actors than the two logical clients, so the audit did not treat the pass as evidence that large simulations fit. The status was PARTIALLY READY, and the missing GPU path became an open item; the model workstream later trained on a hosted studio with an L40S GPU and the main study ran on Lobot (Section 4.6). On 2026-08-28 the workstream also simplified its first versions on purpose: the repository-safety task fell from 10 files and 963 insertions to a net +53 and −925 lines, the hashing task from a 441-line module with two JSON schemas and a Node parity script to a small module, and the threat model from 314 lines to a version with 106 insertions and 280 deletions.

4.3.2 Repository safety, threat model and secret handling

PR #65 set the platform rules: a pull request is required but no approval is mandatory; the author merges after validation; the rule binds administrators; force-pushes and branch deletion are blocked; and secret scanning, push protection, dependency alerts, private vulnerability reporting and read-only Actions permissions are on. Review stayed optional so that changes remain visible and reversible without team members waiting on each other, and a CODEOWNERS rule that requested all three members was removed on 2026-09-01 (PR #73).

The threat model (PR #66) covers R1, simulated R2A, the data and the artefacts, and is a design document, not a vulnerability report. Its only supported claim is that R2A, as designed, reduces the centralisation of raw data. It does *not* claim real device isolation, protection against leakage through updates, differential privacy, secure aggregation, anonymity of hashed or namespaced identifiers, or that the repository contains no user identifiers; the last is recorded as an exception, because the fixture holds 23,109 events, 1,499 distinct `rees46:user:*` identifiers and 4,865 sessions of the real file, and the repository is public. R1

gives one process every batch and must not be described as privacy-preserving; the coordinator sees each update before aggregation; and checkpoints are loaded with unsafe PyTorch deserialisation only under a trusted-artefact contract after hash and semantic checks. The secret-handling rules forbid committing credentials, tokens, keys, environment files and raw histories, deny secrets to fork pull requests, require synthetic data in tests, and prescribe four steps after a leak: rotate or revoke, notify the team privately, remove the secret from tree and logs, and rewrite shared history only with the administrator. The ignore file is a safety net, not an access-control boundary.

4.3.3 Deterministic features

Product hashing contract

For a namespaced identifier s such as `rees46:item:<id>`, let d = BLAKE2b(s; digest size 8, key = seed as 8 bytes big-endian, person = `ppsi-item-v1`) [53]. For a bucket count B the bucket is $b = 1 + \big(\text{int}(d) \bmod (B-1)\big)$; bucket 0 is padding. The module never uses Python's `hash()`, and B, the seed and the residual dimension have no production default.

A unit test pins `rees46:item:44600062` to bucket 33 at $B = 257$ and seed 13. The model batch used 50,000 product and 5,000 brand buckets; the collision report counted four bucket sizes on 113,765 items, and the 50,000-bucket choice implies 60.5% forced collisions and was never recorded as a decision in the architecture ADR. (*descriptive*) Price enters through the frozen edges: a price exactly on an edge goes to the band above, and a missing, non-finite, negative or zero price gives band 0 (one catalogue item priced exactly 194.17 sits one band lower, and a test names it). The time gap is $g = \log\big(1 + \min(\Delta t,\ 86{,}400)\big)$ with Δt in seconds, so a gap is capped at one day; the first event and equal timestamps give 0, and out-of-order history is rejected, not repaired. Issue #26 had asked for $\log(1 + \text{price})$ with a missing mask; the module bands the price because the edges were already frozen.

4.3.4 Measurement discipline

PR #71 fixed three measurements: serialised file size in bytes, tensor payload (elements times element size, explicitly *not* network traffic) and CPU latency with discarded warm-ups, p50 and p95. The byte ledger (PR #83) counts the length of the serialised arrays Flower transmits, read at the two strategy points that see every transmission. It is not a parameter count times a width: for a `Linear(16,8)` module of 136 float32 parameters an estimate gives 544 bytes while the measured payload is 800 bytes, because each array carries a header, so the estimate is 32% low. (*descriptive*) In the smoke run (two clients, two rounds, seed 13, byte-identical on repetition) each round moved 1,600 B in each direction, 3,200 B per run, in 4 transmissions each way. A `CommunicationRecord` holds only a round, an opaque client identifier, a direction and a byte count, refuses a raw numeric user identifier, and records nothing for a client whose reply never arrives: that client did not transmit zero bytes, it did not transmit, and a zero row would read as a measurement.

The Quality Retention registry (PR #72) computes QR as the ratio of a comparison arm to R1 for approved MAXIMIZE metrics, paired by seed, task, cohort, metric, direction, unit and a frozen compatibility tuple; other metrics stay absolute and a missing R1 is reported as pending. At the end of the prototype stage the artefact folder held only seven R2A smoke records and no R1 record, so the pilot retention came from a committed pilot pair. The temporal leakage validator (PRs #81 and #87; 10 tests) rejects history that is not strictly before the decision, a T1 label used as input, a T2 query in the history, examples crossing a split and immature outcomes treated as negatives. Running it on real rows in PR #87 showed that it demanded a maturity time equal to the T1 target time while the builder writes the decision time, so it would have rejected every real T1 row; the tests had used hand-built rows no producer emits. No pipeline calls the validator in CI yet.

4.3.5 ONNX export and the dynamic-batch fix

The export (opset 17, CPU execution provider, ONNX Runtime 1.24.1 [47]) fixes one signature: five history channels, the gap, the lengths, four query tensors and the candidate identifiers, category, price band and rank go in; `t1_logits`, `t2_logit` and `t3_scores` come out as raw activations, so any probability shown in a demonstration is the demonstration's claim and not the model's. Batch, history length and candidate width are dynamic, and nothing else is. (*descriptive*) On ten shape combinations of the 5-channel prototype (3,664,919 parameters) the maximum absolute differences to PyTorch were 1.42e-07, 1.79e-07 and 0.00e+00 for the three heads, against a tolerance of 10^{-4}, about six hundred times inside it and at float32 rounding.

The first export, however, worked only at the batch size it was traced on; other sizes failed in ONNX Runtime with "Attempting to broadcast an axis by a dimension other than 1. 2 by 4". A row index was built from `batch.batch_size`, a Python integer the tracer froze into the graph. The fix replaced it with `sequence.gather`; old and new forms agreed exactly (difference 0.0) on 20 combinations, and a regression test exports at batch 4 and checks 1, 2, 7 and 16, because the failure was loud in ONNX Runtime and invisible in PyTorch. The final export (PR #94) reads its architecture from a configuration, requires 2,379,263 parameters, and verifies the weights' SHA-256 (beginning `6a322126`) before loading, because an untrained model, a search checkpoint or a library-default model all export perfectly and mean nothing. The committed evidence has status "architecture only" with `weights_loaded` false. The model consumes two of the five history channels, so the graph has 12 inputs, not 15; parity over eight shapes was 1.79e-07, 1.49e-07 and 0.00e+00.

4.3.6 INT8 quantisation and the reversal

CP-DEPLOY accepted INT8 only if it was at least 2× smaller, lost at most 1% relative on each headline metric, and was not slower. The benchmark used dynamic INT8 quantisation of weights in ONNX Runtime [47], which uses the 8-bit affine scheme of [46] but computes activation scales at run time, so no calibration set was needed, because static quantisation needs real task examples, which carry

user identifiers and are not public; latency used one thread, five discarded warm-ups and 30 repetitions (Table 4.2).

Table 4.2: FP32 against dynamic INT8 on the prototype and on the final candidate ((*descriptive*)). Source: `docs/deployment-int8-benchmark.md` and `docs/deployment-final-export.md`.

Model	FP32 (B)	INT8 (B)	Size ratio	Median FP32 / INT8 (ms)	p95 FP32 / INT8 (ms)
Prototype (5 channels)	14,678,873	4,003,834	3.67	1.22 / 1.14	2.28 / 2.90
Candidate (20-step history, 100 candidates)	9,535,393	2,715,815	3.51	1.88 / 2.36	–

On the prototype INT8 was 3.67× smaller and 1.07× faster at the median, though slower at p95, and the documents recommended accepting it on size and speed with quality unanswered. On the real candidate the result reversed: the size ratio held (3.51) but the median rose from 1.88 ms to 2.36 ms (INT8 26% slower), so the "not slower" condition fails and FP32 is shipped (ADR-003, status Proposed, readiness GO with three conditions). Dynamic quantisation pays a per-call cost to quantise activations, and at this shape that cost exceeds what the smaller weights save. Both runs were kept side by side, because a decision on the prototype would have been wrong. The quality condition stayed unanswered, not passed: the maximum output disagreement of the quantised prototype (0.0908 for `t1_logits`, 0.0765 for `t2_logit`) bounds how far outputs move and is not a metric delta, and the T3 zero is no evidence of safety because that path is dominated by the frozen retrieval rank.

4.3.7 Continuous integration and the demonstration specification

The workflow "Quality checks" (PR #70) pins every action by commit SHA, uses `uv` 0.11.12, and runs Ruff, pytest, the environment smoke and the contract validator with read-only permissions; concurrency cancels superseded runs. (*descriptive*) The suite grew from 257 tests (PR #67) to 313, 591, 710 and 867 (PR #93). On 2026-09-11 the model branch accumulated 11 red runs from three causes: `pandas==3.0.5` added without regenerating `uv.lock`; 116 Ruff findings; and a multi-line f-string valid only on Python 3.12 or later, although 3.11.14 is pinned. The workstream posted a diagnosis on PR #79 (18:42:59Z); PR #85 added one annotation per failing step and a README section "When CI fails"; and PR #93 made the lock check the first step and added `scripts/check.py`, which runs the same six checks locally, cheapest first, and prints the fix.

PR #89 specified the data flow of a local run against what was implemented. Inference is local and makes no network call; federated training transmits model payloads in both directions and keeps session events local. No raw product identifier or user identifier is an input. Offline use is unchanged; an empty history contributes exactly zero; an unknown product is hashed like any other, so a collision is indistinguishable from a match; and a missing or corrupt model file makes ONNX Runtime refuse to start, with no fallback model and no silent degradation. The demonstration logs no raw history, emits no telemetry and writes no inputs to disk.

What we learned

Gained. A deterministic feature contract pinned by tests; measurement conventions that separate payload from traffic and absence from zero; an export with parity evidence across shapes; a CI that returns the fix; and a threat model that lists what is *not* protected. **Did not work or stayed open.** INT8 was accepted on the prototype and rejected on the real candidate, and its quality condition was never answered because the evaluation set could not be public. The leakage validator was wrong against real rows until PR #87 and is not yet called in CI. The product, brand and price channels gave no measurable T1 gain, so hashing and price bands reach the model only through the query and candidate block. The committed export evidence describes an architecture, not a trained model.

4.4 The main-study research platform

The re-plan of 2026-09-16 demoted the prototype to a preliminary experiment, and the main study was adopted on 2026-09-26 with a "research and engineering company" model and five sequential gates, beginning with the data census gate (G0). The main study ran until 2026-10-11 under a single study lead working with AI agents. From the prototype it kept the retention ratio against a centralised reference, the discipline of seeds and run identities, FedAvg and FedAdam with τ recorded as a control, the sealed-TEST discipline, and the ONNX opset 17 and dynamic INT8 choices; it did not keep the 588-category task, the 50,000 buckets, the pilot retentions or the October-only data.

4.4.1 Packages and frozen code trees

Seven packages were ported to the public repository on 2026-10-03 (PRs #96–#102, code only); Table 4.3 lists them. The server ran *frozen code trees*, each deployed once and made read-only, and every run records its tree: `code_p3` for the contest runs, `code_ds` for the small-model tuning stages, `code_ds3r2` to `code_ds3r6` for the A40 receipts, PRE, C_FULL, the warm MAIN line and the DP chains, `code_ds3r7` for the endgame (evaluator digest beginning `61b64e61`), and `code_p2r1` for the follow-up analysis. The deploy pattern was: freeze local code with all suites exiting 0 and the negative controls failing when their guard is removed; pack with a script that asserts the reviewed hash of every changed file; extract into a *new* directory beside the old one; verify every file against the SYNC_MANIFEST; make the tree read-only; run targeted server tests; and compare the digests of a tiny run of seven federated arms, which were identical from tree `r3` to `r7b` (DP `08383b6f`, FA `807eadfb`, FA_1024 `878fc357`, FA_Q8 `651f0eb0`, FP `825b27f8`, PF `991bd6a4`, S_CAL `ff1b5150`).

4.4.2 The evaluation harness

The harness ranks all 158,486 products for each decision, because sampled candidate lists can change the order of models [6]. The primary metric is the client-macro MRR@20, and retention is 100× an arm's score over a named reference. The TEST set has 26,254 users and 229,742 decisions, and intervals come

Table 4.3: Main-study packages ((*descriptive*)). "Lines" were added when the package was ported on 2026-10-03; "Tests" are test functions at `origin/main` (negative controls in brackets), counted with `gitgrep`, not a test run. Source: the public-release reader report.

Package	Role	Lines	Tests
`fedsim`	federated simulator: FedAvg, FedAdam, sampling, DP, 8-bit uploads	12,368	344 (72)
`seqrec`, `central`	SASRec models; central trainer for the references	2,767; 802	53 (13); 18 (4)
`fullrank`, `evaluation`	full-catalogue scorer; evaluation harness and contrast statistics	3,219; –	–; 117 (21)
`ondevice`, `deployment`	ONNX and WASM path; prototype package, unchanged	3,025; –	74 (12); 42
`benchmarks`, `benchrun`	benchmark definitions; benchmark runner	927; 3,214	16; 56 (7)
`privacy`	secure-aggregation simulation and membership inference	–	12

from a paired user bootstrap [50] with a Holm family [51] and TOST equivalence tests [52]. The realistic edge system FA_WARM_FROZEN retains 88.49% [87.73, 89.18] of C_FULL (contrast K20b, (*confirmatory*)). The harness treats the denominator as part of a number's definition: both seeds of K20b (88.59 and 88.39) are scored against the seed-2026 C_FULL of 0.180388. The retention report was fixed twice as a separate tool (digest `d0247370` to `9a3b2c2a`), not as an edit to a frozen tree.

4.4.3 The federated engine: FedAdam and the DP accountant

The simulator implements the server loop of Figure 3.4. In the FedAdam step [25] the aggregated pseudo-gradient feeds moment estimates m_t and v_t and the model moves by $\eta\, m_t/(\sqrt{v_t}+\tau)$, so τ sets a floor under the denominator. The registered $\tau = 10^{-3}$ met client pseudo-gradients of about 10^{-7}, so the server step was 20–100× too small ((*exploratory*)); the engine's remedy was a validated parameter (Section 4.8), not an explanation of any gain, because the re-runs changed several ingredients at once. DP arms clip each client update, add noise and sample clients by Poisson sampling, and an accountant reports the resulting ε; the standard analysis of such aggregation is Rényi accounting [35, 43–45].

4.4.4 Privacy tooling and the Edge device bench

The secure-aggregation arm simulates masked aggregation of uploads [36, 37]. It is a *simulation only*: it checks that the masked sum is bit-identical to the plain sum and measures upload overhead (Figure 5.14); only the identity control inside the update-leakage audit is limited to 4,608 coordinates, and it is not production cryptography. The membership-inference audit tests whether an attacker can tell members from non-members [38] and reports an AUC (Figure 5.13); per-user scores are kept out of the public tables.

The update-leakage audit was built in the technical closure. It asks whether one client's upload reveals the items the client viewed, and whether a 64-client aggregate, as secure aggregation would produce, removes the signal. It runs on CALIB, the 32,768-client calibration subset of MAIN, replays rounds 0 to 49 of a bound recipe, and uses 50 cohorts and 512 predeclared targets; items can be read only from the input item

table, because the output embedding is dense. (*exploratory*) A single upload identifies the viewed item set with an F1 of 1.0 against 0.0564 for a popularity baseline. The aggregate-input attack advantage is about 2.625 percentage points (95% CI 2.312–2.953), above the fixed 2 pp threshold, so the update-leakage audit gives a *negative* finding: secure aggregation is needed but not shown to be sufficient. A category probe found no material advantage, which is not proof of privacy, and the frozen-item variant is not applicable because frozen tables upload zero deltas.

The `ondevice` package continues the prototype choices: opset 17, dynamic INT8, a fixed batch of 1 and a byte-identical re-export, run in a browser WebAssembly runtime [47, 49]. The device model is the SASRec of the federated arms with $d = 64$ and 2 blocks (20.7M parameters) [2]; the cloud reference has $d = 256$ and 3 blocks (83.9M). The bench times FP32 against INT8 on a laptop, a phone and a server (Figure 5.15), with device models generalised in the public table. The GRU export adapter was stopped under the condition "build only if GRU wins G2".

Limitations

(1) **No formal DP guarantee.** The legacy DP implementation used a disclosed public seed, so its noise and Poisson streams can be reconstructed; the utility numbers remain measurements, but those runs do not establish the randomised-DP guarantee. (2) Secure aggregation is a simulation. (3) The canary design has a ceiling of $\varepsilon \approx 5.213$, below 8, and cannot reject $\varepsilon = 8$. (4) L6 latency is single-thread CPU latency on a shared server, not a laptop or a phone, and the GRU against `C_SMALL` comparison is a CPU rehearsal. (5) (*stopped*) DP-FTRL and the canary restart were stopped by the study lead, and the ColdDP calibrations failed before training. (6) No TEST2 (December 2019) data exist.

4.5 Governance tooling

The main study treated governance as software, because a sealed TEST set can be read only once. The *decision log* is append-only: every action is a new row and old rows are never edited; it reached row 603 by 2026-10-13T19:53:30Z. The *contracts* hold the pre-registration [7], amendments 06A to 06P and 07, and the second registration (PREG2). *Lock files* and *registers* live on the server and must not be written by teammates or their agents. The *TEST exposure register* holds 19 entries with 0 errors (SHA-256 beginning `dfdcbaea`): entries 10 and 11 are the confirmatory test evaluation (2026-10-05T21:03Z to 2026-10-07T10:09Z), entries 12 to 19 are the exploratory looks of the main study, and the closure-phase extension of the register added entries 20 to 23 in new register mirrors, leaving the 19-entry register untouched.

Reading TEST data requires a fixed sequence: a new lock; an independent review; a pre-scoring registration entry, which fixes the exact code and inputs before anything is scored; one scoring through the gated wrapper; a log row with hashes and no scores; an inventory of the protected trees before and after with an empty difference; a post-scoring disclosure entry, which records that the single scoring happened; and an independent post-review. No score is read before the post-scoring disclosure entry. Every exploratory

number is labelled (*exploratory*) and shown beside the confirmatory one, and the labels of this book ((*confirmatory*), (*confirmatory, follow-up*), (*exploratory*), (*validation*), (*descriptive*)) mirror the labels stored with the results. A server README of eight rules extended these habits to teammates and their agents: no stopping or renicing of listed processes, no writes to run, register, lock or code trees, no TEST read outside the gate, no tokens or passwords in any artefact, no new GPU job while listed jobs run, and pushes to GitHub only from the study lead's branch. The independent reviews in this chapter are AI reviews and not expert human reviews (see Chapter 3).

4.6 Infrastructure and operations (Lobot)

All main-study training and scoring ran on *Lobot*, Queen's shared JupyterHub group server `group-digi2026-g8`. (*descriptive*) It offers no SSH or scp, and every teammate works as the same Linux user, so home-directory contents do not identify a person and nobody touches another teammate's directories or processes. The hardware is two NVIDIA A40 GPUs (46 GB each), 128 visible CPUs, a memory limit of 384 GiB and a 345 GB disk (about 268 GB free on 2026-10-03), with PyTorch 2.13.0+cu130; every job uses one prebuilt Python environment into which nobody installs packages, and the frozen dataset is read-only.

The study lead drove the server from Windows through a small client that starts a Jupyter kernel over the REST API, runs one command or script over the kernel websocket and deletes the kernel. Each teammate needs a personal token, and a token is never shared, printed, committed or copied to the server. The recorded traps are practical: Git Bash rewrites Linux paths, Windows line ends must be stripped on the server, long jobs must be launched detached (`nohupsetsid`) because a kernel call dies when its websocket closes, and an inline `pkill` once matched the executing shell itself. In the closing phase the set-up of the team's own access to the cluster was completed.

Work runs in *job queues*: detached bash scripts that execute registered jobs in order, may wait for a file, log to a queue log and write a `LANE_STOP` marker on failure or refusal. A queue never decides a value. Monitoring is value-free: checks read status and round fields, never metrics, together with `nvidia-smi`, process lists, counts of run summaries and free disk. A running queue file is never rewritten; to change a plan the queue is withdrawn, the withdrawal is recorded, and a new queue starts. The run directories separate the contest runs, C_SMALL, PRE, the deployment study (C_FULL, every MAIN federated run, device runs, baselines, ONNX and analysis freezes), the follow-up analysis, the benchmark waves and the immutable code trees.

4.7 Verification and testing

The main-study suites were audited in a technical closure on 2026-10-12 and 2026-10-13. It worked only on a copy, never on the original workspace, under rules that forbade edits to frozen trees, run trees, TEST outputs, locks and register history. Table 4.4 gives the counts. They are engineering evidence, not

statistical results, and no confirmatory number was produced or changed.

Table 4.4: Technical-closure verification counts ((*descriptive*)). NC is an expected negative control, a test that must fail when its guard is removed. Source: `DELIVERY_REVIEW/evidence/` and `EXECUTION_FINAL_STATUS.json`.

Check	Result
Registered-artefact audit	44 checks PASS, repeated after restoration: 44 PASS
Protected tree, first closure	69,381 file entries before and after, none added, removed or changed (metadata comparison, not a rehash of 125 GB)
Protected archive before optional TEST scoring	78,413 entries (SHA-256 beginning `b13c8d8a`); empty difference after all steps
Suites FL, ANA, INT, update-norm analysis	283 + 79 NC; 249 + 91 NC; 473 + 158 NC; 12 PASS
Total after first closure	1,017 PASS and 328 expected NC, 0 unresolved
Total at final closure	1,087 PASS, 328 expected NC, 1,415 scoped cases, 0 unresolved (named suites only)
Python files parsed under 3.11.14	676, then 688, then 710
Source against the handover	604 to 612 Python files: 597 unchanged, 7 modified, 8 new, 0 removed; 27 server-only support files

Three incidents belong to the record. The first test run on the frozen copy lacked support contracts, so the closure assembled an unfrozen work root with 27 server-only support files and altered nothing in the frozen copy. Second, an operator error: legacy tests write receipts beside themselves, and five receipt files were written into `code_x1`; each was restored from its source after a SHA-256 equality check, keeping inode and modification time, and no confirmatory or TEST result, lock or register changed. Third, a nested f-string valid only on Python 3.12 broke test collection in one module under the pinned 3.11, the same class of defect that had turned the prototype CI red, and the closure then parsed all 676 Python files under Python 3.11.14. Linux digests were reproduced bit-identically from the unchanged historical tree `code_p2r4` (all seven cold toy server digests and four warm digest pairs), and the Windows constants were kept.

The closure also added the missing summary of the update-norm logs ((*descriptive*)): 29 MAIN and warm-start runs, 100,942 rounds, no non-finite step, no training and no TEST. For the registered tuned warm-start federated model on MAIN, seed 2026, the median RMS of the server step is 7.307×10^{-8} and the final relative drift of the trained parameters is 0.06003%; for seed 2027 it is 0.05985%. The logged norm is a group L2 norm, not an individual update, so the fraction of coordinates below τ can only be bounded from below, and the frozen item tables are excluded from the drift denominator.

What we learned

Gained. A read-only proof that the registered study was intact before further work, a documented operator error with a clean restoration, platform-independent digests, and the update-norm evidence that the registered server step barely moved the model. **Open.** The counts are not uniform across documents (1,017 against 1,087 passes): both are correct for their scope, and we do not break down the 70 additional passes beyond the update-leakage and optional-gate suites added later. The 710 parsed files exceed the 612 files of the canonical tree, and the sources we used do not reconcile them.

4.8 Delivery and consolidation

4.8.1 The merged folder of 2026-10-17

On 2026-10-17 the closure phase produced the delivery entry documents (the guide `START_HERE_DELIVERY.md`, the technical backlog, the closure status and a delivery manifest), and the final consolidation merged the main-study research, the technical closure, the GitHub repository at commit `89b61f2` and the dataset into one merged folder, leaving the originals unmodified. (*descriptive*) Of 338,858 files, 338,857 were verified, so one file is not verified. Files are hardlinked rather than copied, so the folder adds no second copy of their bytes, and a provenance manifest records where each part came from. A curated integrity manifest and a read-only verifier accompany the delivery, and where an older guide differs from the entry guide, the entry guide wins. The merged folder does not hold the GitHub-only text (issues, pull requests, CI runs), which was exported separately for this book.

4.8.2 The public release, PRs #103–#114

On 2026-10-10, the day before the study lead left the project, the team made the public repository the scrubbed public record of the main study. (*descriptive*) Twelve pull requests (#103–#114) were opened; 11 merged within 5 to 10 minutes of creation (for example #103 in 5 min 54 s and #107 in 6 min 34 s) and one (#113) was closed unmerged by the study lead's decision. All ran the single check "Quality checks". From `43fbbc1` to the head `89b61f2` the diff is 60 files, +3,829 and −24 lines, and the repository grew from 99 commits at `bfc90a0` to 188. A trial merge of six branches gave 0 conflicts and 707 passing tests. PR #103 validated the parameter `fl.server_tau` (finite and above 0, 4 unit tests, `docs/fedadam-tau.md`), keeping the default 10^{-3} for reproducibility and recommending 10^{-6}. PRs #104 and #105 added 9 CSV tables each (18 in all) with READMEs, #106 added the results report (+546/−1) and a notebook with three figures, and #108–#113 came from a gap analysis. Per-user membership-inference scores, exact device models, checkpoints and tokens were deliberately excluded.

Before the tables stayed public, a read-only QA pass checked them line by line against the sources: all confirmatory numbers matched, and the label and definition errors it found became PR #107 (+50/−36, 7 files; no confirmatory number changed). These included a renamed column `pre_on_fl_users`, the ML-1M C_FULL of the capacity table corrected from 0.135347 to the three-seed mean 0.1361 (0.136110 in the CSV; (*validation*)), a cold non-private DP reference sourced as the two-seed mean 0.13064, descriptive and exploratory labels on the audit, device and post-test-look results, the removal of an unmeasured sentence on the cause of the cold-line DP cost, and the stated retention reference (seed-2026 C_FULL 0.1804).

Limitations

The public record has gaps that this book corrects rather than inherits. (1) The 12 pull requests have 0 reviews and 0 comments on GitHub; the independent control was the server QA pass, itself an AI-run

check, and this book cites that pass and not GitHub review. (2) The QA pass missed items. The public sentence that K20b "averages the two seeds, each against its own C_FULL" is wrong: both seeds use the seed-2026 reference, and each seed's own reference (0.179743 for seed 2027) would give 88.71 and a mean of 88.65, not 88.49% ((*confirmatory*)). K12, the DP arm at $\varepsilon = 8$ with noise only, is 98.52% [97.70, 99.38] ((*confirmatory*)); the server flagged it as seed-sensitive (spread 4.40, user-and-seed interval [94.11, 102.92]) and the public CSV dropped the flag, and since the legacy DP runs carry no formal DP guarantee (Section 4.4) the number is a utility measurement. (3) The REES46 launchers and data build are not public, and `docs/data-access.md` describes the prototype files, so the repository documents the REES46 results but does not regenerate them. (4) The credential policy changed within 40 minutes on 2026-10-10: first the pull requests were blocked because the operator would not use the study lead's token, then, on the study lead's instruction, they were opened through the REST API with the stored git credential, read in process and never printed; later rules forbid tokens in every artefact. (5) The pull-request texts mention no AI assistance, by a rule in the decision log, and PR #114 removed the name of an AI tool from the public documentation; the disclosure is made in Chapter 3 and in the appendices instead.

Chapter 5: Results and Evaluation

This chapter reports every experiment of the project in the order in which it was run, because each result shaped the next decision. The first half follows the path from the first prototype to the sealed confirmatory test: the prototype stage, the pilot studies, the foundations of the main study, the backbone contest, the calibration of the federated recipe and the single confirmatory test evaluation. The second half reports what we learned after that test evaluation: why federated training added so little, what happens when the recipe is corrected, how much federation is worth when the server has less data, what the privacy audits found, whether the models run on real devices, and what the closure experiments added.

Every number carries the label of the analysis that produced it. (*confirmatory*) and (*confirmatory, follow-up*) results come from the single pre-registered test evaluation and are quoted exactly as printed in the result files. (*exploratory*) results were designed after the confirmatory results were seen; they explain those results but do not replace them. (*validation*) results were read on validation or calibration data, and (*descriptive*) results characterise the measured setup without a registered claim. Each experiment section closes with a short "What we learned" box that records what the experiment gained and what it did not.

5.1 Overview of the evaluation

The experiments of this project were not designed in one go. Fourteen phases of work, from the first prototype in August 2026 to the closure experiments in mid-October, produced the evidence that the rest of this thesis interprets, and each later phase was designed because of what an earlier one had shown. This section gives the map: the three evaluation regimes of the chapter and why their numbers must not be compared, the vocabulary used in every table, and the fourteen phases against the sections that report them. The sections of this part cover the foundations: the prototype stage, the centralised pilot studies, the census and feature work of the main pre-registered study, and the contest that chose the model family.

The first regime is the *prototype stage*, a sprint that built a complete pipeline on one month of data and three prediction tasks. The second is the group of *pilot studies*, in which a centralised next-product model was built on a frozen two-month dataset before the main study existed. The third is the *main study*, which carries the confirmatory evidence of the thesis. The regimes differ in task, catalogue, data window and evaluation split (Table 5.1), so a number measured in one has no counterpart in another. We never place a prototype or pilot-study number beside a main-study number.

Four quantities recur throughout the main study and are defined once here.

Quantities used throughout the chapter

- **Client-macro MRR@20.** For each user, the reciprocal rank of the true next product if it lies within

Table 5.1: The three evaluation regimes of this chapter. Sources: the prototype repository documentation, the pilot-study dataset record `splits.json`, and the main-study data handover (`02_DATA.md`). (*descriptive*)

Aspect	Prototype stage	Pilot studies	Main study
Task	next category (T1, 588 categories), purchase (T2), rerank (T3)	`Next_Item`: next product over the full catalogue	`Next_Item`, same definition
Data	REES46, October 2019	REES46, October and November 2019	same frozen dataset; matched users with 5 to 100 eligible TRAIN examples
Catalogue	588 categories	158,486 products	158,486 products
Selection split	VALIDATION 2019-10-22 to 10-27, 25% slice of cohort C1	full VS: 8,826,488 decisions, 790,459 users	`MAIN_VALIDATION`: 40,764 users, 519,714 decisions
Unbiased split	TEST sealed, not scored	TEST scored once (GRU), retrospectively (SASRec)	TEST of the confirmatory test evaluation: 26,254 users, 229,742 decisions
Main metric	MRR@20 on the category-change slice; PR-AUC; NDCG@5	client-macro MRR@20	client-macro MRR@20

the top 20 of the full catalogue, and 0 otherwise; the mean over users, so that each user weighs equally. Out-of-vocabulary targets count as misses (the end-to-end denominator).

- **Retention.** $R(x/y) = 100 \times \text{metric}(x)/\text{metric}(y)$, computed per seed and then averaged. Intervals are 95% intervals of a paired user bootstrap with 1,000 resamples [50]. They are conditional on the trained models and do not measure variation across training seeds.
- **Budget, PB and CB.** The training budget is counted in passes over the training data, one pass being one exposure of every eligible TRAIN example of the cohort; every main-study arm trains for 6.0 passes and is evaluated at twelve marks, one every 0.5 passes. PRACTICAL_BEST (PB) is the best of the twelve marks (a new best must exceed the previous one by more than 0.0002) and is optimistic on validation data. CONTROLLED_BUDGET (CB) is the fixed 6.0-pass endpoint.
- **Populations.** MAIN is the main population of 131,072 users; CALIB is a nested 32,768-user subset used only for tuning; the PRE band holds 1,217,610 further users assumed to consent to server training; MAIN and the PRE band together, 1,348,682 users, form the population of the cloud reference C_FULL. TEST is the held-out window of MAIN. "MAIN" never means TEST.

The roadmap in Table 5.2 lists the fourteen phases in the order in which they were run, with the question each one asked and the labels of the results it produced. A phase that produced no result of its own, such as the engineering handover of phase 12, still appears, because it shaped what the following phases could do.

The first four phases contain no confirmatory TEST number: the single sealed look at TEST belongs to phase 10, and everything before it was selection, tuning or design. The one exception in kind is the backbone decision of phase 4, a registered rule read once, but on validation data. The sequence is also a chain of consequences rather than a plan executed as written: the prototype of phase 1 was demoted by the re-plan that produced phase 2, and the findings of phases 2 and 3 fixed the design of the contest in phase 4.

Reading rules for the rest of the chapter

Table 5.2: Roadmap of the fourteen phases of the evaluation. Periods are UTC dates in 2026. Source: `EXPERIMENT_TIMELINE.md`, Part 1, and the 14 phase files. (*descriptive*)

#	Period	Question and outcome	Reported in	Labels
1	05-24; 08-11 to 09-12	Proposal; a working end-to-end prototype on one dataset	Section 5.2	(*validation*) (*descriptive*)
2	09-16 to 09-26	A trustworthy centralised baseline; first GRU and SASRec campaigns; feature study	Section 5.3	(*validation*) (*exploratory*) (*descriptive*)
3	09-26 to 09-27	Main-study operating model, census, population, catalogue, input features	Section 5.4	(*descriptive*) (*validation*)
4	09-27 to 09-28	Backbone contest, GRU against SASRec; architecture lock	Section 5.5	(*confirmatory*) (*validation*) (*descriptive*)
5	09-28 to 10-01	Calibration of the federated recipe; cloud references and MAIN runs	Section 5.6	(*validation*) (*exploratory*)
6	10-01	Benchmark design and the cold-start chain	Section 5.7	(*descriptive*)
7	10-01 to 10-03	MAIN completion, the single VALIDATION read, design of the follow-up analysis	Section 5.8	(*validation*) (*exploratory*)
8	10-03 to 10-04	Build and review of the follow-up analysis; first handover; port to GitHub	next sections	(*descriptive*)
9	10-04 to 10-05	Follow-up analysis runs; device measurements; membership-inference audit; server-data band	next sections	(*validation*) (*descriptive*)
10	10-05 to 10-07	The confirmatory test evaluation; benchmark TEST looks	confirmatory sections	(*confirmatory*) (*confirmatory, follow-up*)
11	10-06 to 10-10	The FedAdam τ finding; corrected recipe; exploratory TEST looks; privacy redo	Section 5.11 and the following sections	(*exploratory*)
12	10-10 to 10-11	Final handover and public release	exploratory sections	(*descriptive*)
13	10-12 to 10-13	Technical closure: verification, update norms, the update-leakage audit, further exploratory looks	exploratory sections	(*exploratory*) (*descriptive*)
14	10-13 to 10-17	Extensions and project closure	exploratory sections	(*exploratory*) (*descriptive*)

- A number is meaningful only with its split, metric, checkpoint rule (PB or CB) and evidence label; numbers from different regimes are never compared.
- Validation values read at PB are the maximum of twelve marks and are optimistic; the only unbiased numbers are those of the single confirmatory test evaluation.
- We report what failed beside what worked, because the failures fixed the design of the later phases.

5.2 The prototype stage on GitHub

The proposal that opened the project asked one question: how much shopping-recommendation quality is retained when models move from the cloud to the device or to federated training (Chapter 1). Before committing to a larger study we wanted to know where the real difficulties lay, so the three team members built a complete pipeline in a sprint on a public GitHub repository. The sprint was planned for two weeks, ran from 2026-08-11 to 2026-09-12, about 32 days, and merged 39 pull requests and 99 commits. The work ran in three parallel workstreams: data and models, federated training with the pilots, and system, deployment and security. We call the sprint the *prototype stage*.

The plan had three tasks and six training regimes on one shared model. Task T1 predicts the category of the next different item viewed, T2 predicts whether the first viewed item of a session is bought later in the same session, and T3 reranks a fixed candidate list. The regimes were R1 centralised, R2A FedAvg [3], R2B FedAdam [25], R3 personalised federated learning, R4 strict local training, and R5 centralised pretraining followed by local adaptation. What was delivered is a scoped subset: pilots of T1 and T2 that compare R1 with R2A at one seed. R2B, R3, R4 and R5 were never run, eight of the 56 planned issues stayed open, and neither differential privacy nor secure aggregation was attempted. Every model-quality number in this section is a (*validation*) number on the window 2019-10-22 to 10-27, which was also used for model selection and is therefore not held out; counts and system measurements are (*descriptive*). The TEST window was sealed and not used for selection.[1]

5.2.1 Data audit, cohort and task headroom

A model result is only as good as the data and splits beneath it, so the first deliverable of the data workstream was an audit (Table 5.3). The raw REES46 file [8] was converted to Parquet without rejecting a row, time was split into half-open UTC windows, and the client cohort was defined from the TRAIN window alone, so that no future information decides who is eligible. The cohort rule, at least 10 events, 2 sessions and 3 active days, selects 388,789 clients: 16.81% of the TRAIN users but 60.21% of the TRAIN events. The cohort holds 17.6 million TRAIN events, which did not fit in memory, so all modelling used a deterministic 25% slice of complete clients. A data-freeze gate (G1) closed the data workstream; its barrier asserts that the example builders reproduce the frozen validation counts before any TEST row is read, and

[1] "Sealed" is not "never opened": a measured audit found that one validation load decoded 384,203 TEST rows from one Parquet row group, none of them used, and that the public smoke fixture holds 3,321 rows in the TEST window.

it stopped the first run because the T3 builder produced 719,983 examples against 1,096,774 frozen ones (an older eligibility rule still lived in one file). After the fix the gate returned GO on nine of nine criteria.

Table 5.3: Prototype data audit, temporal split and cohort (REES46, October 2019). Source: `s1-ds-04-data-audit.md`, `s1-ds-05-06-cohort-temporal-protocol.md`, `s1-ds-09-freeze-and-g1.md`. (*descriptive*)

Quantity	Value
Raw file	42,448,764 events, 3,022,290 users, 9,244,772 sessions, 166,794 products; view 96.07%, cart 2.18%, purchase 1.75%
Users' activity	median 4 events per user; 22.9% of users have one event; top 10% of clients own 68.15% of VALIDATION events
Event split	TRAIN 29,218,702 (68.83%); VALIDATION 6,897,095; TEST 5,087,488; label-observation day 1,245,479; 4,995 boundary-crossing sessions (0.054%) excluded
Drift	cart share 2.240% in TRAIN, 1.579% in TEST
Cohort C1 and slice	388,789 clients; 25% slice 97,279 clients with 3,113,814 TRAIN and 438,185 VALIDATION decisions (31,576 clients)

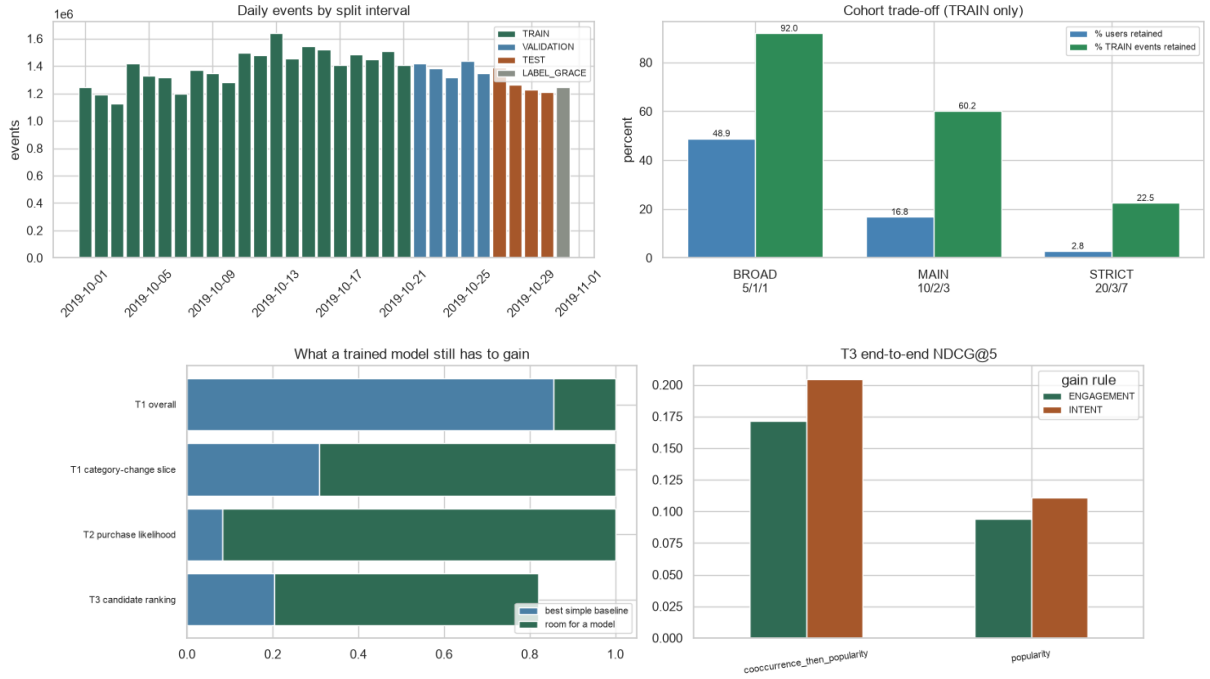


Figure 5.1: Output of the prototype's data notebooks (`S1_DS_05_06_Cohort_Temporal_Protocol.ipynb` and `S1_D1_DS_07_T3_Protocol_and_Task_Examples.ipynb`). Top left: daily events of October 2019, coloured by the interval of the temporal split. Top right: the share of users and of TRAIN events kept by three candidate cohort rules, computed on TRAIN only. Bottom left: the share of each task that the best simple baseline already solves, and the room left for a learned model. Bottom right: end-to-end NDCG@5 of two retrieval orders for T3 under the two gain rules. (*descriptive*)

Before training anything we measured how much room a learned model has above trivial rules (Table 5.4). For T1 the result was sobering: 82.32% of next-category transitions stay in the same category, so repeating the current category already scores MRR@20 of 0.8232 and any overall metric is saturated. We therefore fixed, before any comparison number existed (evaluation protocol of 2026-09-01), that the T1 headline is the macro MRR@20 on the category-change slice, the 77,457 decisions from 16,096 clients in which the category does change. A table of first-order category transitions fitted on TRAIN scores 0.3143 there, and this is the bar a model must clear. Metrics were computed exhaustively over all 588 categories, because

sampled metrics can change the ranking of models [6]. A second protocol decision proved important later: the model workstream scored with a convention that suppresses the current category from the ranking, the frozen harness does not, and on one checkpoint the two differ by a factor of about 1.6 (baseline 0.3143 against 0.1892, model 0.3479 against 0.2174). We adopted the rule of quoting baseline and model from the same harness run. Figure 5.1 reproduces the audit notebooks' own plots of the split, the cohort rules and the task headroom.

Table 5.4: Model-free references on the VALIDATION window (25% slice). Source: `HANDOFF_TO_LANES.md`, `FINDINGS_FOR_S1_LANE.md`, `t3-final-component-001/summary.md`. (*validation*)

Task	Reference	Value	Metric and population
T1	repeat the current category	0.8232	overall MRR@20, 438,185 decisions
T1	transition table; globally popular category	0.8560; 0.2841	overall MRR@20, same decisions
T1	transition table, category-change slice	0.3143	macro MRR@20 (micro 0.3085), 77,457 decisions
T2	prevalence; item popularity; item and category	0.0288; 0.0657; 0.0760	PR-AUC, 392,554 decisions, 11,297 positives
T3	frozen retrieval order (100 candidates)	0.2707	macro NDCG@5 (micro 0.2046), 18,814 queries

5.2.2 The GRU model workstream and the architecture comparison

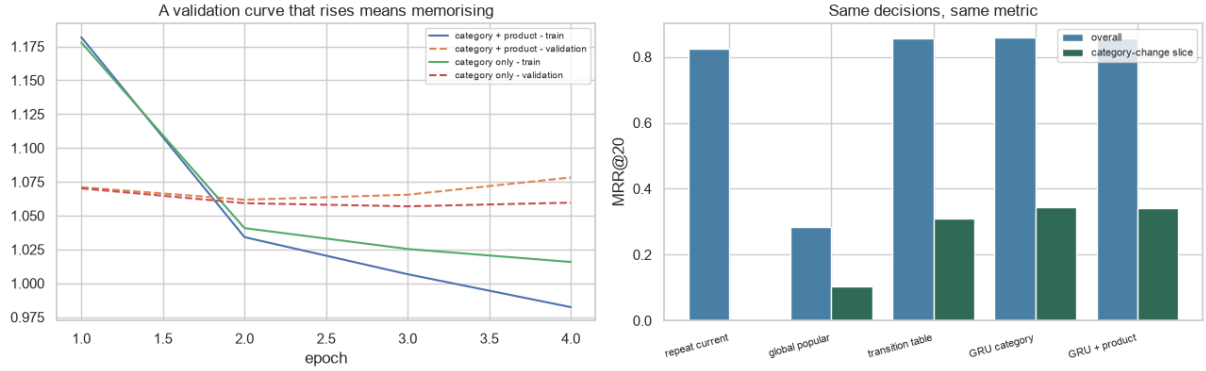


Figure 5.2: The first GRU run on real REES46 sessions, as plotted by the notebook `S2_SMOKE_GRU_On_Real_Data.ipynb`. Left: training and validation loss over four epochs for the model with and without product inputs; the validation loss of the product variant rises after epoch 2 while its training loss keeps falling, which is memorisation. Right: MRR@20 of three simple references and the two GRU variants on the same decisions, overall and on the category-change slice. (*validation*)

The model workstream built one shared session encoder with three heads: a one-layer GRU [1] with hidden size 128 over the last 20 events of the session, fed by the category, the event type and a capped log time gap, $\log(1 + \min(\text{gap}, 86400))$. With the T1 head it reaches a slice macro MRR@20 of 0.3474 over three seeds (0.3479, 0.3472, 0.3472) against 0.3143 for the transition table, a gain of +0.0331 that is about 47 times the seed spread of 0.0007 (*validation*). The gain is flat across clients with 10–19, 20–49, 50–99 and 100 or more TRAIN events (+0.0331, +0.0346, +0.0308, +0.0357), which matters because learning on the device is interesting only if the benefit does not sit with the heaviest users. A series of single changes found nothing beyond the run-to-run spread of 0.0033: hashed product identity −0.0006, brand +0.0008, price band +0.0015, a second layer −0.0020. Product identity entered through a hash [53]

into 50,000 buckets that forces 68,831 of the 113,765 catalogue items (60.5%) into an already occupied bucket (Table 5.6), so that null result is weak evidence. Figure 5.2 shows the first run of this model on real sessions.

We also compared four sequence cores under one frozen protocol (same data, split, evaluator, budget and seed; Table 5.5). The GRU run reproduced the 0.3479 of the model workstream as 0.3477, which was the gate for the whole comparison. The GRU and the LSTM are tied, since 0.0006 lies inside the seed spread, and the LSTM is 27% slower; the Transformer [14] and the temporal convolutional network trail by about 5 and 11 seed spreads. The comparison favours the GRU, and we said so: the budget was the GRU's own, so the claim we allowed ourselves is that, under the GRU's budget, the GRU is not worse than the alternatives, not that it is the best architecture. The same question returns, with a controlled design, in Sections 5.3 and 5.5.

Table 5.5: Architecture comparison under the GRU's own training budget (seed 13, T1 slice macro MRR@20). Source: `MVP_REPORT_AR.md`, `scripts/t5` summary. (*validation*)

Core	Slice macro MRR@20	Difference to GRU	Minutes	Parameters
GRU	0.3477	—	27.5	2,379,263
LSTM	0.3471	−0.0006	34.8	2,412,287
Transformer	0.3440	−0.0037	46.0	2,420,863
Temporal convolutional network	0.3398	−0.0079	28.9	2,329,727

5.2.3 The T2 label defect and its correction

The T2 head's first published result, a PR-AUC of 0.1797, looked strong. An independent review of our own results then found a defect in the upstream label rule. The rule said that a decision without an observable horizon is censored and never negative; the code instead censored every negative that was the last event of its session. Because sessions crossing a window boundary had already been excluded, every remaining session is complete inside its window, so the correct number of censored decisions is zero. The defect withheld 549,185 decisions (478,718 from TRAIN and 70,467 from VALIDATION), all of them negatives, so the published base rate was 0.0351 instead of 0.0288. Retraining on the restored labels moved the corrected PR-AUC by only +0.0013 (0.1411 to 0.1424): the model had always been this good and the number had been wrong. On the corrected labels the fine-tuned encoder scores 0.1424 against 0.1104 for a frozen encoder with a head and 0.0760 for the best simple rule, and a paired client-cluster bootstrap on 42,312 clients gives a gain over that rule whose 95% interval, $[+0.0582, +0.0752]$, lies wholly above zero (*validation*). The sealed TEST file for T2 still carries the mask on 47,636 rows; we disclose it rather than repair it, since TEST was never scored.

5.2.4 Negative transfer and the joint loss

Because the architecture shares one encoder, we tested whether a head can be added after the encoder was trained for T1. It cannot. Fine-tuning the encoder sequentially for T2 lowered the T1 slice macro

MRR@20 from 0.3479 to 0.1791 and the overall micro MRR@20 from 0.8604 to 0.4728, below the model-free references of 0.3143 and 0.8232 (*validation*). With the encoder frozen the T1 score did not move, so the drop is a property of the encoder and not of the evaluator. The remedy was a joint loss with the T2 weight $\lambda \in \{0.1, 0.3, 1.0\}$ and the selection rule fixed before the run: maximise T2 PR-AUC subject to a T1 slice macro within 0.003 of 0.3479. No weight broke the constraint (T1 0.3457, 0.3471, 0.3467 and T2 PR-AUC 0.1197, 0.1250, 0.1337 for $\lambda = 0.1, 0.3, 1.0$), and $\lambda = 1.0$ kept T1 within 0.0012 of the reference while raising T2 to 0.1337, which is 87% of the gain over the best simple rule that the destructive sequential route (0.1424) reached. A separate T2 model reached 0.1430, so sharing costs T2 0.0093 in exchange for half the parameters (2,379,263 against 4,758,526). The best weight sits at the edge of the grid, so we did not find the breaking point; widening the grid after seeing the result would have been tuning until the number pleased us, and we declined.

5.2.5 T3 learned heads and the miss decomposition

For T3 the model reranks a candidate list built by a frozen retrieval order, with gains of 2 for a purchase, 1 for a cart and 0 for a view. The reranker is residual: before training it reproduces the retrieval order exactly (macro NDCG@5 0.270687 for both), so any later departure is measured. No learned loss beat the frozen order (*validation*): the listwise reranker reaches 0.2505 (gain −0.0202, 95% interval $[-0.0265, -0.0142]$), the pointwise one 0.2186 and a query-aware variant about 0.259, against 0.2707, so the shipped T3 component is the model-free retrieval order. A first verdict had been withdrawn after review for three defects: the model could not see the query item, an error in the ideal-ranking denominator (0.2056 against the correct 0.2046) hid behind a tolerance five times too wide, and the candidate lists had been built using VALIDATION outcomes. Rebuilding the lists from TRAIN anchors (47,948 to 107,484 anchors) left the baseline unchanged.

The headroom is nevertheless large. An ideal ordering of the same lists scores 0.8782, and of the 18,814 queries 3,207 (17.05%) are lost by retrieval, 9,835 (52.27%) have the purchased item in the list but outside the top five, and 5,772 (30.68%) already have it in the top five. More than half of the queries fail at the ordering step where learning should help, yet our models took nothing from that room. We excluded three explanations by measurement: incomplete lists (13% more data moved the score by about 0.001), an unlucky seed (three seeds, spread 0.0009) and the need to rediscover the order (a frozen, zero-initialised residual).

5.2.6 Federated plumbing and system measurements

Federated training ran on the Flower framework [20] with FedAvg [3], under experiment contracts (13 schemas, immutable terminal attempts, three fixed seeds) that made each run reproducible by identifier. A real-data smoke run of three rounds with four clients reproduced an independent weighted average of the client updates to within 2.384×10^{-7}, and a scale profile with 1,000 to 5,000 simulated clients showed that a five-fold larger population adds 23% to the round time (Table 5.6). These are plumbing checks and

carry no scientific claim (*descriptive*). The most instructive result of the system workstream is a reversal. On the five-channel prototype, dynamic INT8 quantisation (8-bit affine scheme of [46], run through [47]) made the exported model 3.6662 times smaller and 1.0709 times faster, and we accepted it. On the real candidate it was 3.5111 times smaller but slower, with a median of 2.3623 ms against 1.8826 ms for FP32, so the "not slower" condition failed and FP32 was shipped. The ONNX export [47] matched the PyTorch outputs [48] to within 1.79×10^{-7} at opset 17, after a defect in which the first export held only at the traced batch size was found and fixed. The privacy boundary of the whole phase was data locality: no differential privacy, no secure aggregation and no formal guarantee.

Table 5.6: Prototype federated plumbing and system-workstream measurements. Source: `federated-real-smoke.md`, `federated-scaling.md`, `product_hash_collisions.v1.json`, `int8_benchmark.v1.json`, `final_export.v1.json`, `communication_bytes.v1.json`. (*descriptive*)

Measurement	Result
Scale profile, 20 clients per round	median warm round 0.525 s (1,000 clients), 0.618 s (2,500), 0.644 s (5,000); peak memory 1.775, 1.779, 1.784 GiB
Product-hash collisions, 113,765 items	excess collisions 60.5028% at 50,000 buckets; 18.8371% at 262,145; 10.1639% at 524,289; 5.3461% at 1,048,577
INT8, prototype (3,664,919 parameters)	14,678,873 to 4,003,834 bytes; median 1.2228 to 1.1419 ms
INT8, real candidate	9,535,393 to 2,715,815 bytes; median 1.8826 ms (FP32) against 2.3623 ms (INT8)
Communication ledger	measured 800 bytes per transmission against an estimate of 544 bytes (32% low)

5.2.7 The scoped prototype and the 88.38% quality retention

The scoped T1 pilot compares a centralised model that replays exactly the federated schedule (R1) with FedAvg (R2A). The configuration was declared before the runs: seed 13, 1,000 TRAIN-eligible clients, 20 rounds of 50 clients, the last round as the selected model, the raw evaluator over all 438,185 validation decisions, and one declared difference (the centralised Adam state persists, while each client's Adam state is reset every round). Three attempts were needed (the first failed the frozen aggregation tolerance of 10^{-6} at 1.43×10^{-6}, float32 noise at 50 clients; the second exposed a defect in the result contract); no model, data or training setting changed between them and the failed attempt was kept as evidence. On the headline metric the pilot gives a quality retention of 88.38% (*validation*): the macro MRR@20 on the category-change slice is 0.112683 for the centralised pilot and 0.099591 for FedAvg, a difference of −0.013093 (Table 5.7).

Three cautions limit what this number means. First, both pilot models are weak: they sit below the Markov baseline of 0.139256 fitted on the pilot clients, and far below the 0.189224 of the same baseline fitted on the full TRAIN window. Second, the ratio depends on the metric: the same pair gives 73.49% on the micro MRR@20, 47.53% on the overall macro MRR@20 and 322.09% on macro accuracy at rank one, where the centralised pilot is so poor (0.024552) that FedAvg (0.079080) exceeds it; that figure reveals a weak reference, not a federated advantage. Third, the denominator is a matched pilot, not an all-data upper bound. The T2 pilot, with 200 clients and 10 rounds of 20, retained 58.71%, and T3 shipped as a shared fixed component with no federated training. We never set these ratios beside the confirmatory retention of

the main study, which measures another task, another population and another denominator.

Table 5.7: Scoped prototype pilots (seed 13, VALIDATION window, raw evaluator; R1 is the matched centralised pilot, not a final reference). Source: `pilot_quality_retention.v1.json`, `final-t2-t3-closeout.json`, `comparison.v1.json`. (*validation*)

Task and metric	R1 pilot	R2A FedAvg	Retention
T1, macro MRR@20, category-change slice (headline)	0.112683	0.099591	88.38%
T1, micro MRR@20, category-change slice	0.108290	0.079587	73.49%
T1, overall macro MRR@20	0.719584	0.341987	47.53%
T1, macro accuracy at rank one, category-change slice	0.024552	0.079080	322.09%
T2, average precision (200 clients, 10 rounds)	0.065448	0.038425	58.71%

Participation of the T1 pilot ((descriptive)): 1,000 scheduled participations by 636 distinct clients, 33,253 contributing examples, 19,041,272,000 bytes of model payload (9,520,636,000 down, 9,520,636,000 up).

5.2.8 Why the project moved to a stronger, pre-registered main study

The sprint ended on 2026-09-12 with a data foundation, a working federated pipeline, a shared encoder for T1 and T2, scoped pilots and a measured system workstream. It did not deliver the multi-task reference runs, any privacy mechanism or a freeze document. A re-plan of 2026-09-16, the "Protocol V2", demoted the prototype to a preliminary experiment that is not a mandatory basis for the thesis. Its reasons are visible above: next category over 588 categories is a weak proxy for the shopping decision, and the product channel was degraded by forced hash collisions; the labels, masks and evaluator needed to be rebuilt after the T2 and T3 defects and the two scoring conventions; the October TEST window of 27 to 30 would have been reused; and the data were to grow to October and November, with next-item prediction and purchase intent as separate tasks. The 88.38% and 58.71% figures were barred from carrying over.

Some things did carry over: the retention ratio against a centralised reference, the seed and run-identity discipline, the sealed-TEST discipline, the ONNX route at opset 17, and a note on FedAdam that listed its stabiliser τ as a quantity to record, which returns in the τ finding of the later sections. The 588-category task, the 50,000-bucket hash, the three-head design and the October-only data did not. We state three limits of the prototype plainly. The validation window served for selection, so no number above is held out. No formal pull-request review took place, because the sprint's workflow made review optional; approvals recorded in the decision documents are assertions without signed evidence, and the deployment decision record (ADR-003) stays at the status "Proposed". And the prototype supports a data-locality claim only.

What we learned

Gained.

- A reproducible data foundation, with a gate that stopped a stale rule before any TEST row was read, and evaluation rules fixed before the first comparison number.
- A working federated pipeline whose aggregation matches an independent weighted average to 2.384×10^{-7}, and measured, not estimated, sizes, latencies and bytes.

- A catalogue of failure modes that the main study designed against: censored labels, two scoring conventions, leaked candidate selection, negative transfer between heads.

Did not work.

- Product, brand, price and depth gave no gain beyond the run-to-run spread; learned T3 rerankers lost to the frozen order; sequential multi-task fine-tuning destroyed the T1 encoder; INT8 was accepted on the prototype and rejected on the real candidate.
- The plan itself: about 32 days instead of 14, R2B to R5 never run, and a retention of 88.38% whose pilot reference is weaker than a Markov rule.

The prototype showed that the plumbing worked and that its task, model and metrics could not carry a study. The next section tells how a stronger centralised baseline was built on a cleaner foundation.

5.3 The pilot studies

The prototype ended with a verdict: its task, labels and evaluator could not carry a thesis. The re-plan of 2026-09-16 ("Protocol V2") therefore restarted from the raw data and defined the task as `Next_Item`, the next different product viewed on a consecutively de-duplicated view sequence, scored against the full catalogue, on October and November 2019. Between 2026-09-20 and 2026-09-26 the pilot studies, run with AI agents, built one immutable dataset, trained a GRU and then a SASRec model, and ran a feature study; nothing federated, private or on-device was run. Labels are (*validation*) for gates, selection values and the single look at the confirmation window after each lock, (*exploratory*) for what was decided after validation values were seen, and (*descriptive*) for counts and every retrospective number on the held-out window. The question handed on was whether attention is really better than recurrence, and whether attention or side information is responsible.

5.3.1 A frozen dataset

Every later model had to share one definition of "a decision", so the first deliverable was the dataset `rees46_nextitem_v1` (Table 5.8). Sessions were rebuilt with an inactivity gap of 3,600 s, only views were kept, consecutive views of the same product were compressed (A, B, A is kept), and the target was fixed *before* the catalogue was filtered to the $K = 158{,}486$ products with at least five TRAIN occurrences, which avoids 38,858 adjacencies that filtering first would manufacture. All identifiers are stored as strings, because an early build rounded category identifiers near 2.05×10^{18} through a double-precision type. The windows were TRAIN [2019-10-01, 2019-11-16), VALIDATION_SELECT (VS) [11-16, 11-20), VALIDATION_CONFIRM (VC) [11-20, 11-23) and TEST [11-23, 11-30); VS serves selection and VC is opened once after a lock. The frozen dataset passed 82 final-gate checks with none failed, and its cold rebuild was identical.

The rising share of targets outside the catalogue (0.141% in TRAIN, 3.383% in TEST) is a drift that the

Table 5.8: The pilot-study dataset `rees46_nextitem_v1`. Source: `VERSION.json`, `splits.json`, `TRAINING_READINESS.json`, `FINAL_FULL_GRU_RESULTS.json`. (*descriptive*)

Quantity	Value
Raw rows	109,950,743 (2019-10: 42,448,764; 2019-11: 67,501,979); 5,316,649 users; 206,876 products
Decision-ledger rows by split	TRAIN 52,760,314; VS 10,930,090; VC 3,427,555; TEST 7,965,332
Decisions used for the training loss	40,180,608
Evaluated decisions (end-to-end)	VS 8,826,488 (790,459 users); VC 2,461,568 (373,953 users); TEST 5,862,968 (746,662 users)
Targets outside the catalogue	TRAIN 0.141%; VS 0.561%; VC 1.923%; TEST 3.383%

evaluator must not hide. Such targets count as misses, so every metric of this chapter is *end-to-end*: the denominator is all observed targets, and dividing the TEST count by all decisions would give the wrong 2.490%.

5.3.2 The GRU pilot study

ContextGRU-v1 embeds items in 128 dimensions, applies a one-layer GRU [1] of hidden size 256 and ties the output layer to the item embedding, so the loss is an exact cross-entropy over all 158,486 products; we avoided sampled evaluation on purpose [6]. Arm `F0` uses the item sequence only (20,791,190 parameters) and `F4` adds all side channels (20,964,062). Before any training an independent AI reviewer returned FIX_REQUIRED with eight corrections; the most consequential was that grouping the macro average on the client identifier would have merged about 500,000 unseen users into one group. A further bug was silent: decision identifiers reach 100,032,590,990, so 432,143 of the first 1,000,000 rows overflowed 32-bit storage.

Acceptance thresholds were written down before any result [7]. The pilot trained each arm on 1,000,000 TRAIN decisions for two passes (about 5% of one epoch) and evaluated on a fixed cohort of 10,000 validation users (112,365 decisions) (*validation*). After the second pass the macro MRR@20 was 0.100795 for `F0` and 0.101160 for `F4`, against 0.13308 for a first-order transition rule (TRANS) and 0.02189 for popularity (POP); `F4` minus `F0` was +0.000365 (95% interval $[-0.001005, +0.001698]$). The gate GO-2 required 0.13308 for approval and 0.11977 for a conditional approval, so the verdict was NO_GO, and we kept it on record. The failure signature was under-training, not a weak architecture, so the study lead authorised a full-data run after one bounded review.

The full run followed a frozen protocol (two to six epochs, one pre-declared learning-rate step from 10^{-3} to 10^{-4} on a plateau). Each arm made 941,736 updates on 241,083,648 exposures, about 33.3 GPU-hours in all. Table 5.9 shows the learning curves. `F0` stalled after epoch 4, its rate was stepped to 10^{-4} and it gained +0.01249 in one epoch (0.161386 to 0.173874), whereas `F4` improved by at least 0.001 at every epoch, so its step fired only at the last epoch end. The comparison of the two arms is therefore confounded by the schedule. We locked `F0` (validation 0.173874) at 2026-09-23T17:51:17Z, opened VC once, and then TEST once.

Table 5.9: Full-VS macro MRR@20 by epoch (end-to-end, rounded to six decimals; GRU epochs are counted from 1 as in the reports). The rows of the first failed SASRec run (v2) and of the repaired run (v3) share the same evaluator. Cells marked † and the second seed of A1 were added after epochs 3 and 4 had been seen and are (*exploratory*); all others are (*validation*). Source: `FINAL_FULL_GRU_REPORT_AR.md`, `epoch_records.json`, `FINAL_SASREC_REPORT_EN.md`.

Model	e1	e2	e3	e4	e5	e6
GRU F0	0.154372	0.159049	0.161037	0.161386	0.173874	0.173758
GRU F4	0.154786	0.160550	0.162498	0.163648	0.165699	0.165996
SASRec v2 A0 (stopped)	—	—	0.091332	0.090092	—	—
SASRec v2 A1 (stopped)	—	—	0.117914	0.116805	—	—
SASRec v3 A0, seed 2026	—	—	0.173762	0.172820	0.178585†	—
SASRec v3 A1, seed 2026	—	—	0.181421	0.179761	0.186298†	—
SASRec v3 A1, seed 2027	—	—	0.181504	0.180630	0.186513†	—

5.3.3 The SASRec pilot study

SASRec [2] replaces the recurrence by causal self-attention [14]. We used two arms: A0 with item embeddings only (82,897,430 parameters) and A1 with the rich side information (83,094,368). Compute forced a rescoping: the original plan of seven runs would have needed 3.6 to 6.0 days, the study lead rejected five to seven days, and the pilot study became the pair A0 and A1, one seed, six epochs, trained in TF32 arithmetic. TF32 was accepted for training only (evaluation stays strict FP32) after a parity gate on 3,000 real-data updates, where the loss differed by 1.6007×10^{-5} against a tolerance of 0.04595 (*validation*).

The first full run (v2) failed. At epoch 3 the two arms scored 0.091332 and 0.117914, far below the 0.161037 of the GRU at the same epoch (Table 5.9). The training loss of A0 fell to a minimum of 7.55 at about 12,000 updates and then rose to 9.02 at 320,000, while the item-embedding rows grew to a norm of about 161.7 (the GRU's were 11 to 13) (*descriptive*). We stopped v2 in epoch 5 of 6, repaired the recipe as a bundle of five changes and replaced a miscalibrated 20,000-update criterion, and filed both as an amendment that discloses the original outcome. The repaired run (v3) behaved: at epoch 3, A0 reached 0.173762 and A1 0.181421, which exceeds the locked GRU F0 (0.173874) by +0.00755 (*validation*).

At the study lead's request for the best accuracy, and after the validation values of epochs 3 and 4 had been seen, we added a cooldown epoch 5, a second seed (2027) for A1 and post-hoc ensembles (systems S1 to S4b, fixed before they were scored) (*exploratory*). The cooldown reached 0.186298 and 0.178585 for A1 and A0 with seed 2026, and 0.186513 for A1 with seed 2027; the seed gaps at matched epochs were 0.000083, 0.000869 and 0.000216, a range from two seeds and not a standard deviation. A pre-registered rule locked the headline A1_RICH256_s2027 at epoch 5 at 2026-09-26T05:05:18Z, and only afterwards were VC and TEST read, once. These looks are retrospective, because the GRU's values were already known, and we label them (*descriptive*).

Table 5.10 gives the verdict of the pilot study, and it is modest. The GRU beats the transition rule on TEST by +0.021091 (95% interval $[0.020835, 0.021345]$) and F4 is below F0 by 0.012191 ($[-0.012446, -0.011948]$), so side information did not help the recurrent model. The GRU's gain is

Table 5.10: Macro MRR@20 of the pilot-study models on the selection split (full VS), on the confirmation split (VC, one look after the lock) and on the held-out window (TEST, one look). VS values are (*validation*); VC and TEST values are (*descriptive*) (retrospective, previously inspected, not main-study confirmatory evidence). Source: `FINAL_FULL_GRU_RESULTS.json`, `FINAL_SASREC_RESULTS.json`, `FINAL_SASREC_REPORT_EN.md`.

Model	VS	VC	TEST
POP	0.021579	0.020168	0.021118
TRANS	0.161638	0.148740	0.147604
GRU F4	0.165996	0.156429	0.156504
GRU F0 (locked)	0.173874	0.168988	0.168695
SASRec A0, seed 2026, epoch 5	0.178585	—	0.166484
SASRec A1, seed 2026, epoch 5	0.186298	—	0.170225
SASRec A1, seed 2027, epoch 5 (headline)	0.186513	0.171339	0.170475

concentrated where there is history (0.122611 and 0.101593 at prefix lengths 20 to 49 and 50 or more, against 0.096051 and 0.076684 for the rule), whereas the rule wins when the user has no prior view (0.159537 against 0.152686) (*descriptive*). The SASRec headline exceeds F0 by +0.012639 on VS, +0.002351 on VC and +0.001780 on TEST (95% interval $[0.001538, 0.002022]$); the margin shrinks at each look and its cause was not isolated. The item-only A0 is 0.002211 below F0 on TEST, a comparison that was neither pre-registered nor given an interval. These intervals are conditional on the fitted models and carry no seed variance. We did not claim a state-of-the-art result, a privacy property or a cause.

A staged feature study, run in parallel to decide whether the side-information columns and a possible repeat-channel signal were worth keeping, ended with the decision KEEP_CURRENT at 2026-09-26T08:28Z. Both pilot studies also looked at the held-out window: the GRU on 2026-09-23 and the SASRec model retrospectively on 2026-09-26. The main study reused the frozen dataset and records these looks as a historical legacy exposure (entry 3 of its exposure register, Section 5.4). No main-study selection used these numbers.

What we learned

Gained.

- An immutable, hash-covered dataset with a full-catalogue, end-to-end evaluator, and the look discipline that the main study inherited: lock, read VC once, read TEST once, with claim files.
- Silent defects found before they reached a result: 32-bit overflow of decision identifiers, a macro average that merged about 500,000 unseen users, and a target class that conflated two meanings.
- A first estimate of the gap between the families: small, shrinking at each look, and at most +0.001780 on TEST.

Did not work.

- The GRU pilot failed its gate (NO_GO), the first SASRec run failed outright (v2), and side information did not help the GRU (a comparison confounded by the learning-rate schedule).
- The architecture question stayed open, because the GRU's learning-rate step and the SASRec five-

change bundle were not matched. **Handed on:** is attention better than recurrence once both are tuned with equal effort? The main study opened with a controlled contest for that reason.

5.4 Foundations of the main study: census, population, catalogue and features

On about 2026-09-26T13:20Z the study lead adopted the main study (ADR-001): a central contest between the GRU and SASRec on one clean protocol, then *one* winner carried to every later arm. FedBN, Purchase Intent and all downstream runs of the loser were out of scope. This section covers the first 32 hours of the main study, to 2026-09-27T21:38Z. It produced no confirmatory or TEST number; its labels are (*descriptive*) for counts and engineering measurements and (*validation*) for the feature screen. The programme was run by a single study lead with AI agents in defined roles (the AI operations agent that kept the log, the AI gate reviewer whose rulings bound the gate, AI worker agents and an independent reviewer); Chapter 3 describes the governance and the AI-use statement.

The study was organised as a "research and engineering company" with five gates (plan and data protocol, G0; production readiness, G1; architecture choice, G2; and two later gates, G3 and G4), task packets, architecture decision records and a literature register. Recurring controls were exclusive-create lock files, SHA-256 manifests re-hashed on verification, fail-closed launchers, self-hashed registries and a hash-chained exposure register for the TEST data. The independent reviewer reversed work that had been accepted (INT-02). With the study lead offline, the gate reviewer acted as proxy for blocking decisions within hard limits: no push, no deletion of evidence, TEST never used for selection, and the 5–100 rule and the history policy never reopened.

5.4.1 Census, population and catalogue

The proposal never said whom to simulate, so the main study counted. A census of the TRAIN window found 4,804,933 users with 40,180,608 eligible examples and split them by the number of eligible examples (Table 5.11). The study lead's population amendment fixed the main stratum as the pool of users with 5 to 100 eligible TRAIN examples: 1,446,986 users supplying 28,623,386 examples, 71.24% of the supervision. The earlier population and manifests became historical evidence about infrastructure only, which is why a second data lock (LOCK_V2) exists. The pool is cut into five disjoint rank bands: the main population MAIN (131,072 users), the server-side PRE band (1,217,610), two bands of 32,768 users that no training band contains (UNSEEN_VAL and UNSEEN_HOLDOUT) and a feature-search band (FS) of 32,768. MAIN plus the PRE band, 1,348,682 users or 93.2% of the pool, is the population of the cloud reference `C_FULL`. CALIB, a nested subset of 32,768 MAIN users, is used for tuning, and the catalogue is fixed at the $K = 158{,}486$ products with a global TRAIN support of at least five.

Limitations

The 5–100 restriction is a design limit and not a finding. Users with fewer than five or more than 100 eligible TRAIN examples never take part in training, local or federated; they serve only as evaluation

Table 5.11: Main-study census and populations (TRAIN-only definitions). Manifest columns: users, TRAIN decisions, users with end-to-end validation rows, validation decisions. Source: `CENSUS-02.md`, `02_DATA.md`. (*descriptive*)

Stratum or population	Users	Examples	Note
All strata	4,804,933	40,180,608	TRAIN-only eligible examples
Pool of 5 to 100 examples	1,446,986	28,623,386	71.24% of supervision; median 13
Sparse (1 to 4)	1,197,003	2,503,629	
Heavy (> 100)	51,009	9,053,593	
True cold, with TRAIN events	1,509,381	0	no eligible example
MAIN manifest	131,072	2,587,776	40,764 users, 519,714 validation decisions
CALIB manifest	32,768	646,067	10,070 users, 127,530 validation decisions
PRE band	1,217,610	—	server-side pretraining
C_FULL population	1,348,682	—	MAIN plus the PRE band
Evaluation panels with VALIDATION decisions	—	—	true cold 381,295; sparse 155,059; heavy 36,373; warm truncation 16,384 users

panels. The conclusions of the thesis therefore concern users with a moderate amount of history, and the main study did not reopen this rule.

The pilot studies used a window limited to a single session, which cannot describe a device that has accumulated history over many sessions. The study lead therefore required a history policy that crosses session boundaries (USER_HISTORY_L50) and a new builder (DATA-05). An exhaustive check on the pool found 0 causality violations, and 87% of the validation rows now see events from the TRAIN period. The census also taught a hygiene lesson: a schema check read the TEST status and split columns (counts per status only, nothing used), and a first census run contained mean reciprocal ranks of the earlier models and the GRU–SASRec difference, which nobody had asked for. The first event led to the exposure register, whose entry 2 records it (counts only, no feedback); the superseded run was quarantined in place. Entries 1 (census counts), 3 (the historical look of the pilot studies) and 4 (the history builder, counts only) complete the early register.

5.4.2 The feature search and the input schema

The input schema was the last open design question before the contest. A cleaning pass fixed the contract at 59 columns: 51 KEEP, 6 DROP_REDUNDANT, 0 DROP_SPEC_DEAD and 2 RE_ENCODE. A screen then asked which groups of input channels earn their place. The rule, registered in ADR-005, accepts a reduced schema only if the *worse* of the two families loses at most 1.0% of client-macro MRR@20 in the seed mean and the upper 95% bound of the relative drop is at most 2.0%. Stage A trained 8 runs (about 3.05 h on two GPUs) and the final stage 4 runs (about 1.13 h), 12 in all, each at 6.0 passes over the training data, for seeds 2026 and 2027 on 10,269 FS validation users (*validation*). The importance of a group is the absolute MRR@20 lost when its values are permuted among length-matched users (Table 5.12).

The reduced schema S* is better for the GRU, with a relative drop of −0.8956% (95% interval $[-1.635, -0.145]$; a negative drop is a gain), and indistinguishable for SASRec (−0.1316%, upper

bound +0.8603%), so the worse family, SASRec, passes the rule and no fallback was needed (*validation*). Near-duplicate removal alone (L3S) was within noise for both families (GRU −0.668%, SASRec −0.077%). Five groups were dropped because both families found them within noise: G3 (weekday), G4 (hour of day), G7 (item age), G11 (gap since the previous session) and G12a (previous counts). S* has 25 information channels, 24 input columns plus the output catalogue. One group is instructive: permuting G8 *improves* the GRU by 0.000508 (interval $[-0.000776, -0.000242]$) but costs SASRec 0.000980 ($[0.000560, 0.001408]$). The log records this as a mixed effect: the value of an input depends on the model family. The log describes the contents of only some groups, so we name the others by label.

Table 5.12: Permutation importance of the input groups, absolute MRR@20 lost when the group is permuted (10,269 FS validation users, both seeds pooled). ‡ the 95% interval includes 0. Source: `FS_FINAL.json` (`final_decision.importance`). (*validation*)

Group	GRU	SASRec	Outcome
G1 (required)	0.023937	0.040778	required
G2	0.006394	0.008609	kept
G3 weekday	−0.000006‡	0.000036‡	dropped
G4 hour of day	0.000032‡	−0.000003‡	dropped
G5	0.000235	0.004228	kept
G6	0.004903	0.003161	kept
G7 item age	0.000061‡	0.000435‡	dropped
G8	−0.000508	0.000980	kept; mixed effect
G9	0.000072‡	0.000440	kept; one family
G10	0.000487	0.001687	kept
G11 gap since previous session	−0.000107‡	0.000156‡	dropped
G12a previous counts	0.000033‡	−0.000037‡	dropped
G12b	0.000582	0.000080‡	kept; one family
G12c	0.001274	0.001091	kept
G13 session intent	0.001534	0.000974	kept

5.4.3 Engineering measurements and the first tuning result

Because the contest compares two models by a difference of a few thousandths, the numerical set-up had to be shown not to move the answer. Strict deterministic training was bitwise reproducible from run to run and identical to the warn-only mode, at $K = 2{,}000$ and $K = 158{,}486$ and for both families. Scoring in chunks differs from a full matrix product by up to about 5×10^{-5}, enough to move a near-tie by one rank, so the scoring block size became a registered constant of every comparison. The data path ran at 0.986 and 0.994 of the speed of the pure GPU step, with cached and streamed paths bitwise equal, and the full catalogue trained at 17.5 (GRU) and 9.2 (SASRec) updates per second on the A16 GPUs of the cluster (*descriptive*). The phase ended with the first tuning result, the GRU recipe `P2_GRU_D128_S1_s4` (width 128, schedule S1, peak learning rate 0.001687, dropout $[0.1, 0.05]$) at 2026-09-27T21:31:28Z; its value is reported in Section 5.5.

Process faults cost time but changed no result: hand-typed times ran ahead of the clock (so timestamps now come only from the system clock or file times), a monthly spending limit of the AI service stopped two agents mid-task (no partial output used), and two feature-search launches failed closed and were kept.

What we learned

Gained.

- A defensible population, a fixed catalogue of 158,486 products, a causal history policy, and a 25-channel schema chosen by a registered rule on two seeds and two families.
- Controls (locks, hashes, a hash-chained exposure register) that made later claims auditable, and the observation that an input can help one family and hurt the other (G8).

Did not work.

- The 5–100 rule restricts the conclusions to moderate-history users and was never relaxed.
- The late change of population and history policy made earlier locks historical and delayed the feature search by a few hours; process faults cost time but changed no result.

5.5 The backbone contest and the architecture lock

Which sequence-model family, the GRU or SASRec, should carry every later arm of the study, when both are tuned with equal effort on one protocol?

The pilot studies could not settle the question, because the two families had been trained under unmatched schedules, and published comparisons of sequential models depend on the loss and on negative sampling [16, 17]. We therefore gave both families the same exact full-catalogue cross-entropy, data, 6.0-pass budget and selection rule. The design had three stages: equal-effort tuning on CALIB (P2); a contest on MAIN with the tuned recipes at seeds 2026 and 2027 (P3); and one mechanical decision, G2, which selects within each family and then between families by the rule T1. The study lead removed any anchoring on the pilot studies: model size was part of the search, selection used data only, and a tie goes to the smaller model. TEST stayed sealed. Labels are (*confirmatory*) for values that a registered rule read once on MAIN validation data (not the final number of the study; the confirmatory test evaluation comes later), (*validation*) for tuning and post-decision reads, and (*descriptive*) for baselines, diagnostics and configuration.

5.5.1 Equal-effort tuning

The GRU grid had 12 trials (width $\{128, 256\}$, schedule $\{S0, S1\}$, dropout $\{0.1, 0.2, 0.3\}$); the SASRec grid had 12 trials over three sizes (256 wide with 2 blocks, 256 wide with 3 blocks, 384 wide with 2 blocks), two schedules and two trials per cell. The best configuration of each size was re-run at seed 2027, and the tuned recipe is the one with the higher two-seed mean. Table 5.13 gives the outcomes. Width 128 beat width 256 for the GRU by 0.0096, and a checkpoint average was not carried because its gain, +0.00016, was below 0.0005. On CALIB, SASRec scored 0.1324 against 0.1248, but the gate reviewer ruled that this order is “tuning only” and may not decide the contest. Two edge effects were disclosed and not corrected: all three SASRec sizes were tuned at the lowest learning rate of the grid (0.000400), and the

GRU winner sits at the top of its S1 sub-grid.

Table 5.13: Equal-effort tuning outcomes (CALIB validation, client-macro MRR@20, two-seed means). Source: `runs_p2`. (*validation*)

Family	Recipe	Value
GRU	P2_GRU_D128_S1_s4 (width 128, peak learning rate 0.001687, dropout [0.1, 0.05])	0.124844
GRU	best width-256 recipe	0.115286
SASRec	D256_B3_S0_s0 (256 wide, 3 blocks, learning rate 0.000400)	0.132402
SASRec	D256_B2_S0_s0	0.130120
SASRec	D384_B2_S0_s0	0.127326

5.5.2 The eight contest runs on MAIN

The study lead objected that the tuning had been done on CALIB only, and asked for all training on MAIN. After a ruling ("design A modified") CALIB stayed as a screen and the contest moved to MAIN with two candidates per family, the best configurations of the top two sizes: GRU width 128 (C1) and width 256 (C2), and SASRec with 3 blocks (C1) and 2 blocks (C2), each at two seeds, eight runs in all. Amendment 05 was registered before any MAIN launch or read, and the decision runner was reviewed and frozen seven hours before the unsealing; the review caught a high-severity defect, a CALIB-disjoint diagnostic that would have scored the wrong users while the user count still matched. The runs took about 13.1 h (2026-09-27T22:18:51Z to 2026-09-28T11:22:23Z), against a pre-run estimate of about 24 h. Because a GRU still improving at 6.0 passes over the training data continues automatically, all four GRU runs continued to 7.0 passes without any value being read. The G2 script unsealed the eight runs once, at 2026-09-28T11:25Z (Table 5.14).

Table 5.14: The eight contest runs on MAIN validation (client-macro MRR@20). PB is the best of the evaluation marks, with the mark in brackets; the endpoint is the fixed 6.0-pass checkpoint. The PB values of the first candidate of each family are the G2 inputs ((*confirmatory*)); all other cells were read after G2 and are (*validation*). Source: `G2_RESULT.json`.

Candidate and rule	Seed 2026	Seed 2027
GRU C1 (width 128), PB (mark 6.0) (*confirmatory*)	0.157141	0.156733
GRU C2 (width 256), PB (mark 6.0)	0.15412	0.15345
SASRec C1 (3 blocks), PB (mark 3.0) (*confirmatory*)	0.161196	0.161221
SASRec C1 (3 blocks), endpoint 6.0	0.15287	0.15258
SASRec C2 (2 blocks), PB (mark 3.0)	0.16059	0.16006
SASRec C2 (2 blocks), endpoint 6.0	0.15086	0.15011

Within each family the first candidate won on the two-seed endpoint mean, with no run excluded: 0.156937 against 0.153785 for the GRU and 0.152724 against 0.150484 for SASRec. Between families the rule T1 compares the PB values per seed. SASRec scored higher in both seeds, and the mean difference (GRU minus SASRec) was −0.004271 (95% interval [−0.005180, −0.003261]), which the rule classes as DECISIVE_STABLE for SASRec. The CALIB-disjoint diagnostic on the 38,496 MAIN validation users outside CALIB agrees in sign (−0.004103), and NDCG@10 gives 0.19105 for the GRU and 0.19668 for SASRec (*descriptive*). The SASRec mean of 0.1612 is about 2.72% above the GRU's 0.156937. In our

contest SASRec scored higher; we draw no general conclusion about the two families from it.

Two conditions qualify this result. First, SASRec peaked at the mark of 3.0 passes and then declined, by about 0.0085 to the endpoint, while the GRU was still improving at 6.0 passes. The rule compares best marks, an optimistic reading, because the best of twelve marks is chosen on the same validation data; at the fixed endpoint the order reverses (GRU minus SASRec +0.004213, interval [+0.003283, +0.005216], a reported diagnostic (*descriptive*)). Both checkpoints were therefore kept for the later arms. Second, both SASRec candidates had width 256, so the 384-wide size did not reach MAIN, and the tuned learning rates sat at grid edges. The seed gaps were small (0.00041 for the GRU and 0.00002 for SASRec at PB).

5.5.3 Baselines, the lock and the incidents

The model-free baselines, run on the CPU and read only after G2, anchor the scale on MAIN validation (*descriptive*): session-kNN ($k = 500$) reaches MRR@20 0.11138 (NDCG@10 0.1485), repeating the last item 0.07111 (0.1016; about 44% of the SASRec value) and popularity 0.01943 (0.0249). Much of next-product ranking is repetition.

The decision was locked as `ARCHITECTURE_LOCK.json` at 2026-09-28T11:34Z. The cloud model `C_BIG` is `SASREC_D256_B3` (83,881,764 parameters; per-visit payload 671,054,112 bytes in FP32), trained on MAIN for 6.0 passes over the training data, with its PB checkpoint at mark 3.0 and its controlled-budget checkpoint at the endpoint. The device-size model is `SASREC_D64_B2` (20,710,048 parameters; 165,680,392 bytes per visit, under the cap of 170 MB), registered after the lock. The GRU leaves the downstream plan; its export adapter and its re-evaluation were abandoned.

Process incidents were small and changed no result: the GPU index is part of the run configuration, so a launch on GPU 1 differed from a dry run on GPU 0 in one field (INC-025), and a test fixture restored with the wrong line endings broke 13 later tests (INC-023).

What we learned

Gained.

- A mechanical, pre-registered decision: SASRec scored higher in our contest (−0.004271 for GRU minus SASRec, interval [−0.005180, −0.003261]), robust to the CALIB-disjoint check, and a lock that fixed the cloud model, the device-size model and the checkpoints for every later arm.
- Baselines that anchor the scale: repeating the last item reaches about 44% of the winner.

Did not work or remains open.

- The result depends on the checkpoint rule: at the fixed 6.0 endpoint the GRU is higher by +0.004213. We therefore make no claim that SASRec is the better family; its choice is a design decision of the study.
- Tuning sat at grid edges, only width-256 SASRec candidates reached MAIN, and the CALIB order

was declared “tuning only”.

The lock closed the foundations. The next phases asked what a federated recipe can retain of this cloud model, and began with the calibration of that recipe.

5.6 Calibration and the federated recipe tuning stages

The backbone contest ended with the choice of SASRec [2] under the registered rule and with the architecture lock. The federated arm was still the recipe registered before any federated run existed. The study lead therefore asked that it be made “as strong as realistically possible” before any federated value was read on the main population, so that the report could state *when we did X, federated learning improved by Y*. This section follows that work from 28 September to 1 October 2026 in the order in which it happened.

Unless stated otherwise, the performance numbers below are CALIB selection reads (*descriptive*): peaks over the evaluation marks, measured on a 32,768-client calibration subset and used to choose a recipe. They are not claims, and they need not transfer to the main population or to the TEST set. No central or federated value on the main population (MAIN) had been read; the first such read is the VALIDATION read of Section 5.8.

Terms used from here on

Training budget (passes over the training data): one pass is one exposure of every eligible training example of the cohort. For the standard budget of 6.0 passes over the training data, a federated run with N clients and P local passes takes $\lceil 60N/(576P) \rceil$ rounds of 64 clients: 13,654 rounds on MAIN (131,072 clients) and 3,414 on CALIB (32,768 clients) with one pass, and 6,827 and 1,707 with two passes.

CALIB is the 32,768-client subset of MAIN used only for tuning; **UNSEEN_VAL** is a validation panel of users who were never in training, used only to select on-device recipes.

PB / CB. PRACTICAL_BEST (PB) is the best of the 12 evaluation marks, taken every 0.5 passes over the training data up to 6.0, and is optimistic by construction. CONTROLLED_BUDGET (CB) is the fixed endpoint at 6.0 passes over the training data.

5.6.1 References and rules of the tuning stages

The device-size model (SASREC_D64_B2: width 64, two blocks, 20.7M parameters) was tuned by a four-point learning-rate screen on CALIB (28 September, seeds 2026 and 2027), which registered the *SMALL recipe*: a peak learning rate of 0.000711 and a size cap of 165,680,392 bytes per client visit. The recipe trained the matched central reference C_SMALL on MAIN for 6.0 passes over the training data. The cloud references use width 256 and three blocks (83,881,764 parameters): C_BIG on the 131,072 MAIN users and C_FULL on all 1,348,682 users (26,677,600 training examples); both seeds of C_FULL finished on 30 September with their values sealed. Downstream runs used two NVIDIA A40 cards, available from

21:17 UTC on 28 September, while the central contest had run on A16 cards. The evaluation device class is part of the comparison key, so the references were re-evaluated on the A40 class; the two classes rank differently on 12,635 and 14,063 rows for the GRU and SASRec checkpoints, so pooling them would have been invalid.

The federated recipe tuning stages are numbered 0 to 8, and each was screened on CALIB with the same metric: client-macro MRR@20 at PB for the FA arm (the FedAvg-family federated arm), seed 2026, with a full schedule of 6.0 passes over the training data (1,707 rounds with two local passes, 3,414 with one). Four rules were registered before any stage was read. Stages run strictly one after the other, and a stage's value enters the record only after its selection file exists, written by a reviewed tool. A challenger replaces the incumbent only if it gains at least 0.002 on the raw values. Differences inside a tie band of 0.0005 go to the simpler option (FedAvg, then FedAvgM, then FedAdam). A winner on the edge of its grid triggers one registered outward extension. Table 5.15 lists the stages and Figure 5.3 shows the cold-start tuning stages 0 to 4.

Table 5.15: The federated recipe tuning stages on CALIB (*descriptive*). Metric: client-macro MRR@20, PB, FA, seed 2026, 6.0 passes over the training data. Stages 0 to 4 start from random weights; stages 7 and 8 start from the server-pretrained model at the stage-4 recipe. Gains are against the previous stage's winner (stages 0 to 4) or the grid incumbent (stages 7 and 8). Source: `FL_IMPROVEMENT_LADDER.md` and the selection files `FL_TUNING_S2.json`, `FL_TUNING_S3.json`, `FL_TUNING_SELECTION.json`, `FL_WARM_SELECTION_S7.json`, `FL_WARM_SELECTION_S8.json`.

Tuning stage	What changes	Amend.	Winner	Gain
0	FedAvg as registered: server rate 1, fresh client AdamW, batch 16, two passes, central rate shape, local rate = SMALL peak	06, 06B	0.023473	reference
1a	local-rate multiplier {0.5, 1, 2}; winner ×2 on the grid edge	06B	0.025562	+0.002089 (+8.9%)
1b	multiplier extended to {4, 8, 16}; winner ×8	06F	0.028796	+0.003234 (+12.7%)
2	server optimiser: FedAdam $\eta_s = 0.01$, $\tau = 10^{-3}$	06G, 06J, 06L	0.034247	+0.005451 (+18.9%)
3	constant rate shape and client SGD with momentum 0.9, rate 0.1	06I, 06J, 06L	0.094936	+0.060689 (+177%)
4	one local pass instead of two, batch 16	06J	0.107562	+0.012626 (+13.3%)
5	FA_X2: a budget of 12 passes over the training data	06D, 06I	—	not run (declared)
6	personalisation variants (FT_FA, PF)	06, 06A	—	run later as device arms
7	pretrained start, all weights trainable (FA_WARM); client factor 0.25, server factor 0.1	06K, 06M_B	0.170582	+0.034585 over 0.135997
8	pretrained start, item tables frozen (FA_WARM_FROZEN); client factor 1, server factor 0.1	06K, 06M_B	0.170709	+0.002248 over 0.168461

5.6.2 The local learning rate and the grid edge (tuning stages 0 and 1)

The as-registered recipe (tuning stage 0) reached 0.023473 (*descriptive*). Its one tuned quantity was the local learning rate, screened at 0.5, 1 and 2 times the SMALL peak (0.020263, 0.023473, 0.025562). The best value sat on the top edge of the grid and the curve was still rising, but the rule that a grid-edge winner must never be accepted was not yet in force: six MAIN runs (FA, FedProx, 8-bit uploads and the clip

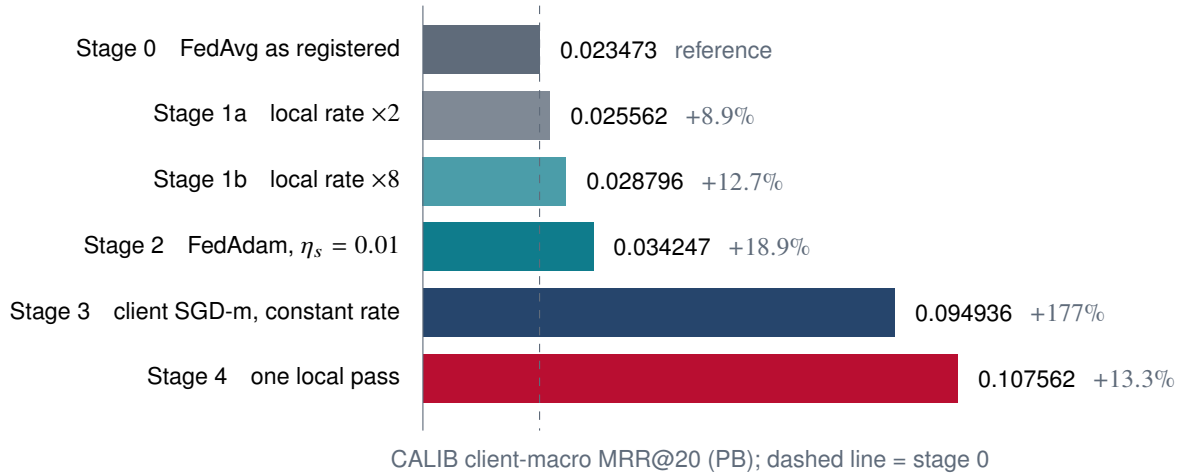


Figure 5.3: The cold-start federated recipe tuning stages 0 to 4 on CALIB (*descriptive*). Bars give the winner of each stage (client-macro MRR@20, PB, FA, seed 2026, 6.0 passes over the training data); the percentage is the gain over the previous stage's winner. The cumulative gain is a factor of 4.58 (0.107562 against 0.023473). Source: `FL_IMPROVEMENT_LADDER.md`.

calibration, all at ×2) started at 09:12 UTC on 29 September. The study lead and the independent design gate stopped them about five minutes later, before any evaluation mark had been written; their directories are sealed as superseded and are not evidence. Amendment 06F then extended the screen to ×4, ×8 and ×16, and the winner became ×8 at 0.028796 (0.027570 at ×4, 0.022909 at ×16), an interior point and +0.003234 over ×2. The text of 06F was itself overwritten once by a second agent session and restored byte-identically before any run read it; from then on, agents coordinated only through the decision log. For orientation, the tuning-stage register estimates (INFERENCE, not a result) that a matched central device-size model reaches about 0.11 to 0.12 on the same CALIB users, so tuning stage 0 kept about a fifth of it.

5.6.3 Server optimiser, client optimiser and local computation (tuning stages 2 to 4)

Tuning stage 2. The research diagnosis recorded before this stage named three limits of the recipe (INFERENCE): the server step was capped by about four fresh-AdamW local steps with server rate 1 and no momentum; each client update carried dense noise on the 158,486 output rows; and the central rate shape, written for epochs, wasted about 53% of the rounds. Tuning stage 2 replaced the server step by FedAdam [25], which divides a running mean of client updates by the square root of a second moment plus a constant τ (registered at 10^{-3}), and by server momentum (FedAvgM [21]). The best cell was FedAdam with $\eta_s = 0.01$ at ×1 with 0.034247, +0.005451 (+18.9%) over tuning stage 1b and +0.010774 (+45.9%) over tuning stage 0. All other cells were much lower: FedAdam at 0.016630 ($\eta_s = 0.01$, ×8), 0.015632 ($\eta_s = 0.1$, ×1) and 0.016130 ($\eta_s = 0.1$, ×8); the edge runs 0.027238 ($\eta_s = 0.003$), 0.021876 ($\eta_s = 0.001$) and 0.024374 ($\tau = 10^{-5}$); and FedAvgM at ×0.5, ×1, ×2 and ×8 with 0.024236, 0.027541, 0.022626 and 0.008489. The winner peaked at mark 3.0 (0.0342) and fell to 0.0251 at 6.0, 27% below its peak, exactly when the central rate shape reached 0.1×. Moreover τ was screened once, here, against a client optimiser that was about to change; it stayed at 10^{-3} in every later selection of the registered study and was not

screened again after tuning stage 3. The following sections return to that constant.

Tuning stage 3. Two suspects for the decline were tested separately. A constant rate shape with the fresh client AdamW scored 0.033104 and still peaked early, so the shape alone was not the cause. Replacing the client optimiser by SGD with momentum 0.9 changed the picture: 0.058775 at rate 0.01, **0.094936** at 0.1 and 0.082381 at 1.0. The winner is +0.060689 (+177%) over tuning stage 2 and 4.04 times tuning stage 0, and its curve rose through the whole budget (0.0285 at the first mark, 0.0758 at 3.0, 0.0949 at 6.0), so the registered contingency, tuning stage 3b (decline $D = 0$) was not triggered. Our reading (INFERENCE) is that the decline came mostly from the fresh AdamW, whose sign-like updates touch every output row, and not from overfitting; with client SGD the model is under-trained at 6.0 passes over the training data.

Tuning stage 4. Local computation was varied at equal exposure (passes in $\{1, 2, 4\}$, batch in $\{8, 16\}$), so that fewer passes mean more rounds. One pass with batch 16 won with **0.107562**, against 0.094936 for two passes, 0.088351 for batch 8 and 0.077878 for four passes (+0.012626, +13.3%). The cumulative gain from tuning stage 0 to tuning stage 4 is a factor of 4.58. The DP-labelled arms and the clip calibration kept two passes and batch 16, because their number of rounds ($T = 384$) and noise multipliers were registered earlier. The budget of 12 passes over the training data (tuning stage 5, about 21 hours per process) was registered but declared not run.

5.6.4 The warm line: pretraining and frozen item tables (tuning stages 7 and 8)

By the end of tuning stage 4 the cold arm had improved fourfold, but a question from the study lead changed the plan: an untrained model on the device is unrealistic. The deployment headline should start from a model trained server-side, and the cold line should become the ablation that measures what the pretrained start is worth. The pretrained model PRE (written θ_{pre}) was trained with the SMALL recipe for 6.0 passes over the training data on 1,217,610 users who are not in MAIN (24,089,824 decisions); this is about nine times more updates than the recipe was tuned for, which can only understate PRE. No arm is warm-started from data it later trains on. Consequently PRE uses other users' data (a different data-sharing tier), and the warm line is read against C_FULL and not C_BIG, because PRE has seen nine times the data of the federated users.

Seven descriptive runs at the leader point of tuning stage 2 showed what the cold recipe does to a pretrained model (*descriptive*). With the item tables frozen, the warm arm stayed flat at about 0.166 (0.1673 at mark 0.5). With every weight trainable and the fresh client AdamW, it collapsed: the gentlest run fell from 0.1582 at mark 0.5 to 0.038 at 6.0 and the others ended near 0.03 or below, against 0.034247 for the cold arm at the same point. The dense noise that limited the cold arm thus destroys a pretrained model when the item tables are trainable (INFERENCE), and freezing them protects it. Two item tables hold about 98% of the parameters, so freezing them also removes almost all of the upload: only the encoder is trained and sent, 263,822 values (MEASURED; the registered figure of 265,162 was to be re-measured). The realism audit had also found that non-DP updates reveal the items a user viewed, so "weights only" is data

minimisation and not privacy.

Table 5.16: The registered warm-start screens on CALIB (*descriptive*) at the tuning-stage-4 point (client-macro MRR@20, PB, seed 2026). "Client" and "server" are factors applied to the learning rates of tuning stage 4. Winners in bold. Source: `FL_WARM_SELECTION_S7.json`, `FL_WARM_SELECTION_S8.json`.

Stage-7 arm	Client / server	CALIB	Stage-8 arm	Client / server	CALIB
FA_WARM	1 / 1	0.135997	FA_WARM_FROZEN	1 / 1	0.168461
FA_WARM	1 / 0.1	0.167746	FA_WARM_FROZEN	**1 / 0.1**	**0.170709**
FA_WARM	0.25 / 1	0.156175	FA_WARM_FROZEN	4 / 0.1	0.170351
FA_WARM	**0.25 / 0.1**	**0.170582**	FA_WARM_FROZEN	4 / 1	0.165227

The registered screens (Table 5.16) selected a server factor of 0.1 in both stages. Tuning stage 8 beat the grid incumbent by +0.002248, just above the 0.002 rule, and the two winners were 0.000127 apart in favour of tuning stage 8, so the tie rule made the frozen-table arm the warm candidate (FA_WARM*). Its CALIB value is at the level of the pretrained model itself: PRE scored 0.173081 on a different, non-MAIN validation panel (not directly comparable). The study lead and the AI operations agent therefore expected, already here, that the gain of the federated rounds over θ_{pre} would be small (INFERENCE); amendment 06N later registered exactly that gain (K21) as one of the two primary contrasts.

5.6.5 Device arms, DP constants and the MAIN launches

On-device recipes were selected once each on UNSEEN_VAL from the matched central model, with no metric printed: the fine-tuning cell FT_C_f0.3_P4 (a rate 0.3 times the SMALL peak, four passes) and the device-only cell LO_P4. The fine-tuning arm on the pretrained model (FT_PRE) was added later and uses the FT_C selection.

The privacy arms were calibrated before any run. The registered accountant assumes Poisson sampling of clients, which the first sampler (fixed-size, without replacement) did not implement; review caught this before any DP run, since at the registered noise multiplier that sampler would have corresponded to $\varepsilon \approx 1{,}388$ instead of 8. Amendment 06C therefore moved the non-noised twin (FA_1024), the clip calibration and the DP arms to Poisson sampling with rate $q = 1{,}024/131{,}072$, a fixed denominator of 1,024, $T = 384$ rounds and $\delta = 5 \times 10^{-6}$ [35, 44, 45]. The noise multipliers are $z = 0.5642$ for $\varepsilon = 8$ and $z = 1.3094$ for $\varepsilon = 1$ (and $z = 0.7244$ for $\varepsilon = 4$, achieved 3.99986), reproduced independently in multiple precision to 2.6×10^{-11}. The clip norm $S = 0.284363$ is the median client-update norm over rounds 1 to 20 of the non-private run (20,625 norms). These ε values are accountant targets: a randomness audit after the confirmatory test evaluation found that the legacy implementation drew its noise from a disclosed seed, so the runs do not establish a randomised-DP guarantee. We call them DP-labelled throughout and report their utility as a measurement (Chapter 6).

The MAIN federated runs were launched from the selection file of tuning stage 4 on 30 September at 18:37 UTC. A few hours later the study lead questioned the cold-start headline, and the design gate was asked for a run-allocation memo. The decision (amendment 06N) was to finish the pretrained line first and to run the random-start line afterwards, as the measurement of what pretraining brings. The cold FedProx runs

of both seeds, the cold non-noised twin of seed 2026 and the queued cold privacy and personalised arms were stopped before any value had been read and carry the marker STOPPED_REALLOCATED; they are never evidence, and the cold arms were later re-run from scratch (Section 5.7). The warm runs were launched at 23:54 UTC the same day. Ten concurrent federated jobs each ran at about 550 rounds per hour (about 5,500 in total) and six at about 950 (about 5,700), so the two cards were already compute-saturated.

Table 5.17: Registered amendments that changed the recipe or the reading rules before the values they govern were read. Source: `PREG1_AMENDMENT_06*.md`.

Amendment	Effect on the recipe
06B	local-rate screen on CALIB; contrasts K1 to K16; privacy levels; 8-bit upload arm
06C	Poisson sampling for the non-noised twin, the clip calibration and the DP arms (sampler and accountant made consistent)
06D	pinned A40 device class; TEST manifests; ε contrasts read at the controlled budget; C_FULL fixed at 1,348,682 users
06F	grid-edge rule: local-rate screen extended to ×4, ×8, ×16; the ×2 MAIN runs superseded
06G, 06H	FedAdam [25] as server optimiser with $\tau = 10^{-3}$, second moment started at τ^2; an optional DP-sensitivity run (later declared not run)
06I, 06J	realism audit, client-optimiser wave (fresh AdamW against SGD with momentum), contrasts K17 to K19; tuning-stage rules (sequential stages, the 0.002 rule, tie band 0.0005)
06K, 06M	warm start: tuning stage 7 (PRE plus FA_WARM) and tuning stage 8 (frozen tables), contrasts K20 and K21; never warm-start from a MAIN-trained model; descriptive warm preview
06L	server-momentum and SGD screens, privacy matrix, contrasts v3; no clipped count recorded under DP
06N, 06O	pretrained line primary (K20b with K21), cold line the ablation (K2 with K_INIT); descriptive C_FULL and PRE table on a non-MAIN panel

What we learned

Gained. The cold federated arm improved by a factor of 4.58 on CALIB, from 0.023473 to 0.107562; every step was registered before it was read and selected by a reviewed tool (Table 5.15). The warm line was fixed with an explicit design reason (frozen tables protect θ_{pre}), and the sampler and accountant mismatch was caught before any DP run.

Did not work. A grid-edge winner (tuning stage 1a) forced a re-run after six MAIN runs had been started on it. FedAvgM, large server rates and four local passes lost to the incumbents. With the fresh client AdamW, FedAdam peaked and fell, and full-model warm starts collapsed. The budget of 12 passes over the training data was not run.

Caution. All gains are CALIB peaks. The constant $\tau = 10^{-3}$ was screened once, with a client optimiser that was later replaced, and it was not screened again.

5.7 Benchmark design and the cold-start chain

The REES46 study could say how a federated recommender behaves on one data set, but not whether the recipe is specific to it, and it could not be compared with published work. On 1 October 2026, while the pretrained line was training, the study lead therefore asked for external benchmarks that would give “the strongest results”. This section reports how the benchmark study was designed before any benchmark run,

what the independent design review changed, and how the cold-start runs of REES46 were scheduled in parallel. No benchmark value and no REES46 MAIN, VALIDATION or TEST value was read in this period; the numbers below are counts, protocol checks and timings (*descriptive*).

5.7.1 Choosing the data sets and cutting the scope

The first plan listed four data sets, two evaluation tracks and many comparison systems, and two external reviews proposed about ten further components (knowledge distillation, text features, more models). The study lead cut them: no method that was not used on REES46 is added to the benchmarks. The study therefore runs only the REES46 recipe, takes one lock and one TEST look per data set, and writes nothing into any REES46 file. Wave 1 uses three Amazon data sets (Beauty, Sports, Toys [10]) and MovieLens-1M [12] in the form released with eSASRec [18]; our copies are byte-identical to that release (8 of 8 file hashes). Wave 2 uses Video Games and Baby Products from Amazon Reviews 2023 [11] (official 5-core files, last item held out); they have no published reference on this split, so they are compared only against our own arms and are descriptive by rule. Table 3.5 lists the six data sets; Table 5.18 lists the candidates that were dropped, with the measurement that decided each case.

Table 5.18: Benchmark candidates dropped before any run (*descriptive*). Counts are MEASURED. Source: `BENCHMARK_PLAN_V1.md`, `FABLE_AUDIT_BENCHMARKS_20261001.md`.

Candidate	Measurement that decided it	Decision
Amazon 2023 All_Beauty	253 users at 5-core	dropped
Taobao, full	1,708,530 items; about 875 MB of item tables on a phone against 83 MB on REES46	dropped
MBHT Tmall (`ijcai_beh`)	782,694 items; about 400 MB of phone item tables	dropped
MBHT Taobao (`tmall_beh`)	a training-instance dump: 462,945 rows, 437,186 session ids, 108,437 exact duplicates, 407,478 chain heads against 147,892 users in the source paper	sessions cannot be clients
Amazon 2023 Beauty and Personal Care	729,576 users; 75,998 federated rounds; a tenth of an epoch of `C_FULL` took 2,986 s (about 8.3 h per epoch on 16 threads)	dropped after timing

5.7.2 Protocol and the reproduction of published baselines

The protocol is leave-one-out with full-catalogue ranking: every item is a candidate except those the user has seen, ties go to the smaller item identifier, and a test target that never occurs in training counts as a miss. NDCG@10 is primary. Model selection uses only an inner validation item (the last item of the training history), and the TEST item is scored from the full training history without retraining. This is the protocol of the published full-ranking tables, so published results on identical splits can be compared without running the published methods. Most federated recommendation papers report sampled metrics, which are not comparable with full ranking [6], and none of the seven federated or on-device papers checked in the design review used these splits. Leave-one-out shares users between training and test and can leak at the population level, whereas the temporal protocol of REES46 does not; benchmark ratios are therefore never pooled with REES46 retentions. We checked the copy of data and protocol: three deterministic baselines of the Spotify capacity-probe paper (MC, SeqRules and PCTM) [19], run on our

four Wave-1 splits, reproduced 12 of 12 NDCG@10 cells of its Table 1 to four decimals. These models are comparison baselines and not part of our method.

5.7.3 Amendment 07, the design gate and the runner

The draft amendment went to the independent design gate. The verdict was to register with fixes (R1 to R9), appended as section B7. The main fixes were these. Federated clients are all users of the data set (the 15% pretraining band keeps its devices; consent covers only the server-side copy), and with 64 clients per round, 10% drop-out and one pass the number of rounds is T = 630, 2,330, 2,023 and 3,709 for ML-1M, Beauty, Toys and Sports. The DP mechanism is the REES46 mechanism unchanged (Poisson rate $q = 2^{-7}$, T = 384, z = 0.5642) with a fixed denominator of 47, 175, 152 and 278 clients per round on the same four data sets. Training uses the training file minus the inner-validation row for every arm. C_FULL is the width-256, three-block model on all users, and PRE is the device-size model on the pretraining band at its endpoint of 6.0 passes over the training data. A claim against a published single run is made only if the difference exceeds 0.005 NDCG@10 with every seed on the same side; otherwise it is stated as "on par". A secondary protocol (the tuned protocol) explores a small grid of maximum sequence length and dropout (and, for the central arms, of training budget) and is never mixed with the headline protocol (the matched-recipe protocol), which uses the REES46 recipe unchanged.

The gate also recorded two warnings that proved relevant. The DP mechanism does not transfer unchanged: with 47 to 278 clients per round the same (q, T, z) gives 4 to 22 times the relative noise of REES46. And C_FULL at 6.0 passes over the training data may sit below published central models for budget reasons alone. The gate also measured the coverage of the pretraining band: a seeded 15% band sees 67.2%, 66.0%, 65.9% and 92.7% of the items of Beauty, Sports, Toys and ML-1M and covers 75.2%, 74.5%, 71.9% and 99.8% of their holdout targets. A frozen model cannot rank targets outside the band, so the frozen warm arm is capped on the Amazon sets. The choice between the frozen and unfrozen warm arm (FA_WARM*) was therefore left to the registered rule (higher inner-validation NDCG@10, ties to the frozen arm) and not to expectation.

The runner was written by AI worker agents and reviewed by separate AI agent instances; no human expert reviewed the code. The first review failed it (one blocker, seven major findings), and the defects in Table 5.19 are the best evidence in this study that independent review changed the design. After Revision 3 the amendment was registered and Wave 1 was launched on the CPU at 23:18 UTC: 234 jobs, at most six slots of 16 threads, each job write-once, a failure skipping its dependents, and the plan stopping before any lock or TEST look.

5.7.4 The cold-start chain and amendment 06P

In parallel the REES46 line continued. Both seeds of the cold FA arm finished on 1 October (by 20:07 UTC) and were sealed with their values unread. The cold re-runs of the arms stopped by the reallocation

Table 5.19: Specification and code defects caught by review before the first benchmark run (*descriptive*).

Defect	Found by	Consequence if undetected
Fine-tuned arms rebuilt the scoring context from the inner sequences, and the last user raised an index error after the TEST file had been written	first runner review (blocker)	the single TEST look could have been lost
DP sampling at rate m/N instead of exactly 2^{-7}	first runner review	sampler and accountant inconsistent, as in 06C
"The registered z" is valid only if (q, T, δ) equal those of REES46	design gate	different noise and different ε
The strict lock refused single-seed grid cells; clip records not pinned; reruns overwrote result files	first runner review	unusable lock, lost evidence
ML-1M validation population 6,039 instead of 6,038 (the tail must be the session length plus one, 201)	second review	wrong denominator
Population of the clip calibration ambiguous (band or clients)	design audit	wording withdrawn and corrected

of Section 5.6 (personalised FedAvg, FedProx, 8-bit uploads, and the cold DP chain) were queued so that the cards would never sit idle. These N33 runs from random weights are the registered cold runs; the stopped directories stay marked STOPPED_REALLOCATED and are refused by the analysis tool. Under the study lead's delegation, the AI operations agent and the design gate also decided to run the cold non-noised twin and the cold DP arm for both seeds, so that the contrast K12 keeps its two registered seeds. The last cold run finished on 3 October.

Amendment 06P (registered with no MAIN or TEST value read) aligned the contrasts with the pretrained line. It fixed FA_WARM* to be FA_WARM_FROZEN in K20, K20b and K21; made K20b and K21 the primary pair, read unadjusted; made K2 a secondary contrast; and added K_INIT (FA_WARM_FROZEN minus FA), the warm-line variants K4w, K5w, K12w, K13w and K14w, and the fine-tuning contrast P5. A counting error (the text spoke of a Holm family of 19 while the list had 18 members) was caught by the thesis-draft AI worker agent and fixed by an erratum. The contrast file holds 33 contrasts: 18 secondary contrasts in one Holm family [51], the two primary contrasts outside it, and a tertiary band of 13 read without adjustment. The amendment also fixed what would be run before the freeze (INT8 for K16 on the CPU, the fine-tuning selection and runs for FT_FA, and K9b on users the base model never saw) and what was declared not run: the DP-sensitivity run, the budget of 12 passes over the training data, the device panels on sparse, heavy and truly cold users, and the truncated-history studies. Two engineering fixes closed the period: the INT8 job for K16 had refused because the central device-size model was trained on an extended feature view while the tool built the default view (the tool now builds the module from the checkpoint's recorded view), and the code tree `code_ds3r7` was deployed beside `code_ds3r6` with identical tiny-run digests for all seven federated arms.

What we learned

Gained. A benchmark study whose splits are byte-identical to a published release, whose baselines are reproduced to four decimals, and whose design was reviewed before the first run: nine fixes to the draft amendment and the defects of Table 5.19 removed before any TEST look existed.

Did not work. Five candidate data sets could not be used as federated benchmarks, and an early plan with about ten extra components was cut. The DP mechanism transfers to small data sets only with much more relative noise.

Carried forward. The frozen warm arm is capped on the Amazon sets by the band coverage, and the central reference at 6.0 passes over the training data may be under-trained against published models; both are tested in Section 5.10.

5.8 MAIN completion, the VALIDATION read and the follow-up analysis design

At 23:41 UTC on 1 October the pretrained line was still training and nothing had been read. This section covers the next 41 hours: the last MAIN runs, the gates that guarded the one VALIDATION read, what the read said, and the design of the follow-up study that the read made necessary. We call the registered REES46 analysis of the previous sections *the main analysis* and the registered follow-up designed here *the follow-up analysis*.

5.8.1 Finishing the MAIN runs and the endgame gates

The four warm MAIN runs (FA_WARM and FA_WARM_FROZEN, two seeds each) finished by 03:22 UTC on 2 October; the fine-tuning arms, the warm DP chain (09:22 UTC), the whole warm method set (20:22 UTC) and the cold re-runs followed, and the last MAIN run (cold FA_Q8, seed 2026) finished on 3 October, before the gate chain started. Every output was sealed at completion with its value unread. The INT8 job for K16 exceeded its two-hour limit on the full population under the benchmark load, so the registered fallback fired: 16,384 users and 206,445 decisions, disclosed; its latency was not measured.

The VALIDATION read was designed as a single, irreversible step, guarded by five scripts run in order (Figure 3.5): D freezes the analysis tools and contrasts, E checks them against the data without reading a metric, F reads once, G locks every method that may later be scored on TEST, and H builds the TEST manifests. The design gate approved the scripts with fixes, and the study lead authorised the AI operations agent to run D to H on the condition that a refusal stops the chain and nothing is improvised around a gate. Three refusals followed before the read (Table 5.20). All three were tool-definition defects and none was a data problem; no metric value was read in any of them. Both defects of step E had been masked by unit-test fixtures whose numbers happened to agree, and after the second one a value-free dry check of the candidate tool on the read-only block became a step before the next freeze.

5.8.2 The single VALIDATION read

Every number in this subsection is (*validation*). The population is MAIN_VALIDATION: 40,764 users with at least one evaluation row (519,714 decisions). The checkpoint rule is PB, the best of 12 marks, which is optimistic; the intervals are 95% paired user-bootstrap intervals (1,000 resamples, seed 20260926). Only the confirmatory test evaluation of Section 5.10 gives unbiased numbers.

Table 5.20: The gate chain of 3 October 2026 (*descriptive*). Times are UTC.

Step	Time	Outcome
D, run 1	13:58	refused: the freeze script loaded the analysis tool without registering the module, so a data class failed; one-line fix; freeze v1 written at 14:00 and later superseded, never read
E, run 1	14:04	refused: the tool compared the numbers of users with at least one evaluation row (40,764; 10,070; 30,694) with the registered manifest counts (51,211; 12,715; 38,496), two different registered quantities; the tool now enforces the manifest counts and reports the evaluation counts beside them (hash `1d50f7f2` to `d0247370`); freeze v2 superseded, never read
E, run 2	14:30	refused: a check required the number of clip norms to equal the number of calibration rounds, whereas the registered record has 20,625 norms over 20 rounds (`9a3b2c2a`); dry check passed; freeze v3 (`59c83c86`) written at 14:59
E, run 3	15:03	passed; no value read
F	15:04–15:14	the single VALIDATION read, exit code 0; from here every new look is post-read exploratory
G	15:27	lock `04a3cd19`: 56 federated checkpoints sealed, 131 arms listed, 9 items declared not run, 5 TEST manifests to build
H	15:30–15:36	gate passed, five TEST manifests built (MAIN, unseen holdout, sparse, heavy, truly cold), 0 failures, no model scored; the study lead put the TEST look on hold

Table 5.21: Headline contrasts on the VALIDATION read (*validation*). Client-macro MRR@20, PB, two seeds; retention $R(a/b) = 100\,a/b$. Source: `VALIDATION_v3/REPORT.md`, HANDOVER/04.

ID	Contrast	Value	95% CI
K20b	R(FA_WARM_FROZEN/C_FULL), primary	88.81	[88.27, 89.36]
K21	FA_WARM_FROZEN − PRE, primary	−0.000433	[−0.000758, −0.000101]
K20c	R(PRE/C_FULL)	89.03	[88.49, 89.57]
K_INIT	FA_WARM_FROZEN − FA	+0.033704	[+0.032688, +0.034760]
K9c	FT_PRE − PRE	+0.003976	[+0.003329, +0.004640]
K2	R(FA/C_BIG), cold federated line	86.07	[85.56, 86.62]

The absolute values put the contrasts in scale. The warm frozen-table arm scored 0.172465 and 0.172461 for the two seeds, the pretrained model 0.172896, the full-data cloud model 0.194192 and 0.194716, and the pretrained model fine-tuned on each device 0.176993 and 0.176752. The reading (INFERENCE) is that the deployable pretrained system keeps about 89% of the full-data cloud model, that pretraining is the largest lever (K_INIT), that per-device fine-tuning adds a small reliable gain (K9c), and that on this recipe the federated rounds add nothing over the pretrained model: K21 is slightly negative, with an interval below zero. A diagnostic on the 30,694 MAIN users outside CALIB gave K21 = −0.000465, so CALIB lying inside MAIN does not drive the result. Freezing the tables cut the per-visit traffic from 165.7 MB to 2.11 MB (ratio 0.0127), at the price of a one-time download of 81.78 MB (*validation*). The cold DP-labelled arm kept 21.40% of C_BIG against 21.68% for its non-noised twin: the Poisson schedule of 384 rounds, not the noise, is what is weak on the cold line (INFERENCE).

5.8.3 Was the warm recipe at fault?

The study lead asked whether the warm federated training had been done inappropriately. Reading the registered selection file of tuning stage 8, the AI operations agent found two facts (MEASURED). The screen was a 2×2 grid (client factor $\{1, 4\}$, server factor $\{1, 0.1\}$) run on the cold-start recipe of tuning stage 4, and its winner sat at the smallest server rate, a grid edge; on VALIDATION every frozen-table warm arm scored 0.1724 to 0.1727 (the unfrozen FA_WARM 0.1716 and 0.1717) against 0.1729 for PRE (*validation*). A diagnostic of the checkpoints, run after the read and without any evaluation (D1, (*exploratory*)), measured how far each model had moved from θ_{pre}: the relative parameter distance was 0.53% at the best checkpoint and 0.67% at the endpoint for FA_WARM_FROZEN, 0.09% and 0.26% for FA_WARM, 0.013% and 0.077% for the DP-labelled DP8_FROZEN, and 0.004% and 0.030% for FA_1024_WARM_FROZEN. The change was concentrated in small gate, bias and normalisation tensors (up to 20 to 45%), whereas the large attention and feed-forward matrices barely moved, and the best checkpoints were selected near round zero. The first reading (INFERENCE) was that FedAdam at a small server rate moves each parameter by about that rate per step whatever the gradient; the mechanism was measured later and is refined in Figure 5.7. What D1 already showed was enough: the registered recipe left the pretrained model almost unchanged, so K21 ≈ 0 is a statement about the recipe and not a proof that federated training cannot add value. The study lead decided to solve the weaknesses by design and code, to run nothing, and to hold the TEST look so that both analyses could share one test evaluation.

5.8.4 The follow-up analysis design

The follow-up analysis is a separate pre-registered study with its own contrasts Q1 to Q14, bootstrap seed 20261004, lock and script freeze. Its question is whether federated training on the users' data adds quality once a pretrained model is shipped, and at what communication, privacy and device cost. A firewall protects the main analysis: selection uses CALIB only, the main-analysis code stays byte-identical with every new option off, and frozen tools are imported and never edited. To make a flat result interpretable, a

no-op test is defined on the trained subset (below 1% relative distance, no large tensor group moved by 1% or more, and a top-20 Jaccard index of at least 0.95 on a functional probe), and every run logs per-round update norms.

The design has three controls and a recipe grid. C0 is θ_{pre} itself. C1 is a central fine-tune of θ_{pre} on MAIN (all weights or frozen tables, learning rate screened over 10^{-5} to 3×10^{-4}). C2 is the device-size model trained centrally on the pretraining and MAIN users, a size-matched reference, because C_FULL is wider and C_FULL − PRE confounds capacity with data. The grid keeps the settings of the main analysis (pretrained start, 64 clients per round, 10% drop-out, one pass, 6.0 passes over the training data) and varies the server optimiser and client rate: WA with frozen item tables (server FedAvg, FedAvgM or FedAdam at two rates each, client SGD rate in $\{0.003, 0.01, 0.03\}$, 18 runs plus outward extensions and the main-analysis incumbent), WB with trainable tables at a reduced rate (one variant with sparse item uploads), WC with low-rank adapters or a personal vector, and a DP chain on MAIN. Selection uses CALIB_VALIDATION with a tie band of 0.001 (about three paired standard errors), preferring fewer upload bytes, then fewer trained parameters, then smaller steps; a futility rule skips WB and WC if neither C1 nor the best WA gains 0.001 over PRE on CALIB. The selected recipe yields the tuned warm-start federated model.

The primary contrast Q1 is the tuned warm-start federated model minus PRE. Its reading is registered: SUCCESS if the lower limit of the 95% interval is above zero and both seeds are above zero; EQUIVALENT if the 90% interval lies inside ±0.0005 (two one-sided tests [52]); FAILURE if the upper limit is below zero; otherwise INCONCLUSIVE. The minimum detectable effect is about 0.00047, while the plausible gain was about 0.001 (INFERENCE), so the design promises no positive result (disclosure D-h). The design gate allowed one TEST evaluation to score both analyses, provided the follow-up analysis has its own lock and freeze before the first TEST re-evaluation and nothing is fetched before both result files exist; a second evaluation is refused. Two decisions of the study lead shaped the scope: no December 2019 data exist on the server, so there is no fresh-period replication (Q14 is reported as not available); and the size-matched control C2 was run, dropped and restored within one day before any run of the follow-up analysis (contrast file versions v1, v2 and v3), a history that is disclosed.

5.8.5 Building the follow-up analysis

The follow-up code was written as packets by AI worker agents and checked by separate AI reviewers: 13 verdicts by our count (one FAIL, one PASS and eleven PASS-WITH-MINORS). The FAIL shows what review bought. The tests of the selection tool used a monkeypatched reader that hid the fact that every real follow-up run would have been refused, and its anchor flag used 0.0005 instead of the registered 0.001; both were found before any run. Byte-identity was proved before any run as well: 56 pairs of arm specification and run configuration, 13 pins and 21 CALIB arm-specification digests of the main analysis reproduce exactly, and tiny trajectories were identical before and after the changes (22 trajectories and 4,704 configurations after the final wiring). The first run batch started at 18:24 UTC on 3 October; its

bit-identity receipt matched, so the main-analysis incumbent recipe was reused and not re-run.

Operations taught two lessons. Raising concurrency from 4 to 26 follow-up jobs lifted GPU utilisation from 12–21% to 65–91% but not the aggregate throughput (about 54 against 50 rounds per minute), because a single federated run is latency-bound and the cards time-slice; we measured before adding more load. An unattended chain controller stalled for about seven hours at the first C1 selection, because it allowed one extension round per step while the registered rule extends each variant once; the stop was logged but raised no alert, and the fix was both the logic and the rule that a stop marker needs an alert. In parallel, the code was ported to the public repository as seven packages (pull requests #96 to #102, merged on 3 October between 19:34 and 20:49 UTC by git time) by an AI worker agent acting through the study lead's account; the details and the AI-use statement are in Chapter G.

What we learned

Gained. Every REES46 MAIN run was complete and sealed before any value was read; the gate chain read VALIDATION exactly once; and the read produced a precise question (why did K21 sit at zero?) together with a registered, firewall-protected design for answering it.
Did not work. Three tool-definition defects refused the chain before the read, and the warm screen had been run on a cold-start recipe at a grid edge. Concurrency did not raise throughput, and a controller stalled unattended.
Caution. The VALIDATION values are best-of-12-mark values. They are not final, and they were used for nothing except the decision to design the follow-up analysis.

5.9 Follow-up analysis runs, device measurements and the membership audit

The follow-up analysis had been designed, reviewed and ported to the public repository, but no follow-up model had been trained on MAIN (Section 5.8). This section covers the next 31 hours, from 14:50 UTC on 4 October to 21:20 UTC on 5 October 2026. In that time the confirmatory MAIN runs of the follow-up analysis were trained, the Part-B analyses that survived a budget cut were run, and the single TEST evaluation was prepared. The section contains no confirmatory result. Every number is a selection read on the calibration subset, an engineering measurement or an audit status (*descriptive*), and the section ends where the test evaluation begins.

5.9.1 Part B reduced

The design of the follow-up analysis had separated a critical path (Part A: the controls, the recipe grid, the tuned warm-start federated model at two seeds, the fine-tuning arms and the DP chain) from a Part B that must never delay the test evaluation (a secure-aggregation simulation, DP-FTRL, local DP, three audits and the device measurements). At 14:50 UTC on 4 October the study lead accepted a proposal to cut Part B; the cut was declared before any follow-up result existed and recorded in the method lock. Three items were run: the membership-inference audit, a user-level audit of the final models; E4, a simulation of secure

aggregation; and the device measurements. Five items were declared not run: DP-FTRL (contrasts Q12a and Q12b), the E2 canary or one-run audit [42], the update-leakage audit, the feasibility of local DP, and the optional secure-aggregation arm on MAIN (Q13). The recorded reason is that Part B was reduced to fit the schedule and the budget. The lock therefore lists 26 sealed (arm, seed, rule) triples, 2 declared-not-run arms and 10 reused runs of the main analysis. Some of these items (update leakage, and the design ceiling of the canary, whose restart the study lead stopped; secure aggregation stays a simulation) were taken up in the closure work of 12 to 17 October, and their results appear with the privacy redo in Section 5.14.

5.9.2 Selection, futility and the confirmatory MAIN runs

The WA grid and the central fine-tune C1 were screened on CALIB before any MAIN run. Table 5.22 lists what the screen produced. All 22 cells of the WA grid lie within ±0.00012 of the pretrained model's own CALIB score, far inside the registered tie band of 0.001, and the best central fine-tune is −0.00004 below it. The registered futility rule therefore returned its decision word, FUTILITY_SKIP_WB_WC, so the trainable-table and low-rank variants WB and WC were skipped by rule and the WA winner became the tuned warm-start federated model.

Table 5.22: Follow-up selection on CALIB and the facts of the confirmatory MAIN runs (*descriptive*). CALIB values are client-macro MRR@20 on end-to-end decisions. Source: `HANDOVER/05_PHASE2_FOLLOWUP.md`.

Quantity	Value
PRE on CALIB (anchor)	0.171302
Spread of the 22 WA cells around the anchor	within ±0.00012
Registered tie band	0.001 (about 3 paired standard errors at about 10,070 CALIB users)
Best central fine-tune C1 against the anchor	−0.00004
Selected recipe W*	FedAdam with server rate 3×10^{-6} (`fedadam|eta=3e-06|c=0.1`); 263,822 trained and uploaded parameters
MAIN budget	13,654 rounds (6.0 passes over the training data); CALIB 3,414 rounds
DP chain	Poisson sampling, m = 1,024 clients per round, T = 384, noise multipliers z = 0.5642 (nominal ε = 8) and z = 1.3094 (nominal ε = 1), clip S = 0.27539
Jobs of the confirmatory batch (RUN-3)	14 of 14 completed, none failed (10 launched at 17:42:50 UTC on 4 October, plus 4 C1 runs on MAIN)
Realised budget at the endpoint of W* (passes over the training data)	6.005918 (seed 2026), 5.999705 (seed 2027)

Because every cell sat inside the tie band, the registered tie order (fewer upload bytes, then fewer trained parameters, then a smaller effective server step, then a smaller client rate) selected the smallest server step. The tuned warm-start federated model therefore stays close to θ_{pre} by construction. We disclosed this before the test evaluation, and Q1 is read exactly as registered; the cause of the flat grid is the subject of Section 5.11. The confirmatory batch started at 17:42:50 UTC on 4 October with the tuned warm-start federated model at both seeds and the clip calibration, followed by the non-private twin and the DP arms that wait for the clip. The seed-2026 run of the tuned warm-start federated model finished at about 08:14 UTC on 5 October, the DP and non-private twins between 15:19 and 15:43 UTC, and the last fine-tuning run (64 shards of 4.1 to 5.3 minutes) at 20:16 UTC. The endpoint of the seed-2027 run of the tuned

warm-start federated model fell at 5.999705 passes over the training data, a hair below 6.0, which mattered at the test evaluation (Section 5.10).

The study lead's question at this point was whether warm federated learning looked useless because it had been done badly. The answer was declared exploratory work, defined before any result and kept under separate roots and labels, never part of the test evaluation. It had two strands. Eight WB and WC cells were paused, resumed and finally terminated (resumably) to protect the test evaluation, and no result from them is reported. The second strand asked what a weaker server-side pretrain would change, and became the 100k-user pretraining variant: the pretraining set was cut to a band of 100,000 users (1,980,666 eligible examples, PRE ranks $[131{,}072, 231{,}072)$), pretraining started at 19:16:47 UTC on 4 October and the federated run at 20:15:32 UTC. A trial of four exploratory cells slowed the critical-path fine-tuning from 246 to 390 seconds per shard (+59%) and was paused under the rule that the core runs go first. The values of the 100k-user pretraining variant stayed unopened until both analyses of the test evaluation existed; Section 5.13 reports them.

5.9.3 The membership audit and the secure-aggregation simulation

The membership-inference audit asks whether the final models reveal which users were in the training window. It scores membership by the model's loss on the user's own training rows against a matched set of non-members, in the manner of membership-inference attacks [38, 39] and of their use in DP auditing [41], with a validity gate: the audit counts only if the pretrained model θ_{pre}, which never saw the MAIN users, shows no membership signal. The gate was applied to the raw-loss area under the curve, because the calibrated score of PRE is identically zero, which made the raw loss the only non-vacuous reading. Table 5.23 gives the status words of the six audited targets. The audit has a known power limit: with 5,000 non-members in the second half of the split, it could certify at most about 7.2 for the estimated ε, so the nominal $\varepsilon = 8$ model could never fail it.

Table 5.23: Membership-inference audit of the final models, status words on TRAIN-window rows (*descriptive*) (the register files the audit as exploratory).

Target (final model)	Status	Note
Validity gate on PRE (raw-loss AUC)	VALID	required for the audit to count
PRE endpoint	null control	later re-scored VALID_NULL
P2_WSTAR, seed 2026, best checkpoint	NO_MATERIAL_LEAKAGE	the model had barely moved from θ_{pre}
P2_WSTAR_DP8, seed 2026, endpoint	PASS	cannot fail: power ceiling about 7.2 < 8
P2_WSTAR_DP1, endpoint	PASS	nominal $\varepsilon = 1$
P2_C1_FT_CENTRAL_FULL, seed 2026, best checkpoint	MATERIAL_LEAKAGE	a central fine-tune on MAIN users' raw data

The only target that leaks is the one that learned: the centrally fine-tuned model. The federated target reads NO_MATERIAL_LEAKAGE, but this status is a statement about a model that did not move, and we do not read it as evidence that federated training protects. When the audit was repeated later on models that move ((*exploratory*)), the cold federated winner reached an AUC of 0.747, the DP8 arm brought it to

0.502, and the warm winner of the 100k-user pretraining variant read 0.537 (Section 5.14). The same lesson holds for the DP statuses: a model that is nearly inert gives the noise nothing to hide.

E4 simulated the secure aggregation of Bonawitz et al. [36] with the sparse k-regular masking graph of Bell et al. [37], on CPU only. It is a simulation of the protocol's arithmetic and its failure rule, not production cryptography. Table 5.24 shows that masked and plain sums are identical in every completed cell, that the upload overhead is below one percent, and that the protocol fails exactly when too few clients survive.

Table 5.24: E4 secure-aggregation simulation on the update vector of the tuned warm-start federated model (*descriptive*). Simulation only, not production cryptography. Source: `E4_SECAGG.json` and `E4_SECAGG_FULLMODEL.json`.

Quantity	Value
Cells	27 = {full graph, k-regular with $k = 32$, $k = 16$} × drop-out {0, 0.1, 0.3} × 3 seeds
Update dimension, clip	d = 263,822 trained parameters; $S = 0.27539$
Bit-identity violations	0 (masked sum equals plain sum)
Upload overhead against a plain upload	×1.002 to ×1.009
Server unmasking time	4.7 to 6.7 s per cell (loaded CPU)
Failed cells	1 ($k = 16$, drop-out 0.3, one seed): survivors fell below the threshold of 43, a measured outcome by design
Unit tests	26 of 26 for E4; the membership-inference audit and E4 together 73 passed on the server
Full-model vector, side run during the test evaluation (*exploratory*)	dimension 20,708,708; 26 cells ok, all bit-identical

5.9.4 Device measurements on the tuned model

The device measurements closed the loop on the deployment question: what does the tuned model cost on a real device? The tuned warm-start federated model (seed 2026) was exported to ONNX [47] in FP32 (82,863,741 bytes) and as an INT8 dynamically quantised copy [46] (21,243,628 bytes), a payload ratio of about 3.90 (our division of the two byte counts). Both ran in the browser through WebAssembly [49] in Microsoft Edge, with 1,000 timed queries at batch size 1, on a laptop (D1), a phone (D2) and shared server CPUs (SERVER_BROWSER). Table 5.25 gives the ranges over sessions; 68 runs completed and none failed (12 on the laptop, 8 on the phone, 48 on the server).

Table 5.25: Warm latency (ms) of the tuned model in Edge via WebAssembly, range over sessions, 1,000 timed queries, batch 1 (*descriptive*). The server rows run on shared CPUs and are not a consumer device; the server p95 was not recorded in the log row. Source: `DEVICE_D1_WSTAR`, `DEVICE_D2_WSTAR`, `DEVICES.json`.

Device	Threads	FP32 p50	FP32 p95	INT8 p50	INT8 p95	Cold start FP32 / INT8
Laptop (D1)	1	5.02–5.09	7.08–7.40	2.69–3.60	5.18–5.34	834–1,243 / 664–786 ms
Laptop (D1)	4	2.21–2.40	3.34–3.37	1.56–1.71	2.61–2.80	as above
Phone (D2)	1	2.65–2.98	3.97–5.13	3.48–3.92	5.32–6.32	4.8–5.3 / 2.5–2.6 s
Phone (D2)	4	2.17–2.21	4.73–6.18	2.38–2.42	5.75–7.07	as above
Server (shared)	1	4.55–4.57	–	4.26–4.50	–	1.0–1.5 s
Server (shared)	4	2.55–3.06	–	2.79–3.41	–	as above

The table supports three readings, all descriptive. Warm latency is a few milliseconds on every device. INT8 is not uniformly faster: on the laptop with one thread it is faster than FP32 (2.69–3.60 against 5.02–5.09 ms), on the phone it is slower (3.48–3.92 against 2.65–2.98 ms), and on the server it is about equal. INT8 always starts faster, because the file is smaller: 664–786 against 834–1,243 ms on the laptop and 2.5–2.6 against 4.8–5.3 s on the phone. The quality cost of INT8 is a separate question that the confirmatory test evaluation answers (K16, Table 5.27). A later exploratory device run of the four-times-larger cloud model (31 session results, one hung session excluded) is a pointer for the capacity argument and is not tabulated here.

5.9.5 Defects found before the test evaluation

Review again caught defects that would have wasted the test evaluation. The lock tool expected arm labels that the launcher does not write, so every real DP run would have been refused at the seal; the report tool did not admit the labels of the DP chain and would have refused every DP arm at the test evaluation; both were fixed with a run-identity check before any DP run. The first export of the tuned model failed because a bare state dictionary does not satisfy the exporter (incident INC-093); wrapping the weights in the envelope of θ_{pre} fixed it. Finally, a single federated run is latency-bound, so a benchmark filler job (ML-1M) slows it even at low GPU utilisation; the watchdog was rebuilt twice, and its final form holds the filler on each GPU whenever a follow-up job runs there. The study lead's request to parallelise the last fine-tuning run was not carried out: a helper that finished after the main process could have overwritten its completion mark with a partial one, and every helper would have added a launch leg to a sealed run directory that the lock checks. The expected gain was about two hours.

What we learned

Gained. The follow-up analysis trained completely and reproducibly (14 of 14 jobs, none failed) and was sealed before the test evaluation. The device numbers show warm latency of a few milliseconds on a laptop, a phone and a shared server, and a payload about 3.9 times smaller with INT8. The secure-aggregation arithmetic holds in all 26 completed cells.

Did not work. All 22 recipe cells were indistinguishable from the pretrained model on CALIB, so the tuned warm-start federated model is near θ_{pre} by construction. The audit statuses of the DP arms are weak evidence, since the audit cannot fail `P2_WSTAR_DP8` and the models barely moved. DP-FTRL, the canary, update leakage, local DP and the MAIN secure-aggregation arm were not run.

Caution. Exploratory work shared the server with the critical path, and the 59% slowdown of the fine-tuning shards was the price of mixing them.

5.10 Confirmatory TEST results

Every run in the preceding sections was trained to be scored once. This section reports that scoring: the single confirmatory test evaluation of the REES46 study, which produced the two confirmatory result files

(main analysis and follow-up analysis), and the single TEST look of each external benchmark. The labels follow the registration [7]: (*confirmatory*) for the main analysis, (*confirmatory, follow-up*) for the follow-up analysis, (*confirmatory*) for the benchmark looks under their registered claim rule and (*descriptive*) for the absolute REES46 values and for the Amazon 2023 sets, which were registered as descriptive. Where we explain a result with later, exploratory work we say so and label it (*exploratory*).

5.10.1 The test evaluation

The test evaluation had two attempts. The first, under lock v1, stopped at step 3 of the dry run of the follow-up analysis before any TEST row was read: the seed-2027 endpoint of the tuned warm-start federated model lay at 5.999705 passes over the training data, below the registered 6.0, which exposed a tool-definition bug in the re-evaluation script. The lock was superseded and no TEST row was scored under it. The second attempt, under lock v2, ran from 21:03 UTC on 5 October to 10:09 UTC on 7 October 2026, about 37 hours of wall clock (Table 5.26). The order of work followed the exposure register of Figure 3.5: shas first, then the fetch, then the register entries, and only then was any number read.

Table 5.26: The confirmatory TEST evaluation (*descriptive*). Times are UTC. Source: `HANDOVER/18_EXPERIMENT_REGISTER.md`, rows T0-01 and T0-02.

Time	Event
5 Oct, before 21:03	All main-analysis and follow-up models trained and sealed; every TEST population had a manifest and no model had been scored. Attempt 1 (lock v1) stopped at the dry run; no TEST row read.
5 Oct, 21:03	Attempt 2 starts: lock v2 `b3a2f62a`, register `d5a9e1a6` (9 entries, 0 errors), code tree `code_p2r4`.
6–7 Oct	Step 7 (on-device arms, 14 launcher parts): the measured sequential plan would have taken 3 to 4 days (inference). Eleven parts ran in parallel as helpers under a gated amendment with seven conditions; the step finished in 21.2 hours (09:34 on 6 Oct to 06:44 on 7 Oct), all helpers with return code 0.
7 Oct, 06:44	Step 8 refused at launch (return code 2, no job launched): the code tree was writable. After `chmod -Ra-w`, with the tree digest equal to the lock before and after, it was relaunched at 06:50.
7 Oct, to 10:09	Step 8: 26 jobs (24 re-evaluations and 2 on-device runs), none failed. Steps 9 to 12 produced the two result files (main analysis `28b76492`, follow-up analysis `c44382c5`).
7 Oct, 10:09:20	Test evaluation complete; the result tar was fetched and both shas re-verified. Register entries 10 (main analysis) and 11 (follow-up analysis) were written before any number was read: 11 entries, 0 errors.

The TEST set is the REES46 time window that follows the training window: the later decisions of the MAIN users, plus the unseen-user and fully cold-start panels. The main population has 26,254 evaluated users and 229,742 decisions. The unseen-holdout contrast K9b uses 6,650 users, and the INT8 contrast K16 a subset of 16,384 users. The statistics were registered before the test evaluation. The unit of resampling is the user; the paired bootstrap uses 1,000 resamples [50] with seed 20260926 for the main analysis and 20261004 for the follow-up analysis. The secondary family of 18 contrasts of the main analysis is tested with the Holm step-down at $\alpha = 0.05$ [51]; the primary pair K20b and K21 is read unadjusted outside it, and 13 tertiary contrasts carry only a descriptive band. In the follow-up analysis, Q1 is a success if the lower limit of its 95% interval exceeds zero and both seeds are positive, equivalent if its 90% interval lies inside ±0.0005 in a two one-sided test [52], and a failure if the upper limit is below zero.

Q2 to Q8 form a Holm family of size 7. A retention is 100× the model's client-macro MRR@20 divided by the reference's, and a delta is a difference of client-macro MRR@20.

5.10.2 The main analysis: the key contrasts

Table 5.27 gives the fifteen contrasts that carry the story, quoted from the published contrast file of the main analysis; Figure 5.4 shows the retention contrasts with their intervals. The full set of 33 contrasts is in Chapter B. The Holm family of 18 had 15 members rejected; the three that were not rejected are the DP and 8-bit contrasts on the warm line (K12w 100.00 [99.95, 100.07], K13w 100.03 [99.92, 100.15] and K14w 99.85 [99.72, 99.98]).

Table 5.27: Key contrasts of the main analysis on MAIN TEST (*confirmatory*). Unit: retention in percent (R) or difference of client-macro MRR@20; 95% paired-bootstrap interval; PB = best validated mark, CB = fixed endpoint at 6.0 passes over the training data. Both seeds unless stated. K20b and K21 are the primary pair; K1, K2, K3, K8, K9c, K12, K17, K20c and K_INIT are secondary and rejected in the Holm family; K11, K13, K14 and K16 are tertiary and unadjusted. Source: `results/rees46/confirmatory_phase1_contrasts.csv`.

Id	Contrast	Rule	Point	95% CI	Reading
K1	size cost, R(C_SMALL/C_BIG)	PB	94.69	[94.03, 95.31]	below 100
K2	cold federated model, R(FA/C_BIG)	PB	85.11	[84.30, 85.90]	below 100
K3	federation cost at matched size, R(FA/C_SMALL)	PB	89.88	[89.15, 90.62]	below 100
K8	device-only training, R(LO/C_BIG)	PB	2.10	[1.88, 2.34]	below 100
K9c	on-device fine-tune of the pretrained model, FT_PRE − PRE	PB	+0.002283	[0.001588, 0.003076]	above 0
K11	collaboration value, FA − LO	PB	+0.123378	[0.121064, 0.125358]	above 0
K12	nominal $\varepsilon = 8$, R(DP8/FA_1024)	CB	98.52	[97.70, 99.38]	below 100
K13	nominal $\varepsilon = 1$, R(DP1/FA_1024), seed 2026	CB	94.38	[92.53, 96.00]	below 100
K14	8-bit uploads, R(FA_Q8/FA)	PB	100.11	[99.85, 100.38]	includes 100
K16	INT8 against FP32, C_SMALL, ONNX CPU, 16,384 users, seed 2026	PB	99.98	[99.75, 100.23]	includes 100
K17	cold federated model, R(FA/C_FULL), seed 2026	PB	70.10	[69.19, 70.94]	below 100
K20b	**realistic edge system, R(FA_WARM_FROZEN/C_FULL)**	PB	**88.49**	[87.73, 89.18]	below 100
K20c	pretrained model alone, R(PRE/C_FULL), seed 2026	PB	88.77	[88.03, 89.47]	below 100
K21	**federated training on top of θ_{pre}, FA_WARM_FROZEN − PRE**	PB	**−0.000508**	[−0.000933, −0.000037]	below 0
K_INIT	value of the pretrained start, FA_WARM_FROZEN − FA	PB	+0.033119	[0.031774, 0.034516]	above 0

Read in the order of the deployment stages, the table says the following. The federated model trained from a random start with the registered FA recipe at the settings of tuning stage 4 keeps 85.11% of the large central model on the same users (K2) and 70.10% of the full-data cloud reference (K17); it beats the no-training last-item heuristic by +0.050933 [0.048639, 0.052886] (K11b) and the device-only control by +0.123378 (K11). The matched-size central model keeps 94.69% of the large one (K1), so the cost of shrinking the model is about five points and the cost of federation at that size about ten more (K3). The realistic edge system, FA_WARM_FROZEN (a federated model that starts from the server-pretrained θ_{pre}

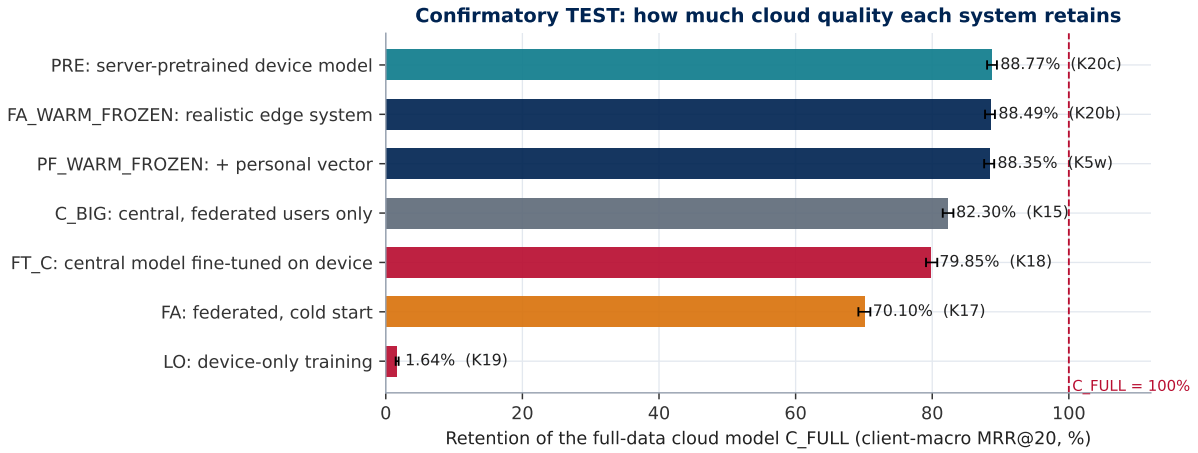


Figure 5.4: Confirmatory retention (100 × model / reference, client-macro MRR@20) on MAIN TEST with 95% paired-bootstrap intervals (*confirmatory*). The reference is the full-data cloud model C_FULL (trained on all 1,348,682 users) unless the contrast names another one. Source: `confirmatory_phase1_contrasts.csv`.

and keeps the item tables frozen), keeps 88.49% of C_FULL (K20b), and the pretrained model alone keeps 88.77% (K20c). The pretrained start is worth +0.033119 against the cold federated model (K_INIT), so the largest lever in the whole study is the server-side pretrain.

The difference that matters for the research question is K21. Federated training on top of θ_{pre} changes MRR@20 by −0.000508 [−0.000933, −0.000037]: the point estimate is slightly below zero and the interval excludes zero, but the effect is negligible in size (the absolute MRR@20 of both models is about 0.160, Table 5.29). The sign confirms the VALIDATION read, which was also below zero. The only positive deployable gain is the on-device fine-tune: +0.002283 over the pretrained model (K9c), and +0.002045 [0.001324, 0.002807] when it is applied after warm federated training (contrast P5). Table 5.28 sets the biased VALIDATION values next to the unbiased TEST values; every TEST value is a little lower, the intervals overlap, and no sign changed.

Table 5.28: The same contrasts read twice. The VALIDATION values (*validation*) are best-of-12-mark reads and are optimistic; the TEST values (*confirmatory*) are the unbiased ones. Source: `HANDOVER/04_RESULTS_VALIDATION.md` and `confirmatory_phase1_contrasts.csv`.

Contrast	VALIDATION (*validation*)	TEST (*confirmatory*)
K2 R(FA/C_BIG)	86.07 [85.56, 86.62]	85.11 [84.30, 85.90]
K20b R(FA_WARM_FROZEN/C_FULL)	88.81 [88.27, 89.36]	88.49 [87.73, 89.18]
K21 FA_WARM_FROZEN – PRE	−0.000433 [−0.000758, −0.000101]	−0.000508 [−0.000933, −0.000037]
K9c FT_PRE – PRE	+0.003976 [0.003329, 0.004640]	+0.002283 [0.001588, 0.003076]
K_INIT FA_WARM_FROZEN – FA	+0.033704 [0.032688, 0.034760]	+0.033119 [0.031774, 0.034516]

In absolute terms (seed 2026, MAIN TEST, PB, (*descriptive*); Table 5.29) C_FULL scores 0.180388, the pretrained model PRE 0.160129, FA_WARM_FROZEN 0.159797 and the cold FA 0.126459, against 0.075569 for the last-item rule LAST.

Privacy and compression arms cost little in this registered recipe: nominal $\varepsilon = 8$ retains 98.52% of its non-private twin (K12; per seed 100.72 and 96.31, so the result is seed-sensitive), nominal $\varepsilon = 1$ retains

Table 5.29: Absolute client-macro MRR@20 on MAIN TEST, seed 2026, PB (*descriptive*). Source: `RETENTION_RESULT.json` of the test evaluation of the main analysis (per-seed numerators and denominators) and `p2_out_test_v1/REPORT.md`.

Arm	MRR@20	Arm	MRR@20	Arm	MRR@20
C_FULL	0.180388	PRE	0.160129	FA	0.126459
C_BIG	0.14846	FA_WARM_FROZEN	0.159797	FA_Q8	0.126703
C_SMALL	0.141164	FT_PRE	0.162496	FA_1024	0.034199
FT_C	0.144046	FT_FA_WARM_FROZEN	0.161629	LAST	0.075569
FT_FA	0.130332	FA_1024_WARM_FROZEN	0.159787	LO	0.002964

94.38% (K13, one seed), 8-bit uploads retain 100.11% (K14) and INT8 inference retains 99.98% (K16). On the warm line, the DP and 8-bit contrasts are all within 0.15 points of 100. These small measured costs are partly a symptom of the same cause as K21: a federated model that barely moves from its start gives noise and quantisation little to destroy.

Limitations

No formal DP guarantee. The legacy DP implementation used a disclosed public seed, so its noise and Poisson-sampling streams can be reconstructed. The utility numbers of K12, K13 and the DP arms are measurements, but those runs do not establish the randomized-DP guarantee; we call the arms DP-labelled and write nominal ε throughout. Secure aggregation is a simulation, not production cryptography. A later update-leakage audit of the aggregate-input attack is a negative finding: the attack advantage is about 2.625 percentage points (95% CI 2.312–2.953), above the fixed threshold of 2 points. The canary design has a ceiling of $\varepsilon \approx 5.213$ and cannot reject $\varepsilon = 8$. These audits belong to the closure work (Section 5.14).

5.10.3 The follow-up analysis: Q1 is equivalent

Table 5.30 gives the contrasts of the follow-up analysis. Every statement of the follow-up analysis carries the disclosure registered with it: its hypotheses were generated after the VALIDATION read of the main analysis, the arms were selected on CALIB, and everything was locked before any TEST value existed.

The primary contrast Q1 is -0.000081 $[-0.000179, 0.000024]$, with per-seed values -0.000075 and -0.000087. Its 90% interval, $[-0.000164, 0.000007]$, lies inside the registered margin of ± 0.0005, so the registered reading is EQUIVALENT (*confirmatory, follow-up*): the tuned federated recipe cannot be told apart from the pretrained model it started from. The Holm family Q2 to Q8 of size 7 rejects none; Q3 and Q4 read above zero by a few ten-thousandths but are not rejected after adjustment. The registered deployment-grade verdict, which requires Q1 to succeed and Q5 and Q7 to be Holm-positive, is DEPLOYMENT-GRADE = False. Q9 shows that the tuned model keeps 88.72% [87.95, 89.43] of C_FULL, close to the 88.77% of the pretrained model alone (K20c). Q10 (headroom capture) is undefined, because the point estimate of the central headroom Q2 is not positive. Q12a, Q12b and Q13 were not run (Section 5.9), and Q14 is not available: no December 2019 data exist on the server, so there is no fresh-period replication.

Table 5.30: Contrasts of the follow-up analysis on MAIN TEST (*confirmatory, follow-up*). Seeds 2026 and 2027 unless stated; PB or CB as shown; Q9 and Q11 are tertiary and unadjusted. Source: `results/rees46/confirmatory_phase2_contrasts.csv`.

Id	Contrast	Rule	Point	95% CI	Reading
Q1	**tuned warm model against the pretrained model, W* – PRE**	PB	**−0.000081**	[−0.000179, 0.000024]	**EQUIVALENT**
Q2	central fine-tune of θ_{pre} on the users' data, minus PRE	PB	−0.000212	[−0.000628, 0.000208]	includes 0
Q3	W* – central fine-tune with frozen tables	PB	+0.000166	[0.000027, 0.000300]	above 0, not Holm-rejected
Q4	W* – FA_WARM_FROZEN	PB	+0.000427	[0.000007, 0.000893]	above 0, not Holm-rejected
Q5	on-device fine-tune: FT_W* – FT_PRE	PB	−0.000006	[−0.000093, 0.000075]	includes 0
Q6	R(W*_DP8/W*_FA_1024), nominal $\varepsilon = 8$	CB	100.00	[100.00, 100.00]	includes 100
Q7	W*_DP8 – PRE	CB	−0.000037	[−0.000097, 0.000020]	includes 0
Q8	size-matched central model on all users minus PRE, seed 2026	PB	+0.000541	[−0.000457, 0.001591]	includes 0
Q9	R(W*/C_FULL)	PB	88.72	[87.95, 89.43]	below 100
Q11	R(W*_DP1/W*_FA_1024), seed 2026	CB	99.99	[99.98, 100.00]	includes 100

Key finding of the confirmatory test evaluation

With a strong server-side pretrain on 1,217,610 users, federated rounds add nothing measurable on REES46. The shipped pretrained model keeps 88.77% of the full-data cloud reference (K20c), the realistic edge system FA_WARM_FROZEN keeps 88.49% [87.73, 89.18] (K20b), and federation on top of the pretrain changes MRR@20 by −0.000508 (K21) (*confirmatory*). The tuned warm-start federated model of the follow-up analysis is EQUIVALENT to the pretrain (Q1 −0.000081 [−0.000179, 0.000024]) (*confirmatory, follow-up*), it keeps 88.72% (Q9) and the registered verdict is DEPLOYMENT-GRADE = False. The deployable gains are the pretrain itself (+0.033119 over a cold federated start, K_INIT) and the on-device fine-tune (+0.002283, K9c). Without a pretrain, federation is what makes the model work at all: the cold FA recipe is +0.123378 above device-only training (K11) and keeps 70.10% of C_FULL (K17).

Why is the federated gain zero? The confirmatory test evaluation cannot say, because it measured outcomes, not mechanisms; the mechanism was measured after the looks and is therefore (*exploratory*). The registered FedAdam adaptivity floor was $\tau = 10^{-3}$, while the client pseudo-gradients were of the order of 10^{-7} (FedAdam [25]; Section 5.11 gives the measurements). On REES46, 99.3 to 100% of update values were below τ, the median update was 6×10^{-8}, and the tuned warm-start federated model on MAIN moved only about 0.06% from θ_{pre} over 13,654 rounds. A tau-fixed warm run ($\tau = 10^{-6}$) minus PRE is +0.00022 [−0.00011, 0.00057], which includes zero (*exploratory*), so the confirmatory answer is robust to the setting. The re-runs changed several ingredients at once, and we therefore do not attribute any gain to τ alone. What the later work does show is that the value of federated training depends on how strong the server-side pretrain is: with a weak 100,000-user pretrain, federated training adds +0.0074 [0.0064, 0.0084] over that pretrain (*exploratory*) (Section 5.13).

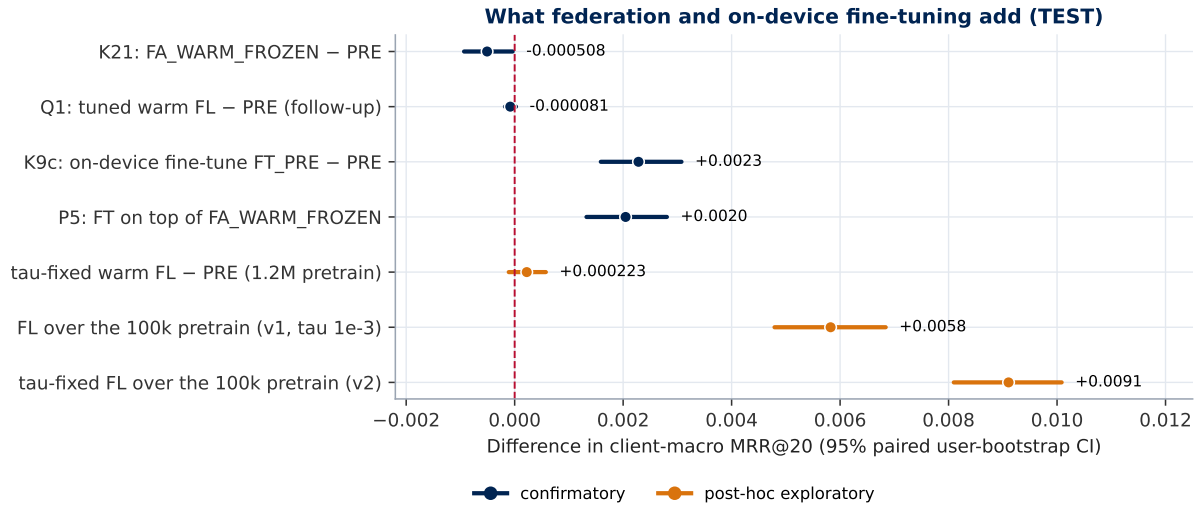


Figure 5.5: Forest plot of key differences in client-macro MRR@20 with 95% intervals. The confirmatory deltas K21, Q1, K9c and P5 are (*confirmatory*) or (*confirmatory, follow-up*) and come from the confirmatory test evaluation; the deltas labelled B1, A1f and C1 are (*exploratory*) and come from the later sections (Sections 5.12 and 5.13). Source: `confirmatory_phase1_contrasts.csv`, `confirmatory_phase2_contrasts.csv` and the exploratory result files.

5.10.4 The benchmark TEST looks

The external benchmarks asked a different question: does the REES46 recipe, run unchanged, transfer to public data, and is our cloud reference a credible model? The looks were taken after the REES46 test evaluation had started, one per dataset. The study lead released the five CPU data sets (Beauty, Sports and Toys in the S3 split, and Amazon 2023 Video Games and Baby Products [11]) at 21:15 UTC on 5 October; their finals ended with return code 0 at 05:29 UTC on 6 October, and their values were read only after the result directories had been made read-only. The ML-1M look was released at 11:00 UTC on 6 October but deliberately held until the test evaluation was complete, because the same GPU was needed by step 8; it was launched at 10:20 UTC on 7 October on the A40 device class (NVIDIA_A40-FP32-DET, switched from CPU before any ML-1M value was read, for schedule reasons). The matched-recipe protocol (M) is read at the fixed endpoint of 6.0 passes over the training data (CB); the tuned protocol (T) adds larger-budget cells for the central model and the pretrain and is read at the best validated mark (PB). A cell is written as [budget in passes over the training data, maximum sequence length, dropout]. The population is ALL users, which is the 15% pretraining band plus the others, the metric is NDCG@10 and values are means over seeds 2026 and 2027.

Table 5.31 shows the first result: the cloud reference C_FULL is the best arm on all six data sets under the matched-recipe protocol, and under the tuned protocol on the three S3 sets. Under the matched-recipe protocol, every federated, warm, DP and fine-tuned arm lies below C_FULL on all five CPU data sets (S3 (*confirmatory*), Amazon 2023 (*descriptive*)) with both seeds on the same side and an absolute difference above 0.005 (the registered claim rule), with differences between −0.0069 and −0.0384 NDCG@10, and most arms sit close to the pretrained model PRE. On ML-1M the arms of the matched-recipe protocol are C_FULL 0.1936 (three seeds), FA 0.1572, FT_FA_WARM 0.1324, FA_WARM 0.1309, PRE 0.1092 and

Table 5.31: Benchmark TEST looks, population ALL, NDCG@10, mean of seeds 2026 and 2027 (ML-1M C_FULL: three seeds). The matched-recipe protocol (M) is read at the fixed endpoint (CB), the tuned protocol (T) at the best validated mark (PB). The S3 sets and ML-1M are (*confirmatory*); the Amazon 2023 sets are (*descriptive*) (ours against ours). Cells of the tuned protocol: C_FULL [24, 50, 0.5], PRE [96, 50, 0.5], FA [6, 50, 0.1]. Source: `HANDOVER/14`.

Data set	Holdout	M: C_FULL	T: C_FULL	T: PRE	T: FA	T: FA_WARM_FROZEN	Runs
S3 Beauty	22,311	0.0437	0.0509	0.0224	0.0253	0.0064	44
S3 Sports	35,539	0.0224	0.0248	0.0087	0.0078	0.0049	44
S3 Toys	19,365	0.0410	0.0543	0.0174	0.0054	0.0050	44
Video Games (2023)	94,762	0.0487	–	–	–	–	22
Baby Products (2023)	150,777	0.0164	–	–	–	–	22
ML-1M	6,038	0.1936	–	–	–	–	63

FA_WARM_FROZEN 0.1039 (*confirmatory*). Below the cloud reference the ordering is not uniform: under the tuned protocol the cold FA is ahead of PRE on Beauty (0.0253 against 0.0224) and behind it on Sports and Toys. The frozen warm arm is the weakest on Beauty (0.0064); this is consistent with its poor pretrain of 6 passes over the training data and with item tables frozen to a pretraining band that sees only 67.2% of the Beauty items and 75.2% of the holdout targets (*descriptive*).

How much of the cloud model does the best federated arm keep? Table 5.32 reads the cold FA arm against C_FULL at PB on both sides. The range is 13 to 77%. An earlier handover text quoted 13 to 81%; that figure mixed the rules, because the ML-1M headline values 0.1936 and 0.1572 are controlled-budget means (their ratio is 81.2%, our division) whereas the PB reading is 77.06%. We use the PB range, name the recipe every time, and treat the earlier figure as superseded.

Table 5.32: The cold federated arm FA as a percentage of C_FULL, matched-recipe protocol, PB on both sides, population ALL, NDCG@10. S3 sets and ML-1M (*confirmatory*); Amazon 2023 sets (*descriptive*). Source: computed from the benchmark TEST values in `results/benchmarks/confirmatory_track_a.csv`.

Data set	FA (PB)	% of C_FULL	Label
S3 Beauty	0.025266	57.84	(*confirmatory*)
S3 Sports	0.007837	35.28	(*confirmatory*)
S3 Toys	0.00538	13.20	(*confirmatory*)
ML-1M	0.15035	77.06	(*confirmatory*)
Video Games (2023)	0.02403	47.77	(*descriptive*)
Baby Products (2023)	0.009945	54.69	(*descriptive*)

The first diagnosis of these weak federated results, made after the looks, is (*exploratory*) and is developed in Sections 5.11 and 5.12. On the benchmarks about 100% of the update values were below τ, the server steps ranged from 10^{-7} (item table) to 1.5×10^{-5} (transformer blocks), and $\tau = 10^{-6}$ gives steps 70 to 100 times larger. In addition, the federated budget was fixed at 6 passes over the training data; on Beauty, 126 thousand of 145 thousand client visits were a single local step, and FedProx was inert (94 to 97% single-step visits). The validation curve of FA on Beauty rises from 0.0031 at round 193 to 0.0324 at round 2,329 and is still rising (*validation*), and the cold arm's best mark is the last one (6.0) on all five CPU data sets except one run (Baby Products, seed 2026, at mark 5.0). A pretrain of 6 passes over the training data reaches about 0.007 on Beauty validation, against about 0.030 for the best pretrain of 96 passes

(*validation*), so the warm start is a handicap there. With the corrected τ the best fixed-recipe federated arm of each data set (FA_WARM_X) reaches 75 to 92% of C_FULL on TEST in the T1 look (the cold FA_X: 72 to 91%) (*exploratory*), which is a pointer to Section 5.12, not a confirmatory claim.

The S3 Toys look carries a recorded incident (I055). An AI worker agent copied a code tree with `cp -a`, which kept its read-only modes, so a patch did not land, and refusal tests then ran on the unpatched tree against the real benchmark directory. A `final` command on S3 Toys then appended a start line and an error line to `TEST_ATTEMPTS.jsonl`, with the message that the one TEST look of the data set had already been taken (13:38:48 UTC), and a CPU smoke run overwrote an old smoke directory from 1 October that the lock does not reference. The gate therefore did what it is built to do: it refused the second look. The AI operations agent measured the hashes of the TEST look, TEST results and lock (`bbc58726`, `6c4cbe7b`, `4b188ff3`) unchanged, no process was stopped and no value was read. The incident is disclosed in the run book, and we count the S3 Toys attempts as two: one look and one refused attempt after it.

5.10.5 Comparison with published full-ranking results

The benchmark data sets were built so that published numbers on the same split are directly comparable. The published results come from Table 1 of the RecSys 2026 capacity study [19], which also reports eSASRec [18] and ranks the full catalogue on leave-one-out splits, as we do. Our copy of the data reproduces the published numbers of three baselines (a Markov chain, SeqRules and PCTM) in all 12 of 12 cells to four decimals (*descriptive*), so the data and the protocol are the same. Table 5.33 and Figure 5.6 place our C_FULL next to the published columns. The comparison is for the cloud reference only; no federated number of ours is compared with a published one, because the published models are centrally trained.

Table 5.33: Our C_FULL against published full-ranking NDCG@10 on the same four data sets and splits. Published columns are the printed values of Table 1 of [19] (single runs according to the authors' repository; the paper reports no spread, and the ±0.003 is our estimate from the gap to the eSASRec authors' published values, Section 2.1) (*descriptive*); the two FMC columns are omitted. Our matched-recipe value is a fixed-budget (CB) mean over seeds, our tuned-protocol value is the tuned cell [24, 50, 0.5] at PB; both (*confirmatory*) for the S3 sets and ML-1M. Source: `results/benchmarks/published_table1_ndcg10.csv` and Table 5.31.

Data set	MC	SASRec+	eSASRec	SeqRules	PCTM	Ours matched	Ours tuned
Amazon Beauty	0.0492	0.0537	0.0524	0.0605	0.0635	0.0437	0.0509
Amazon Sports	0.0251	0.0315	0.0324	0.0371	0.0368	0.0224	0.0248
Amazon Toys	0.0548	0.0575	0.0533	0.0730	0.0738	0.0410	0.0543
MovieLens-1M	0.1240	0.1662	0.1739	0.1505	0.1815	0.1936	–

On ML-1M [12] our C_FULL scores 0.1936, above the printed eSASRec value of 0.1739 and above the clean-room re-run of eSASRec at 0.1767, and above every column of the published table, including PCTM at 0.1815; the margin to the best published value, 0.0121, is larger than the run-to-run spread of about 0.003 of single published runs. On the three Amazon sets [10] the picture is more modest. Under the tuned-protocol cell our C_FULL reaches 0.0509 on Beauty (eSASRec 0.0524), 0.0248 on Sports (0.0324) and 0.0543 on Toys (0.0533). That is within the spread of eSASRec on Beauty and Toys and below it on

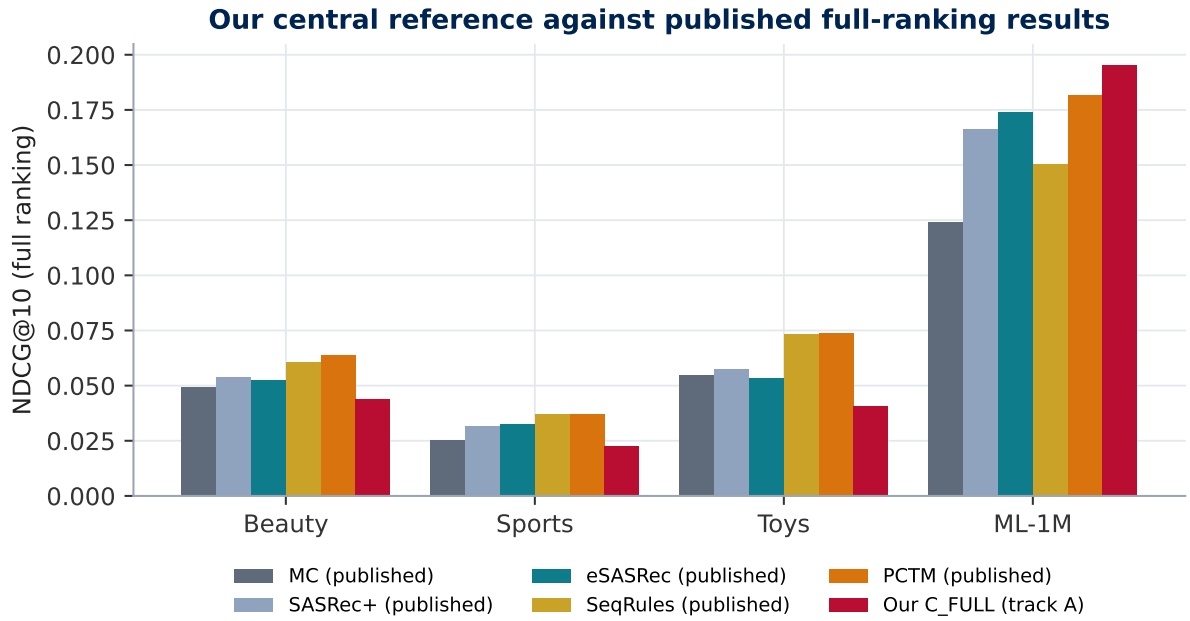


Figure 5.6: Our cloud reference C_FULL against published full-ranking NDCG@10 on the same data sets and splits. Published values are (*descriptive*) (Table 1 of [19]); our values are (*confirmatory*) (S3 sets and ML-1M). Source: `published_table1_ndcg10.csv` and the benchmark TEST looks.

Sports, and it is below SeqRules and PCTM on all three sets, which the published study itself reports as stronger than eSASRec there. At the fixed budget of the matched-recipe protocol our values are lower still. Two honest conclusions follow. Our cloud reference is a credible full-ranking model, strongest on ML-1M and competitive with eSASRec on two of the three Amazon sets, so the retention ratios of the study are measured against a reference that is not a straw man. It is not state of the art on the Amazon sets, and the lighter non-neural models of the published study would beat it there; we did not run them, because the study lead's rule was that no method we had not used on REES46 would be added to the benchmarks. The two eSASRec values for ML-1M differ because 0.1739 is the printed table value and 0.1767 comes from a clean-room re-run of the released code; the eSASRec README run reports 0.1772. The three values differ by at most 0.0033, about the stated run-to-run spread, and none changes the conclusion.

What we learned

Gained. The confirmatory test evaluation gave unbiased answers to the registered questions. The realistic edge system keeps 88.49% [87.73, 89.18] of the cloud reference (K20b), the shipped pretrain alone keeps 88.77% (K20c), and the sign of K21 (−0.000508) confirms the VALIDATION read. The follow-up analysis reports a registered null: Q1 is EQUIVALENT (−0.000081 [−0.000179, 0.000024]) and the deployment-grade verdict is False. The on-device fine-tune (+0.002283, K9c) is the one reliable deployable gain. The benchmarks show that the cloud reference C_FULL is the best of our arms on every data set and exceeds published eSASRec on ML-1M (0.1936 against 0.1767 for the clean-room re-run and 0.1739 as printed). The process held: 26 scoring jobs ran with none failed, the gate refused the second S3 Toys look, and register entries 10 and 11 were written before any number was read.

Did not work. On REES46, federated rounds add nothing over a strong 1.2-million-user pretrain; on the benchmarks the cold federated arm keeps 13 to 77% of C_FULL (S3 Toys lowest, ML-1M highest). The privacy and compression arms cost little because the registered federated model barely moves, and the DP arms carry no formal guarantee. DP-FTRL, the canary, update leakage and the MAIN secure-aggregation arm were not run in the test evaluation, and there is no fresh-period (TEST2) replication.
Caution. Mechanism claims such as the τ effect are exploratory and never replace the confirmatory numbers; the incident I055 shows what a gate is for, and shows that disclosure is part of the result.

5.11 Diagnosing the FedAdam tau effect

Status of the analyses in this part of the chapter

The analyses in Sections 5.11 to 5.17 were designed after the confirmatory TEST results were seen. They are (*exploratory*) (POST_HOC_EXPLORATORY). Their TEST looks are entries 12 to 19 of the test-exposure register (the looks of the main pre-registered study) and entries 20 to 23 (the closure-phase extension of the register, made at the technical closure). They explain the confirmatory results and are not confirmatory evidence. Each exploratory number is printed beside, never instead of, the confirmatory number it concerns; no confirmatory result was edited, re-run or re-selected; and validation and calibration data were read repeatedly while the exploratory recipes were chosen.

Terms used in this part

Retention is $R(x/y) = 100\times$ (metric of x) / (metric of y) for the client-macro MRR@20; a delta is a raw difference (0.002283 equals 0.2283 percentage points, pp). **MAIN** is the population of 131,072 simulated devices and **CALIB** its 32,768-client subset used for tuning; MAIN VALIDATION and TEST (26,254 users, 229,742 decisions) are two evaluation periods of MAIN users. **Passes over the training data** (epochs, defined in Section 5.1) count how often, summed over all clients, the training examples are presented to local training; the registered 6.0 passes are 13,654 rounds on MAIN and 3,414 on CALIB. **PB** (PRACTICAL_BEST) is the best of 12 evaluation marks under the registered rule and **CB** (CONTROLLED_BUDGET) the fixed 6.0-pass endpoint. **PRE** is the server-pretrained model θ_{pre}, the **tuned warm-start federated model** is the warm recipe of the follow-up analysis (practically equal to PRE), FA the cold-start federated arm, and FA_WARM and FA_WARM_FROZEN the arms that start from PRE. The **100k-user pretraining variant** uses a pretraining band of 100,000 users. A **pre-scoring registration entry** is written to the register before any TEST score exists and fixes what will be scored; a **post-scoring disclosure entry** is appended once the result file exists.

Table 5.34 lists the six exploratory TEST looks of this part, so that every later number can be traced to one look. Entries 12 to 19 were prepared by the study lead with AI agents; entries 20 to 23 were prepared in the technical closure, in the closure-phase extension of the register, which keeps the original 19-entry register as a prefix.

Table 5.34: (*exploratory*) The exploratory TEST looks of this part (the pre-scoring entry is written before scoring, the post-scoring entry after the result file exists). Source: test-exposure register entries 12 to 19 and closure-phase extension entries 20 to 23.

Pre / post entry	Look	What was scored on TEST	Seeds	Section
12 / 13	100k-user pretrain v1	five sealed arms from the 100,000-user pretrain, original recipe ($\tau = 10^{-3}$), 22 contrasts	2026	5.13
14 / 17	T2: REES46, τ fixed	tau-fixed warm and cold federated arms (8 checkpoints)	2026, 2027	5.12
15 / 16	T1: six benchmarks, fixed recipe	FA_X, FA_WARM_X, C_SMALL_X on six datasets (36 runs)	2026, 2027	5.12
18 / 19	T3: 100k-user pretrain v2	tuned, tau-fixed warm arm from the 100,000-user pretrain	2026, 2027	5.13
20 / 22	Disjoint server-data and capacity study look	disjoint server band and capacity cells on four datasets (147 checkpoint-rule pairs)	2026, 2027	5.13
21 / 23	DP8 look	four CB endpoints with nominal $\varepsilon = 8$ (cold and 100k-user pretrain)	2026, 2027	5.14

The question left by the confirmatory test evaluation. The confirmatory test evaluation ended with a registered null. The federated stage added nothing measurable to the shipped pretrained model: K21 (FA_WARM_FROZEN minus PRE) is −0.000508 [−0.000933, −0.000037] (*confirmatory*), and the follow-up analysis contrast Q1 (the tuned warm-start federated model minus PRE) is −0.000081 [−0.000179, 0.000024], read as EQUIVALENT within ±0.0005 (*confirmatory, follow-up*). On the six public benchmarks the best federated arm reached only 13% to 77% of the central reference (*confirmatory*) (matched-recipe protocol, NDCG@10; the Amazon-2023 rows are descriptive). Two earlier signals had pointed the same way. On 2026-10-03 the D1 diagnostic found the warm federated models only 0.53% (best checkpoint) and 0.67% (endpoint) away from θ_{pre} for FA_WARM_FROZEN, and 0.013% and 0.077% for DP8_FROZEN (*exploratory*). And all 22 cells of the server-step grid of the follow-up analysis fell within ±0.00012 of the CALIB score of PRE (0.171302), so the registered tie rule chose the smallest server step and the tuned warm-start federated model equals PRE in effect (*descriptive*).

A null this consistent has two possible explanations: federation has nothing to add to a strong pretrained model, or the registered recipe never trained the model enough to add anything. This section collects the evidence that separates them; Section 5.12 tests the resulting prediction.

What the registered server optimiser does to small updates. FedAdam [25] applies Adam [26] on the server to the weighted mean Δ_t of the client updates of round t, treated as a pseudo-gradient. With moment estimates m_t and v_t it moves $\theta_{t+1} = \theta_t + \eta_s\, m_t/(\sqrt{v_t} + \tau)$, where η_s is the server step and τ a constant that controls the degree of adaptivity. The registered recipe (amendment 06G) fixed $\beta_1 = 0.9$, $\beta_2 = 0.99$, $\tau = 10^{-3}$ and $v_0 = \tau^2$. For a coordinate whose update is much smaller than τ the denominator is essentially τ, so the step is $\eta_s m_t/\tau$: it shrinks with the averaged update instead of keeping the magnitude of order η_s that Adam is designed to take, and the global model hardly moves. The value 10^{-3} was the default. The main-analysis server-optimiser screen on CALIB contained one cell with $\tau = 10^{-5}$ at the winning server step $\eta_s = 0.01$, which scored 0.024374 against 0.034247 for the winner (*validation*); every later tuning stage, the warm recipes and the whole follow-up analysis kept $\tau = 10^{-3}$. We call the registered

setting badly scaled for the observed update sizes, not a bug: it was registered, reviewed and executed as written.

The ranked causes. On 2026-10-06, after the test evaluation and before any exploratory TEST look, an AI worker agent diagnosed the benchmark runs on validation data only and ranked six candidate causes: (1) the constant τ, with about 100% of the update values below 10^{-3} and server steps from 10^{-7} (item table) to 1.5×10^{-5} (blocks), 70 to 100 times larger at $\tau = 10^{-6}$; (2) under-training, since every cold validation curve was best at its last mark on five datasets and the s3_beauty curve rose from 0.0031 at round 193 to 0.0324 at round 2,329 and was still rising; (3) a weak warm start, a 6-pass pretrain scoring about 0.007 against about 0.030 for the best 96-pass one (validation NDCG@10, s3_beauty); (4) a capacity confound ($d = 256$ cloud reference against $d = 64$ federated arms); (5) inert FedProx [22], personal head and 8-bit uploads, which matched plain federated averaging within 10^{-4} because most client visits made a single local step (126,000 of 145,000 on s3_beauty); and (6) no sign of a defect in dropout, schedule, aggregation or loading of the pretrained model. Only the first cause carries over to REES46. There the AI worker agent measured 99.3% to 100% of the update values below τ, a median update value of 6×10^{-8} and a server-step RMS of 5.6×10^{-8} (1.2×10^{-6} at $\tau = 10^{-6}$), and the tuned warm-start federated model (seed 2026) moved only about 0.06% from θ_{pre} in 13,654 rounds. Table 5.35 collects the measurements.

Table 5.35: Measurements behind the diagnosis of the registered FedAdam setting ($\tau = 10^{-3}$). The D1 row is exploratory; the other rows characterise the measured setup and carry no registered claim. Source: the D1 diagnostic of 2026-10-03, `tests/fedsim/test_fedsim_fedadam_tau.py` and the `README.md` of the update-norm analysis.

Quantity	Value	Label
Distance from θ_{pre}, FA_WARM_FROZEN (best / endpoint); DP8_FROZEN	0.53% / 0.67%; 0.013% / 0.077%	(*exploratory*)
Server step, benchmarks, at $\tau = 10^{-3}$	10^{-7} to 1.5×10^{-5}; 70 to 100 times larger at $\tau = 10^{-6}$	(*descriptive*)
Update values below τ, REES46	99.3% to 100%; median 6×10^{-8}	(*descriptive*)
Server-step RMS, REES46	$5.6{\times}10^{-8}$ at $\tau = 10^{-3}$; $1.2{\times}10^{-6}$ at $\tau = 10^{-6}$ (ratio about 21)	(*descriptive*)
Tuned warm-start model, seed 2026, drift from θ_{pre} after 13,654 rounds	about 0.06%	(*descriptive*)
Update-norm analysis, tuned warm-start model, seed 2026: median server-step RMS; final drift of the trained parameters	7.307×10^{-8}; 0.06003% (seed 2027: 0.05985%)	(*descriptive*)

The update-norm analysis. The technical closure added a further descriptive reading that needs no new training. Every run of the follow-up analysis had logged the norms of its aggregated update; the update-norm summary reads 29 such logs (8 MAIN runs and 21 CALIB cells of the server-step grid), 100,942 rounds in all, with no non-finite step. The logs hold L2 norms per tensor group, not coordinates, so the share of coordinates below τ cannot be recovered exactly. It reports the bound $\max\left(0,\ 1 - \|\Delta\|_2^2/(n\tau^2)\right)$, which holds because at most $\|\Delta\|_2^2/\tau^2$ of the n coordinates can reach τ; it is conservative because it depends on the L2 norm alone. Figure 5.7 shows that the tuned warm-start federated model barely leaves θ_{pre} (drift 0.06003% for seed 2026 and 0.05985% for seed 2027) and that its per-round bound lies between

91.4% and 100.0% (median 97.1%, seed 2026). No logged run moved by more than 1.20%. The two cells that came closest, FedAvg with $\eta_s = 1$ and FedAvgM with $\eta_s = 0.1$ (both with client learning rate 0.03), use no adaptive denominator, yet they too stayed within the ±0.00012 band around the CALIB score of PRE. A move of up to 1.2% of the trained parameters thus did not change that score by more than 0.00012, also where no τ is involved, so the registered τ is not the only reason why federation added little on REES46.

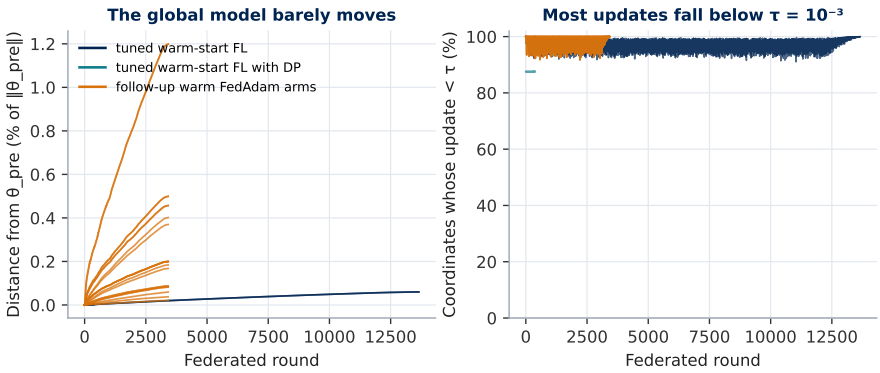


Figure 5.7: (*descriptive*) Update norms of 29 federated runs of the follow-up analysis (100,942 rounds), read from the update-norm logs. Left: relative distance of the trained parameters from θ_{pre}; the tuned warm-start federated model ends at 0.06% after 13,654 rounds. Orange lines are the 21 logged CALIB cells of the server-step grid (FedAvg, FedAvgM and FedAdam, 3,414 rounds each). Right: lower bound on the share of coordinates whose update is below $\tau = 10^{-3}$. Source: `TOTAL_BY_ROUND.csv` of the update-norm summary.

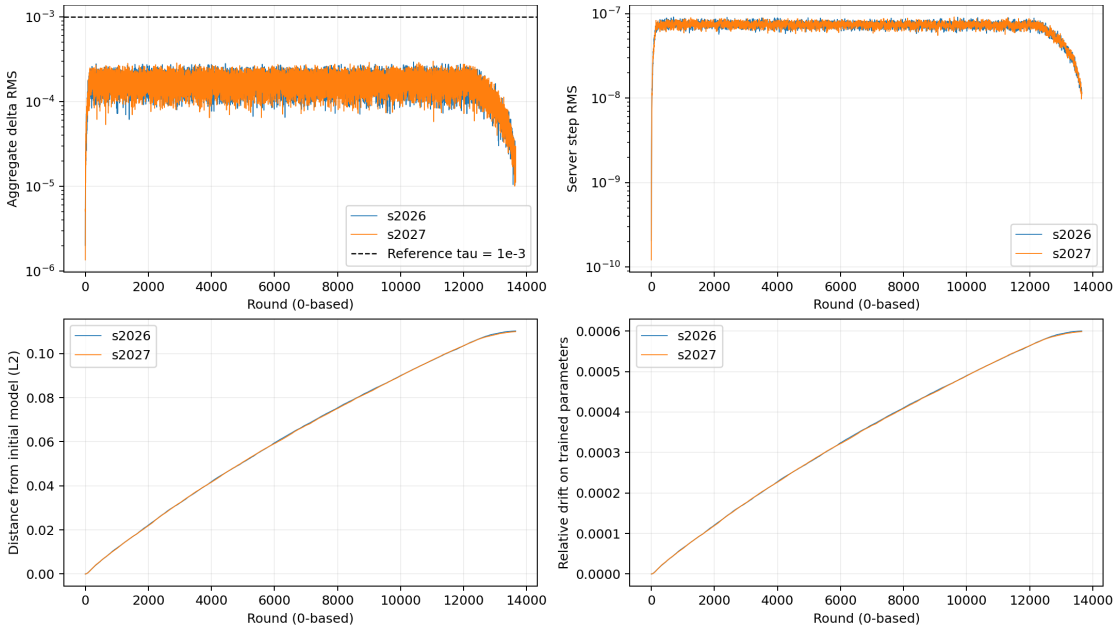


Figure 5.8: (*descriptive*) The raw trajectories of the tuned warm-start federated model on MAIN for both training seeds (2026 and 2027), as plotted by the update-norm analysis code from the training logs. Top left: RMS of the aggregated client update per round, against the reference $\tau = 10^{-3}$ (dashed). Top right: RMS of the server step, about 10^{-7}. Bottom: L2 distance from the initial model and relative drift of the trained parameters, which end near 0.11 and 0.0006 after 13,654 rounds.

What the diagnosis explains and what it does not. The registered FedAdam setting left the server step tiny; this is a measured property of the confirmatory recipe, not an inference. The code default stays

at $\tau = 10^{-3}$ so that the registered runs remain reproducible. In the public repository the setting is now validated (`fl.server_tau` must be a finite number above zero), a unit test documents the effect, and 10^{-6} is recommended (pull request #103, merged 2026-10-10). The diagnosis does not show what a corrected recipe achieves. The re-runs that follow changed several ingredients at once, namely τ together with the server step, the item-table policy and the budget, and we do not claim that τ alone explains any gain. Figure 5.8 shows the raw per-round logs of the tuned warm-start model behind this summary.

What we learned

Gained. A measured mechanism for the registered null (tiny server steps, a model that moved 0.06%), confirmed by three readings: the AI worker agent's coordinate-level measurement, a unit test and the closure update-norm analysis of 100,942 logged rounds; and five further candidate causes that shaped the re-runs.

Did not work or remains open. The diagnosis cannot say how much a corrected τ gains. On REES46 it cannot be the whole story, because cells without an adaptive denominator also stayed within ±0.00012 of PRE. The share of coordinates below τ is a bound, not a measurement. No registered result was changed.

5.12 The fixed recipe on the benchmarks and on REES46

The diagnosis of Section 5.11 makes two predictions that can be tested. If the registered FedAdam setting was a main reason why federation added so little, a recipe with $\tau = 10^{-6}$ should lift the federated arms on the six public benchmarks, where the server-side pretrain is weak and the best federated arm reached only 13% to 77% of the central reference (*confirmatory*). On REES46, where θ_{pre} is trained on 1,217,610 users, the same correction should change little, because the federated stage starts from a model that already captures most of what the data supports. We tested each prediction with one exploratory TEST look, T1 for the benchmarks and T2 for REES46. The recipe of each look was fixed beforehand on validation or calibration data alone, and each look was preceded by an independent AI gate review, a lock file and a pre-scoring registration entry (Table 5.34).

The benchmark sweep. The new code lived in a separate tree (spec tag POST_HOC_X) that refuses to run before the confirmatory test evaluation is complete. By the study lead's correction, the federated arms were trained only at $d = 64$ with two blocks and seeds 2026 and 2027, the cloud reference C_FULL was not retrained, the pretrained models selected on validation for the confirmatory runs were reused read-only, and all tuning used the inner validation split of each dataset. A first full grid of about 330 to 350 runs (3,100 to 3,500 slot-hours, at least 11 days) was stopped after about 13 minutes and replaced by a lean plan with $\tau = 10^{-6}$, FedAdam server steps $\{10^{-3}, 3 \times 10^{-3}\}$ for FA and $\{3 \times 10^{-4}, 10^{-3}\}$ for FA_WARM, and one run of up to 24 passes per cell (41 fixed runs plus up to 24 conditional ones). The frozen warm arm (it scores 0.0 for users whose target lies outside the pretraining band), two local passes and the central models were dropped, and a matched-size central reference, C_SMALL, was added later. When the study lead remarked that a warm start should beat a cold one, twelve more warm cells with client learning rates

0.1 and 0.3 were added.

On inner validation ((*exploratory*), NDCG@10, mean of two seeds) the best fixed federated arm reached 79.5%, 79.6%, 91.3%, 75.9%, 82.2% and 89.4% of the central reference C_FULL (matched-recipe protocol) on Beauty, Sports, Toys, Video Games, Baby Products and ML-1M. Against the matched-size C_SMALL ($d = 64$) it reached 81.5%, 84.8%, 82.1% and 83.4% on Beauty, Sports, Toys and Video Games, a federation cost at matched size of about 15% to 18%, whereas the size cost of $d = 64$ against $d = 256$ lay between about 0% and 14%, depending on which C_FULL reference is used.

The T1 look: six benchmarks on TEST. With the recipe fixed on validation, we looked once at TEST (registration entry 15, disclosure entry 16; two seeds, 36 runs). Table 5.36 gives the result and Figure 5.9 shows it against the registered arms. The corrected recipe lifts the best federated arm from 13% to 77% of the central reference in the confirmatory matched-recipe-protocol table (*confirmatory*) to between 75% and 92% of the look's own C_FULL (75.10% on Sports, 92.41% on Toys) (*exploratory*); the two ranges use the central references of their own looks and are therefore compared only roughly. The gain over the confirmatory federated arm is large on the three S3 Amazon datasets (for instance 0.0366 against 0.0253 on Beauty and 0.0372 against 0.0054 on Toys). On ML-1M the confirmatory arm was already competitive (0.1504), and the pre-set reading rule calls the corrected cold arm (0.1462) on par with it.

Table 5.36: (*exploratory*) TEST NDCG@10, mean of two seeds. FA (conf.) is the confirmatory cold arm with $\tau = 10^{-3}$; FA_X and FA_WARM_X are the fixed-recipe cold and warm arms; the last three columns give the percentage of the look's own C_FULL. Rows marked † are (*descriptive*) (no published reference). Source: `exploratory_fixed_fl_test.csv`, `X_RESULT.json` of the T1 look.

Dataset	C_FULL	C_SMALL_X	FA (conf.)	FA_X	FA_WARM_X	C_SMALL%	FA_X%	WARM_X%
Beauty	0.0437	0.0449	0.0253	0.0366	0.0366	102.67	83.80	83.83
Sports	0.0222	0.0213	0.0078	0.0161	0.0167	95.99	72.25	75.10
Toys	0.0408	0.0449	0.0054	0.0372	0.0377	110.26	91.36	92.41
ML-1M	0.1951	0.1867	0.1504	0.1462	0.1613	95.71	74.95	82.68
Video Games†	0.0503	0.0452	0.0240	0.0368	0.0381	89.78	73.21	75.66
Baby Products†	0.0182	0.0174	0.0099	0.0139	0.0146	95.74	76.66	80.40

Two further readings follow. Warm minus cold at the fixed recipe is small: +0.000012, +0.000633 and +0.000429 on Beauty, Sports and Toys (on par), +0.001234 and +0.000680 on Video Games and Baby Products ((*descriptive*)), and +0.015073 on ML-1M, where the validation order had been the reverse (0.1217 cold, 0.1095 warm). And the matched-size C_SMALL_X scores 89.78% to 110.26% of C_FULL, so the size of the device model is not what limits the federated arms on most datasets (Video Games, at 89.78%, is the exception); the warm arm reaches 78.4% to 86.4% of C_SMALL_X (computed from the table), a federation cost at matched size of about 14% to 22%, and the pre-set rule reads every federated arm as below C_SMALL_X.

The T2 look: the same correction on REES46. The REES46 check (Part B) started from θ_{pre} with $\tau = 10^{-6}$, trainable item tables at 0.1× the learning rate and a server step $\eta_s \in \{3 \times 10^{-6},\ 10^{-5},\ 3 \times 10^{-5}\}$, and was selected on CALIB only. The score fell as the step grew. The winner, $\eta_s = 3 \times 10^{-6}$ at the lower

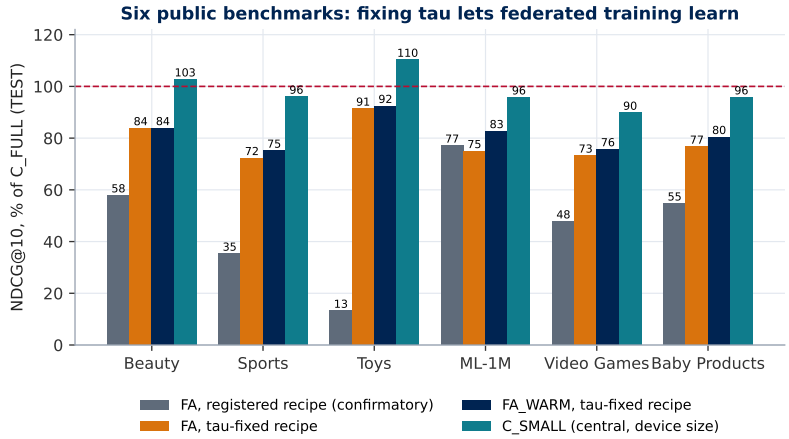


Figure 5.9: (*exploratory*) The six benchmarks on TEST (NDCG@10, mean of two seeds): the registered federated arms, the fixed-recipe arms FA_X and FA_WARM_X, and the matched-size central reference C_SMALL_X. The two Amazon-2023 datasets are (*descriptive*). Source: `exploratory_fixed_fl_test.csv`.

edge of the grid, reached a best CALIB MRR@20 of 0.171311 (0.171155 at 6 passes) against 0.171302 for PRE, whereas 10^{-5} went from 0.171237 to 0.169721 at 6.0 passes and 3×10^{-5} from 0.170536 to 0.166280 (*exploratory*). Fixing τ therefore moved the model further but did not improve it on CALIB, which corrected an earlier suspicion of the AI operations agent that K21 and Q1 were an artefact of τ alone. In parallel the cold REES46 arm was re-tuned on CALIB with $\tau = 10^{-6}$. On MAIN VALIDATION, before the look, the corrected warm arm was about +0.0003 above PRE (interval includes 0) and the corrected cold arm about −0.008 below the confirmatory cold arm.

The T2 look (registration entry 14, disclosure entry 17; seeds 2026 and 2027) confirmed this reading on TEST (Table 5.37). The corrected warm arm X_FA_WARM minus PRE is +0.00022 $[-0.00011, 0.00057]$, an interval that includes 0, so the confirmatory answer is robust to the correction: federated rounds add nothing measurable to the 1.2M-user θ_{pre}, with K21 −0.000508 (*confirmatory*) and Q1 −0.000081 (*confirmatory, follow-up*) as the registered counterparts. As retention, X_FA_WARM reaches 89.05% $[88.37, 89.71]$ of C_FULL and the tuned warm-start federated model 88.88% $[88.20, 89.58]$ in the same analysis (own-seed denominators; the confirmatory Q9 prints 88.72 (*confirmatory, follow-up*)). The corrected cold arm does worse than the confirmatory one: 66.48% $[65.71, 67.18]$ of C_FULL against 70.25% $[69.48, 70.98]$ for FA (K17 prints 70.10 (*confirmatory*)), and 85.05% $[84.33, 85.78]$ of C_SMALL.

Table 5.37: (*exploratory*) The tau-fixed REES46 contrasts of the T2 look (TEST, client-macro MRR@20, paired user bootstrap, seeds 2026 and 2027). X_FA_WARM and X_FA_COLD are the corrected warm and cold arms, FA the confirmatory cold arm and P2_WSTAR the tuned warm-start federated model of the follow-up analysis. Source: `exploratory_test_contrasts.csv`, `X_RESULT.json` of the T2 look.

ID	Contrast	Δ MRR@20	95% CI	Reading
B1	X_FA_WARM – PRE	+0.00022	[−0.00011, 0.00057]	includes 0
B2	X_FA_WARM – P2_WSTAR	+0.00030	[−0.00004, 0.00068]	includes 0
D1	X_FA_COLD – FA (conf.)	−0.00680	[−0.00778, −0.00587]	below
D2	X_FA_COLD – PRE	−0.04043	[−0.04191, −0.03889]	below

The re-runs changed several things at once

The corrected arms differ from the registered ones in more than τ: the warm arm also uses a different server step (3×10^{-6}), trainable item tables at $0.1\times$ the learning rate and, on the benchmarks, a longer budget, and D1 changes τ together with the server step of a re-tuned cold recipe. The contrasts therefore compare whole recipes, and we do not claim that τ alone explains any gain; D1 and D2 show that a corrected cold recipe is not automatically better.

What we learned

Gained. The corrected recipe moves the benchmark arms from 13% to 77% to 75% to 92% of the central reference on TEST ((*exploratory*), different references) and shows that the remaining gap is a federation cost of about 14% to 22% at matched size, not a size cost; on REES46 the confirmatory answer survives the correction (+0.00022 $[-0.00011, 0.00057]$).

Did not work or remains open. Warm start adds little to cold start at the fixed recipe on the Amazon datasets, the ML-1M order reverses between validation and TEST, and the corrected cold REES46 arm is worse than the registered one. Because several ingredients changed at once, no gain can be assigned to τ alone.

5.13 How much federation is worth depends on the server's data

Section 5.12 left a contrast. With the corrected recipe federation lifted the benchmark models by a large margin, yet it added nothing measurable to the 1.2M-user model on REES46. The benchmarks pretrain the server model on a band of only 15% of the users (904 users on ML-1M), REES46 on 1,217,610 users. This suggests that the value of the federated stage is a function of how good the server-side pretrain is. We tested the suggestion in three ways: by shrinking the REES46 pretrain to the band of 100,000 users (the 100k-user pretraining variant), by making the server band disjoint from the federated users on the benchmarks (the disjoint server-data study), and by asking whether model capacity could explain the gap (the capacity study).

A weak pretrain on REES46. The 100k-user pretrain (PRE100K) uses 100,000 users, 8.2% of the users of PRE (computed). In the first look (v1; registration entry 12, disclosure entry 13; seed 2026) the original recipe with $\tau = 10^{-3}$ was applied to it, with trainable item tables (FA_WARM_PB100K) or frozen ones (FA_WARM_FROZEN_PB100K). On validation the trainable arm rose from 0.1478 at mark 1 to 0.1513 at mark 12 and was still rising, and the frozen arm peaked at 0.1490 at mark 5 (*exploratory*). After the tau finding a second look (v2; registration entry 18, disclosure entry 19; seeds 2026 and 2027) scored the arm X2, selected on CALIB with server step 3×10^{-5}, client learning rate 0.1, item tables at $0.1\times$, $\tau = 10^{-6}$ and one local pass (CALIB MRR@20 0.14704). Table 5.38 gives the TEST contrasts.

With a weak pretrain the federated stage adds a clear gain: +0.00743 over its own starting point in v1 and +0.00911 in v2, where the corrected recipe adds a further +0.00167 over v1. With the strong pretrain the

Table 5.38: (*exploratory*) The 100k-user pretraining variant on TEST (client-macro MRR@20, paired user bootstrap). v1 is seed 2026 with the original recipe; v2 is the tau-fixed, tuned arm X2 over seeds 2026 and 2027. The lower block gives retention against C_FULL in percent. Source: `exploratory_test_contrasts.csv`, `X_RESULT.json` of the v1 and v2 looks.

Look	Contrast	Δ MRR@20	95% CI
v1	FA_WARM_PB100K – PRE100K	+0.00743	[0.00644, 0.00838]
v1	FA_WARM_FROZEN_PB100K – PRE100K	+0.00582	[0.00479, 0.00684]
v1	PRE100K – PRE	−0.02702	[−0.02845, −0.02530]
v1	FA_WARM_PB100K – PRE	−0.01959	[−0.02091, −0.01818]
v1	FA_WARM_PB100K – W_STAR	−0.01952	[−0.02084, −0.01806]
v2	X2 – PRE100K	+0.00911	[0.00810, 0.01009]
v2	X2 – FA_WARM_PB100K (v1)	+0.00167	[0.00113, 0.00221]
v2	X2 – PRE	−0.01792	[−0.01924, −0.01648]
v2	X2 – P2_WSTAR	−0.01784	[−0.01917, −0.01640]

Retention (%): PRE100K 73.79 [72.95, 74.66]; v1 FA_WARM_PB100K 77.91 [77.08, 78.72]; v2 X2 78.98 [78.22, 79.78]; PRE 88.77 [87.99, 89.48]; P2_WSTAR 88.73 [87.95, 89.43] (v1 look).

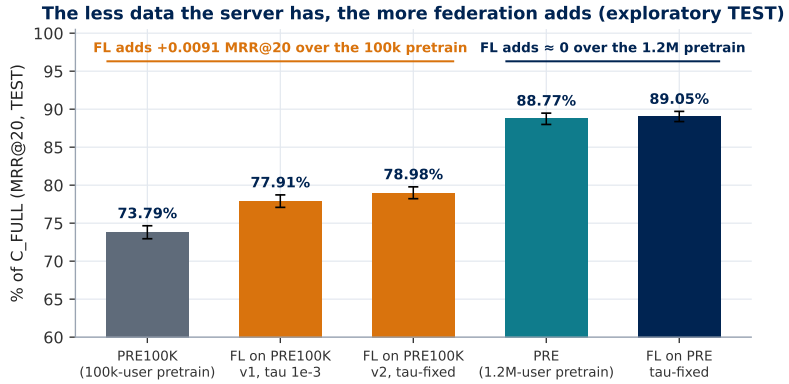


Figure 5.10: (*exploratory*) Value of federation against the size of the server-side pretrain on REES46 (TEST, retention against C_FULL): the 100,000-user pretrain (PRE100K), its federated versions (v1 and v2), the 1,217,610-user pretrain (PRE) and the tau-fixed warm run from PRE. Source: `exploratory_test_contrasts.csv`.

same stage adds +0.00022, an interval that includes 0. Federation therefore recovers about a third of the gap between the two pretrains (computed: 0.00911 of 0.02702), lifting retention from 73.79% to 78.98%. It does not replace server data: the 100,000-user model with federation stays 0.01792 below the 1.2M-user model without it, a difference of 9.79 retention points (computed), and the confirmatory REES46 retention of 88.5% (*confirmatory*) is already reached by pretraining alone (PRE 88.77%, K20c (*confirmatory*)). FL adds nothing over a strong 1.2M-user pretrain on REES46, and it adds a lot when the pretrain is weak.

A disjoint server band on the benchmarks. In the benchmark pretraining of the confirmatory runs, the federated users overlapped with the band on which the server model was trained. The disjoint server-data study removes the overlap: the server pretrains on a band of 15% or 50% of the users (DJ15, DJ50), and the arms are evaluated on the remaining federated users F. The four arms are the server model alone (PRE_S), cold federated learning on F (FA_F), warm federated learning from the server model (FA_WARM_F) and a central $d = 64$ model trained on F only (C_SMALL_F). Figure 5.11 gives the validation result (*exploratory*). Warm beats cold by 6.86%, 7.65% and 4.84% on Beauty, Sports and Toys at the 15% band, and by 29.65%, 54.52% and 27.46% at the 50% band, so the gain from a warm start grows with the size of the server band. The cold arm gets weaker from 15% to 50% (Beauty 0.044144 to 0.037393 on 18,962 and 11,153 evaluated users), consistent with a smaller pool of federated users, while the warm arm keeps its level (0.047173 to 0.048481). At the 50% band the warm arm exceeds the central model trained on F alone on these three datasets (for instance 0.048481 against 0.044988 on Beauty); at the 15% band it does not (0.047173 against 0.054229).

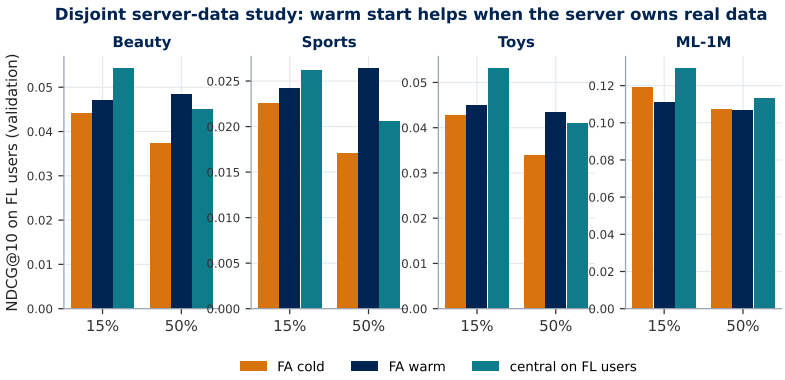


Figure 5.11: (*exploratory*) Disjoint server band on the four benchmarks (inner validation, NDCG@10 on the federated users): the server model, cold and warm federated learning and the central $d = 64$ model on the same users, for a 15% and a 50% server band. Source: `validation_disjoint_server_band.csv`.

ML-1M is the exception on validation: warm is below cold by 6.95% at the 15% band and by 0.44% at the 50% band (0.110859 against 0.119138 and 0.106795 against 0.107266 on 5,132 and 3,020 users). ML-1M is also the smallest dataset (6,038 holdout users and a 904-user band), and the server model alone is competitive there (0.106131 at the 50% band). The exploratory TEST look (registration entry 20, disclosure entry 22; seeds 2026 and 2027; NDCG@10, practical-best rule) covered 80 cells and 147 available checkpoint-rule pairs (80 practical-best and 67 controlled-budget; 13 controlled-budget endpoints were unavailable because of early stopping), with 90 contrasts computed and 22 unavailable. Table 5.39

gives the primary contrasts of the disjoint server-data study. An independent AI post-review validated that warm starts increase NDCG@10 relative to cold federated learning in both bands on all four datasets (all eight intervals above zero), and that against C_SMALL the warm arm loses at every 15% band and wins at every 50% band (all eight intervals exclude zero). On TEST the ML-1M exception disappears: warm beats cold at both bands. The sign of the ML-1M contrast thus differs between validation and TEST, as it did in T1; we do not have a tested explanation and treat the small dataset as a warning against selecting on its validation split.

Table 5.39: (*exploratory*) The disjoint server-data study on TEST (NDCG@10, practical-best rule, paired user bootstrap, seeds 2026 and 2027): FA_WARM minus cold FA, and FA_WARM minus the central C_SMALL trained on the same federated users. Source: `BENCH_INTERPRETATION_v4.json`, `primary_selected_contrasts`.

Dataset	Band	Users	vs FA	95% CI	vs C_SMALL	95% CI
Beauty	DJ15	18,962	+0.00216	[0.00111, 0.00329]	−0.00655	[−0.00789, −0.00523]
Beauty	DJ50	11,153	+0.00940	[0.00740, 0.01155]	+0.00386	[0.00187, 0.00610]
Sports	DJ15	30,203	+0.00201	[0.00121, 0.00278]	−0.00239	[−0.00340, −0.00137]
Sports	DJ50	17,765	+0.00745	[0.00617, 0.00871]	+0.00437	[0.00312, 0.00559]
Toys	DJ15	16,460	+0.00166	[0.00057, 0.00268]	−0.00610	[−0.00741, −0.00476]
Toys	DJ50	9,682	+0.00997	[0.00795, 0.01189]	+0.00278	[0.00082, 0.00476]
ML-1M	DJ15	5,134	+0.01410	[0.00944, 0.01903]	−0.02393	[−0.02858, −0.01943]
ML-1M	DJ50	3,020	+0.02477	[0.01844, 0.03105]	+0.02485	[0.01800, 0.03144]

Capacity. The capacity question came from the fourth candidate cause of Section 5.11: the cloud reference has $d = 256$ and three blocks, every federated arm $d = 64$ and two blocks. On validation, Figure 5.12 separates the two effects (*exploratory*). The central $d = 64$ model reaches 94.04% to 111.10% of the central $d = 256$ model (97.42%, 94.04%, 111.10% and 95.65% on Beauty, Sports, Toys and ML-1M), the cold $d = 64$ federated arm 81.01% to 93.48% of the central $d = 64$ model, and a $d = 256$ federated arm trained from random weights is worse than the $d = 64$ one on all four datasets (for instance 0.046539 against 0.047118 on Beauty), while from the pretrained model the two stay within ±5%. A larger device model thus gives no reliable gain for federated learning, whereas federation costs 7% to 19% at matched size.

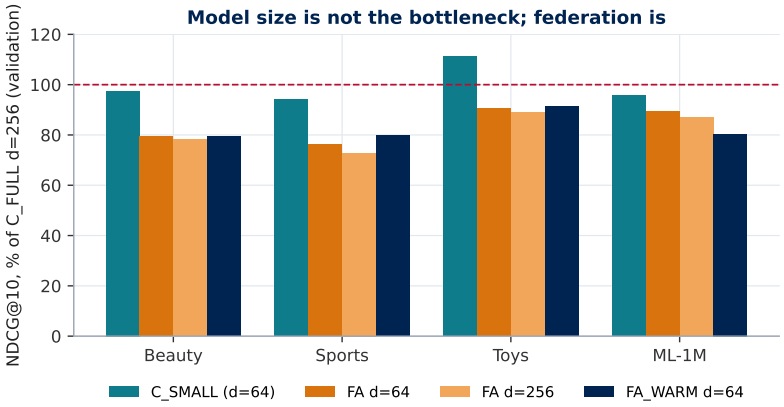


Figure 5.12: (*exploratory*) Capacity decomposition on the four benchmarks (inner validation, NDCG@10): central $d = 256$, central $d = 64$, and cold and warm federated arms at $d = 64$ and $d = 256$. Source: `validation_capacity.csv`.

The closure-phase capacity-study cells on TEST (same look as the disjoint server-data study; NDCG@10, practical-best rule) support the validation reading only in part. A $d = 256$ warm arm scores below the $d = 64$ central reference on all four datasets (-0.00747 $[-0.00854, -0.00616]$ on Beauty, -0.00518 $[-0.00607, -0.00425]$ on Sports, -0.00631 $[-0.00774, -0.00490]$ on Toys and -0.03427 $[-0.03897, -0.02947]$ on ML-1M). Raising the warm arm from $d = 64$ to $d = 256$ reduces NDCG@10 on ML-1M (-0.00883 $[-0.01290, -0.00461]$) and has intervals that include zero on the three Amazon sets (Beauty +0.00076, Sports −0.00054, Toys +0.00096), so a general capacity benefit is not demonstrated. The cold-capacity contrast is positive on ML-1M (+0.01534 $[0.01171, 0.01895]$) and negative on the Amazon sets (Beauty −0.00127, Sports −0.00094, Toys −0.00323), again opposite to the ML-1M validation order (0.1171 and 0.1196 for $d = 256$ against 0.1208 and 0.1225 for $d = 64$); it is not among the findings validated by the post-review.

What we learned

Gained. The value of federation is a function of the server pretrain (retention 73.79% to 78.98% with the 100,000-user band, against 88.77% without federation on the full pretrain); the gain from a warm start grows with the size of the server band; capacity does not explain the gap on TEST.

Did not work or remains open. Federation does not replace server data. The ML-1M contrasts change sign between validation and TEST, the v1 look used a single seed, and no look covers a server band of intermediate size on REES46.

5.14 Privacy evaluation

Privacy was a stated aim of the project, but the confirmatory measurements of differential-privacy (DP) cost had been taken on federated models that barely moved: the D1 diagnostic had put the DP-noised frozen arm only 0.013% and 0.077% away from θ_{pre} (Table 5.35), so noise had little to destroy. Once the corrected recipe made federated learning learn, the privacy questions had to be asked again. This section restates the confirmatory DP cost, reports the closure-phase DP8 TEST look (disclosure entry 23), the membership-inference audit, the update-leakage audit and the secure-aggregation simulation, and states what none of them shows.

What the privacy evidence does not establish

(i) The legacy DP implementation used a disclosed public seed, so its noise and Poisson participation streams can be reconstructed. The utility numbers remain measurements, but those runs do not establish the randomized-DP guarantee: they carry no formal DP guarantee, and their nominal ε values are labels of the configuration. We therefore call the arms DP-noised, never private. (ii) Non-DP federated learning and local fine-tuning carry no formal privacy claim. (iii) Secure aggregation was simulated only and is not production cryptography. (iv) The canary design has an ε ceiling of about 5.213, below 8, and cannot reject $\varepsilon = 8$. (v) The aggregate-input attack of the update-leakage audit exceeds its fixed threshold: that is a negative finding.

The confirmatory DP cost. The confirmatory retention of DP-noised arms against the same schedule without noise [35, 43] is near 100% on the frozen recipe: K12w 100.00 [99.95, 100.07] for $\varepsilon = 8$ and K13w 100.03 [99.92, 100.15] for $\varepsilon = 1$ (*confirmatory*), and Q6 100.00 [100.00, 100.00] and Q11 99.99 [99.98, 100.00] for the tuned warm-start federated model (*confirmatory, follow-up*). The cold arms show a cost, K12 98.52 [97.70, 99.38] and K13 94.38 [92.53, 96.00] (*confirmatory*), with a seed-sensitivity flag on K12: per-seed retentions of 100.72 and 96.31, a spread of 4.40, and a user-and-seed interval [94.11, 102.92] that includes 100. These numbers say less about DP than about the federated stage: noise is nearly free on the frozen recipe because the stage moved the model by under 1% and added nothing to PRE (K21, Q1), so there was little signal for noise to remove. They are not a claim that DP is free.

DP8 on models that learn: the closure-phase TEST look. The tau-fixed cold arm and the tau-fixed 100k-user pretraining warm arm were rerun with nominal $\varepsilon = 8$ (the DP run applies clipping and noise over 384 Poisson-sampled rounds). On validation the cold arm kept 29.42% (0.03843 against 0.13064) and the 100k-user pretraining warm arm 95.29% (0.1461 against 0.1533) (*exploratory*). The closure-phase DP8 look (registration entry 21, disclosure entry 23) scored the four controlled-budget endpoints once on TEST: 26,254 paired clients, 229,742 decisions, seeds 2026 and 2027, 1,000 bootstrap draws (client-macro MRR@20, seed means). The cold arm scores 0.119700 without DP and 0.037468 with DP8, a difference of -0.082232 $[-0.084017, -0.080490]$ and a retention of 31.30% [30.39, 32.34]; the 100k-user pretraining warm arm scores 0.142186 and 0.134665, a difference of -0.007521 $[-0.008401, -0.006666]$ and a retention of 94.71% [94.10, 95.32]. Both agree with the validation values. The analysis flags the cold retention as seed-sensitive (30.78% and 31.83% by seed); the warm retention is not (94.69% and 94.73%). Both comparisons set the DP8 run against a non-DP run of the same recipe and are whole-recipe contrasts, not isolated noise. The pattern is the one the corrected recipe predicts: a cold start pays heavily for noise (about 69% of the score lost) and a warm start from a pretrain pays little (about 5%).

Membership inference. The membership-inference audit checks whether a model reveals that a training row was used [38, 39]. It scores 10,000 member and 10,000 non-member rows of the TRAIN window with a calibrated membership score and reports the area under the ROC curve (AUC, where 0.5 means no signal), the true-positive rate at 1% false positives and a lower bound on ε derived from the attack, as in DP auditing [41]. Table 5.40 lists the models. The models of the confirmatory run show no material leakage, except the fully fine-tuned central model (AUC 0.6255, lower bound 0.974). The corrected cold federated arm, which actually learns, leaks: AUC 0.7466 [0.7400, 0.7533], a true-positive rate of 0.1426 at 1% false positives and an ε lower bound of 2.112. DP8 brings it to AUC 0.5019. The corrected warm arm from the 100,000-user pretrain shows no material leakage (0.5370 [0.5295, 0.5453]). The audit has limited power: its lower-bound ceiling in the follow-up-analysis record is about 7.2, below 8, so a DP8 run could not fail it, and a pass is the absence of a detected signal, not a proof.

Update leakage: a negative finding. The membership-inference audit examines the final model; the update-leakage audit examines what one client's upload reveals. It replays the first 50 rounds of a tau-fixed

Table 5.40: Membership-inference audit (10,000 members against 10,000 non-members, TRAIN-window rows). The first six rows come from the confirmatory-run models ((*descriptive*)), the last four from the tau-fixed models ((*exploratory*)). Verdicts are those of the audit; DP rows are legacy DP. Source: `privacy_membership_inference.csv`.

Model	Role	AUC	95% CI	TPR@1%	Verdict
FA_WARM_FROZEN	non-DP	0.5036	[0.4957, 0.5112]	0.0088	no material leakage
PRE_FT_CENTRAL	non-DP	0.6255	[0.6176, 0.6332]	0.0290	material leakage
W_STAR	non-DP	0.4971	[0.4893, 0.5053]	0.0118	no material leakage
W_STAR_DP1	DP, $\varepsilon = 1$	0.4973	[0.4893, 0.5053]	0.0074	pass
W_STAR_DP8	DP, $\varepsilon = 8$	0.4972	[0.4893, 0.5053]	0.0068	pass
PRE	null control	0.5000	[0.5000, 0.5000]	0.0000	valid null
FA_COLD_TAUFIX	non-DP	0.7466	[0.7400, 0.7533]	0.1426	material leakage
FA_COLD_TAUFIX_DP8	DP, $\varepsilon = 8$	0.5019	[0.4937, 0.5096]	0.0098	pass
FA_WARM_PB100K_TAUFIX	non-DP	0.5370	[0.5295, 0.5453]	0.0198	no material leakage
FA_WARM_PB100K_TAUFIX_DP8	DP, $\varepsilon = 8$	0.5042	[0.4965, 0.5121]	0.0172	pass

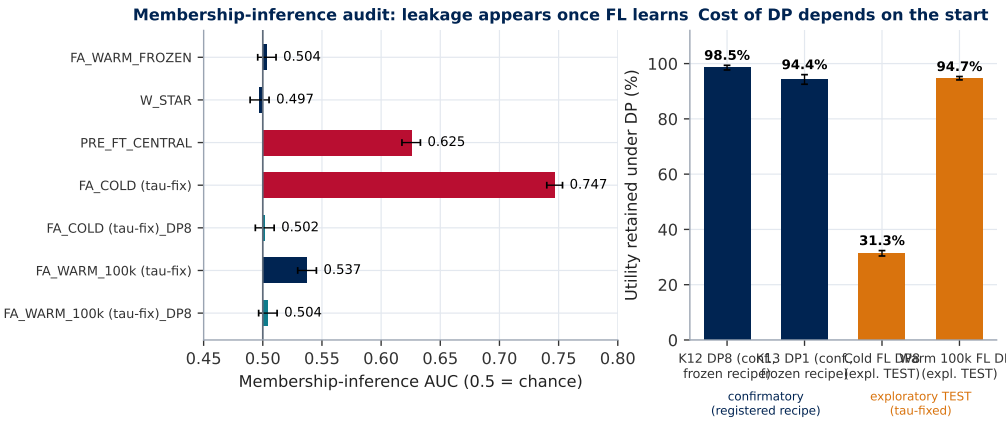


Figure 5.13: Left: membership-inference AUC of the models in Table 5.40 ((*descriptive*) for the confirmatory-run models, (*exploratory*) for the tau-fixed ones). Right: utility retention of DP-noised arms, confirmatory K12 and K13 (*confirmatory*) and the cold and 100k-user pretraining DP8 TEST look (*exploratory*). All DP rows are legacy DP without a formal guarantee. Source: `privacy_membership_inference.csv`, `privacy_dp_cost.csv`.

warm recipe on CALIB data (512 predeclared target clients, 50 cohorts of 64) and asks whether one client's update reveals the items it viewed, and whether hiding it inside a 64-client aggregate, as secure aggregation would, removes that. The pre-declared rule required a single-upload advantage above 10 percentage points (pp) and an aggregate advantage of at most 2 pp (Table 5.41, variant with trainable item tables). A single upload reveals the viewed item set almost exactly: an item-set F1 of 1.0 against 0.0564 for a popularity baseline of matched size, an advantage of 94.36 pp. The ideal 64-client aggregate reduces this to an F1 of 0.0379 against 0.0117, an advantage of 2.625 pp (95% CI 2.312 to 2.953), above the 2 pp threshold. The verdict is that the aggregate attack remains and that secure aggregation is needed but not shown to be sufficient; no privacy certificate is issued. The frozen-table variant uploads no item-table deltas, so the item-set attack does not apply there, and zero deltas are not privacy evidence. A category probe finds no material advantage (single accuracy 0.3636 against a majority baseline of 0.4336), an absence under this probe and not a proof of privacy. The first runs stopped at the optimiser's 2,000-iteration limit; a predeclared, independently reviewed amendment raised the cap to 20,000 before fresh replays, and the first runs were preserved.

Table 5.41: (*exploratory*) The update-leakage audit on CALIB data (trainable item tables; 50 cohorts, 15 holdout cohorts; holdout 154 users for the item set, 143 for the category). Advantages are attack minus baseline in percentage points; intervals are 1,000 resamples of holdout cohorts. The frozen variant is not applicable to the item-set attack. Source: `ANALYSIS_RESULT.json` of the amended runs.

Probe	Single upload	64-client aggregate	Rule
Item set, attack F1	1.0	0.0379 [0.0354, 0.0406]	–
Item set, baseline F1	0.0564 [0.0443, 0.0706]	0.0117 [0.0103, 0.0133]	–
Item set, advantage (pp)	94.36 [92.94, 95.57]	**2.625** [2.312, 2.953]	single $>$ 10 met; aggregate $\leq$ 2 **not met**
Item-set support size (mean)	18.3 (3 to 93)	930.6 (753 to 1,051)	–
Category, advantage (pp)	−6.99 [−13.51, −2.03]	−4.20 [−12.08, 0.00]	no material advantage

Secure aggregation: simulation only. The secure-aggregation experiment (E4) simulated a masking protocol of the family of [36, 37] for 64 clients, dropout rates of 0, 0.1 and 0.3 and a complete or a sparse neighbour graph (16 or 32 neighbours), three cells per setting (*descriptive*). The upload overhead ratio of the 263,822-value encoder update is 1.0089 for the complete graph, 1.0023 for 16 neighbours and 1.0046 for 32; for the 20,708,708-value full-model update it is 1.0001, 1.0000 and 1.0001 (Figure 5.14). All cells decoded bit-identically, except that 16 neighbours at a dropout of 0.3 decoded 2 of 3 cells. The identity control of the update-leakage audit, which sums masked updates in 50 of 50 rounds with a bit-identical result, covers 4,096 raw coordinates and a 512-dimensional sketch, not the full model. These are simulations: they are not production cryptography, and we make no claim about the cost or the security of a deployed protocol.

The canary ceiling. A planned canary audit tests whether inserted canary examples can be distinguished, which turns a DP claim into a measurable lower bound on ε [41, 42]. Its attainable ceiling with 1,000

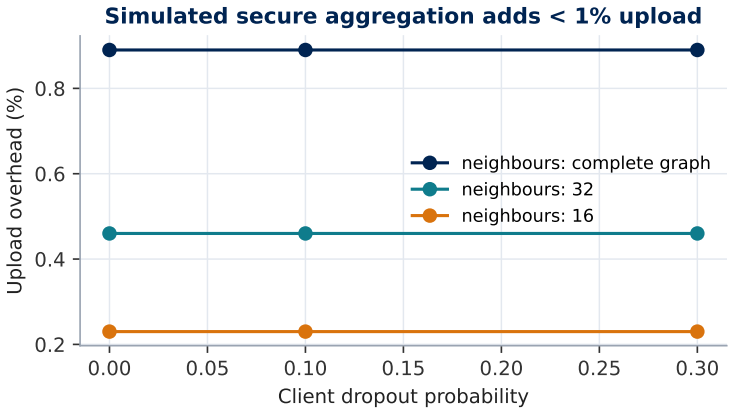


Figure 5.14: (*descriptive*) Simulated secure-aggregation upload overhead ratio for the encoder update (263,822 values) and the full-model update (20,708,708 values), 64 clients, by neighbour graph and dropout. A simulation only; no production cryptography. Source: `secure_aggregation.csv`.

canaries and 9 tests is $\varepsilon \approx 5.213376$ (4.5174 for 500 guesses, 3.5925 for 200), below the target of 8 (*descriptive*). The design therefore cannot reject $\varepsilon = 8$; it is a limit on the power of the audit, not a result about the models. The DP canary restart that would have used it was stopped (Section 5.16).

What we learned

Gained. Federated learning that really learns leaks membership (cold arm AUC 0.7466); a 100,000-user pretrain removes most of that leak (0.5370) and DP8 the rest (0.5019 and 0.5042), at a price that depends on the start: retention 31.30% [30.39, 32.34] cold and 94.71% [94.10, 95.32] for the 100k-user pretraining warm start (TEST, whole-recipe contrasts). The update-leakage audit measured the update leak directly.
Did not work or remains open. The DP results carry no formal guarantee; the aggregate advantage in the update-leakage audit, 2.625 pp, is above its 2 pp threshold; secure aggregation is a simulation; the canary ceiling of about 5.213 cannot reject $\varepsilon = 8$. We make no claim of privacy for any arm.

5.15 On-device feasibility

Local processing only protects behaviour if the model can run where the behaviour happens. The question for the browser is therefore practical: does compressing the model for the device cost accuracy, how large is it, and how fast does it answer? The export path, ONNX at opset 17 with INT8 dynamic quantisation [46, 47], was built in the prototype stage. There INT8 had been accepted on the prototype but rejected for the then-current candidate model (ADR-003 ships FP32), so the question had to be asked again for the SASRec device model.

Accuracy and size. Two confirmatory contrasts bear on compression. K14 compares the 8-bit quantised upload arm (Q8) with the full-precision one, and K16 compares the INT8 model with its FP32 original; their retentions are 100.11 and 99.98, and in both cases the interval includes 100 (*confirmatory*). Compression of the upload and of the model therefore costs no measurable accuracy in the registered design. The

exported tuned warm-start model is 82,863,741 bytes in FP32 (82.9 MB) and 21,243,628 bytes in INT8 (21.2 MB), about 3.9 times smaller (computed).

Latency in Edge. We measured the exported model in Microsoft Edge with WebAssembly [49] on a laptop (Intel i7-12700H, 16 GB, Windows 11; 3 runs), a phone (Snapdragon 8 Gen 3, 12 GB, Android; 2 runs) and a server CPU (Xeon Gold 6338, 4 cores; four browser versions pooled, 12 runs), with one thread and with several (*descriptive*). Table 5.42 and Figure 5.15 give the medians. Warm median latency is between 1.66 and 5.09 ms on the laptop, between 2.19 and 3.70 ms on the phone and between 2.66 and 4.56 ms on the server. INT8 shortens the cold start on all three devices (for instance 670 ms against 915 ms on the laptop and 2,573 ms against 5,168 ms on the phone, one thread), and it is faster in the warm state on the laptop (3.51 ms against 5.09 ms, one thread). It is not faster on the phone (3.70 ms against 2.82 ms, one thread; 2.40 ms against 2.19 ms with several threads), and it is slower than FP32 in the multi-thread server runs (3.02 ms against 2.66 ms), so the benefit of INT8 is size and start-up time, not a guaranteed gain in warm speed. With two or three runs per cell on the laptop and phone, the numbers describe the setup and carry no inference.

Table 5.42: (*descriptive*) Edge WebAssembly latency, FP32 (82.9 MB) against INT8 (21.2 MB), cold start in ms and warm median (p50) and 95th percentile (p95) in ms. Source: `device_latency.csv`.

Device	Threads	Cold FP32	Cold INT8	p50 FP32	p50 INT8	p95 FP32	p95 INT8	Runs
Laptop	1	915	670	5.09	3.51	7.22	5.26	3
Laptop	several	895	695	2.24	1.66	3.36	2.64	3
Phone	1	5,168	2,573	2.82	3.70	4.55	5.82	2
Phone	several	4,863	2,495	2.19	2.40	5.46	6.41	2
Server CPU	1	1,283	1,017	4.56	4.30	6.13	7.56	12
Server CPU	several	1,332	1,089	2.66	3.02	9.21	10.32	12

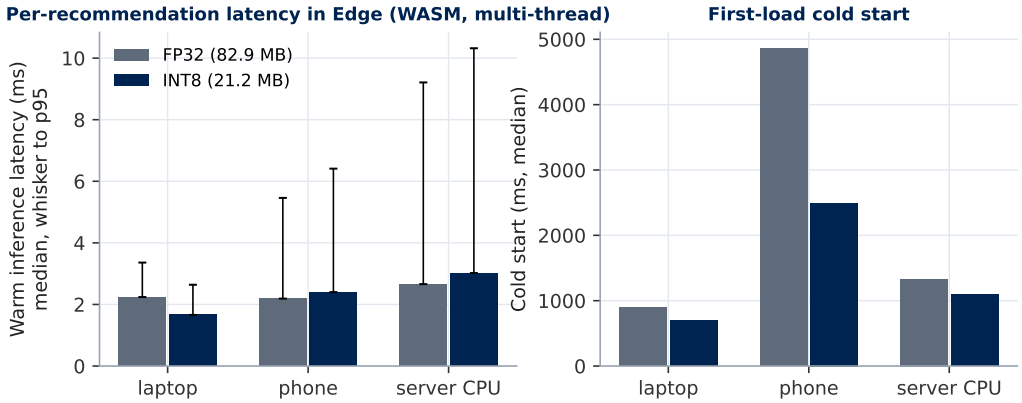


Figure 5.15: (*descriptive*) Edge WebAssembly latency of the exported FP32 and INT8 model on a laptop, a phone and a server CPU, with one and with several threads. Source: `device_latency.csv`.

What we learned

Gained. The device model ships at a quarter of its size (82.9 to 21.2 MB) without a measurable loss (K16 99.98, K14 100.11, both with intervals that include 100) and answers in a few milliseconds in Edge.

Did not work or remains open. INT8 is not faster than FP32 on the phone or in the multi-thread server

runs; the runs per cell are few, and energy, memory over a session and product integration were not measured.

5.16 Closure experiments

The previous study lead left the project on 2026-10-11. After the technical closure had been completed (the update-norm analysis, update-leakage audit, disjoint server-data study, capacity study and DP8 results above), the study lead instructed on 2026-10-13T20:52Z that the master-plan experiments should continue until told to stop. The optional experiments that followed ran in an isolated tree with a write guard: a second pretrain seed, fine-tuning on moving models, training-free baselines, more local passes, FedProx and a personal head, a GRU comparison and a latency pass. Each job had a design note, a source-bound protocol and independent reviewer controls, all carried out by AI agents; these reviews are AI reviews, not external peer review. Every new number in this section is (*exploratory*) or (*descriptive*) (one audit is (*validation*)), read on CALIB or MAIN VALIDATION; confirmatory numbers appear only as anchors, and no TEST data was opened. On 2026-10-16T22:39Z the study lead stopped all experiments, and the project closed at 22:43Z.

A second pretrain seed. All earlier warm runs shared one pretrain, seed 2026. A second pretrain from a different seed (2027; same data, 1,217,610 users, same recipe, 6.0 passes, 564,606 updates) was therefore combined with both federated seeds and with both warm recipes, the frozen tuning-stage-8 recipe and the strong unfrozen recipe, giving four new runs next to the four reused ones. Table 5.43 gives the matched contrasts on MAIN VALIDATION (40,764 users, 519,714 decisions). All eight matched-seed contrasts are negative: the warm score falls by 0.0996 to 0.1153 percentage points when only the pretrain seed changes, with 99.334% to 99.422% retained. The intervals are pointwise and conditional on the four fixed model realisations. This is a measurement of how far the pretrain realisation moves the warm score, and it is larger than the federated effect of the registered design, K21 being −0.0508 pp. It does not repeat the confirmatory design, so it does not tell whether K21 would keep its sign under another pretrain.

Table 5.43: (*exploratory*) Warm federated runs from pretrain seed 2027 minus pretrain seed 2026 (MAIN VALIDATION, client-macro MRR@20, mean over two federated seeds; paired user bootstrap, 1,000 draws; differences in percentage points). CB is the controlled budget, PB the practical-best rule. Source: `PAIRED_CONTRASTS.csv` of the independent review 03.

Recipe	Rule	Pretrain 2026	Pretrain 2027	Δ (pp)	95% CI	Retained (%)
Frozen	CB	0.1725	0.1714	−0.1019	[−0.1707, −0.0244]	99.409
Frozen	PB	0.1725	0.1715	−0.09963	[−0.1748, −0.02218]	99.422
Unfrozen	CB	0.1731	0.172	−0.1099	[−0.1797, −0.03656]	99.365
Unfrozen	PB	0.1732	0.1721	−0.1153	[−0.1895, −0.03872]	99.334

Fine-tuning on moving models. The confirmatory design fine-tuned the shipped model on the device (K9c, +0.002283 [0.001588, 0.003076], (*confirmatory*)). The question here was whether fine-tuning (FT) still helps when it starts from a moving federated model, the cold one or the 100k-user pretraining

one, rather than from PRE (Figure 5.16). On MAIN VALIDATION, FT lifts the cold base from 0.1306 to 0.139, a gain of +0.8324 pp [0.7819, 0.8832] or 106.372% of the base [105.982, 106.794], and the 100k-user pretraining base from 0.1533 to 0.158, a gain of +0.4707 pp [0.41, 0.5347] or 103.070% [102.667, 103.505]. The two seeds agree (cold +0.8279 and +0.8368 pp; 100k-user pretraining +0.4599 and +0.4815 pp). The gain is larger the weaker the base model (0.83 pp cold, 0.47 pp for the 100k-user pretraining variant, 0.2283 pp on the strong PRE), although the last figure is a TEST value and the first two are validation values, so the order is suggestive only. All four arms, 256 shards and 384 metric values were verified; TEST was not accessed.

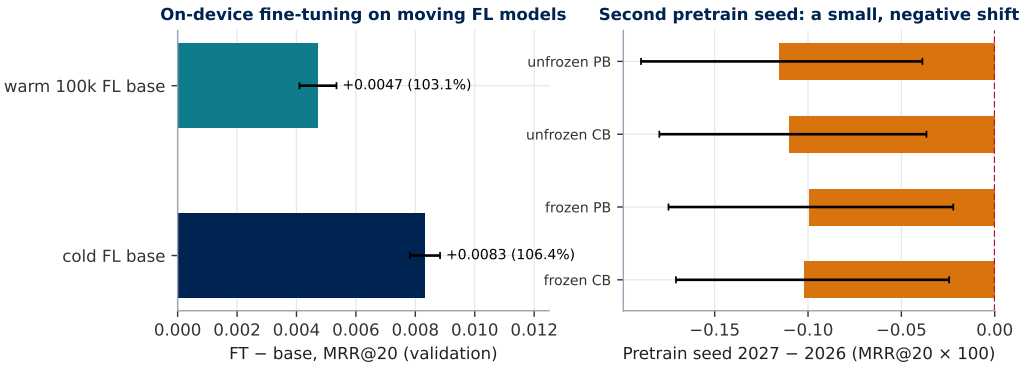


Figure 5.16: (*exploratory*) Closure extensions on MAIN VALIDATION (client-macro MRR@20): fine-tuning on the moving cold and 100k-user pretraining federated models, and warm runs from the second pretrain seed. Source: `PAIRED_SEED_MEAN_CONTRASTS.csv` and `PAIRED_CONTRASTS.csv`.

Training-free baselines. Three training-free sequence baselines, a first-order Markov chain (MC, counts of item-to-next-item transitions), sequential rules (SeqRules) and a third published method (PCTM), were run with the published configuration of the Beauty dataset and no tuning on REES46, ranking the full catalogue of 158,486 products without a seen-item filter or candidate sampling; the counts are causal, so the target of a decision is never part of its query history. The CALIB run (127,530 decisions, 10,070 users) came first as a diagnostic, and MAIN (519,714 decisions, 40,764 users) followed (Figure 5.17). On CALIB the three score 0.1147, 0.1028 and 0.1067; on MAIN VALIDATION 0.1366, 0.1107 and 0.1155 (*exploratory*). Against the session-based nearest-neighbour method SKNN (0.11150 in this closure run; the contest baselines of Section 5.5 gave 0.11138) the paired differences are +0.02506 [0.02372, 0.02631] for MC, −0.0007643 [−0.001767, 0.0003468] for SeqRules and +0.004038 [0.002973, 0.005217] for PCTM (1,000 client bootstrap draws, seed 20260926). The plain Markov chain is a strong baseline: it also scores above the cold federated base (0.1306; the difference −0.005923 is derived by subtraction and has no interval). Fine-tuning on the cold base scores +0.0024 [0.001325, 0.003471] above MC and on the 100k-user pretraining base +0.02148 [0.02031, 0.02282]; the fine-tuned cold base is +0.02823 above SeqRules and +0.02342 above PCTM. The other references on MAIN are POP 0.01943, LAST 0.07111, RULE 0.03769, KNN_EMB_SMALL 0.07294 and KNN_EMB_BIG 0.08197. A federated model from scratch therefore passes a counting baseline only after on-device fine-tuning, and then by a small margin.

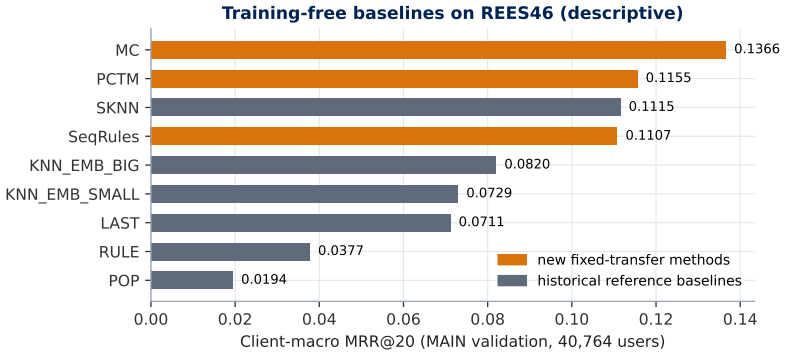


Figure 5.17: (*exploratory*) Training-free baselines and reference methods on REES46 (MAIN VALIDATION, client-macro MRR@20). Source: `METHOD_METRICS.csv`.

More local passes, FedProx and a personal head. The diagnosis of Section 5.11 had found most client visits to be a single local step, which suggested that more local passes might help. On CALIB, with one seed, eight candidates of two and four local passes (P2, P4) were compared with the same-server-step run of one pass (P1). On the practical-best rule all eight are below their matching P1 reference, by 0.0418 pp (100k-user pretraining variant, two passes, step 10^{-4}) to 4.696 pp (cold, four passes, step 3×10^{-4}). The cold winner, two passes at server step 10^{-3}, scores 0.106017, which is 0.7921 pp below its P1 reference (0.1139); the 100k-user pretraining winner, two passes at 3×10^{-5}, is 0.08484 pp below. The only positive entry is a controlled-budget difference of +0.01542 pp for the 100k-user pretraining variant at 10^{-4}. The two 100k-user pretraining candidates differ by 0.000605, inside the tie band of 0.001, so the lower server step wins. The comparison is of whole recipes and not of passes alone, because the source digest of the P1 kernel differs. FedProx (FP, $\mu = 0.01$) and the personal head (PF, $\lambda = 10^{-4}$) differ from the federated-averaging winner by at most 0.00883 pp; the only positive entry is PF on the 100k-user pretraining variant (+0.005005 pp), and neither variant was promoted to MAIN (*exploratory*). Both agree with the earlier reading that the registered local training leaves little for these variants to change.

GRU against C_SMALL: a CPU rehearsal. The backbone contest of Phase 4 had chosen SASRec under the registered rule, with the order reversed at the fixed 6.0-pass endpoint (GRU +0.004213 [0.003283, 0.005216] on validation, (*descriptive*)). As a closing check, a GRU of width 128 (`GRU_D128`) was compared with the matched-size SASRec `C_SMALL` on MAIN VALIDATION with two fixed seeds. The run was a CPU rehearsal, descriptive and without a significance claim, and the budgets differ under the practical-best rule (SASRec 3 passes, GRU 6 passes; both 6 under the controlled budget). The GRU scores 0.157141 and 0.156733 MRR@20 for seeds 2026 and 2027; SASRec scores 0.154250 and 0.154613 at its practical-best mark and 0.152870 and 0.152597 at 6.0 passes (*exploratory*). The GRU is higher by +0.2506 pp in MRR@20 and +0.2109 pp in NDCG@10 under practical-best (seed means), and by +0.4203 pp and +0.423552 pp at 6.0 passes; the advantage is not uniform across metrics, since for seed 2027 the practical-best HR@5 difference is −0.03368 pp. In our contest SASRec scored higher under the registered practical-best rule, and in this rehearsal the GRU scored higher; we draw no general ranking of the two families. A first review of the run asserted equal serialised checkpoint bytes and failed; the

state digests and saved ranks coincide, so this was a review error and not an experimental failure, and the second review passed (1,152 rank-derived checks, maximum error 0.0).

L6: latency on a shared server CPU. The last latency pass (L6) measured the FP32 and INT8 models with one thread on a shared server CPU, with 1,000 queries after 50 warm-ups and seed 2026 (*descriptive*). The FP32 median is 2.250102 ms (p95 2.319876 ms) and the INT8 median 0.993833 ms (p95 0.998065 ms), a median ratio of 2.264065. This is single-thread CPU latency on a shared server and not a laptop or a phone, and the raw per-query times were not retained. It shows INT8 about 2.3 times faster than FP32 on one thread of that CPU, a different setting from the Edge measurements of Table 5.42, and the two must not be read as one result. A separate audit of seed 2028 on 12 fixed federated pairs found none above the strict difference limit of 0.002 (largest 0.00170 for cold ML-1M and 0.00118 for warm Toys) (*validation*).

What was stopped, failed or never run. Several planned experiments produced no result. The study lead interrupted the first DP-FTRL and canary trainings and the four shared-GPU DP-FTRL cells (*stopped*); the partial models were not adopted and are not scientific negatives, and the four DP-FTRL outputs, composed, would give an ε of about 18.0056 and not a joint 8. The DP canary restart was paused and then stopped (*stopped*). The three Cold-DP calibration runs failed before training because a MAIN-only recipe identifier was combined with the calibration split, so the four Cold-DP training cells never started (*stopped*). Local DP and production secure aggregation were not run, and no TEST2 data (December 2019) exists, so no later-period confirmation was possible. Four launches were refused or failed before any GPU training and were repaired by metadata-only changes with independent controls, among them a latent missing-runner defect in the canary operator, the DP-FTRL trainer and its scorer; no result was lost.

What we learned

Gained. A second pretrain seed moves the warm score by about 0.1 pp (all eight matched contrasts negative), more than the registered federated effect of −0.0508 pp; fine-tuning on a moving model gains +0.8324 pp on the cold base and +0.4707 pp on the 100k-user pretraining base; a plain Markov chain (0.1366) sets a floor that a cold federated model passes only with fine-tuning; the GRU rehearsal and the L6 pass complete the comparison and latency picture.

Did not work or remains open. More local passes were below one pass in all eight CALIB cells, FedProx and a personal head changed almost nothing, DP-FTRL, the DP canary restart and Cold-DP calibration produced no results, the GRU run is a CPU rehearsal, L6 is shared-server CPU latency, and no TEST2 data exists. The closure covers the selected code and the completed results, not every option of the master plan.

5.17 Summary of all experiments

Table 5.44 lists the experiments of all 14 phases in the order in which they were run, with the label of the evidence, the headline result, what the experiment gained and what did not work. The confirmatory rows are the two registered test evaluations; every other row prepared them or explains them. The table repeats

numbers from the sections above and from earlier sections of this chapter and does not replace them.

Table 5.44: Summary of all experiments, phases 01 to 14. Evidence labels as defined at the start of the chapter. Source: experiment timeline, phase files 01 to 14.

Ph.	Experiment	Label	Headline	Gained	Did not work
01	Prototype stage (GitHub)	(*validation*)	T1 88.38%, T2 58.71% against matched centralised pilots (not comparable with K20b)	Data foundation, hashing, CI, ONNX and INT8 path	T2 label censoring; negative transfer; INT8 rejected on the real candidate (ADR-003)
02	Pilot studies	(*descriptive*)	GRU 0.1739 (VS); SASRec lock A1_RICH256_s2027, VS 0.186513	Frozen data and evaluator	GRU pilot failed its gate (0.100795 against 0.11977); SASRec v2 drift; VS margin +0.012639 shrank to +0.001780 on TEST
03	Main study launch, census, features	(*descriptive*)	4,804,933 users, 40,180,608 examples; 5-100 pool 1,446,986 users	Protocol lock, S* with 25 channels	Five features dropped; no reportable model
04	Backbone contest	(*confirmatory*) (*validation*)	GRU − SASRec −0.004271 [−0.005180, −0.003261]; reversed at 6.0 passes (+0.004213)	SASRec selected under the registered rule	Not parameter-matched (83.9M against 21.0M)
05	Calibration, federated recipe tuning stages, MAIN runs	(*validation*)	Cold tuning stages 0.023473 to 0.107562 (factor 4.58); warm tuning stage 8 0.170709	Registered recipes	Grid-edge winner, superseded runs; legacy-DP constants without a formal guarantee
06	Benchmark design, cold-start chain	(*descriptive*)	12 of 12 published NDCG@10 baselines reproduced to 4 decimals	Verified pipeline	First runner review failed (1 blocker, 7 major) before any run
07	MAIN completion, VALIDATION read	(*validation*)	K20b 88.81% [88.27, 89.36]; K21 −0.000433 [−0.000758, −0.000101]	FL adds nothing to PRE; FT_PRE best deployable arm	Three tool bugs; validation is best of 12 marks (optimistic)
08	Follow-up analysis build, handover, GitHub port	(*descriptive*)	13 review verdicts (1 FAIL, 1 PASS, 11 PASS_WITH_MINORS)	Byte-identity to the prototype	Chain stalled about 7 h (INC-088)
09	Follow-up analysis runs: server-step grid, devices, membership-inference audit, E4, 100k-user pretraining band	(*descriptive*)	22 server-step grid cells within ±0.00012 of 0.171302; ONNX 82,863,741 B to 21,243,628 B	Tuned warm-start model equals PRE in effect; membership-inference audit, E4, device data	DP-FTRL, canary, update-leakage audit, LDP not run; scoring bug stopped the first test evaluation
10	Confirmatory test evaluation, REES46	(*confirmatory*) (*confirmatory, follow-up*)	K20b 88.49% [87.73, 89.18]; K21 −0.000508 [−0.000933, −0.000037]; Q1 −0.000081 [−0.000179, 0.000024] EQUIVALENT	K9c +0.002283 [0.001588, 0.003076]	Federated rounds added nothing; DEPLOYMENT-GRADE False; Holm family rejected nothing in the follow-up analysis
10	Benchmark TEST looks	(*confirmatory*)	Best federated arm 13% to 77% of C_FULL (matched-recipe protocol)	Central reference best everywhere	Federated arms far below; incident I055

Ph.	Experiment	Label	Headline	Gained	Did not work
11	Tau diagnosis, T1, T2	(*exploratory*)	Server step 20 to 100 times too small; T1 75% to 92% of C_FULL; T2 +0.00022 [−0.00011, 0.00057]	Measured mechanism; K21 and Q1 not a tau artefact alone	Several ingredients changed at once; corrected cold arm worse (−0.00680)
11	100k-user pretraining v1 and v2, disjoint server-data study, capacity study	(*exploratory*) (*validation*)	Retention 73.79%, 77.91%, 78.98% against PRE 88.77%	Value of FL depends on server data	ML-1M exceptions; FL does not replace server data
11	Privacy redo: DP8, membership-inference audit	(*exploratory*)	Cold membership-inference AUC 0.7466; DP8 0.5019	FL that learns leaks; DP8 lowers the measured AUC to 0.5019	Legacy DP, no formal guarantee
12	Final handover, public release	(*descriptive*)	PRs #103 to #114; results CSVs and tau fix published	Reusable modules	No human PR review; some inconsistent documents
13	Technical closure: update-norm analysis, update-leakage audit	(*descriptive*) (*exploratory*)	44 checks PASS; update-norm analysis 29 logs, 100,942 rounds; update-leakage aggregate 2.625 pp	Measured update leak	Update-leakage audit negative; one operator error restored
13	Disjoint server-data and capacity studies and DP8 TEST looks	(*exploratory*)	Warm beats cold in all disjoint server-data settings; DP8 31.30% cold, 94.71% 100k-user pretraining	Server band and DP cost on TEST	No capacity gain; legacy DP
14	Second pretrain seed, FT, training-free, local passes, FP/PF, GRU, L6	(*exploratory*) (*descriptive*)	8 of 8 negative matched contrasts; FT +0.8324 pp cold; MC 0.1366; 8 of 8 local-pass cells below P1; GRU − SASRec +0.2506 pp	Seed variance, FT gain, baseline floor	CPU rehearsal; shared-server latency
14	DP-FTRL, DP canary restart, Cold-DP calibration	(*stopped*)	No result	Canary ceiling of about 5.213 documented	Stopped by the study lead or failed before training

What we learned

Gained. A single confirmatory test evaluation with a documented outcome (the edge system FA_WARM_FROZEN, federated training from the server pretrain with frozen item tables, keeps 88.49% [87.73, 89.18] of the cloud reference C_FULL, and federated rounds add nothing to the 1.2M-user server model on REES46), a measured explanation (the registered FedAdam setting), a quantified role of the server data, an honest privacy picture and a feasibility check on three devices.

Did not work. Federated rounds on top of a strong 1.2M-user pretrain (they add nothing there, but a lot when the pretrain is weak), more local passes, FedProx and the personal head, a larger device model, DP-FTRL, the DP canary, the Cold-DP calibration, and any claim of a formal DP guarantee. The exploratory results explain the confirmatory ones; they do not replace them.

Chapter 6: Discussion

Chapter 5 reported the experiments one by one, in the order in which they were run. This chapter steps back. It first answers the research question and checks the work against the objectives set in Section 1.4, then interprets the main findings, draws practical implications for a browser-based shopping assistant, relates the results to prior work, and states the limitations that bound every claim. It closes with lessons about the research process itself.

6.1 Answering the research question

The research question asked how much shopping recommendation quality can be retained when moving from cloud-based to on-device or federated models. On the pre-registered recipe, measured once on the sealed TEST population of 26,254 users and 229,742 next-item decisions, the answer is:

- The *realistic edge system*—a device-size SASRec trained with FedAdam, warm-started from a model pretrained on the server-side users, with frozen item tables (FA_WARM_FROZEN)—retains **88.49%** of the full-data cloud model C_FULL in client-macro MRR@20 (95% CI 87.73–89.18) (*confirmatory*).
- The pretrained model alone (PRE) already retains 88.77%, and federated training on top of it changes MRR@20 by −0.000508 (CI −0.000933 to −0.000037) (*confirmatory*). The tuned recipe of the follow-up analysis is statistically equivalent to the pretrained model (−0.000081, read as EQUIVALENT within ±0.0005) (*confirmatory, follow-up*).
- Without a server-side start, a federated model trained from scratch retains 70.10% of C_FULL, and a model trained on each device alone retains 1.64% (*confirmatory*).
- Fine-tuning the shipped model on the device adds +0.002283 MRR@20 (CI 0.001588–0.003076) (*confirmatory*), a small but reliable gain that needs no federated traffic.

In short: most of the cloud quality survives on the device, but on REES46 it survives because of the server-side pretraining, not because of federation. The post-hoc work qualifies the answer. The registered FedAdam setting almost froze federated training, although correcting it leaves the REES46 answer unchanged. With the corrected recipe, federation reaches 75–92% of the cloud model on six public benchmarks, and it adds +0.0091 MRR@20 over a pretrained model built from only 100,000 users (*exploratory*). How much federation is worth therefore depends on how much the server already knows.

6.2 Were the objectives met?

Table 6.1 checks each objective of Section 1.4 against the evidence.

Objective O4 is the only one that was not fully met, and the shortfall is substantive rather than cosmetic.

Table 6.1: Assessment of the research objectives defined in Section 1.4.

Obj.	Objective	Status	Evidence
O1	Retention relative to cloud references, full-catalogue ranking	Met	33 confirmatory contrasts on REES46 TEST; six benchmarks (Section 5.10)
O2	Value of server pretraining and of federation on top	Met	K20c, K21, K_INIT, follow-up Q1; post-hoc 100k-user pretraining variant and disjoint server-data studies
O3	On-device adaptation	Met	K8 (device-only), K9c and P5 (fine-tuning); closure study of fine-tuning on moving models
O4	Utility cost of DP, leakage, secure-aggregation overhead	Partly met	DP cost, membership and update audits measured; formal DP guarantee not established; secure aggregation simulated only
O5	Deployability (size, latency, upload)	Met, with qualifications	INT8 and 8-bit uploads without measurable cost; browser latency on laptop, phone and server CPU
O6	Pre-registered, auditable protocol and reproducible code	Met	Locks, gated looks, test-exposure register, decision log; public repository and merged archive

The utility cost of differential privacy and the leakage of non-private updates were measured, but the closure audit found that the legacy implementation does not establish a formal guarantee (Section 6.6).

6.3 Interpreting the main findings

The strength of the pretrain decides the value of federation. The most important single ingredient of the edge system was not federation but the warm start. Starting federated training from the server-pretrained model instead of a random initialisation added +0.0331 MRR@20 (*confirmatory*), which is far larger than any federated or on-device increment we measured. Once the start was strong, federation had nothing left to add on REES46, with the registered recipe (K21) or with the corrected one (+0.00022, CI including zero) (*exploratory*). When the pretrained model was weaker, built from 100,000 instead of 1.2 million users, the corrected federated recipe added +0.0091 MRR@20 (*exploratory*). The picture is consistent with work showing that pretraining and initialisation substantially improve the outcome of federated optimisation [27, 28]. Federation is valuable in proportion to what the server does *not* already know.

Optimiser scale as a silent failure mode. The registered recipe was valid by every rule of the protocol, yet badly scaled. FedAdam divides the server step by $\sqrt{v} + \tau$ [25]. With pseudo-gradients around 10^{-7} and $\tau = 10^{-3}$, the server step was 20–100 times smaller than intended, and the tuned model moved only about 0.06% away from its starting point in 13,654 rounds. Nothing in the confirmatory protocol could detect this, because the protocol controlled *which* recipe was evaluated, not whether the recipe was sensible. The lesson generalises: adaptive server optimisers need their stabilising constant set relative to the actual update magnitudes, and update norms should be monitored as a routine health check.

Learning and leakage arrive together. While federated training was effectively frozen, it was also nearly harmless: the membership-inference audit found no material leakage for the tuned warm-start federated model (*descriptive*). Once federation learned, the cold federated model leaked membership with an attack AUC of 0.747, and differential privacy at $\varepsilon = 8$ brought the measured attack back to chance (*exploratory*). The utility price of that protection depended on the starting point. Under the exploratory TEST look, the warm 100k-user model kept 94.71% of its non-private quality, while the cold model kept only 31.30% (*exploratory*) (legacy DP with no formal guarantee; the cold contrast also differs from its non-private reference in recipe and budget). A protection that looks free on a model that does not learn is not evidence that it is free in general. This is why the confirmatory DP results (98.52% and 94.38% retention for $\varepsilon = 8$ and $\varepsilon = 1$) must not be read as "DP is free".

On-device fine-tuning is the deployable gain. Fine-tuning the shipped model on each device improved MRR@20 reliably, both on the confirmatory pretrained model and, in the closure study, on federated base models. It needs no upload at all, so it carries no update-leakage risk of the kind measured in the membership-inference and update-leakage audits. Training on the device alone, by contrast, is unusable: one user's history is far too short to learn a catalogue of 158,486 products.

Federation, not model size, is the gap. On the six public benchmarks, a central model of device size reached between 89.78% and 110.26% of the large central model on TEST (*exploratory*), so shrinking the model to fit a device costs little. With the corrected recipe, warm federated models reached between 78% and 86% of a central model of the same size (*exploratory*). That gap of roughly 14–22% is the real cost of federation under our simulated conditions. Investment should therefore go into better federated optimisation and stronger server-side starts rather than into larger device models.

6.4 Practical implications for a browser shopping assistant

For a browser vendor considering on-device shopping recommendation, our results suggest a concrete design.

1. **Ship a strong pretrained model.** Build it from consenting or otherwise legitimately available server-side data, and quantise it to INT8: the model shrinks from 82.9 MB to 21.2 MB with no measurable quality loss, and a recommendation takes a few milliseconds in the browser runtime.
2. **Fine-tune locally.** This gives a small, reliable gain with no upload.
3. **Federate only when it can pay off.** Use federated rounds when the server's data is weak or unrepresentative, and only with an optimiser whose scale has been checked against the update magnitudes.
4. **Engineer and audit privacy.** If updates are uploaded, use real secure aggregation and differential privacy with secret randomness and an audited accountant. Do not rely on the absence of raw-data upload as a privacy argument.

6.5 Relation to prior work

Our central reference is competitive with published full-ranking results on the public benchmarks. On MovieLens-1M it scored above the published eSASRec value [18]. On the Amazon datasets it scored below the strongest published models, including simple transition-based methods that a recent study found to be competitive on standard benchmarks [19]. Comparisons with most federated-recommendation papers are only qualitative, because those papers usually rank sampled candidates, which can reorder methods [6]. Our privacy findings agree with the broader literature that model updates and trained models can leak membership [38, 39] or even training inputs [40], and that empirical audits can bound but not certify privacy [41, 42].

6.6 Limitations and threats to validity

Every claim in this report is bounded by the following limitations, which we state explicitly.

- **Simulated clients and one main dataset.** Devices were simulated on GPU servers, and the main study uses one e-commerce log from two months of 2019. A December-2019 replication (TEST2) was planned but not run.
- **Single pretraining seed.** Both warm federated seeds share one server-pretrained model, so their spread measures federated variance only. A second pretraining seed, run during the closure, shifted the warm results by a small negative amount.
- **Recipe details.** FedProx is inert at one local step. Arms with frozen item tables score zero for target items outside the pretraining band.
- **Extra test exposures.** Register entries 12–23 are exploratory looks at TEST after the confirmatory test evaluation. They are disclosed and labelled, but they create selection pressure on the test set.
- **Confounded re-runs.** In the corrected re-runs, τ changed together with the learning rate, the training budget or the item-table policy, so the gain cannot be attributed to τ alone.
- **Benchmark qualifications.** MovieLens-1M ran on a GPU and the other benchmarks on CPUs, so ratios are comparable only within a dataset. The Amazon-2023 datasets are descriptive. Sampled-metric results from the literature are not comparable with our full-ranking numbers.
- **Privacy implementation.** The legacy DP implementation used a disclosed seed that allows its noise and sampling streams to be reconstructed, so the DP-labelled runs do not establish a formal randomised-DP guarantee. The aggregate attack in the update-leakage audit exceeded its 2-percentage-point threshold (2.625 pp). Secure aggregation was a simulation. The canary audit could not reject $\varepsilon = 8$ by construction (ceiling about 5.213).
- **Measurement settings.** The closure GRU comparison was a CPU rehearsal, and the closure latency study used one thread on a shared server.

- **AI-assisted reviews.** The independent reviews of code and decisions were carried out by AI agents, not by human domain experts.

6.7 Lessons about the research process

What pre-registration bought. Locking the analysis before the test was read made the confirmatory answer credible. It would have been easy to promote a favourable exploratory variant after seeing the results, and the protocol made that impossible. The gates also caught real defects before any test value was read. Two analysis refusals before the validation read exposed definitions that disagreed with the registered writers, and a refusal at the start of the test evaluation exposed an endpoint error.

What it cost and what it missed. The protocol was expensive. It required write-once locks, independent reviews, register entries and inventory checks for every look, and it slowed the project in its final weeks. It also could not detect a recipe that was valid but badly scaled; the τ problem was found only by measuring update norms after the test evaluation. A protocol controls choices, not their quality.

Working with AI agents. AI agents helped write and review code and analyses of the main study under the team's supervision. This made a study of this size feasible in a few weeks, but it introduced its own incidents: concurrent writers duplicating decision-log rows, an AI worker agent running tests against a protected tree, and sessions cut by usage limits. The defences that worked were the same as for human teams: state kept in files, append-only logs, anchored process control and independent checks.

What we would do differently. We would monitor optimiser health (update norms and the share of updates below τ) from the first federated run. We would train more than one server-pretraining seed. We would treat secret randomness for differential privacy as a tested requirement. And we would plan the temporal replication on fresh data early enough to run it.

Chapter 7: Conclusion and Future Work

7.1 Summary of the work

This project set out to measure how much next-product recommendation quality can be retained when shopping behaviour stays on the user's device. We built a prototype with a full data pipeline, three shopping tasks, federated plumbing, and tooling for privacy-motivated feature encoding (hashing and price bands) and ONNX/INT8 deployment. From its lessons we designed the main pre-registered study.

The main study ranks the full catalogue of 158,486 products of the REES46 log for 131,072 simulated devices. The device-size models are compared with cloud references trained on up to 1,348,682 users. A sealed TEST population of 26,254 users was scored once. After that confirmatory test evaluation we investigated why federation added so little, corrected the optimiser setting in a series of gated exploratory looks, measured the value of server-side data, audited privacy leakage and measured browser latency. In a closure phase we verified every saved result, ran additional seed, fine-tuning and baseline studies, documented what was stopped, and merged the whole project into a single verified archive.

7.2 Answer to the research question

On the registered recipe, the realistic edge system—federated training warm-started from a server-pretrained device model—retains **88.49%** of the full-data cloud model (95% CI 87.73–89.18, client-macro MRR@20) (*confirmatory*). That quality comes from the server-side pretraining. Federation adds nothing over the pretrained model (−0.000508 MRR@20) (*confirmatory*), and a tuned follow-up is statistically equivalent to it (*confirmatory, follow-up*). On-device fine-tuning adds a small, significant +0.002283 (*confirmatory*).

After the test evaluation, we found that the FedAdam constant $\tau = 10^{-3}$ was mis-scaled for update magnitudes around 10^{-7} and had almost frozen federated training; correcting it does not change the REES46 answer. With the corrected recipe (several ingredients changed together, not τ alone), federated models reach 75–92% of the large central model `C_FULL` on six public benchmarks (*exploratory*). Federation adds +0.0091 MRR@20 when the server's pretraining used only 100,000 users, and nothing measurable when it used 1.2 million (*exploratory*).

The answer to the research question is therefore twofold. Most of the cloud quality can be retained on the device. Whether federated training contributes to that depends on how much the server already knows and on a correctly scaled optimiser.

7.3 Contributions

The project contributes: (i) a pre-registered, full-catalogue comparison of cloud, federated and on-device sequential recommendation on a large real shopping log; (ii) the evidence that the server-side pretrained start, not federation, carries the edge system's quality when the pretrain is strong; (iii) the diagnosis of a silent optimiser-scale failure in adaptive federated optimisation, and evidence that the value of federation depends on server-side data; (iv) a privacy evaluation that pairs utility costs with membership and update-leakage audits and records the implementation limits honestly; (v) an on-device pipeline with INT8 export and browser latency measurements; and (vi) a reproducible code base, a complete decision log and a verified project archive.

7.4 Future work

Several directions follow directly from our findings and limitations.

- **Teacher distillation.** A large teacher model trained on opted-in users could be distilled into the device model before any federated round. This would strengthen the shipped model without moving further data.
- **Differential privacy done right.** A future study should use secret randomness, an audited accountant and the full client pool, so that a formal guarantee can be claimed. It should also complete the DP-FTRL and canary-audit studies that were stopped.
- **Real secure aggregation.** A cryptographic implementation, measured on real networks, should replace the simulation.
- **Temporal replication.** The confirmatory comparison should be repeated on fresh data, such as December 2019 (TEST2), to test whether the conclusions survive a change of period.
- **More pretraining seeds and optimiser monitoring.** Update norms should be tracked from the first round, and several server-pretraining seeds should be trained to measure the variance the current design cannot see.
- **A field trial.** A controlled deployment inside the browser on real devices would test latency, battery and user-facing quality beyond offline metrics.

7.5 Closing remarks

The most useful outcome of this project may be the combination of a clear confirmatory answer with an equally clear account of its limits. A shopping recommender can live mostly on the device. Federated learning is a tool whose value depends on context and configuration rather than a guarantee of better models, and privacy must be engineered and audited rather than assumed. We have tried to report not only what worked, but also what did not and why. We believe that record is as valuable to the next team as the

final numbers.

References

[1] B. Hidasi, A. Karatzoglou, L. Baltrunas, and D. Tikk, "Session-based recommendations with recurrent neural networks," in *International Conference on Learning Representations (ICLR)*, 2016.

[2] W.-C. Kang and J. McAuley, "Self-attentive sequential recommendation," in *IEEE International Conference on Data Mining (ICDM)*, 2018, pp. 197–206.

[3] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 54, 2017, pp. 1273–1282.

[4] P. Kairouz, H. B. McMahan *et al.*, "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[5] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Foundations and Trends in Theoretical Computer Science*, vol. 9, no. 3–4, pp. 211–407, 2014.

[6] W. Krichene and S. Rendle, "On sampled metrics for item recommendation," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2020, pp. 1748–1757.

[7] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, "The preregistration revolution," *Proceedings of the National Academy of Sciences*, vol. 115, no. 11, pp. 2600–2606, 2018.

[8] M. Kechinov, "eCommerce behavior data from multi category store," Kaggle dataset (REES46 Marketing Platform), https://www.kaggle.com/datasets/mkechinov/ecommerce-behavior-data-from-multi-category-store, 2020.

[9] S. Wang, L. Cao, Y. Wang, Q. Z. Sheng, M. A. Orgun, and D. Lian, "A survey on session-based recommender systems," *ACM Computing Surveys*, vol. 54, no. 7, pp. 1–38, 2022.

[10] J. McAuley, C. Targett, Q. Shi, and A. van den Hengel, "Image-based recommendations on styles and substitutes," in *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2015, pp. 43–52.

[11] Y. Hou, J. Li, X. Fu, Z. He, A. Yan, X. Chen, and J. McAuley, "Bridging language and items for retrieval and recommendation: Benchmarking LLMs as semantic encoders," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, San Diego, CA, USA, 2026, pp. 3251–3265, arXiv:2403.03952.

[12] F. M. Harper and J. A. Konstan, "The MovieLens datasets: History and context," *ACM Transactions on Interactive Intelligent Systems*, vol. 5, no. 4, pp. 19:1–19:19, 2015.

[13] B. Hidasi and A. Karatzoglou, "Recurrent neural networks with top-k gains for session-based recommendations," in *Proceedings of the 27th ACM International Conference on Information and Knowledge Management (CIKM)*, 2018, pp. 843–852.

[14] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[15] F. Sun, J. Liu, J. Wu, C. Pei, X. Lin, W. Ou, and P. Jiang, "BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer," in *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*, 2019, pp. 1441–1450.

[16] A. Klenitskiy and A. Vasilev, "Turning dross into gold loss: is BERT4Rec really better than SASRec?" in *Proceedings of the 17th ACM Conference on Recommender Systems (RecSys)*, 2023, pp. 1120–1125.

[17] A. V. Petrov and C. Macdonald, "gSASRec: Reducing overconfidence in sequential recommendation trained with negative sampling," in *Proceedings of the 17th ACM Conference on Recommender Systems (RecSys)*, 2023, pp. 116–128.

[18] D. Tikhonovich, N. Zelinskiy, A. V. Petrov, M. Spirina, A. Semenov, A. V. Savchenko, and S. Kuliev, "eSASRec: Enhancing transformer-based recommendations in a modular fashion," in *Proceedings of the 19th ACM Conference on Recommender Systems (RecSys)*, 2025, pp. 1175–1180.

[19] A. V. Petrov, P. Chandar, P. N. Bennett, H. Bouchard, and M. Lalmas, "Do sequential recommendation benchmarks really require higher-order sequence modelling?" in *Proceedings of the 20th ACM Conference on Recommender Systems (RecSys)*, 2026, pp. 1698–1701, arXiv:2608.19833; code: https://github.com/spotify-research/sequential-capacity-probes.

[20] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, J. Fernandez-Marques, Y. Gao, L. Sani, K. H. Li, T. Parcollet, P. P. B. de Gusmão, and N. D. Lane, "Flower: A friendly federated learning research framework," *arXiv preprint arXiv:2007.14390*, 2020.

[21] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," *arXiv preprint arXiv:1909.06335*, 2019.

[22] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 2, 2020, pp. 429–450.

[23] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 119, 2020, pp. 5132–5143.

[24] X. Li, M. Jiang, X. Zhang, M. Kamp, and Q. Dou, "FedBN: Federated learning on non-IID features via local batch normalization," in *International Conference on Learning Representations (ICLR)*, 2021.

[25] S. J. Reddi, Z. Charles, M. Zaheer, Z. Garrett, K. Rush, J. Konečný, S. Kumar, and H. B. McMahan, "Adaptive federated optimization," in *International Conference on Learning Representations (ICLR)*, 2021.

[26] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.

[27] J. Nguyen, J. Wang, K. Malik, M. Sanjabi, and M. Rabbat, "Where to begin? On the impact of pre-training and initialization in federated learning," in *International Conference on Learning Representations (ICLR)*, 2023.

[28] H.-Y. Chen, C.-H. Tu, Z. Li, H.-W. Shen, and W.-L. Chao, "On the importance and applicability of pre-training for federated learning," in *International Conference on Learning Representations (ICLR)*, 2023.

[29] M. Ammad-ud din, E. Ivannikova, S. A. Khan, W. Oyomno, Q. Fu, K. E. Tan, and A. Flanagan, "Federated collaborative filtering for privacy-preserving personalized recommendation system," *arXiv preprint arXiv:1901.09888*, 2019.

[30] G. Lin, F. Liang, W. Pan, and Z. Ming, "FedRec: Federated recommendation with explicit feedback," *IEEE Intelligent Systems*, vol. 36, no. 5, pp. 21–30, 2021.

[31] V. W. Anelli, Y. Deldjoo, T. Di Noia, A. Ferrara, and F. Narducci, "FedeRank: User controlled feedback with federated recommender systems," in *Advances in Information Retrieval: 43rd European Conference on IR Research (ECIR)*, ser. Lecture Notes in Computer Science, vol. 12656. Springer, 2021, pp. 32–47.

[32] K. Muhammad, Q. Wang, D. O'Reilly-Morgan, E. Tragos, B. Smyth, N. Hurley, J. Geraci, and A. Lawlor, "FedFast: Going beyond average for faster training of federated recommender systems," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2020, pp. 1234–1242.

[33] C. Zhang, G. Long, T. Zhou, P. Yan, Z. Zhang, C. Zhang, and B. Yang, "Dual personalization on federated recommendation," in *Proceedings of the 32nd International Joint Conference on Artificial Intelligence (IJCAI)*, 2023, pp. 4558–4566.

[34] W. Yuan, C. Yang, G. Ye, T. Chen, Q. V. H. Nguyen, and H. Yin, "FELLAS: Enhancing federated sequential recommendation with LLM as external services," *ACM Transactions on Information Systems*, vol. 43, no. 6, pp. 1–24, 2025.

[35] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang, "Learning differentially private recurrent language models," in *International Conference on Learning Representations (ICLR)*, 2018.

[36] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017, pp. 1175–1191.

[37] J. H. Bell, K. A. Bonawitz, A. Gascón, T. Lepoint, and M. Raykova, "Secure single-server aggregation with (poly)logarithmic overhead," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2020, pp. 1253–1269.

[38] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *IEEE Symposium on Security and Privacy (S&P)*, 2017, pp. 3–18.

[39] N. Carlini, S. Chien, M. Nasr, S. Song, A. Terzis, and F. Tramèr, "Membership inference attacks from first principles," in *IEEE Symposium on Security and Privacy (S&P)*, 2022, pp. 1897–1914.

[40] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[41] M. Jagielski, J. Ullman, and A. Oprea, "Auditing differentially private machine learning: How private is private SGD?" in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 22 205–22 216.

[42] T. Steinke, M. Nasr, and M. Jagielski, "Privacy auditing with one (1) training run," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 49 268–49 280.

[43] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep learning with differential privacy," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016, pp. 308–318.

[44] I. Mironov, "Rényi differential privacy," in *IEEE 30th Computer Security Foundations Symposium (CSF)*, 2017, pp. 263–275.

[45] Y.-X. Wang, B. Balle, and S. P. Kasiviswanathan, "Subsampled rényi differential privacy and analytical moments accountant," in *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 89, 2019, pp. 1226–1235.

[46] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[47] ONNX Runtime developers, "ONNX Runtime: cross-platform, high performance ML inferencing and training accelerator," https://onnxruntime.ai, 2021.

[48] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

[49] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. F. Bastien, "Bringing the web up to speed with WebAssembly," in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2017, pp. 185–200.

[50] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.

[51] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[52] D. J. Schuirmann, "A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability," *Journal of Pharmacokinetics and Biopharmaceutics*, vol. 15, no. 6, pp. 657–680, 1987.

[53] J.-P. Aumasson, S. Neves, Z. Wilcox-O'Hearn, and C. Winnerlein, "BLAKE2: Simpler, smaller, fast as MD5," in *Applied Cryptography and Network Security (ACNS)*, ser. Lecture Notes in Computer Science, vol. 7954. Springer, 2013, pp. 119–135.

[54] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019.

Chapter A: Pre-registration lineage, locks and gates

This appendix lists, in order, the registered rules under which the confirmatory results of Chapter 5 were produced. Pre-registration [7] was implemented with amendments to the first pre-registration (PREG1) and a second one (PREG2), kept as hashed files under `contracts/`; with locks and freezes written once under `company/gates/`; and with rows of the decision log (DL), the append-only project journal (the hand-over archive of the main study closes at DL 591). The DL row serves as the clock: DL 31 was written on 2026-09-26, DL 188 on 2026-09-29, and the single VALIDATION read (DL 400) on 2026-10-03. The "study lead" is the person who led the main study (Appendix G).

A.1 Governing rules

The following rules were binding unless a later registered file superseded them.

1. *Register first, read later.* Any rule, grid, arm, contrast, seed set or interpretation is written into an amendment or a DL row, hashed and logged before a value it governs exists or is read. Amendments are never edited; corrections go into a new file, an ERRATUM appended before any read, or a new DL row. The one accidental overwrite (amendment 06F, DL 185) was restored byte-identically (DL 188).
2. *Tuning only on CALIB* (the 32,768-client calibration subset of MAIN) and device-recipe selection only on the UNSEEN_VAL selection panel. MAIN VALIDATION was read once, and the TEST set was looked at once, in the confirmatory test evaluation.
3. *Selections are mechanical and written once, and frozen code stays frozen:* the evaluator (hash `61b64e61...`) has not changed since the A40 block began (DL 180); new code goes into a new tree after tests and an independent review.

A.2 Amendment timeline

Table A.1 lists every amendment with its DL rows, what it fixed and why (shas: first eight hexadecimal characters). The unit of the registered training budget is a pass over the training data (full budget 6.0 passes); K, Q and P codes are registered contrasts (Appendix B); the AI gate reviewer is the AI agent that acted as chief scientist (Appendix G).

Table A.1: Amendment timeline of the pre-registration. Source: hand-over file 03 (governance and pre-registration), section 2.

Id	DL	What it registered	Why
01	26–27, 55	Clarifications: tie chain, manifest ids, endpoint tolerance, strict determinism, evaluation stream	make the contest rules executable

Id	DL	What it registered	Why
02, 02R1, 02R2	59–62, 69–70	Backbone-contest design; model size inside the search; full budget of 6.0 passes; no legacy anchoring; learning-rate grid stratified within each size (`0afab4a0...`, `1fe3258a...`)	study lead: a fair contest (DL 60); a review found size confounded with learning rate (DL 69)
03	84	BIG and SMALL as one family in two sizes; device cap of 170 MB per visit at most (`23600aac...`)	study lead's downstream design (DL 83)
04	87	Realistic downstream plan: retention against C_BIG, no threshold; arms; DP-FedAvg; baselines; ONNX (`cb94d3e3...`)	answers "how much is retained" (DL 86)
05	95	CALIB stage as screen only; 2 candidates × 2 seeds = 8 MAIN runs; one sealed G2 decision (`35d42070...`)	study lead's objection to CALIB-only selection
06	133	Downstream execution after G2: SMALL recipe, FL questions Q1–Q5, C_FULL specification, strata	follows the architecture lock
06A	146	Device selection on UNSEEN_VAL; cold-start bins; communication curves; A40 block; ONNX INT8 subset; analysis freeze	full review of the plan
06B	147–148	FL local-learning-rate screen; contrasts K1–K16; privacy ladder; DP $\varepsilon = 1$ core; FA_Q8 (`96cf91c8...`)	thesis-level research review
06C	150	DP sampling changed to Poisson with a fixed denominator	the fixed-size sampler did not match the accountant (ε would be about 1,388, not 8)
06D	159	A40 as second device class; an executable TEST look; K12, K13 at CONTROLLED_BUDGET; K2 primary (`fac97ce4...`)	two blockers found by deep review (INC-033)
06E	169b–171	LO selection on UNSEEN_VAL; registered seed sets; K16 in an ONNX-CPU block; comparison-key rules	reviews REV-FL-01, REV-ANA-01
06F	184–188	FL learning-rate screen extended to ×4, ×8, ×16 (`182f879b...`)	winner of the tuning stage 1a screen at the grid edge
06G, 06H	183, 186, 193; 189	FedAdam server-optimiser grid with $v_0 = \tau^2$; DP server-optimiser sensitivity (`9c10f132...`)	production practice; FedAdam can turn DP noise into a random walk
06I	196, 199	K17–K19 against C_FULL; "FA" means the tuned recipe; FL_CONST; FA_X2; DP8_FULL; realism disclosures (`1b627f22...`)	realism audit
06J	202, 204	FL tuning stages 2, 3 and 4 strictly sequential; incumbent rule ≥ 0.002; tie band 0.0005 (`162cc01f...`)	tuning-stage research
06K	206–207	tuning stage 7 = PRE + FA_WARM; tuning stage 8 = frozen item tables; K20, K21; never warm-start from a model trained on MAIN	real systems warm-start from permitted data
06L, _C, _D, _E	226, 227, 237, 240	FedAvgM cells; privacy matrix; threat model; LAST anchoring; contrasts v3; K9b, K9c, K20b, K20c; no clipping record under DP	multi-expert review gates
06M, 06M_B, 06O	252, 253, 286	Early warm-start preview on CALIB; tuning stage 7 and 8 grids; FT_FA_WARM; a descriptive C_FULL and PRE table on the non-MAIN UNSEEN_VAL panel	study lead: "train the model first, as real systems do"; a picture without opening MAIN
06N	282	Pretrained line primary (K20b, K21); cold line as ablation; cold FP, FA_1024 and DP chains stopped (labelled (*stopped*))	study lead decision after the run-allocation memo
06P	317 (ERRATUM 1: 324; sha: 331)	FA_WARM* = tuning stage 8; primary pair {K20b, K21} unadjusted; Holm family of 18; warm-method contrasts; K_INIT; P5; contrast registry v4 (`75f8dbad...`)	end-game review D1; the study lead delegated to the AI operations agent and the AI gate reviewer (DL 316)
07	341	External benchmarks, sections B1–B11 (`29beb5e3...`)	study lead: external benchmarks (DL 296, 302)
07 B12–B16	357, 368, 377, 401, 433	DP record per PRE seed; ML-1M tuned-protocol backstop (`9baa0f4e...`) rescinded by B15 (`6044dd04...`); Wave 2 on Amazon 2023 (`d5b062f1...`); ML-1M moved to the A40 class (`85ba5e18...`)	schedule; measured timing; the study lead preferred the full grid; re-review by the AI gate reviewer (DL 404)
PREG2	411	Follow-up analysis (`be0561ab...`); contrast registry P2_CONTRASTS v1, v2 (DL 424), v3 (DL 427)	K21 close to 0 and the D1 diagnostic (DL 405–407)

A.3 Locks, freezes and result files

Table A.2 lists the gates and locks in the order in which they were written, followed by the identifiers needed to verify the delivered records. Contracts are stored under `contracts/` and gates under `company/gates/`.

Table A.2: Locks, freezes and result files with sha prefixes. Source: hand-over file 03, section 3, and the key-identifier table of the master plan (appendix A).

Item	DL	File and sha prefix
Data protocol lock v2 (feature search locked at DL 53)	54–55	`DATA_PROTOCOL_LOCK_V2.json` (`bc699a97...`)
G2 script freeze (the single unsealing); architecture lock; A40 device pin	129–131, 167, 169	`3893a49a...`; `fd201013...`; `db744ca3...`
FL tuning selection (tuning stage 4); warm selections (tuning stages 7 and 8)	271; 292	`9bca186e...`; `d67a0a85...`, `8b79d305...`
Contrast registry v4	336	`ANA_CONTRASTS_v4.json` (`f1769f0b...`)
Analysis freeze v3 (v1 and v2 superseded, never read)	388, 393, 397	`59c83c861ff1`
The single VALIDATION read (check-only pass at DL 398)	400	result file `0d765880...`
ALL_METHODS_LOCK	402	`04a3cd196149`
PREG2; P2_CONTRASTS_v3	411, 427	`be0561ab...`; `673b547d...`
P2_METHODS_LOCK v2; P2 script freeze	510; 476	`b3a2f62a7d98`; `3077963f...`
Main-analysis TEST result file (self sha in report header `516a1e8e...`)	537	`28b7649231ee`
Follow-up-analysis TEST result file	537	`c44382c5c4c0`
TEST exposure register after entry 19	580	`dfdcbaea9055`
Exploratory locks: EXP_PB100 (v1), EXP_XREES46 (T2), EXP_BENCH_X (T1), EXP_PB100 v2 (T3)	540, 562, 564, 576	`6ee1c65b...`; `a67b19ca5052`; `025a59dc2b7e`; `c3a67aafdb3b`
Benchmark TEST result files: s3_beauty, s3_sports, s3_toys, video_games, baby_products, ml_1m	521, 523, 542	`93603eb9...`, `df7f330b...`, `6c4cbe7b...`, `a210da2e...`, `fa5e15d0...`, `76286a8d...`
Server-side pretrained model θ_{pre}; frozen dataset `rees46_nextitem_v1`	—	`cf08588c...`; `860e5a1c3a81`
Final hand-over archive; repository head at hand-over (pull request #112)	591; 588–590	`6dff1f6fb882`; `50d9cea`

A.4 The single VALIDATION read and the confirmatory test evaluation

The end-game ran as steps D (freeze), E (a check-only run that reads no values), F (the single VALIDATION read), G and H, with scripts reviewed by the AI gate reviewer. The value-free check was refused twice, on definitions that could never pass on real data (DL 389, 394); each refusal led to a reviewed fix and a new freeze (v2 `d96116d8...`, v3 `59c83c86...`), and freezes v1 (`03028475...`) and v2 were never read. After the v3 check passed (DL 398), the single VALIDATION read took place on 2026-10-03 (DL 399–400); everything new from then on was labelled post-validation exploratory. ALL_METHODS_LOCK (DL 402) fixed 131 arms, 56 sealed FL checkpoints and the TEST plan (Appendix C).

The main analysis and the follow-up analysis shared one confirmatory test evaluation under conditions set by the AI gate reviewer: both locks and the follow-up-analysis script freeze registered, and two locked-evaluation register entries appended, before the first TEST re-evaluation of any arm; all main-analysis scoring, then all follow-up-analysis scoring, then the two analyses, with nothing under the TEST output

directories opened before both result files existed; no second test evaluation, ever; and no follow-up-analysis MAIN VALIDATION value opened by a human before both analyses were written. A main-analysis look first and a TEST2-only route were refused; the December 2019 REES46 data (TEST2) was declined by the study lead (DL 415), so no such data exist. The test evaluation ran from 2026-10-05T21:03Z to 2026-10-07T10:09Z (register entries 10 and 11, Appendix C).

Chapter B: Complete confirmatory results

This appendix prints the complete confirmatory tables, generated from the public result files, so that every row quoted in Chapter 5 can be checked in context. All are TEST tables, and none mixes in a VALIDATION value. The headline rows are (*confirmatory*) K20b = 88.49% [87.73, 89.18], the retention of the realistic edge system (FA_WARM_FROZEN) against the cloud reference (C_FULL); (*confirmatory*) K21 = −0.000508 [−0.000933, −0.000037], FA_WARM_FROZEN minus the pretrained model PRE; (*confirmatory*) K9c = +0.002283 [0.001588, 0.003076], the gain from on-device fine-tuning; and (*confirmatory, follow-up*) Q1 = −0.000081 [−0.000179, 0.000024], read as EQUIVALENT within ±0.0005. Intervals are 95% paired user-bootstrap intervals with 1,000 resamples [50]; the secondary family of 18 contrasts is Holm-adjusted [51], the primary pair {K20b, K21} is not, and equivalence uses two one-sided tests [52] at the margins ±1% and ±0.0005.

Main-analysis contrasts. Table B.1 lists all 33 main-analysis confirmatory contrasts on the TEST set, each computed on practical-best checkpoints and read under its registered rule; it is the complete version of the headline rows above.

Table B.1: All 33 main-analysis confirmatory contrasts on the REES46 TEST set (client-macro MRR@20, practical-best checkpoints, 26,254 users). Source: `results/rees46/confirmatory_phase1_contrasts.csv`, generated from `RETENTION_RESULT.json` of the confirmatory test evaluation.

ID	Tier	Contrast	Point	95% CI	vs. ref.	Holm	Seeds
K1	secondary	R(C_SMALL / C_BIG)	94.69%	[94.03, 95.31]	below	rejected	2026 2027
K2	secondary	R(FA / C_BIG)	85.11%	[84.30, 85.90]	below	rejected	2026 2027
K3	secondary	R(FA / C_SMALL)	89.88%	[89.15, 90.62]	below	rejected	2026 2027
K4	tertiary	R(FP / C_BIG)	85.11%	[84.30, 85.90]	below	tertiary (unadjusted)	2026 2027
K5	tertiary	R(PF / C_BIG)	85.12%	[84.32, 85.90]	below	tertiary (unadjusted)	2026 2027
K6	tertiary	R(FT_FA / C_BIG)	87.56%	[86.75, 88.33]	below	tertiary (unadjusted)	2026 2027
K7	tertiary	R(FT_C / C_BIG)	96.35%	[95.70, 97.02]	below	tertiary (unadjusted)	2026 2027
K8	secondary	R(LO / C_BIG)	2.10%	[1.88, 2.34]	below	rejected	2026 2027
K9	tertiary	FT_C − C_SMALL	+0.002470	[+0.001955, +0.002993]	above	tertiary (unadjusted)	2026 2027
K9b	secondary	FT_C − C_SMALL	+0.002732	[+0.001512, +0.004171]	above	rejected	2026
K9c	secondary	FT_PRE − PRE	+0.002283	[+0.001588, +0.003076]	above	rejected	2026 2027
K10	tertiary	FT_FA − FA	+0.003637	[+0.003105, +0.004221]	above	tertiary (unadjusted)	2026 2027

ID	Tier	Contrast	Point	95% CI	vs. ref.	Holm	Seeds
K11	tertiary	FA − LO	+0.123378	[+0.121064, +0.125358]	above	tertiary (unadjusted)	2026 2027
K11b	secondary	FA − LAST	+0.050933	[+0.048639, +0.052886]	above	rejected	2026 2027
K12	secondary	R(DP8 / FA_1024)	98.52%	[97.70, 99.38]	below	rejected	2026 2027
K13	tertiary	R(DP1 / FA_1024)	94.38%	[92.53, 96.00]	below	tertiary (unadjusted)	2026
K14	tertiary	R(FA_Q8 / FA)	100.11%	[99.85, 100.38]	includes	tertiary (unadjusted)	2026 2027
K15	tertiary	R(C_BIG / C_FULL)	82.30%	[81.54, 83.10]	below	tertiary (unadjusted)	2026
K16	tertiary	R(INT8 / FP32) for C_SMALL	99.98%	[99.75, 100.23]	includes	tertiary (unadjusted)	2026
K17	secondary	R(FA / C_FULL)	70.10%	[69.19, 70.94]	below	rejected	2026
K18	tertiary	R(FT_C / C_FULL)	79.85%	[79.09, 80.74]	below	tertiary (unadjusted)	2026
K19	tertiary	R(LO / C_FULL)	1.64%	[1.45, 1.88]	below	tertiary (unadjusted)	2026
K20	secondary	R(FA_WARM_FROZEN / C_BIG)	107.39%	[106.50, 108.32]	above	rejected	2026 2027
K20b	primary	R(FA_WARM_FROZEN / C_FULL)	88.49%	[87.73, 89.18]	below	not in a Holm family (read unadjusted)	2026 2027
K20c	secondary	R(PRE / C_FULL)	88.77%	[88.03, 89.47]	below	rejected	2026
K21	primary	FA_WARM_FROZEN − PRE	−0.000508	[−0.000933, −0.000037]	below	not in a Holm family (read unadjusted)	2026 2027
K_INIT	secondary	FA_WARM_FROZEN − FA	+0.033119	[+0.031774, +0.034516]	above	rejected	2026 2027
P5	secondary	FT_FA_WARM_FROZEN − FA_WARM_FROZEN	+0.002045	[+0.001324, +0.002807]	above	rejected	2026 2027
K4w	secondary	R(FP_WARM_FROZEN / C_FULL)	88.49%	[87.73, 89.18]	below	rejected	2026 2027
K5w	secondary	R(PF_WARM_FROZEN / C_FULL)	88.35%	[87.60, 89.07]	below	rejected	2026 2027
K12w	secondary	R(DP8_FROZEN / FA_1024_WARM_FROZEN)	100.00%	[99.95, 100.07]	includes	not rejected	2026 2027
K13w	secondary	R(DP1_WARM_FROZEN / FA_1024_WARM_FROZEN)	100.03%	[99.92, 100.15]	includes	not rejected	2026
K14w	secondary	R(FA_Q8_WARM_FROZEN / FA_WARM_FROZEN)	99.85%	[99.72, 99.98]	below	not rejected	2026 2027

Follow-up contrasts. Table B.2 gives the contrasts of the registered follow-up analysis (PREG2), which asked whether a better-tuned warm-FL recipe could add anything to PRE; Q1 is its primary contrast.

Table B.2: Follow-up confirmatory contrasts (registered follow-up analysis, TEST, client-macro MRR@20). Source: `results/rees46/confirmatory_phase2_contrasts.csv`.

ID	Contrast	Point	95% CI	vs. ref.	Holm / reading
Q1	W_STAR − PRE	−0.000081	[−0.000179, +0.000024]	includes	not in a Holm family (registered reading: equivalent)
Q2	PRE_FT_CENTRAL − PRE	−0.000212	[−0.000628, +0.000208]	includes	not rejected
Q3	W_STAR − PRE_FT_CENTRAL_FROZEN	+0.000166	[+0.000027, +0.000300]	above	not rejected
Q4	W_STAR − FA_WARM_FROZEN	+0.000427	[+0.000007, +0.000893]	above	not rejected
Q5	FT_W_STAR − FT_PRE	−0.000006	[−0.000093, +0.000075]	includes	not rejected
Q6	R(W_STAR_DP8 / W_STAR_FA_1024)	100.00%	[100.00, 100.00]	includes	not rejected
Q7	W_STAR_DP8 − PRE	−0.000037	[−0.000097, +0.000020]	includes	not rejected
Q8	C_FULL_SMALL − PRE	+0.000541	[−0.000457, +0.001591]	includes	not rejected
Q9	R(W_STAR / C_FULL)	88.72%	[87.95, 89.43]	below	tertiary (unadjusted)
Q11	R(W_STAR_DP1 / W_STAR_FA_1024)	99.99%	[99.98, 100.00]	includes	tertiary (unadjusted)

Absolute metrics of every arm. Table B.3 reports the absolute TEST metrics of every main-analysis arm and baseline, the numbers from which all retention values and differences are derived.

Table B.3: Absolute TEST metrics of every main-analysis arm and baseline (mean of the registered seeds). Source: `results/rees46/confirmatory_phase1_arms.csv`.

Arm	Description	MRR@20	NDCG@10	HR@10
C_BIG	central SASRec (d=256, 3 blocks) trained on the 131,072 federated users	0.1486	0.1808	0.3069
C_FULL	central SASRec (d=256, 3 blocks) trained on 1,348,682 users (server users + federated users)	0.1801	0.2169	0.3589
C_SMALL	central device-size SASRec (d=64, 2 blocks) trained on the federated users	0.1407	0.1723	0.2978
FA	federated training (FedAdam server), random start	0.1265	0.1563	0.2748
FP	FedProx (mu 0.01), random start	0.1265	0.1563	0.2748
PF	personalised FedAvg (on-device personal query vector), random start	0.1265	0.1563	0.2748
FA_1024	Poisson-sampled federated training (about 1,024 clients per round, 384 rounds), clipping, no noise	0.0342	0.0423	0.0769
DP8	DP-FedAvg at epsilon 8 (user level), random start	0.0337	0.0420	0.0767
DP1	DP-FedAvg at epsilon 1 (user level), random start	0.0323	0.0408	0.0761
FA_Q8	federated training with 8-bit stochastic-rounding uploads, random start	0.1266	0.1563	0.2748
FA_WARM	federated training of all weights from the pretrained model	0.1584	0.1938	0.3332
FA_WARM_FROZEN	federated training from the pretrained model, item tables frozen (only the encoder is sent)	0.1596	0.1949	0.3342
DP8_FROZEN	DP-FedAvg at epsilon 8 on the warm, frozen-table line	0.1598	0.1950	0.3338
FP_WARM_FROZEN	FedProx on the warm, frozen-table line	0.1596	0.1949	0.3342

Arm	Description	MRR@20	NDCG@10	HR@10
PF_WARM_FROZEN	personalised FedAvg on the warm, frozen-table line	0.1594	0.1947	0.3343
FA_Q8_WARM_FROZEN	8-bit uploads on the warm, frozen-table line	0.1594	0.1948	0.3343
FA_1024_WARM_FROZEN	non-private twin of the warm DP arms (Poisson sampling, clipping, no noise)	0.1598	0.1950	0.3339
DP1_WARM_FROZEN	DP-FedAvg at epsilon 1 on the warm, frozen-table line	0.1599	0.1950	0.3338
PRE	device-size SASRec pretrained on the server on 1,217,610 other users (the shipped warm start)	0.1601	0.1953	0.3341
FT_C	C_SMALL downloaded and fine-tuned on each device	0.1432	0.1751	0.3012
FT_PRE	the pretrained model downloaded and fine-tuned on each device	0.1624	0.1981	0.3382
FT_FA	the cold federated model fine-tuned on each device	0.1301	0.1600	0.2788
FT_FA_WARM	the warm federated model fine-tuned on each device	0.1610	0.1969	0.3377
FT_FA_WARM_FROZEN	the warm frozen-table federated model fine-tuned on each device	0.1617	0.1974	0.3381
LO	local-only training on each device, no collaboration	0.0031	0.0036	0.0056
C_SMALL_FP32	C_SMALL exported to ONNX, FP32, CPU runtime (16,384-user block)	0.1404	0.1723	0.2986
C_SMALL_INT8	C_SMALL exported to ONNX, dynamic INT8, CPU runtime (16,384-user block)	0.1404	0.1721	0.2980
KNN_EMB_SMALL	item kNN on the C_SMALL item embeddings	0.0673	0.0813	0.1376
KNN_EMB_BIG	item kNN on the C_BIG item embeddings	0.0787	0.0942	0.1542
RULE	category / brand affinity plus popularity rule	0.0346	0.0437	0.0843
SKNN	session kNN (k 500, sample 5,000)	0.1109	0.1470	0.2832
LAST	repeat the last viewed item (no training)	0.0756	0.1059	0.2074
POP	global popularity	0.0188	0.0242	0.0481

External benchmarks. Table B.4 reports the TEST results of the six external benchmark datasets of Table 3.5 (the Wave-1 sets are confirmatory under their claim rule; the two Amazon 2023 sets are descriptive) on their TEST sets (matched-recipe protocol, full ranking), averaged over seeds 2026 and 2027.

Table B.4: Confirmatory benchmark results, track A (TEST, full ranking, mean of seeds 2026 and 2027). Source: `results/benchmarks/confirmatory_track_a.csv`.

Dataset	Arm	NDCG@10	HR@10	MRR@20	% of C_FULL
Amazon Beauty	C_FULL	0.0437	0.0699	0.0374	100.0
Amazon Beauty	PRE	0.0056	0.0117	0.0042	12.8
Amazon Beauty	FA	0.0253	0.0418	0.0213	57.8
Amazon Beauty	FA_WARM	0.0063	0.0110	0.0052	14.3
Amazon Beauty	FA_WARM_FROZEN	0.0064	0.0126	0.0051	14.7
Amazon Beauty	FP_WARM_FROZEN	0.0064	0.0126	0.0051	14.7
Amazon Beauty	PF_WARM_FROZEN	0.0064	0.0126	0.0051	14.7
Amazon Beauty	FA_Q8_WARM_FROZEN	0.0065	0.0127	0.0051	14.8
Amazon Beauty	FA_1024_WARM_FROZEN	0.0062	0.0119	0.0049	14.1
Amazon Beauty	DP8_WARM_FROZEN	0.0059	0.0115	0.0047	13.5
Amazon Beauty	FT_FA_WARM_FROZEN	0.0062	0.0120	0.0049	14.2
Amazon Sports	C_FULL	0.0222	0.0372	0.0187	100.0

Dataset	Arm	NDCG@10	HR@10	MRR@20	% of C_FULL
Amazon Sports	PRE	0.0051	0.0097	0.0041	22.8
Amazon Sports	FA	0.0078	0.0142	0.0064	35.3
Amazon Sports	FA_WARM	0.0050	0.0101	0.0039	22.5
Amazon Sports	FA_WARM_FROZEN	0.0049	0.0097	0.0038	22.0
Amazon Sports	FP_WARM_FROZEN	0.0049	0.0097	0.0038	22.0
Amazon Sports	PF_WARM_FROZEN	0.0049	0.0098	0.0038	22.1
Amazon Sports	FA_Q8_WARM_FROZEN	0.0049	0.0097	0.0038	22.0
Amazon Sports	FA_1024_WARM_FROZEN	0.0048	0.0095	0.0038	21.5
Amazon Sports	DP8_WARM_FROZEN	0.0048	0.0096	0.0037	21.5
Amazon Sports	FT_FA_WARM_FROZEN	0.0049	0.0098	0.0039	22.1
Amazon Toys	C_FULL	0.0408	0.0614	0.0357	100.0
Amazon Toys	PRE	0.0037	0.0075	0.0030	9.2
Amazon Toys	FA	0.0054	0.0094	0.0046	13.2
Amazon Toys	FA_WARM	0.0049	0.0089	0.0039	11.9
Amazon Toys	FA_WARM_FROZEN	0.0050	0.0088	0.0042	12.4
Amazon Toys	FP_WARM_FROZEN	0.0050	0.0088	0.0042	12.4
Amazon Toys	PF_WARM_FROZEN	0.0050	0.0087	0.0042	12.3
Amazon Toys	FA_Q8_WARM_FROZEN	0.0050	0.0086	0.0042	12.2
Amazon Toys	FA_1024_WARM_FROZEN	0.0054	0.0095	0.0045	13.2
Amazon Toys	DP8_WARM_FROZEN	0.0052	0.0093	0.0043	12.7
Amazon Toys	FT_FA_WARM_FROZEN	0.0046	0.0082	0.0038	11.2
MovieLens-1M	C_FULL	0.1951	0.3309	0.1608	100.0
MovieLens-1M	PRE	0.1049	0.1876	0.0851	53.8
MovieLens-1M	FA	0.1504	0.2623	0.1227	77.1
MovieLens-1M	FA_WARM	0.1309	0.2286	0.1075	67.1
MovieLens-1M	FA_WARM_FROZEN	0.1039	0.1873	0.0842	53.2
MovieLens-1M	FP_WARM	0.1308	0.2285	0.1074	67.0
MovieLens-1M	PF_WARM	0.1310	0.2286	0.1077	67.2
MovieLens-1M	FA_Q8_WARM	0.1308	0.2286	0.1074	67.1
MovieLens-1M	FA_1024_WARM	0.1208	0.2117	0.0985	61.9
MovieLens-1M	DP8_WARM	0.1063	0.1910	0.0861	54.5
MovieLens-1M	FT_FA_WARM	0.1324	0.2313	0.1090	67.8
Amazon 2023 Video Games	C_FULL	0.0503	0.0880	0.0415	100.0
Amazon 2023 Video Games	PRE	0.0181	0.0344	0.0144	35.9
Amazon 2023 Video Games	FA	0.0240	0.0450	0.0193	47.8
Amazon 2023 Video Games	FA_WARM	0.0192	0.0363	0.0153	38.1
Amazon 2023 Video Games	FA_WARM_FROZEN	0.0179	0.0344	0.0142	35.6
Amazon 2023 Video Games	FP_WARM	0.0192	0.0364	0.0153	38.2
Amazon 2023 Video Games	PF_WARM	0.0192	0.0363	0.0153	38.2
Amazon 2023 Video Games	FA_Q8_WARM	0.0192	0.0364	0.0153	38.2
Amazon 2023 Video Games	FA_1024_WARM	0.0180	0.0344	0.0143	35.8
Amazon 2023 Video Games	DP8_WARM	0.0180	0.0344	0.0143	35.7
Amazon 2023 Video Games	FT_FA_WARM	0.0197	0.0373	0.0157	39.1
Amazon 2023 Baby Products	C_FULL	0.0182	0.0333	0.0149	100.0
Amazon 2023 Baby Products	PRE	0.0088	0.0184	0.0068	48.2

Dataset	Arm	NDCG@10	HR@10	MRR@20	% of C_FULL
Amazon 2023 Baby Products	FA	0.0099	0.0198	0.0079	54.7
Amazon 2023 Baby Products	FA_WARM	0.0094	0.0191	0.0074	51.7
Amazon 2023 Baby Products	FA_WARM_FROZEN	0.0084	0.0173	0.0066	46.5
Amazon 2023 Baby Products	FP_WARM	0.0094	0.0191	0.0074	51.7
Amazon 2023 Baby Products	PF_WARM	0.0094	0.0191	0.0074	51.7
Amazon 2023 Baby Products	FA_Q8_WARM	0.0094	0.0191	0.0074	51.7
Amazon 2023 Baby Products	FA_1024_WARM	0.0081	0.0164	0.0064	44.4
Amazon 2023 Baby Products	DP8_WARM	0.0081	0.0164	0.0064	44.5
Amazon 2023 Baby Products	FT_FA_WARM	0.0095	0.0192	0.0074	52.1

Chapter C: TEST exposure register

Every contact with the TEST data was recorded in the TEST exposure register, a hash-chained file that is never edited. A pre-scoring registration entry is appended *before* scoring and binds the exact lock, run identity and checkpoint hashes that the scorer may use; a post-scoring disclosure entry is appended *after* scoring and records that the values were read. The chain head was `644f6efd...` after entry 6 (DL 407) and `dfdcbaea...` after entry 19 (DL 580). Table C.1 lists all 23 entries, grouped by look from entry 12.

Table C.1: TEST exposure register, entries 1 to 23. Entries 12 to 23 follow the single test evaluation and are post-hoc exploratory. Source: hand-over file 03, section 6, and the exploratory-look records of the main-study team and of the closure-phase extension of the register.

#	When (UTC)	Purpose; observation	What was exposed or bound
1, 2, 4	2026-09-26	structural counts; counts only	census of the TEST-window user and row counts (1); a local schema check (2, incident INC-009) and a data-build step (4) printed TEST status counts
3	2026-09-26	historical exposure; metrics	the earlier GRU and SASRec pilot studies had looked at the legacy TEST window, so every TEST result of the main study is retrospective
5, 6	2026-10-03 15:28Z, 15:43Z	manifest build; integrity check	entry 5 binds ALL_METHODS_LOCK (`04a3cd19...`) before any TEST manifest existed (DL 402); entry 6 records the manifest build (header `bfaa2118...`), no model scored (DL 407)
7, 8, 9	before the test evaluation	locked evaluation	entry 7: main-analysis lock; entry 8: P2_METHODS_LOCK v1 (superseded); entry 9: P2_METHODS_LOCK v2 (`b3a2f62a...`), after the scoring-tool fix of DL 508–510
10	2026-10-05T21:03Z to 2026-10-07T10:09Z	locked evaluation; metrics	(*confirmatory*) test evaluation, main-analysis look: the locked plan of 109 re-evaluations, 39 panel re-evaluations, 14 device runs, 6 baselines and the ONNX K16 step
11	same test evaluation	locked evaluation; metrics	(*confirmatory, follow-up*) test evaluation, follow-up-analysis look under P2_METHODS_LOCK v2
12, 13	after the test evaluation	pre-scoring registration; post-scoring disclosure	(*exploratory*) 100k-user pretraining variant v1 (weak server pretrain), lock EXP_PB100 (`6ee1c65b...`)
14, 17	after the test evaluation	pre-scoring registration; post-scoring disclosure	(*exploratory*) T2, tau-fixed REES46 re-runs, lock EXP_XREES46 (`a67b19ca...`)
15, 16	after the test evaluation	pre-scoring registration; post-scoring disclosure	(*exploratory*) T1, fixed-recipe benchmarks, lock EXP_BENCH_X (`025a59dc...`)
18, 19	after the test evaluation	pre-scoring registration; post-scoring disclosure	(*exploratory*) T3, 100k-user pretraining variant v2, lock EXP_PB100 v2 (`c3a67aaf...`)
20, 22	closure phase	pre-scoring registration; post-scoring disclosure	(*exploratory*) closure-phase benchmark look, disjoint server-data and capacity studies
21, 23	closure phase	pre-scoring registration; post-scoring disclosure	(*exploratory*) closure-phase DP8 look

Entries 10 and 11 are the two confirmatory looks of the one test evaluation; entries 12 to 23 are post-hoc exploratory looks, each a disclosed additional exposure of the TEST data and reported beside, never instead of, the confirmatory numbers. The register shows that the TEST set was sealed and not used for selection; it does not show that it was never opened, and entries 1 to 4 say so.

Chapter D: Incident and fix log

This log condenses the incident record of the hand-over archive (file 19). Every row is something that went wrong, was refused, was stopped or was corrected, and how it was handled. "Impact" has three values: *none* (no result changed; the problem was caught first or the artefact was never used), *rerun* (something was re-run or replaced) and *disclosed* (a result stands but the problem must be stated). "Fail-closed" means that a guard refused to run, so nothing wrong was produced. Early incidents carry the identifier INC-nnn; later events are identified by their DL rows. The AI operations agent ran the server and the decision log of the main study. Routine events such as quota cuts, stale local copies and clock slips are left out; they did not affect results.

Table D.1: Condensed incident and fix log. Source: hand-over file 19 (incident and fix log) and the master plan, section 8.2.

ID / DL	What happened	Impact	Fix and lesson
INC-004 (DL 16, 20)	Chunked FP32 scoring is not bit-identical to a full matrix product (up to about 5×10^{-5}); near-tie ranks can move by 1	disclosed	scoring block size registered as a constant of the comparison key
INC-009 (DL 33)	A local schema check read TEST status columns (counts only); a superseded census run held earlier-model reciprocal-rank levels nobody had asked for	disclosed	register created (entries 1–3); the run was quarantined in place, not deleted
INC-015 (DL 65, 67, 73)	Independent review rejected the first integration packet: every production launch would have failed while building the data providers	none (caught before production)	revised packet re-verified by a delta review
INC-024 (DL 105, 109)	A tuning status file carrying one intermediate value of one SASRec trial was printed	disclosed	no decision depended on it; print fields, not files
INC-031 (DL 144, 145)	Server shut down earlier than the study lead had told the administrator; the kNN-embedding baseline was cut at about 95 min	rerun	baseline re-run after the upgrade
INC-033 (DL 159)	Deep review found two blockers: the trainer refused every device class except A16, and no executable TEST path existed	none (caught before any downstream run)	amendment 06D; test the end of the path before it is needed
INC-035 (DL 162, 164–168)	An overnight plan written with OpenAI Codex ran serially (the learning-rate screen took about 8 h instead of about 2.7 h) and left out FP, LO and FT_C	none on results; about half a day lost	queue stopped before any run started; Claude agents asked to fix and verify
INC-039 (DL 180, 181)	A post-pack edit changed the evaluator code hash (`f1d4c717` instead of `61b64e61`)	none	reverted byte-identically; evaluator frozen
INC-040 (DL 181, 183, 184, 218)	The learning-rate screen's winner (×2) lay at the top edge of the grid and was still rising; MAIN FL runs had started	rerun	amendment 06F extended the screen, winner ×8 (interior)
INC-041 (DL 176, 179, 181)	Three FL runs started on a GPU beside the C_FULL training, against the one-central-job-per-card rule	disclosed	deviation registered; strict numerics do not depend on co-scheduling
INC-042 (DL 183–186, 188)	A file write overwrote amendment 06F, registered minutes earlier	none (no run had read either text)	restored byte-identically (`182f879b...`)
INC-043 (DL 192, 226)	Realism audit: FL as first registered is not a production recipe (no server optimiser, fresh client AdamW, central round schedule); non-DP updates reveal viewed items	disclosed	claims reworded; amendments 06G to 06J added the missing stages

ID / DL	What happened	Impact	Fix and lesson
DL 389, 394	Two value-free analysis refusals before the VALIDATION read: analysis definitions disagreed with the registered writers	none (freezes v1, v2 never read)	reviewed fixes ANA-07 and ANA-08; new freezes v2, v3
DL 497	On ML-1M, process spawn re-imported a module whose import-time line blanked the device variable; 1,300+ workers died	disclosed (B16 erratum)	spawn-safe wrapper, no change to the locked tree
DL 508–510	The test evaluation stopped at its dry run: the follow-up-analysis scoring tool tested the endpoint by passes over the training data ≥ 6.0, while the $\mathsf{W}^\star$ seed-2027 run ended at round T with 5.999705 passes	none (no TEST row had been read)	tool fix, new scoring tree `code_p2r4`, delta review by the AI gate reviewer, lock v2, register entry 9
DL 528 (I055)	An AI worker agent's copy kept read-only modes and its patch did not copy; refusal tests ran on the unpatched tree: a `final` call on s3_toys appended a refusal line to the attempts file, and a CPU smoke overwrote an old smoke directory	disclosed (no value read; TEST files unchanged; s3_toys TEST attempts = 2)	point every test at a scratch root
DL 535, 536	The 14 device parts of the test evaluation would have taken 2–4 days sequentially; the deployed tree `code_p2r4` held 710 writable files at step 8	disclosed; none	11 helpers with identical command and environment under a 7-condition gate (21.2 h); digest verified equal to the lock before and after making the tree read-only
DL 541	Windows temp-file cleanup deleted the stored token path; monitoring went blind	none (all jobs detached)	credentials moved to a stable path
DL 548–551	Budget extensions: the engine hard-codes the endpoint of 6 passes, then the CALIB launch guard refused 2× and 4× rounds	salvaged by resuming	fixes only in the wrapper copy, recorded per run; never patch the engine

The table shows three patterns. First, most incidents were caught by fail-closed guards or independent reviews before a production run (INC-015, 033 and the two refused checks of DL 389 and 394), which is the behaviour the protocol was built to produce. Second, within this table two incidents forced a re-run (INC-031 and INC-040); the others were caught before they could affect a result or are disclosed in the book where a result stands. None is recorded as having changed a rule after the value it governs had been read. Third, the recurring lessons are operational: hash every starting point, never edit a frozen tree, anchor process patterns, keep credentials and state in files, and treat a refusal as information.

Chapter E: Experiment register

This register lists the experiment families in order, with purpose, key outcome and evidence label; it condenses file 18 of the hand-over archive. *CALIB* is the 32,768-client calibration subset of MAIN used for all tuning; *PB* (practical best) is the best validated checkpoint within the budget; passes over the training data (epochs) are the unit of the training budget; the tuned warm-start federated model (table notation W*) is the warm-FL winner of the follow-up analysis; the *100k-user pretraining variant* denotes the exploratory studies on a weak server pretrain of 100,000 users. Metrics are client-macro MRR@20 unless stated; VALIDATION values are maxima over checkpoint marks and are not final.

Table E.1: Condensed experiment register. Source: hand-over file 18 (experiment register); test-evaluation results from the result files of Table A.2.

ID	Experiment	Purpose and outcome	Label
Data and set-up			
D01–D08	census, locks, feature search	The 5–100 pool holds 1,446,986 users; lock v2 built 15 production manifests (MAIN 131,072 users, CALIB 32,768 users); the feature search selected S*, 25 information channels, with a seed-mean relative drop of −0.90% (GRU) and −0.13% (SASRec) against the reference schema	(*descriptive*), (*exploratory*)
D09–D13	cohorts and replay	C_FULL cohort 1,348,682 users and 26,677,600 decisions; pretraining cohort 1,217,610 users and 24,089,824 decisions	(*descriptive*)
Backbone contest and reference models			
C07–C08	tuning on CALIB	Tuned GRU recipe, two-seed mean 0.124844 against 0.115286 for the best D256 run; SASRec two-seed means 0.132402 (D256, 3 blocks), 0.130120 (D256, 2 blocks), 0.127326 (D384, 2 blocks), all at the lowest grid learning rate 0.000400, an edge that was not extended	(*descriptive*)
C10–C12	MAIN contest, G2 decision	Four GRU and four SASRec runs sealed and read once. SASRec was selected under the registered rule T1 (best validated checkpoint within the budget of 6.0 passes): GRU minus SASRec −0.004271 [−0.005180, −0.003261]. At the fixed endpoint of 6.0 passes the order reverses: +0.004213 [+0.003283, +0.005216] (*descriptive*)	(*validation*)
C14–C19	baselines and reference models	On MAIN VALIDATION: session-kNN 0.11138, last item 0.07111, popularity 0.01943 (*descriptive*). Device-size recipe registered (peak learning rate 0.000711; cap of 165,680,392 bytes per visit). On the non-MAIN new-user panel (10,237 rankable users): C_FULL 0.195204 and 0.194014 (seeds 2026, 2027), PRE 0.173081	(*confirmatory*), (*descriptive*)
Main analysis: the registered deployment study on REES46			
P1-05 to P1-10	FL tuning stages on CALIB	PB values: tuning stage 1 learning-rate screen at ×0.5, ×1, ×2: 0.020263, 0.023473, 0.025562 (winner at the grid edge); stage 1b at ×4, ×8, ×16: 0.027570, 0.028796, 0.022909 (winner ×8); stage 2 FedAdam server step ($\tau = 10^{-3}$) 0.034247; stage 3 constant round learning rate with client SGD with momentum 0.094936; stage 4 one local pass 0.107562 (batch 8: 0.088351; four passes: 0.077878)	(*descriptive*)
P1-12 to P1-14	warm start	Preview on CALIB: FA_WARM_FROZEN 0.1673, while full-model FA_WARM collapsed (0.1582 at mark 0.5, 0.038 at mark 6.0) (*descriptive*). The tuning stage 8 screen winner scored 0.170709, a gain of +0.002248 over the incumbent 0.168461, and became FA_WARM*	(*exploratory*)

ID	Experiment	Purpose and outcome	Label
P1-15 to P1-23	device and FL arms on MAIN	Device recipes chosen on UNSEEN_VAL; then, on MAIN and with two seeds each, LO, FT_C, FT_PRE, cold FA, FA_WARM, FA_WARM_FROZEN, fine-tuned variants and the warm methods FP, PF, FA_Q8, all sealed before the VALIDATION read. DP clip norm on the cold line $S = 0.284363$ (20,625 norms); the DP chain FA_1024, DP8, DP1 (see the correction below)	(*confirmatory*)
P1-25	K16, INT8 device-size model (ONNX CPU)	Pass 1 ran as the registered fallback on 16,384 users (the full run timed out at 2 h); INT8 retention 99.98 on TEST	(*confirmatory*)
P1-30	single VALIDATION read	K20b 88.81% [88.27, 89.36]; K21 −0.000433 [−0.000758, −0.000101]; K9c +0.003976 [0.003329, 0.004640]; maxima over marks, not the final numbers	(*validation*)
TEST	confirmatory test evaluation, main and follow-up analyses	K20b 88.49% [87.73, 89.18]; K21 −0.000508 [−0.000933, −0.000037]; K9c +0.002283 [0.001588, 0.003076]; Follow-up analysis: Q1 −0.000081 [−0.000179, 0.000024], EQUIVALENT within ±0.0005	(*confirmatory*), (*confirmatory, follow-up*)
PREG2	Follow-up analysis	Registered after K21 proved close to 0; the winner W* on MAIN moved only about 0.06% from θ_{pre} in 13,654 rounds	(*confirmatory, follow-up*), (*descriptive*)
07 B1–B16	six external benchmarks	Amazon Beauty, Sports, Toys [10]; Video_Games and Baby_Products from Amazon Reviews 2023 [11]; MovieLens-1M [12]. Matched-recipe protocol on TEST, ML-1M: C_FULL 0.1936, FA 0.1572, PRE 0.1092; s3_toys had two TEST attempts (one look, one refusal, I055); values at the fixed endpoint (CB)	(*confirmatory*), (*descriptive*)
After the test evaluation: exploratory studies (register entries 12 to 19)			
tau diagnosis	why K21 is about 0	The registered FedAdam $\tau = 10^{-3}$ met client pseudo-gradients of about 10^{-7}, so the server step was 20 to 100 times too small; with $\tau = 10^{-6}$ FL learns. The re-runs changed several ingredients at once (tau with the server learning rate, unfrozen items or the budget), so the gain is not attributed to tau alone	(*exploratory*)
100k-user variant v1, v2; T1	weak pretrain; fixed-recipe benchmarks	FL adds a clear gain over a weak 100,000-user pretrain (Figure 5.10), unlike over the 1.2M-user pretrain; with the repaired recipe FL reaches 75 to 92% of C_FULL on the benchmarks (Figure 5.9)	(*exploratory*)
T2	tau-fixed REES46	Warm run minus PRE +0.00022 [−0.00011, 0.00057], which includes 0, so the confirmatory answer is robust	(*exploratory*)
disjoint server-data study, privacy redo	validation-only studies	Server-data size and capacity decomposition (Figures 5.11 and 5.12); membership-inference AUC 0.747 on cold FL [38]; the legacy DP8 audit model shows a membership AUC of 0.502 (no formal DP guarantee), with a small utility cost on a warm start and a large one on a cold start (Figure 5.13)	(*validation*), (*exploratory*)
Closure audit, corrections and extensions (2026-10-12 to 2026-10-17)			
update-leakage audit, canary	privacy audits	Update-leakage audit: aggregate-input attack advantage about 2.625 percentage points (95% CI 2.312–2.953), above the fixed 2 pp threshold: a negative finding. The canary design has a ceiling of $\varepsilon \approx 5.213$, below 8, and cannot reject $\varepsilon = 8$	(*exploratory*), (*descriptive*)
corrections	DP, SecAgg, latency	*No formal DP guarantee:* the legacy DP implementation used a disclosed public seed, so its noise and Poisson streams can be reconstructed; the utility numbers remain measurements. Secure aggregation is a simulation only, not production cryptography [36]. The GRU against C_SMALL comparison is a CPU rehearsal. L6 latency is single-thread CPU latency on a shared server, not a laptop or phone	(*descriptive*)
disclosure entries 22, 23	disjoint server-data and capacity benchmarks, DP8	Gated exploratory TEST looks of the closure-phase extension of the register (entries 20 to 23, Appendix C)	(*exploratory*)
Stopped, failed or never run			

ID	Experiment	Purpose and outcome	Label
P1-04 to P1-29; closure	stopped, failed, not run	Serial overnight queue; first MAIN FL launch at ×2 (stopped before any mark was read); early screens of tuning stages 7 and 8; first cold FP and FA_1024 allocation; cold-start Studies A and B; FA_X2 and DP optimiser sensitivity (declared not run, DL 317). In the closure, DP-FTRL and the canary restart were stopped by the study lead, the ColdDP calibrations failed before training, and no TEST2 (December 2019) data exist	(*stopped*)

Chapter F: Reproducibility and source code guide

The delivered folder `2_Source_Code` follows the history of the project in four parts. The project is closed: re-running training or TEST scoring is not part of the deliverable, because a new TEST evaluation would require the full gated protocol of Chapter 3. What a reader can reproduce is the environment, the quality checks and the test suites of the public repository, and every confirmatory table of this report from the saved result files.

1. `1_github_repository`: the public repository at commit `89b61f2` (`main`), made by the whole team between August and October 2026. It holds the prototype platform and the public package of the main study: `ppsi/`, `scripts/`, `tests/`, `docs/`, `notebooks/`, `results/`, `pyproject.toml` and `uv.lock`.
2. `2_v4_research_code`: the research code as it ran on the GPU cluster (the main study, with AI assistance, and the closure extensions): `code/` with its test suites, the frozen analysis and register tools in `tools/`, the exploratory tau-fixed workstreams in `x_rees46/`, the fixed-recipe benchmark code in `code_bench_x*/`, and `deploy_archive/` with deployment manifests and the A40 environment lock.
3. `3_frozen_test_scoring_tree`: `code_p2r4`, the read-only, hash-locked tree that scored the confirmatory test evaluation of 2026-10-05 to 2026-10-07; it is kept for traceability and must not be edited.
4. `4_closure_and_delivery_code`: The technical-closure verifiers (2026-10-12 to 2026-10-17): local suite runners, inventory and delivery verification, update-norm summaries and update-leakage audit replay checks.

F.1 Running the public repository

The repository uses `uv` with a locked environment (Python 3.11.14). The following commands build the environment and run the checks of the continuous-integration job; `scripts/check.py` runs the lock check, the installation, the `ruff` linter, the contract smoke test, the environment smoke test and the pytest suites in that order.

```
cd 1_github_repository; uv python install 3.11.14; uv sync -locked -group
dev; uv run -locked python scripts/env_smoke.py; uv run python
scripts/check.py
```

The suites can also be run one at a time with `uvrunpython-mpytesttests/<component>-q`, for the components `seqrec` (SASRec [2] and GRU4Rec [1]), `fedsim` (federated simulator, FedAdam server step and its τ test), `evaluation/fullrank` (full-catalogue evaluator and paired bootstrap), `ondevice` (ONNX and INT8 benchmark) and `benchrun` (public benchmark runner). After any change to `pyproject.toml`,

the lock is refreshed with `uv lock` and committed with it. By the team's rule nothing is pushed directly to `main`: changes go through branches, pull requests, a green quality check and a merge commit. The head at the hand-over document was `50d9cea`; the delivered snapshot is `89b61f2`.

F.2 Result files, data and checkpoints

The result tables are the CSV files in `1_github_repository/results/rees46/` and `results/benchmarks/` (nine files and a README in each); Appendix B is generated from `confirmatory_phase1_contrasts.csv`, `confirmatory_phase2_contrasts.csv`, `confirmatory_phase1_arms.csv` and `confirmatory_track_a.csv`, and the notebook `notebooks/final_results.ipynb` redraws the figures from them. The CSV files are derived from the primary result files of the test evaluation (Table A.2); if a table and a primary file ever disagree, the primary file wins.

The data is not redistributed. The raw REES46 log is public [8]; `docs/data-source.md` and `docs/data-access.md` explain how to download it and check its SHA-256 value. The frozen derived dataset `rees46_nextitem_v1`, the model checkpoints and the run outputs live on the team's server of the Lobot GPU cluster and are not part of the delivery. The research code of the main study expects that server layout and its GPU environment (PyTorch [48] on NVIDIA A40 cards); the exact package versions are in `deploy_archive/ENV_LOCK_A40.txt` and `ENV_PKGS_A40.txt`. The public repository deliberately omits the REES46 launchers and data build, governance material, per-user files, checkpoints and tokens. The whole folder excludes checkpoints and arrays (`*.pt`, `*.npz`, `*.npy`, `*.parquet`), caches, files holding private run state or raw privacy truth (`*PRIVATE*`) and credentials; a scan for API tokens and private keys found none.

Chapter G: Contribution and AI-use statements

The project is the joint work of the three members of the group named on the title page, who share responsibility for all of it. The problem framing comes from Microsoft Edge: a browser could offer shopping recommendations from behaviour that happens inside it. The supervisors are Prof. Rongxing Lu (Queen's University), Prof. Mona Farouk (Cairo University) and, as industry mentor, Prof. Motaz El-Saban (Microsoft). The work divided as follows.

- *Prototype stage (GitHub, August to September 2026), a team effort.* The sprint ran three workstreams: a data and model workstream; a federated-training workstream, which also ran the pilots; and a system, deployment and security workstream, which delivered blake2b-64 hashing [53], price bands from `price_transform_v1`, the capped log time gap `log1p(min(gap,86400))`, ONNX export at opset 17 [47], INT8 dynamic quantisation [46], the communication byte ledger, the quality-retention registry, continuous integration and `scripts/check.py`.
- *Main study (2026-09-26 to 2026-10-11).* The study was run by a single study lead, who took its binding decisions, working with AI agents.
- *Technical closure and extensions (2026-10-12 to 2026-10-17).* The closure phase verified and closed the delivery and ran the update-leakage audit, the canary and secure-aggregation checks, the corrections of Appendix E and the TEST looks of the closure-phase extension of the register.
- *Final consolidation (October 2026).* The records and this report were consolidated into one verified archive, and a shared Lobot cluster workspace was set up.

Use of AI tools. This project is our own work. We defined the research question, designed the study and its pre-registered protocol, prepared the data, chose and configured the models, ran the experiments, made the decisions and interpreted the results; the few decisions we delegated to an AI agent are recorded in the decision log. We used AI assistants as tools along the way: Anthropic's Claude models and, early in the project, OpenAI Codex. They helped us write and debug code, brainstorm ideas and run routine checks, and they assisted with the writing, layout and proofreading of this report. We reviewed and verified everything they produced against our saved results, and we take full responsibility for all of the work. The technical reviews mentioned in this report were AI-assisted, not reviews by human experts.

The independent reviews in this project were AI reviews, not human expert reviews: AI-assisted technical reviews of code, gates and artefacts, not external peer review and not an independent reproduction. No human expert reviewed the code. The study lead took the binding decisions, but delegated some end-game decisions to the AI operations agent and the AI gate reviewer (DL 316 and 384); rulings made by the AI gate reviewer as the study lead's proxy are not presented as the study lead's own decisions. By team rule, commits to the public repository carry the author's own identity and no AI attribution (DL 454), so the

repository history alone does not show the AI-assisted workflow; this appendix and the decision log do.